Introduction to Neural Shading





Chris Cummings

ccummings@nvidia.com



Benedikt Bitterli

bbitterli@nvidia.com



Kai Zhang

kazhang@nvidia.com



Shannon Woods

swoods@nvidia.com

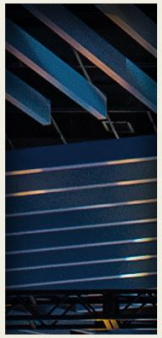
Today's Agenda

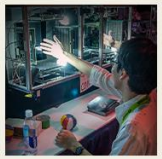
Section
Introduction
Fundamentals
Basic Primitives of Neural Shading
Your First MLP in a Shader
Learning a Neural Texture
Automatic Differentiation
Break
Hardware Acceleration
Simple Neural Materials
From Compression to Learning
What We Didn't Cover
Conclusion and Q&A

Course Material:

https://shader-slang.org/landing/siggraph-26/

© 2026 SIGGRAPH. ALL RIGHTS RESERVED.



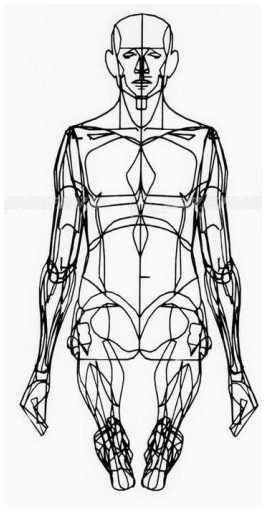




SIGGRAPH 2026
Los Angeles 19–23 JUL

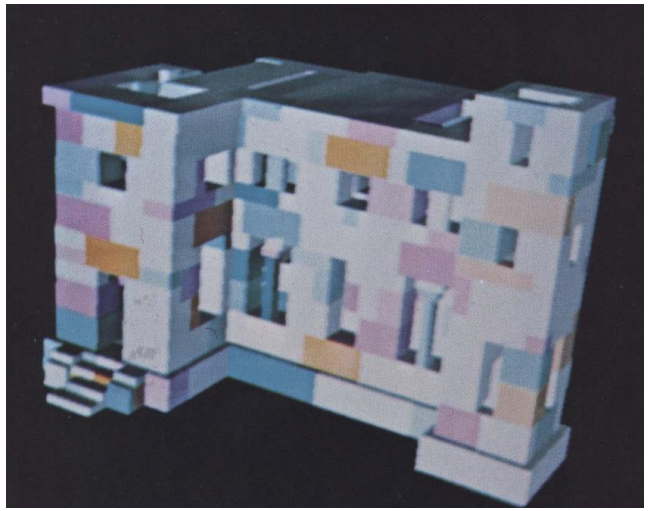
Introduction

The Evolution of Graphics



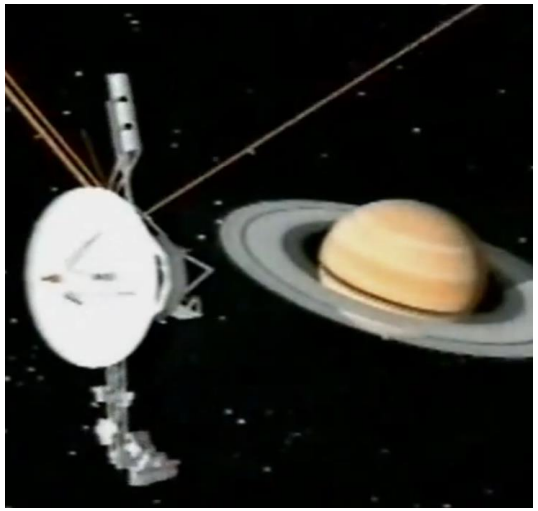
Wireframes

William Fetter, 1964



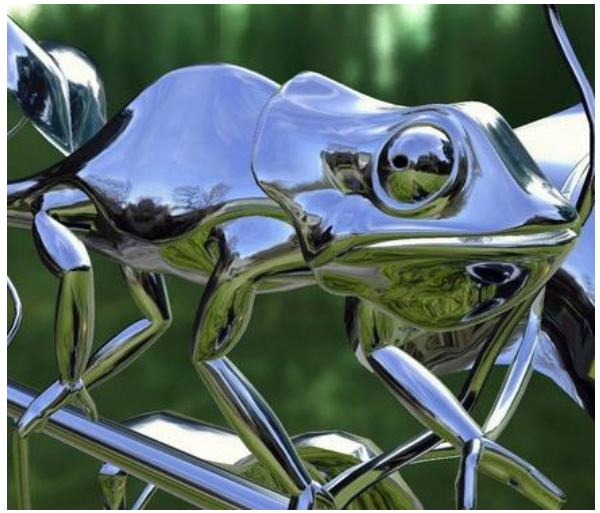
Shaded Surfaces

Gary Watkins, 1970



Texture Mapping

Jim Blinn, Charles Kohlhase, 1981



Programmable Shading

NVIDIA 2001



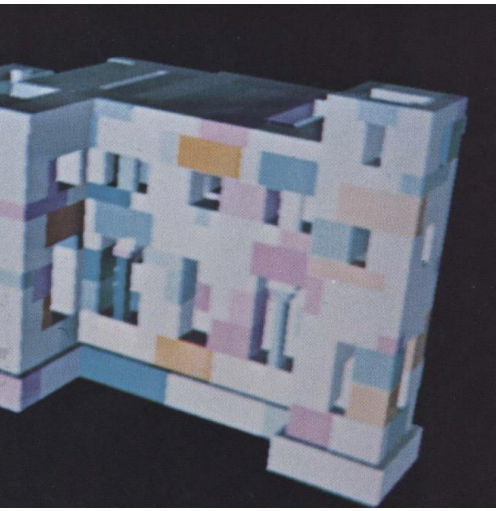
Raytracing

NVIDIA 2019

The Evolution of Graphics

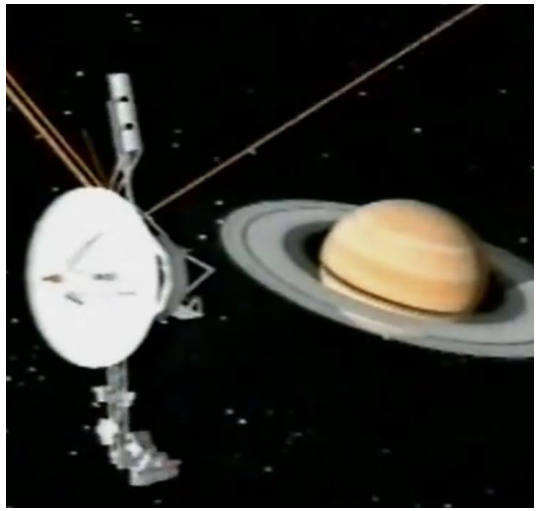


Shaded Surfaces



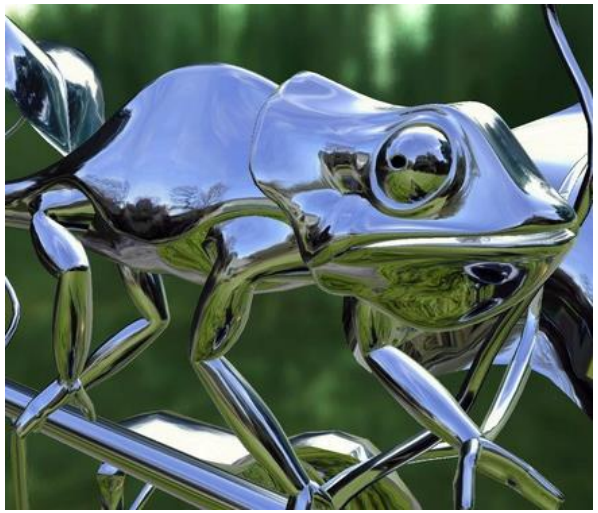
Gary Watkins, 1970

Texture Mapping



Jim Blinn, Charles Kohlhase, 1981

Programmable Shading

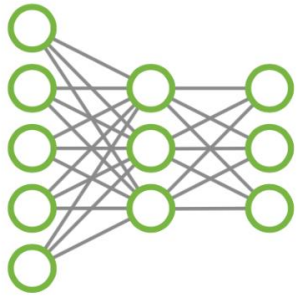


NVIDIA 2001

Raytracing



NVIDIA 2019

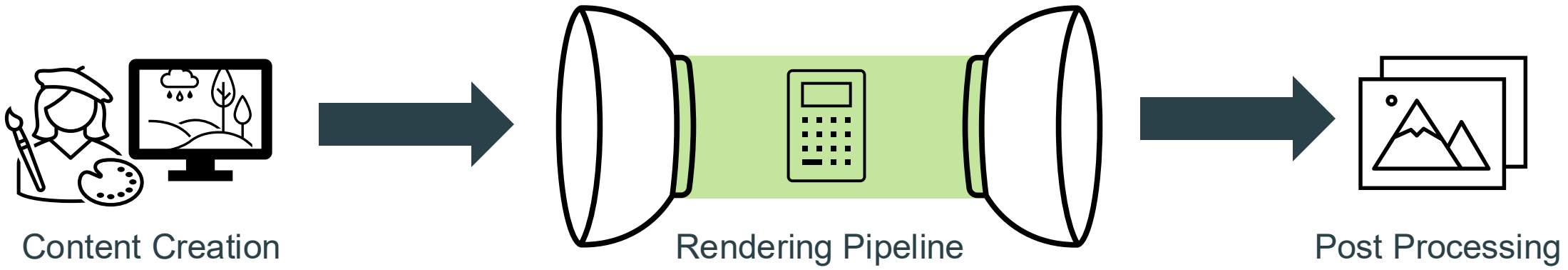






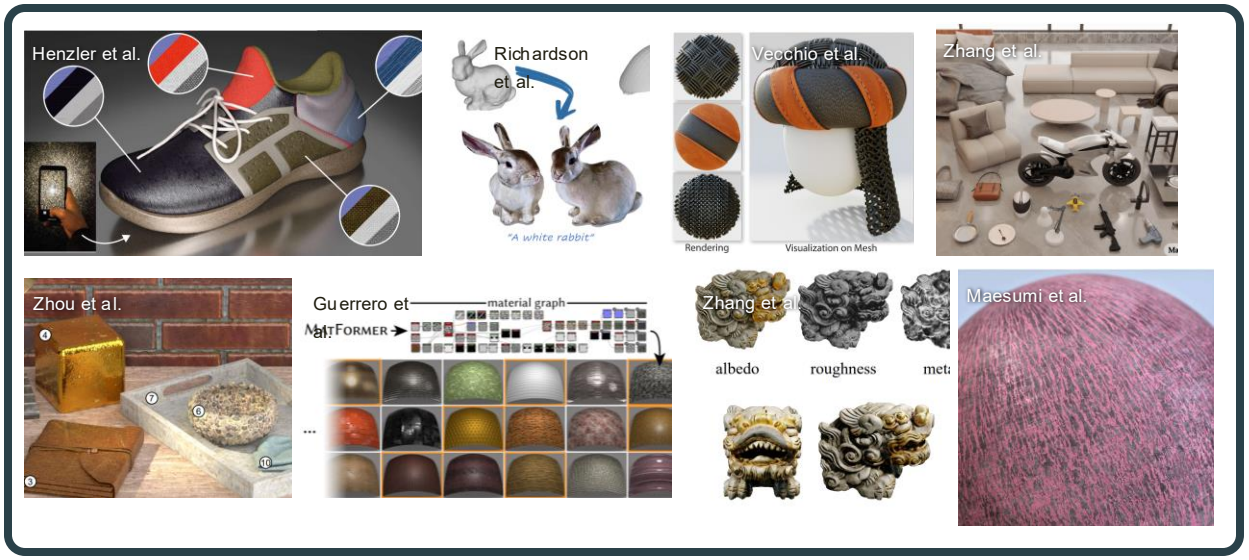


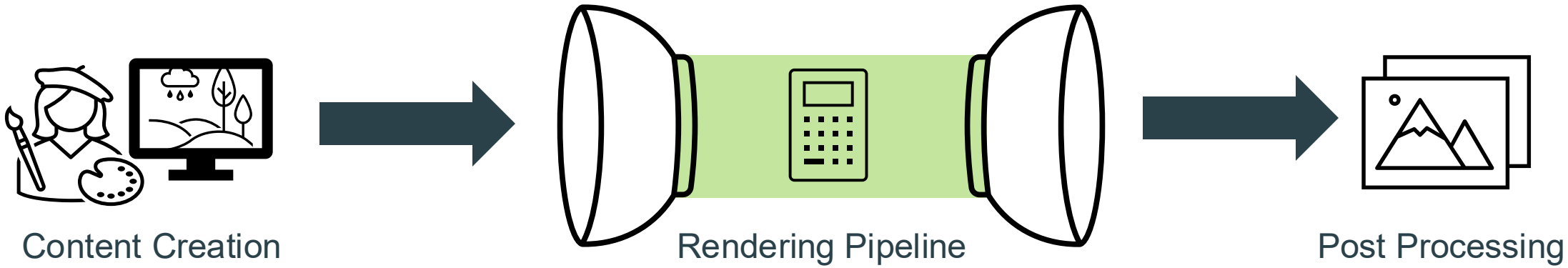
Neural Networks and the Graphics Pipeline



© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Neural Networks and the Graphics Pipeline





© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

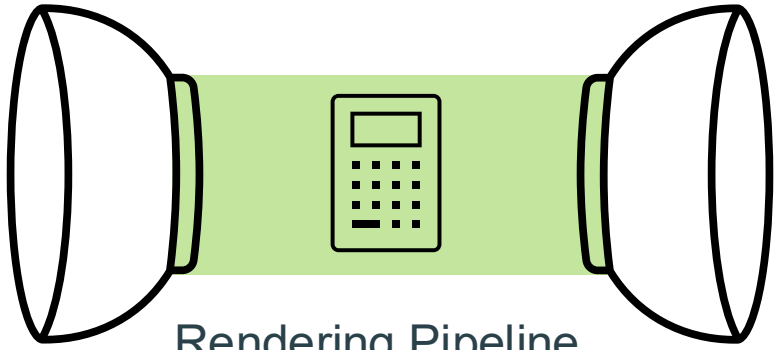
Neural Networks and t



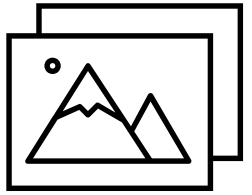




Content Creation



Rendering Pipeline



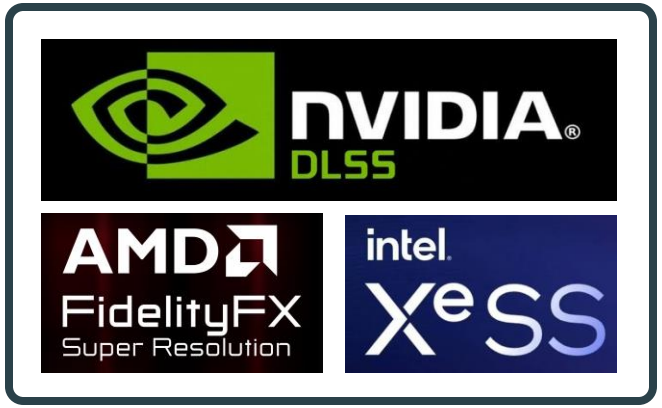
Post Processing

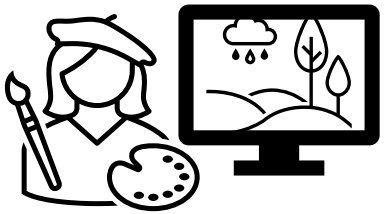
© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Neural Networks and the Graphics Pipeline

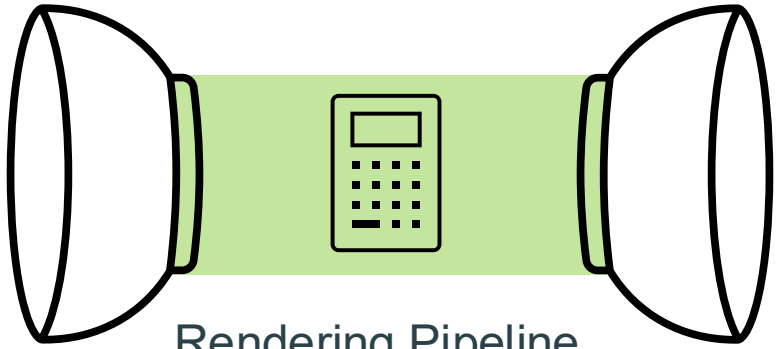




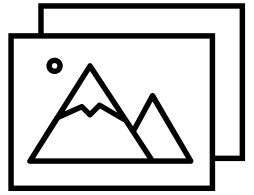




Content Creation



Rendering Pipeline



Post Processing

© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Graphics without the Graphics Pipeline

Genie 3



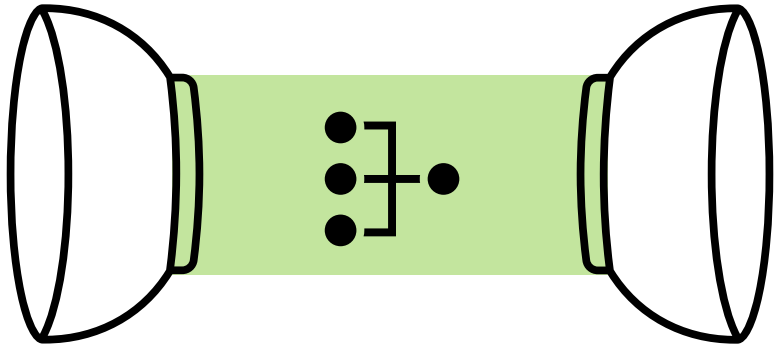
Cosmos 3



OmniDreams

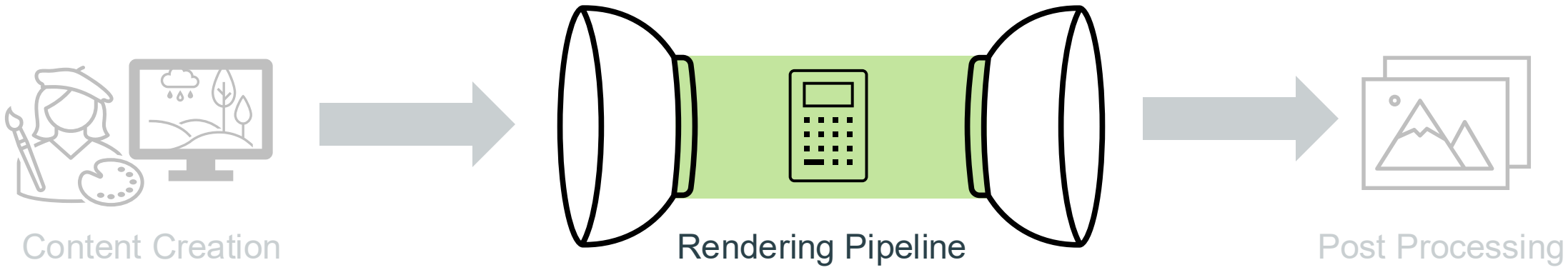






© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Neural Networks and the Graphics Pipeline



© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Neural Networks and the Graphics Pipeline

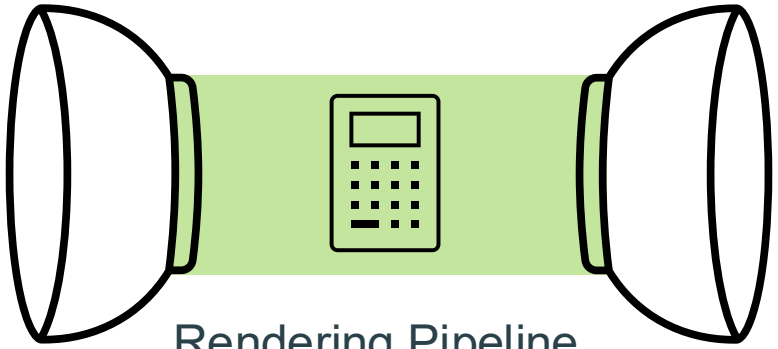


- Usually: Involves Neural Networks
- Anything that's trainable

Neural Shader



Content Creation



Rendering Pipeline



Post Processing

© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Neural Networks and the Graphics Pipeline



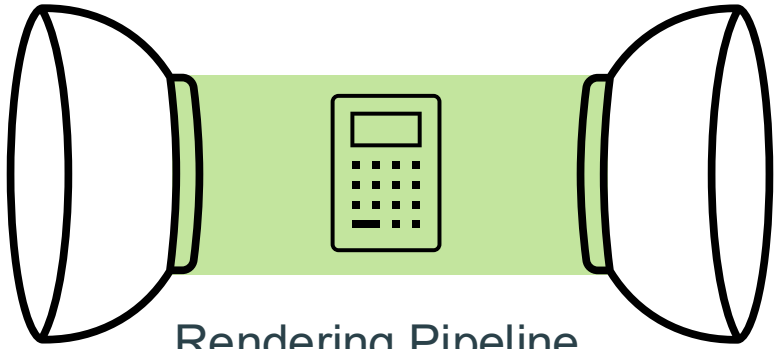
- Usually: Involves Neural Networks
- Anything that's trainable

Neural **Shader**

- Runs in the graphics pipeline
- Part of your normal shader code



Content Creation



Rendering Pipeline



Post Processing

© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

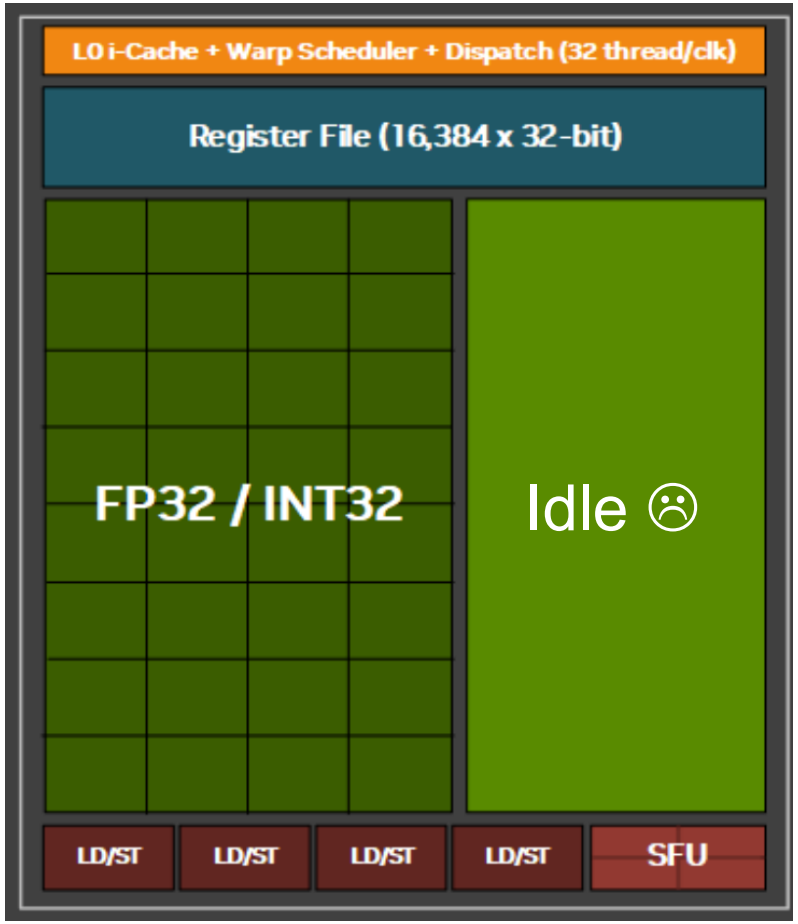
Moore's Law and the Future of Hardware



- Consumer GPUs ship with neural network accelerators
- While you render, these sit at 0% utilization

- These are untapped FLOPS!
- …and very efficient FLOPS

- Accessible in the graphics pipeline with Cooperative Vectors



Summary



Neural Shaders are a powerful new tool in your tool box:

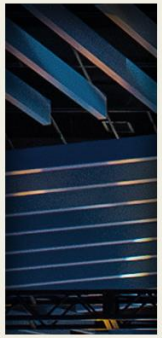
- Renderers that are trainable can solve problems that were hard before
- Neural Networks can run very efficiently
- This is a practical tool worth checking out

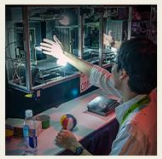
Summary



But: Neural Shaders are not magic

- Things you already do well won’t magically get better
- We need new tools, new programming models and new algorithms
- Leveraging the full power small neural nets requires new skills and clever engineering
- This is what we’re here to talk about!







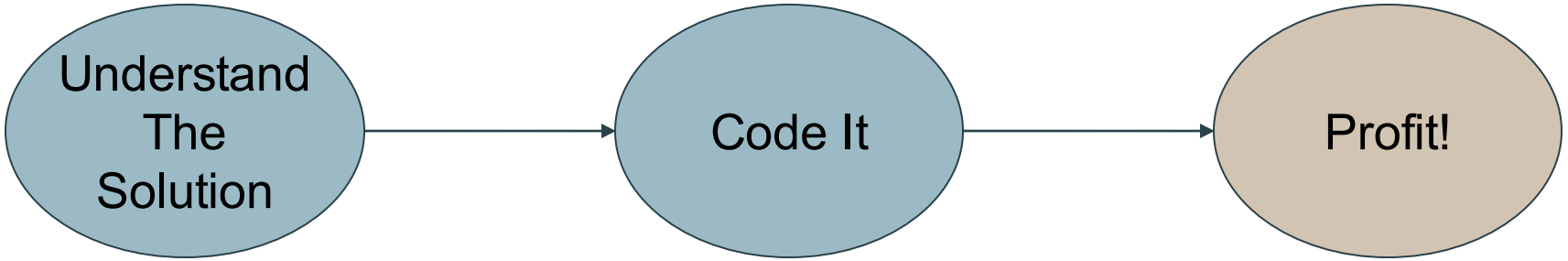
SIGGRAPH 2026
Los Angeles 19–23 JUL

Fundamentals

WHAT IF WE DON'T KNOW THE ANSWER?



Classical Engineering



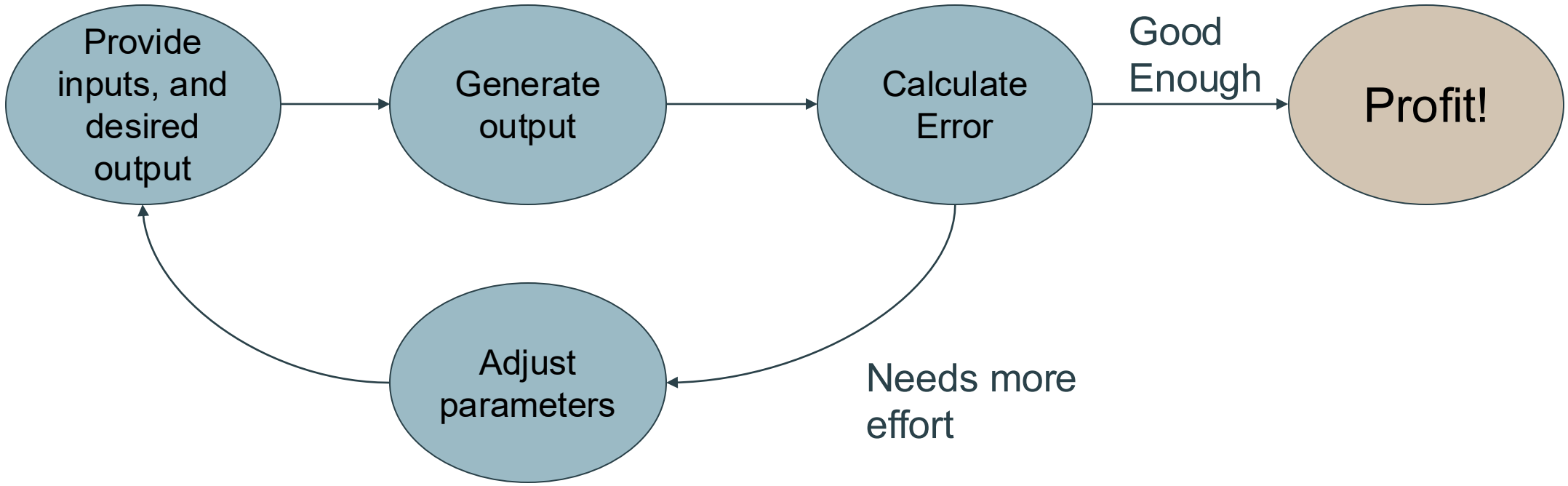
But what if…

- We don't know the solution
- Or there isn't one!
- Or there is, but it's **way** too expensive to compute

LEARNING AN APPROXIMATE SOLUTION.



Optimizer…



TOOLS FOR TODAY



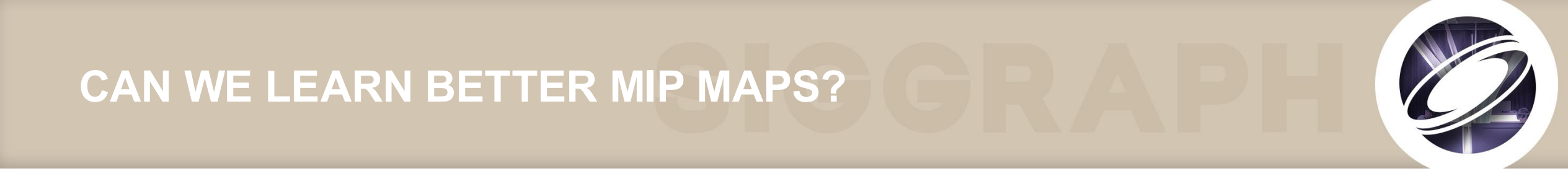
Slang™

- Powerful / flexible shading language
- Write once / run anywhere
- Generics
- Supports ‘auto-diff’ (does calculus for us)

SlangPy

- Python/C++ interface to Slang
- Full featured graphics api
- Cross platform
- Functional api to directly call Slang functions from Python

CAN WE LEARN BETTER MIP MAPS?

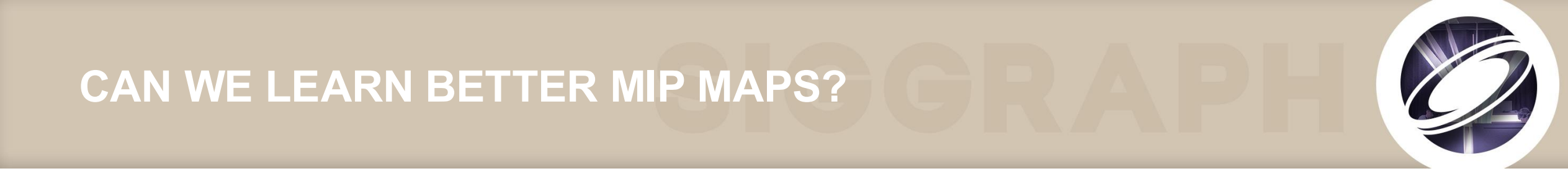


- Mipmaps reduce aliasing **and** improve coherency for distant surfaces
- Color maps downsample well using simple box filter
- Normal maps do not





CAN WE LEARN BETTER MIP MAPS?



1. Render a beautiful PBR material
2. Generate + render low res with **Mip Maps**
3. Generate + render reference with **Supersampling**
4. Calculate the loss

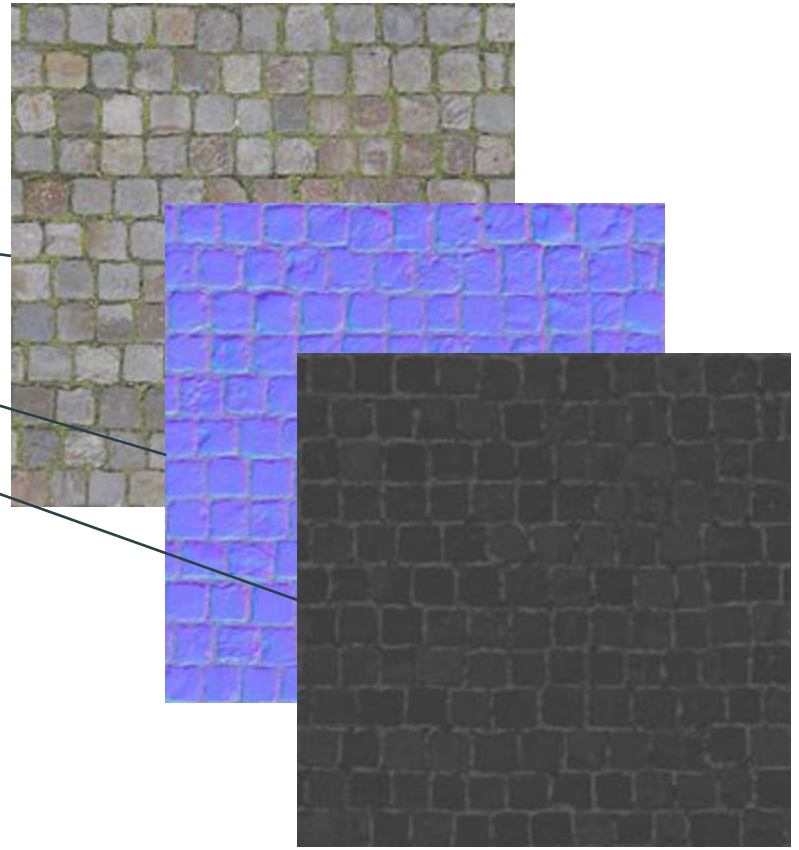


BASIC SLANG PROGRAM



```
struct MaterialParameters
{
    Tensor<float3, 2> albedo;
    Tensor<float3, 2> normal;
    Tensor<float, 2> roughness;

    float3 get_albedo(int2 pixel)
    {
        return albedo.getv(pixel);
    }
    float3 get_normal(int2 pixel)
    {
        return normalize(normal.getv(pixel));
    }
    float get_roughness(int2 pixel)
    {
        return roughness.getv(pixel);
    }
};
```



BASIC SLANG PROGRAM



```
float3 render(int2 pixel, MaterialParameters material, float3 light_dir, float3 view_dir)
{
    // Bright white light
    float light_intensity = 5.0;

    // Sample very shiny BRDF (it rained today!)
    float3 brdf_sample = sample_brdf(
              material.get_albedo(pixel),  // albedo color
              normalize(light_dir),        // light direction
              normalize(view_dir),         // view direction
              material.get_normal(pixel),  // normal map sample
              0.05,                        // roughness
              0.0,                         // metallic (no metal)
              1.0                          // specular
              );

    // Combine light with BRDF sample to get pixel colour
    return brdf_sample * light_intensity;
}
```

BASIC PYTHON PROGRAM



```python
# Create the app and load the slang module.
app = App(width=2048, height=2048, title="Mipmap Example")
module = spy.Module.load_from_file(app.device, "nsc_basicprogram.slang")
```

```python
# Load some materials.
albedo_map = spy.Tensor.load_from_image(app.device, "PavingStones070_2K.diffuse.jpg", linearize=True)
normal_map = spy.Tensor.load_from_image(app.device, "PavingStones070_2K.normal.jpg", scale=2, offset=-1)
roughness_map = spy.Tensor.load_from_image(app.device, "PavingStones070_2K.roughness.jpg", grayscale=True)
```

```python
while app.process_events():
    # Allocate a tensor for output + call the render function
    output = spy.Tensor.empty_like(albedo_map)
    module.render(pixel = spy.call_id(),
                  material = {
                        "albedo": albedo_map,
                        "normal": normal_map,
                        "roughness": roughness_map,
                  },
                  light_dir = spy.math.normalize(spy.float3(0.2, 0.2, 1.0)),
                  view_dir = spy.float3(0, 0, 1),
                  _result = output)

    # Blit tensor to screen.
    app.blit(output)
    app.present()
```

RESULT!



Conveniently Shiny Cobblestones!





DOWNSAMPLING



```python
def downsample(source: spy.Tensor, steps: int) -> spy.Tensor:
    for i in range(steps):
        dest = spy.Tensor.empty(
                    device=app.device,
                    shape=(source.shape[0] // 2, source.shape[1] // 2),
                    dtype=source.dtype)
        module.downsample(spy.call_id(), source, _result=dest)
        source = dest
    return source
```

```
float3 downsample(
      int2 pixel,
      Tensor<float3, 2> source)
{
    float3 res = 0;
    res += source.getv(pixel * 2 + int2(0, 0));
    res += source.getv(pixel * 2 + int2(1, 0));
    res += source.getv(pixel * 2 + int2(0, 1));
    res += source.getv(pixel * 2 + int2(1, 1));
    return res * 0.25;
}
```

DOWNSAMPLED INPUTS



```
# … downsampled output generation/render here

# Quarter res rendered output BRDF from half res inputs.
lr_output = spy.Tensor.empty_like(output)
module.render(pixel = spy.call_id(),
              material = {
                    "albedo":    downsample(albedo_map, 2),
                    "normal":    downsample(normal_map, 2),
                    "roughness": downsample(roughness_map, 2),
              },
              light_dir = spy.math.normalize(spy.float3(0.2, 0.2, 1.0)),
              view_dir = spy.float3(0, 0, 1),
              _result = lr_output)

# … blit the ‘lr_output’ to screen here
```

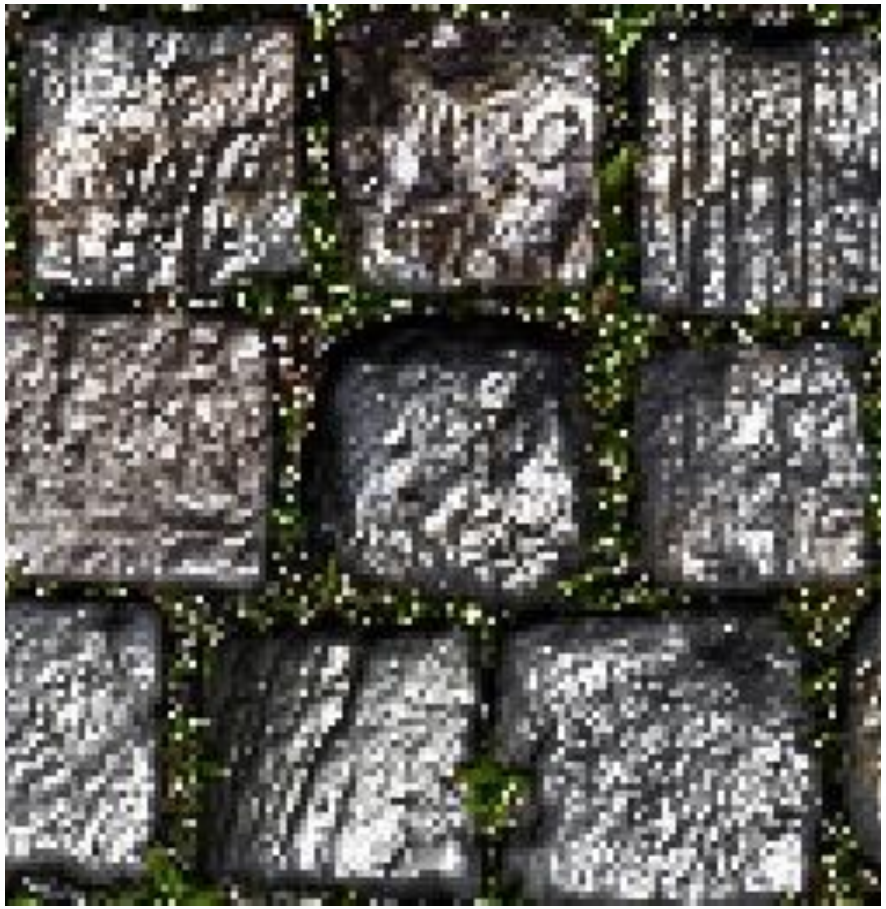
RESULT!



Downscaled inputs

- 2x downsample (Mip level 2)
- 1 sample per map
- PBR function run once per pixel
- Optimal but noisy

Poor normal map downsampling gives nasty surface artifacts.



DOWNSAMPLED OUTPUT



```
while app.process_events():
    # Allocate a tensor for output + call the render function
    output = spy.Tensor.empty_like(albedo_map)
    module.render(pixel = spy.call_id(),
                  material = {
                        "albedo": albedo_map,
                        "normal": normal_map,
                        "roughness": roughness_map
                  },
                  light_dir = spy.math.normalize(spy.float3(0.2, 0.2, 1.0)),
                  view_dir = spy.float3(0, 0, 1),
                  _result = output)

    # Downsample the output tensor to quarter res.
    output = downsample(output, 2)

    # Blit tensor to screen.
    app.blit(output, size=spy.int2(2048,2048))
    app.present()
```

RESULT!



Downscaled output

- 2x downsample (Mip level 2)
- 16 high res albedo, normal + roughness pixels sampled
- Full PBR function run 16 times
- Represents an **ideal** output

If our input mip maps were *perfect*, this is what we'd get



SIDE BY SIDE



The Ideal
Render at full res then downsample



The Reality
Downsample inputs and then render



CALCULATING THE LOSS



```
// Render the difference between the rendered pixel
a reference pixel.
float3 loss(
      int2 pixel,
      float3 reference,
      MaterialParameters material,
      float3 light_dir,
      float3 view_dir)
{
    float3 color = render(pixel, material,
                          light_dir, view_dir);
    float3 error = color - reference;
    return error * error; // Squared error
}
```

```
# Loss between downsampled full res output (the
# reference), and result from quarter res inputs.
loss_output = spy.Tensor.empty_like(output)
module.loss(pixel = spy.call_id(),
            material = {
                  "albedo": downsample(albedo_map,2),
               "normal": downsample(normal_map,2),
               "roughness": downsample(roughness_map,2),
            },
            reference = output,
            light_dir = spy.math.normalize(
                      spy.float3(0.2, 0.2, 1.0)),
            view_dir = spy.float3(0, 0, 1),
            _result = loss_output)
```

CALCULATING THE LOSS



Reference



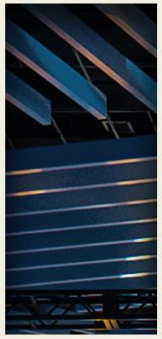
Loss



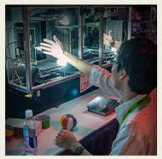
Render



Make mipmap better == Make the loss smaller





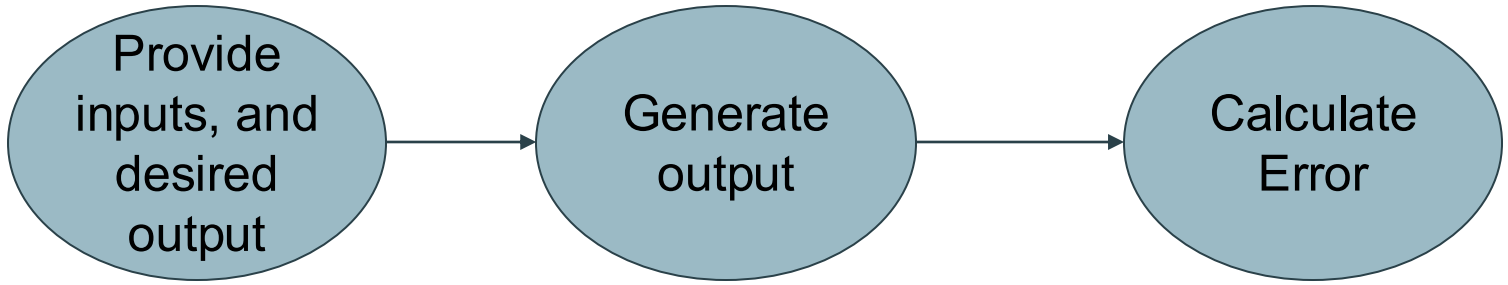


The Premier Conference & Exhibition on
Computer Graphics & Interactive Techniques

SIGGRAPH 2026
Los Angeles 19–23 JUL

Basic Primitives of Neural Shading

Summary: The “Inference” phase

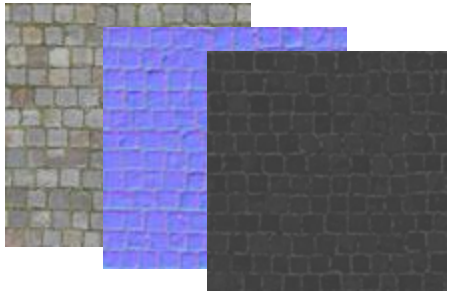


SUMMARY: THE "INFERENCE" PHASE



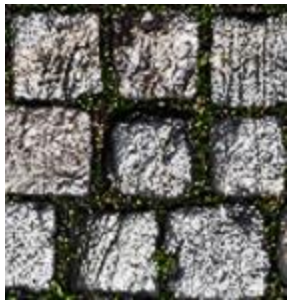
Trainable Parameters

```
MaterialParameters material
```



Desired output

```
float3 reference
```



Generate output

```
float3 render( /* ... */ )
```

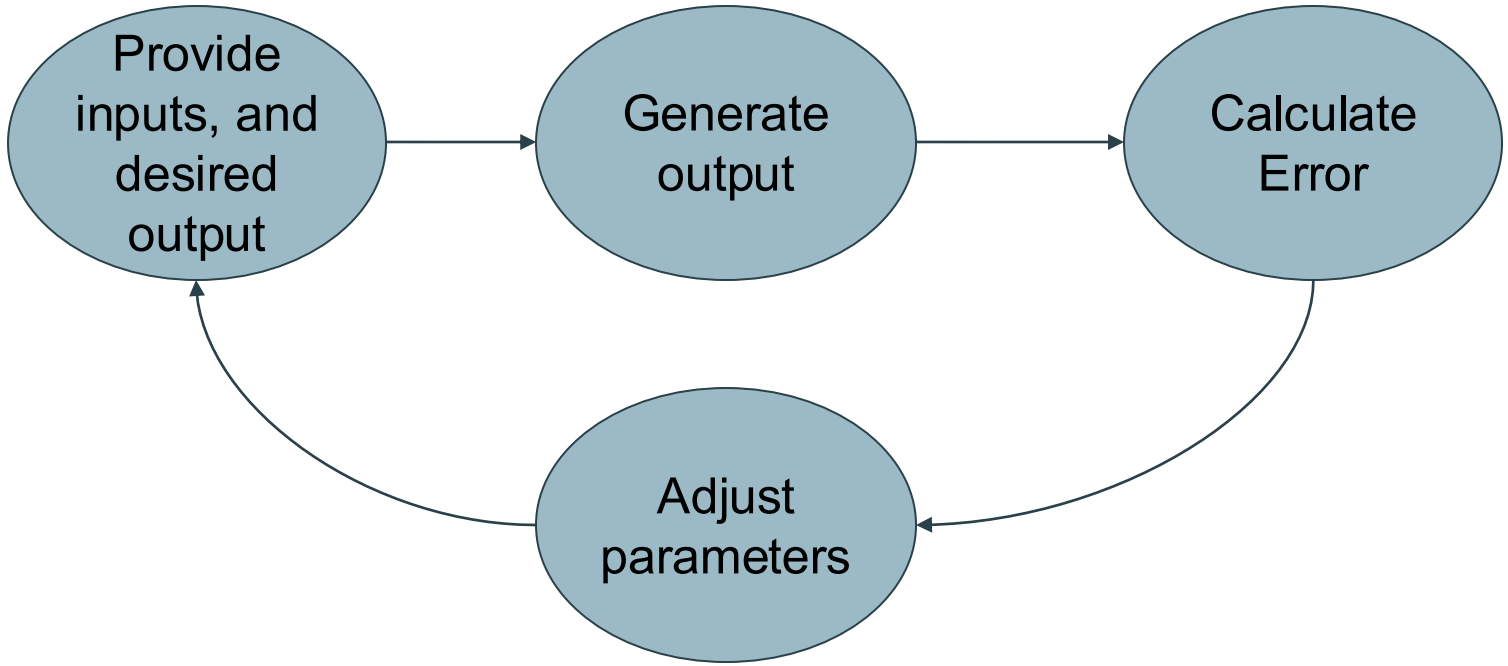


Compute error

```
float3 loss( /* ... */ )
```



Closing the loop: Optimization

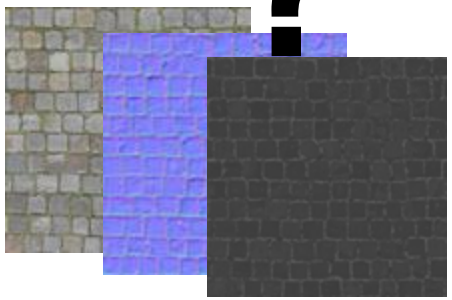


The “Backward” phase



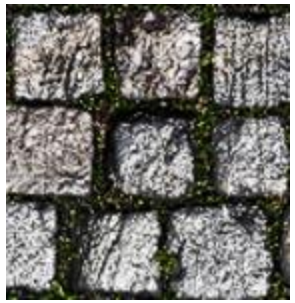
Adjust parameters

```
MaterialParameters material
```



Desired output

```
float3 reference
```



“Backward” render

```
float3 render( /* ... */ )
```



“Backward” loss

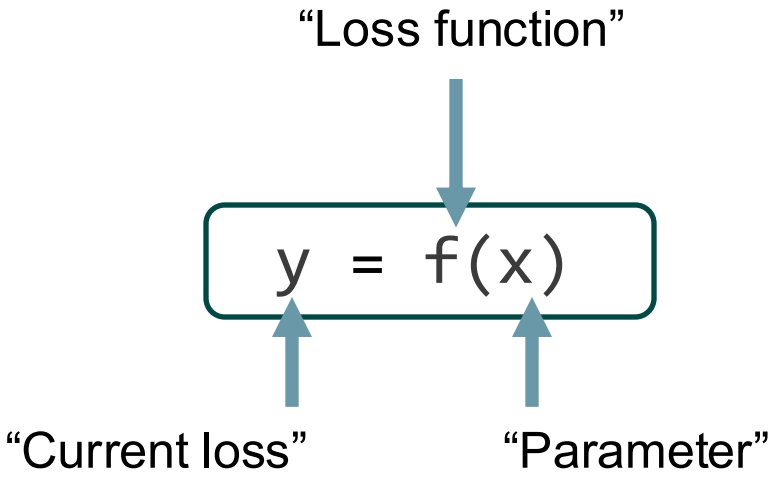
```
float3 loss( /* ... */ )
```



The math view: Derivatives



Inference



Backward

If I want y to go down, how should I change x?

Backwards derivative dx:

```
dx = backwards_deriv_f(x)
```

If you want to change y by `delta`…

```
y - delta    ⇒    x - delta * dx
```

…you need to change x by `delta * dx`

Differentiation



Inference

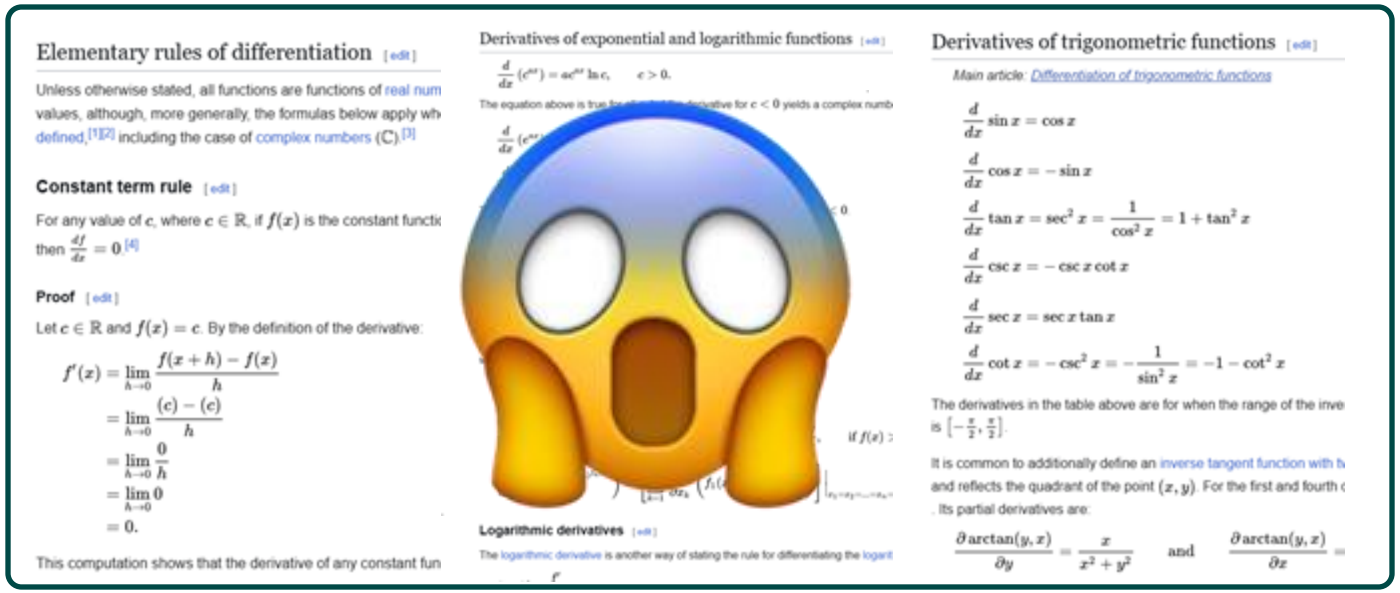
We wrote the inference function:

```
float3 loss(
    int2 pixel,
    float3 reference,
    MaterialParameters material,
    float3 light_dir,
    float3 view_dir)
{
    /* ......  */
}
```

Backward

…but how do we get the backward function?

In HLSL, we need to **manually derive** it



Automatic Differentiation



Inference

We wrote the inference function:

```
float3 loss(
    int2 pixel,
    float3 reference,
    MaterialParameters material,
    float3 light_dir,
    float3 view_dir)
{
    /* ......  */
}
```

Backward

…but how do we get the backward function?

In Slang, we can let the compiler derive it!

```
bwd_diff(loss)( /* ... */ );
```



Generating the derivatives



```
void calculate_grads(
    int2 pixel,
    MaterialParameters material,
    MaterialParameters ref_material)
{
    // Generate random directions
    float3 light_dir = random_direction();
    float3 view_dir = random_direction();

    // Evaluate target value
    float3 reference = eval_reference(ref_material, pixel, light_dir, view_dir);

    // Backpropagate derivatives
    bwd_diff(loss)(pixel, reference, material, light_dir, view_dir, 1);
}
```

Generating the derivatives



```
void calculate_grads(
    int2 pixel,
    MaterialParameters material,
    MaterialParameters ref_material)
{
    // Generate random directions
    float3 light_dir = random_direction();
    float3 view_dir = random_direction();

    // Evaluate target value
    float3 reference = eval_reference(ref_material, pixel, light_dir, view_dir);

    // Backpropagate derivatives
    bwd_diff(loss)(pixel, reference, material, light_dir, view_dir, 1);
}
```

Generating the derivatives



```
void calculate_grads(
    int2 pixel,
    MaterialParameters material,
    MaterialParameters ref_material)
{
    // Generate random directions
    float3 light_dir = random_direction();
    float3 view_dir = random_direction();

    // Evaluate target value
    float3 reference = eval_reference(ref_material, pixel, light_dir, view_dir);

    // Backpropagate derivatives
    bwd_diff(loss)(pixel, reference, material, light_dir, view_dir, 1);
}
```

Generating the derivatives



```
void calculate_grads(
    int2 pixel,
    MaterialParameters material,
    MaterialParameters ref_material)
{
    // Generate random directions
    float3 light_dir = random_direction();
    float3 view_dir = random_direction();

    // Evaluate target value
    float3 reference = eval_reference(ref_material, pixel, light_dir, view_dir);

    // Backpropagate derivatives
    bwd_diff(loss)(pixel, reference, material, light_dir, view_dir, 1);
}
```

Where do the derivatives go?



Inference

Read parameters

```
struct MaterialParameters
{
    Tensor<float3, 2> albedo;

    RWTensor<float3, 2> albedo_grad;

    float3 get_albedo(int2 pixel)
    {
        return albedo.getv(pixel);
    }

    [BackwardDerivativeOf(get_albedo)]
    void get_albedo_bwd(int2 pixel, float3 grad)
    {
        albedo_grad.setv(pixel, grad);
    }
}
```

Backward

Write gradients

```
struct MaterialParameters
{
    Tensor<float3, 2> albedo;

    RWTensor<float3, 2> albedo_grad;

    float3 get_albedo(int2 pixel)
    {
        return albedo.getv(pixel);
    }

    [BackwardDerivativeOf(get_albedo)]
    void get_albedo_bwd(int2 pixel, float3 grad)
    {
        albedo_grad.setv(pixel, grad);
    }
}
```

Where do the derivatives go?



Inference

Read parameters

```
struct MaterialParameters
{
    Tensor<float3, 2> albedo;

    RWTensor<float3, 2> albedo_grad;

    float3 get_albedo(int2 pixel)
    {
        return albedo.getv(pixel);
    }

    [BackwardDerivativeOf(get_albedo)]
    void get_albedo_bwd(int2 pixel, float3 grad)
    {
        albedo_grad.setv(pixel, grad);
    }
}
```

Backward

Write gradients

```
struct MaterialParameters
{
    Tensor<float3, 2> albedo;

    RWTensor<float3, 2> albedo_grad;

    float3 get_albedo(int2 pixel)
    {
        return albedo.getv(pixel);
    }

    [BackwardDerivativeOf(get_albedo)]
    void get_albedo_bwd(int2 pixel, float3 grad)
    {
        albedo_grad.setv(pixel, grad);
    }
}
```

Where do the derivatives go?



Inference

Read parameters

```
struct MaterialParameters
{
    Tensor<float3, 2> albedo;

    RWTensor<float3, 2> albedo_grad;

    float3 get_albedo(int2 pixel)
    {
        return albedo.getv(pixel);
    }

    [BackwardDerivativeOf(get_albedo)]
    void get_albedo_bwd(int2 pixel, float3 grad)
    {
        albedo_grad.setv(pixel, grad);
    }
}
```

Backward

Write gradients

```
struct MaterialParameters
{
    Tensor<float3, 2> albedo;

    RWTensor<float3, 2> albedo_grad;

    float3 get_albedo(int2 pixel)
    {
        return albedo.getv(pixel);
    }

    [BackwardDerivativeOf(get_albedo)]
    void get_albedo_bwd(int2 pixel, float3 grad)
    {
        albedo_grad.setv(pixel, grad);
    }
}
```

Where do the derivatives go?



Inference

Read parameters

```
struct MaterialParameters
{
    Tensor<float3, 2> albedo;

    RWTensor<float3, 2> albedo_grad;

    float3 get_albedo(int2 pixel)
    {
        return albedo.getv(pixel);
    }

    [BackwardDerivativeOf(get_albedo)]
    void get_albedo_bwd(int2 pixel, float3 grad)
    {
        albedo_grad.setv(pixel, grad);
    }
}
```

Backward

Write gradients

```
struct MaterialParameters
{
    Tensor<float3, 2> albedo;

    RWTensor<float3, 2> albedo_grad;

    float3 get_albedo(int2 pixel)
    {
        return albedo.getv(pixel);
    }

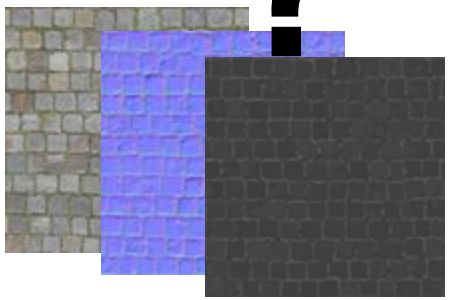
    [BackwardDerivativeOf(get_albedo)]
    void get_albedo_bwd(int2 pixel, float3 grad)
    {
        albedo_grad.setv(pixel, grad);
    }
}
```

The “Backward” phase



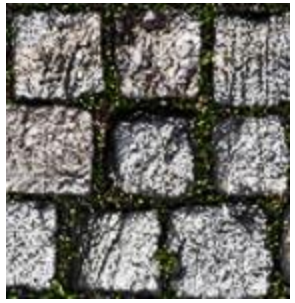
Adjust parameters

```
MaterialParameters material
```



Desired output

```
float3 reference
```



“Backward” render

```
float3 render( /* ... */ )
```



“Backward” loss

```
float3 loss( /* ... */ )
```

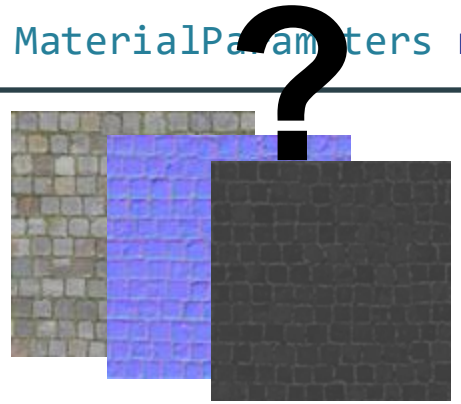


The “Backward” phase



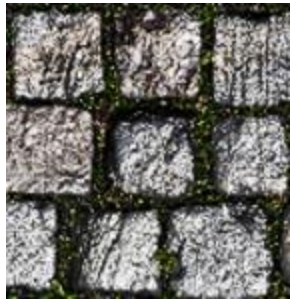
Adjust parameters

```
MaterialParameters material
```



Desired output

```
float3 reference
```



“Backward” render

```
float3 render( /* ... */ )
```



“Backward” loss

```
float3 loss( /* ... */ )
```



Closing the loop: Taking a step



If you want to change y by `delta`…

```
y - delta    ⇒    x - delta * dx
```

…you need to change x by `delta * dx`

```
void optimizer_step(
    inout float3 mipmap,
    float3 derivative,
    float learning_rate)
{
    mipmap -= learning_rate * derivative;
}
```

Closing the loop: Taking a step



If you want to change `y` by `delta`…

```
y - delta    ⇒    x - delta * dx
```

…you need to change `x` by `delta * dx`

```
void optimizer_step(
    inout float3 mipmap,
    float3 mipmap_grad,
    float learning_rate)
{
    mipmap -= learning_rate * mipmap_grad;
}
```

Closing the loop: Taking a step



If you want to change `y` by `delta`…

`y - delta` ⇒ `x - delta * dx`

…you need to change `x` by `delta * dx`

```
void optimizer_step(
    inout float3 mipmap,
    float3 mipmap_grad,
    float learning_rate)
{
    mipmap -= learning_rate * mipmap_grad;
}
```

Closing the loop: Taking a step



If you want to change y by `delta`…

`y - delta` ⇒ `x - delta * dx`

…you need to change x by `delta * dx`

```
void optimizer_step(
    inout float3 mipmap,
    float3 mipmap_grad,
    float learning_rate)
{
    mipmap -= learning_rate * mipmap_grad;
}
```

Putting it together: Generating derivatives



```
module.calculate_grads(
    material = {
            "albedo": trained_albedo_map,
            "albedo_grad": albedo_grad,
            # ...
    },
    # ...
)
```

```
void calculate_grads(
    MaterialParameters material, /*...*/)
{
    /* ... */

    // Backpropagate derivatives
    bwd_diff(loss)(/* ... */);
}
```

Putting it together: Taking an optimizer step



```
module.calculate_grads(
    material = {
            "albedo": trained_albedo_map,
            "albedo_grad": albedo_grad,
            # ...
    },
    # ...
)

module.optimizer_step(
    trained_albedo_map,
    albedo_grad,
    learning_rate)
# ...
```

```
void calculate_grads(
    MaterialParameters material, /*...*/)
{
    /* ... */

    // Backpropagate derivatives
    bwd_diff(loss)(/* ... */);
}

void optimizer_step(
    inout float3 mipmap,
    float3 mipmap_grad,
    float learning_rate)
{
    mipmap -= learning_rate * mipmap_grad;
}
```

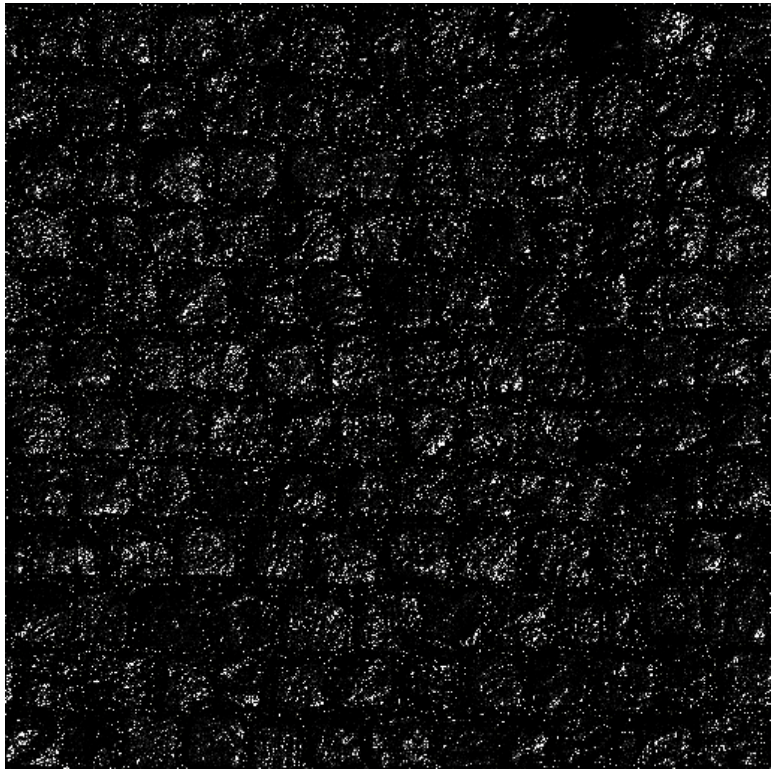
Let’s run it!



Reference



Basic Mip-Mapped Textures

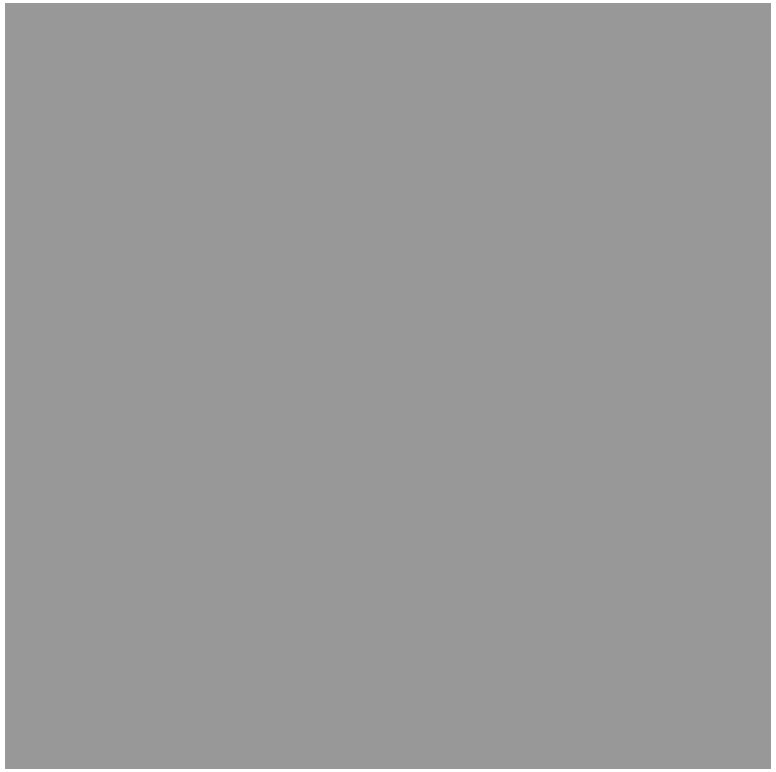


Loss

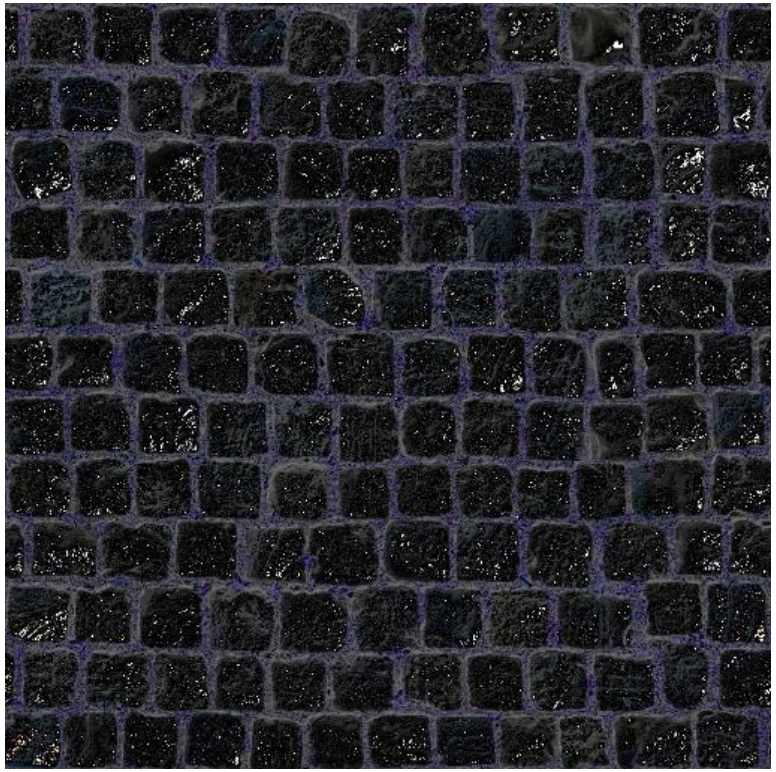
Let’s run it!



Reference



Optimized Textures



Loss

Something is wrong with our derivatives



```
y - delta   ⇒   x - delta * dx
```

```
dx = backwards_deriv_f(x)
```

```
void calculate_grads(
    int2 pixel,
    MaterialParameters material,
    MaterialParameters ref_material)
{
    // Generate random directions
    float3 light_dir = random_direction();
    float3 view_dir = random_direction();

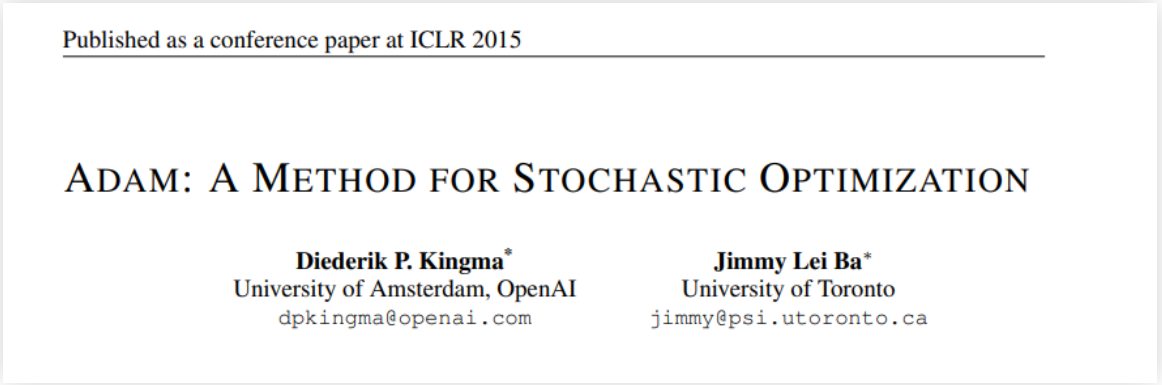
    // Evaluate target value
    float3 reference = eval_reference(ref_material, pixel, light_dir, view_dir);

    // Backpropagate derivatives
    bwd_diff(loss)(pixel, reference, material, light_dir, view_dir, 1);
}
```

Dealing with noisy derivatives



- Our derivatives are noisy. Can we fix it?
- Source of the issue: The optimizer
- Let's swap it out for one that can handle noise!
  - Usual approach: Exponential moving average of gradients
- Even better: Adam optimizer



Published as a conference paper at ICLR 2015

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

Diederik P. Kingma*
University of Amsterdam, OpenAI
dpkingma@openai.com

Jimmy Lei Ba*
University of Toronto
jimmy@psi.utoronto.ca

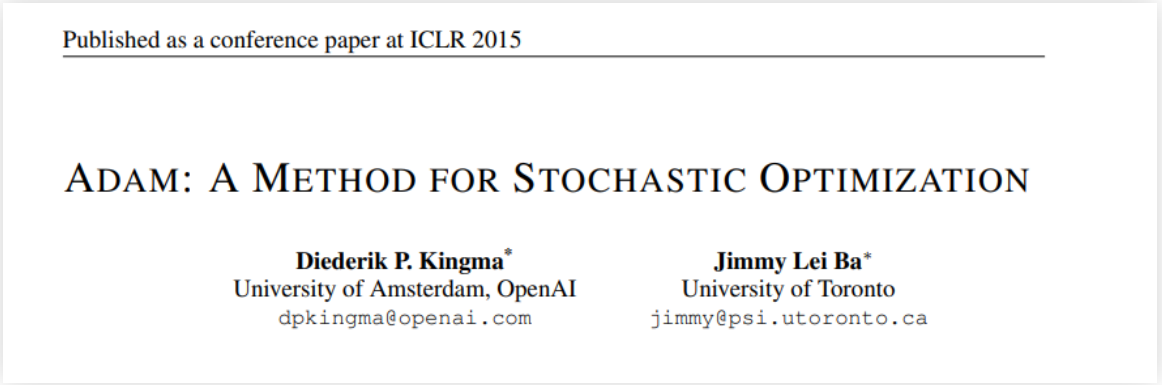
Gradient Descent

```
void optimizer_step(
    inout float3 mipmap,
    float3 mipmap_grad,
    float learning_rate)
{
    mipmap -= learning_rate * mipmap_grad;
}
```

Dealing with noisy derivatives



- Our derivatives are noisy. Can we fix it?
- Source of the issue: The optimizer
- Let's swap it out for one that can handle noise!
  - Usual approach: Exponential moving average of gradients
- Even better: Adam optimizer



Published as a conference paper at ICLR 2015

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

Diederik P. Kingma*
University of Amsterdam, OpenAI
dpkingma@openai.com

Jimmy Lei Ba*
University of Toronto
jimmy@psi.utoronto.ca

Adam Optimizer

```
void optimizer_step(
    inout float3 mipmap,
    inout float3 mipmap_grad,
    inout float3 mean,
    inout float3 variance,
    float learning_rate,
    int iteration)
{
    mean = lerp(mipmap_grad, mean, 0.9f);
    variance = lerp(mipmap_grad * mipmap_grad, variance, 0.999f);

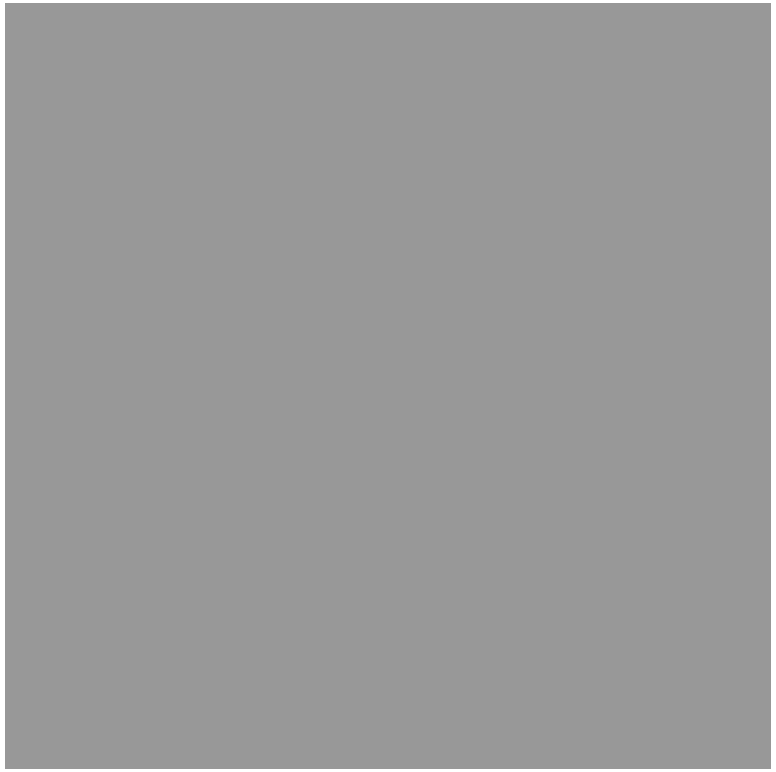
    float3 mHat = mean / (1.0f - pow(beta1, iteration));
    float3 vHat = variance / (1.0f - pow(beta2, iteration));
    float3 step = mHat / (sqrt(vHat) + 1e-8f);

    mipmap -= learning_rate * step;
}
```

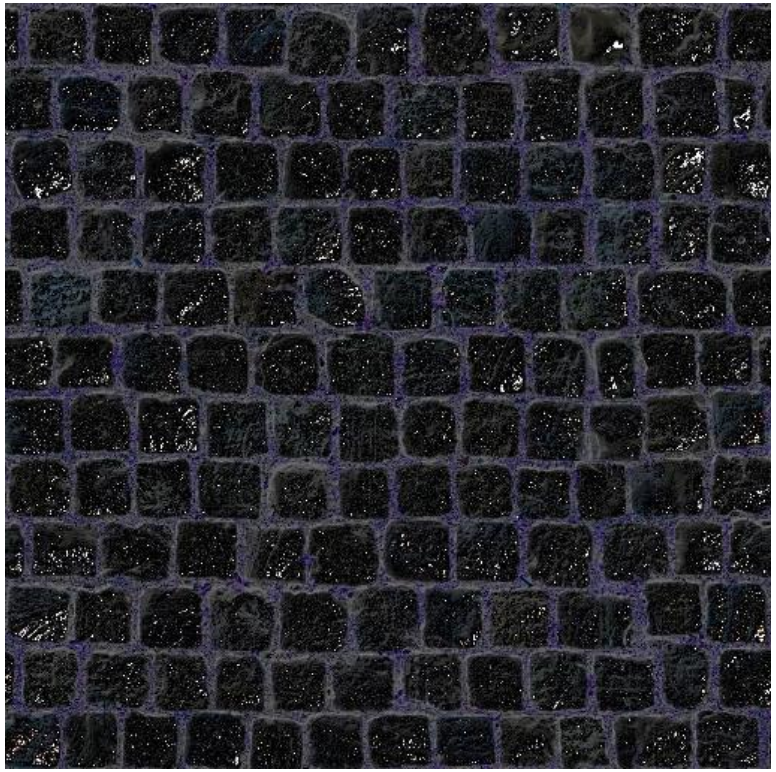
Let’s run it!



Reference



Adam Optimized Textures



Loss

Summary



Summary: The Classical Approach



- We started with a problem that is somewhat tricky
  - Generate texture mip maps without changing the look of the material
- There are techniques developed for this (e.g. Toksvig), but…
  - They are developed for a specific class of material
  - Your material doesn't fit in this mold? You'll have to invent something new
  - That's a lot of work!

Summary: Neural shading

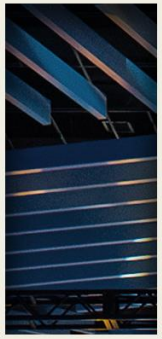


- Alternative: Write down the error, then make it smaller

- In Slang: The bulk of the code you write *is* the inference code
  - I.e., the code you want to ship!
- You can change the inference code and don't need to come up with e.g. a new mip-mapping method

This is the benefit of neural shading:

- Problems you don't know how to solve well (yet), may now be solvable
- With the right tools, you don't have to spend a lot of effort to do this







The Premier Conference & Exhibition on
Computer Graphics & Interactive Techniques

SIGGRAPH 2026
Los Angeles 19–23 JUL

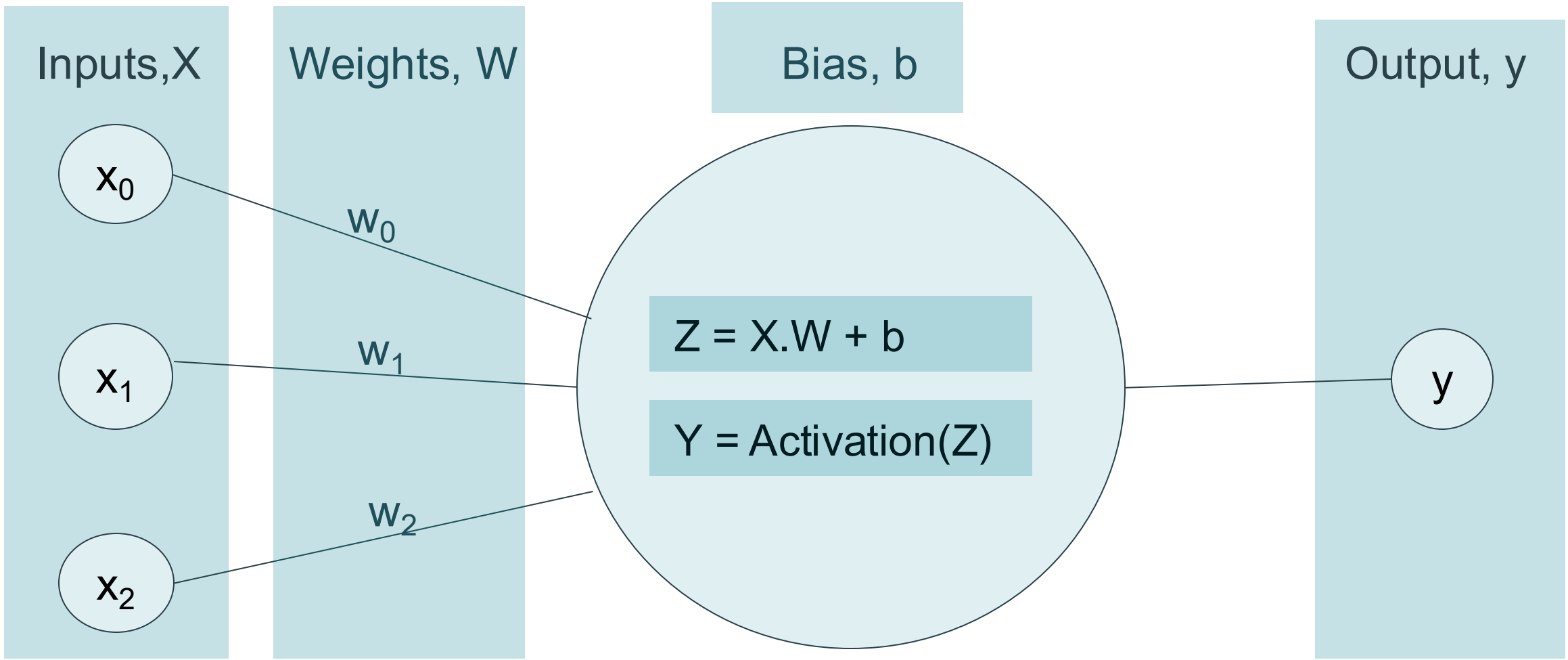
Your First MLP in a Shader

DOES IT HAVE TO BE PIXELS?

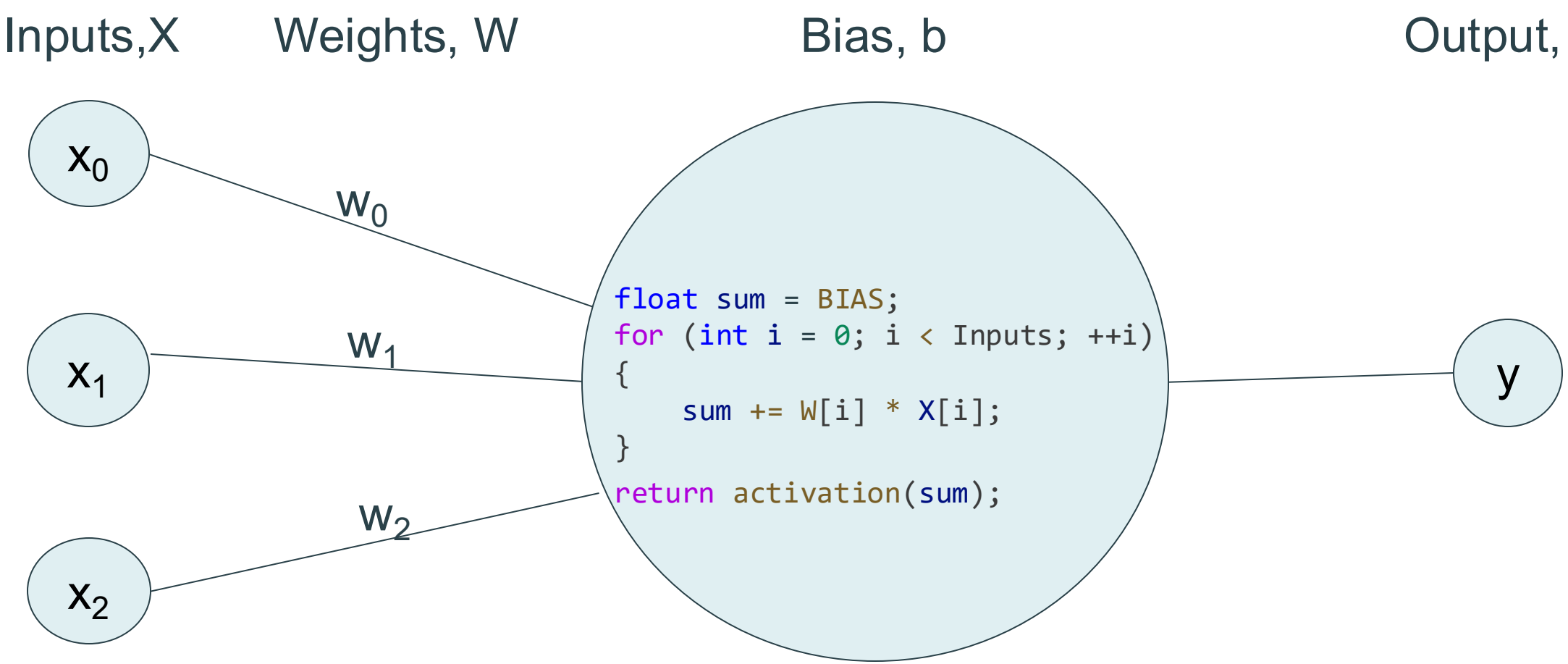


- Training mipmap levels shows we can *learn* the pixels of a texture, based on a desired output.
- Can we *learn* a neural network that generates the pixels of a texture?

THE BUILDING BLOCK



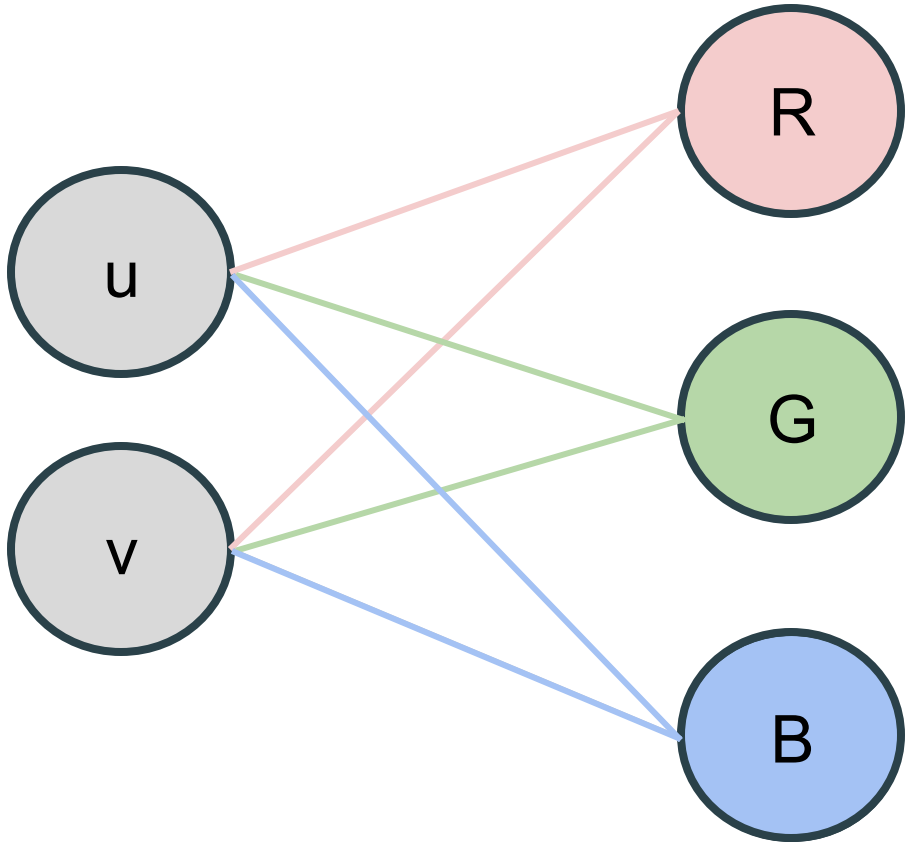
THE BUILDING BLOCK



THE SIMPLEST TEXTURE NETWORK



- Inputs: Texture coordinate (uv)
- Outputs: Color channels (rgb)
- 6 weights (2 inputs * 3 outputs)
- 3 biases
- **9 parameters total**



NETWORK PARAMETERS



```
struct NetworkParameters<int Inputs, int Outputs>
{
    RWTensor<float, 1> biases;
    RWTensor<float, 2> weights;

    AtomicTensor<float, 1> biases_grad;
    AtomicTensor<float, 2> weights_grad;

    [Differentiable]
    float get_bias(int neuron)
    { /* ... */ }

    [Differentiable]
    float get_weight(int neuron, int input)
    { /* ... */ }

    [BackwardDerivativeOf(get_bias)]
    void get_bias_bwd(int neuron, float grad)
    { /* ... */ }

    [BackwardDerivativeOf(get_weight)]
    void get_weight_bwd(int neuron, int input, float grad)
    { /* ... */ }
}
```

```
class NetworkParameters(spy.InstanceList):
    def __init__(self, inputs: int, outputs: int):
        super().__init__( ... )
        self.inputs = inputs
        self.outputs = outputs

        # Biases and weights for the layer.
        self.biases = spy.Tensor.from_numpy( ... )
        self.weights = spy.Tensor.from_numpy( ... )

        # Gradients for the biases and weights.
        self.biases_grad = spy.Tensor.zeros_like(self.biases)
        self.weights_grad = spy.Tensor.zeros_like(self.weights)

# Calls the Slang 'optimize' function for biases and weights
def optimize(self, ... ):
    module.optimize1( ... )
    module.optimize1( ... )
```

NETWORK PARAMETERS



Evaluating 1 layer

```
struct NetworkParameters<int Inputs, int Outputs>
{
    // ... Fields and accessors here ...

    // Evaluate all neurons in the layer
    [Differentiable]
    float[Outputs] forward(float[Inputs] x)
    {
        float[Outputs] z;
        [ForceUnroll]
        for (int row = 0; row < Outputs; ++row)
        {
            // Sum up bias plus weighted inputs for 1 neuron
            var sum = get_bias(row);
            [ForceUnroll]
            for (int col = 0; col < Inputs; ++col)
                sum += get_weight(row, col) * x[col];
            z[row] = sum;
        }

        return z;
    }

}
```

Evaluating the network

```
struct Network {
    NetworkParameters<2, 3>  layer;

    // Evaluate the layer and output activations
    [Differentiable]
    float3 eval(no_diff float2 uv)
    {
        // Convert uv to array of 2 inputs
        float inputs[2] = {uv.x, uv.y};

        // Feed inputs to layer, get outputs back
        float output0[3] = layer.forward(inputs);

        // Apply activation function to each output
        [ForceUnroll]
        for (int i = 0; i < 3; ++i)
            output0[i] = activation(output0[i]);

        // Return the output (to be used an RGB colour)
        return float3(output0[0], output0[1], output0[2]);
    }
}
```

RENDER SOMETHING



Reference



Output
(untrained!)



```
[Differentiable]
float3 render(int2 pixel, int2 resolution, Network network)
{
    float2 uv = (float2(pixel) + 0.5f) / float2(resolution);
    return network.eval(uv);
}
```

Loss
(a lot!)



```
[Differentiable]
float3 loss(int2 pixel, int2 resolution, no_diff float3 reference, Network network)
{
    float3 color = render(pixel, resolution, network);
    float3 error = color - reference;
    return error * error; // Squared error
}
```

TRAINING



Back propagate gradients of loss function to the network's weights and biases

```
void calculate_grads(int2 pixel,
                     int2 resolution,
                     float3 reference,
                     Network network)
{
    bwd_diff(loss)(pixel, resolution,
                   reference, network, 1.0f);
}
```

Calculate gradients, then call optimize to apply them

```
learning_rate = 0.001

for i in range(50):
    module.calculate_grads(
        pixel = spy.call_id(),
        resolution = res,
        reference = image,
        network = network)
    optimize_counter += 1

    network.optimize(learning_rate, optimize_counter)
```

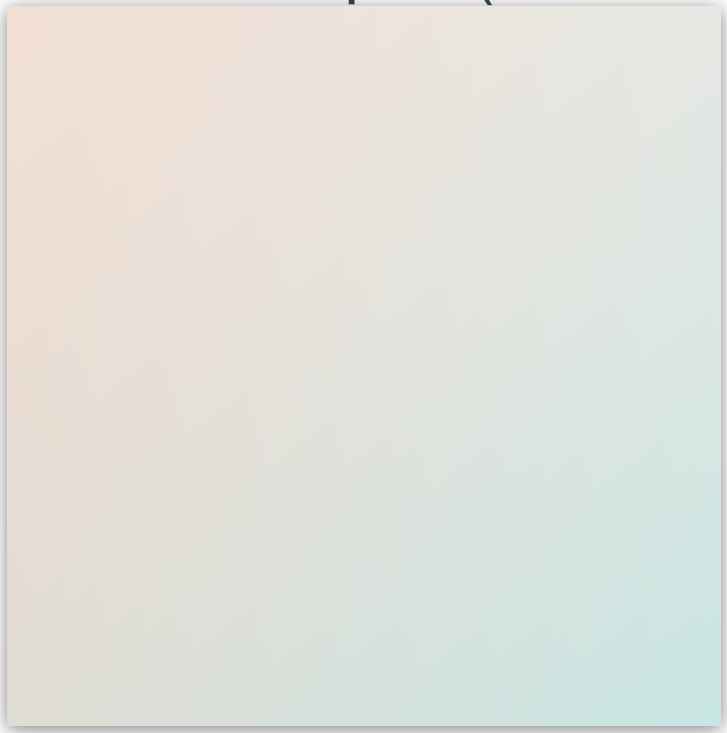
RESULT!



Reference



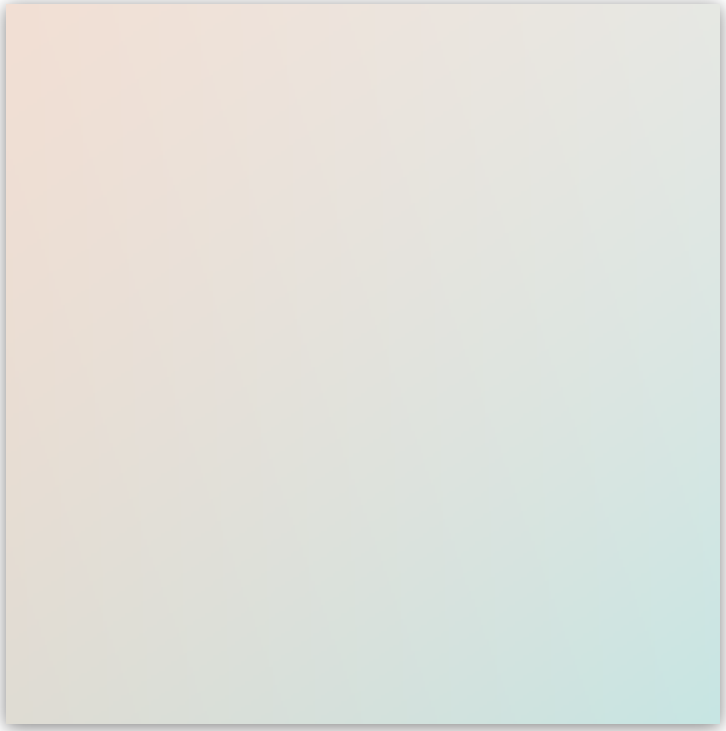
Output :(



RESULT!



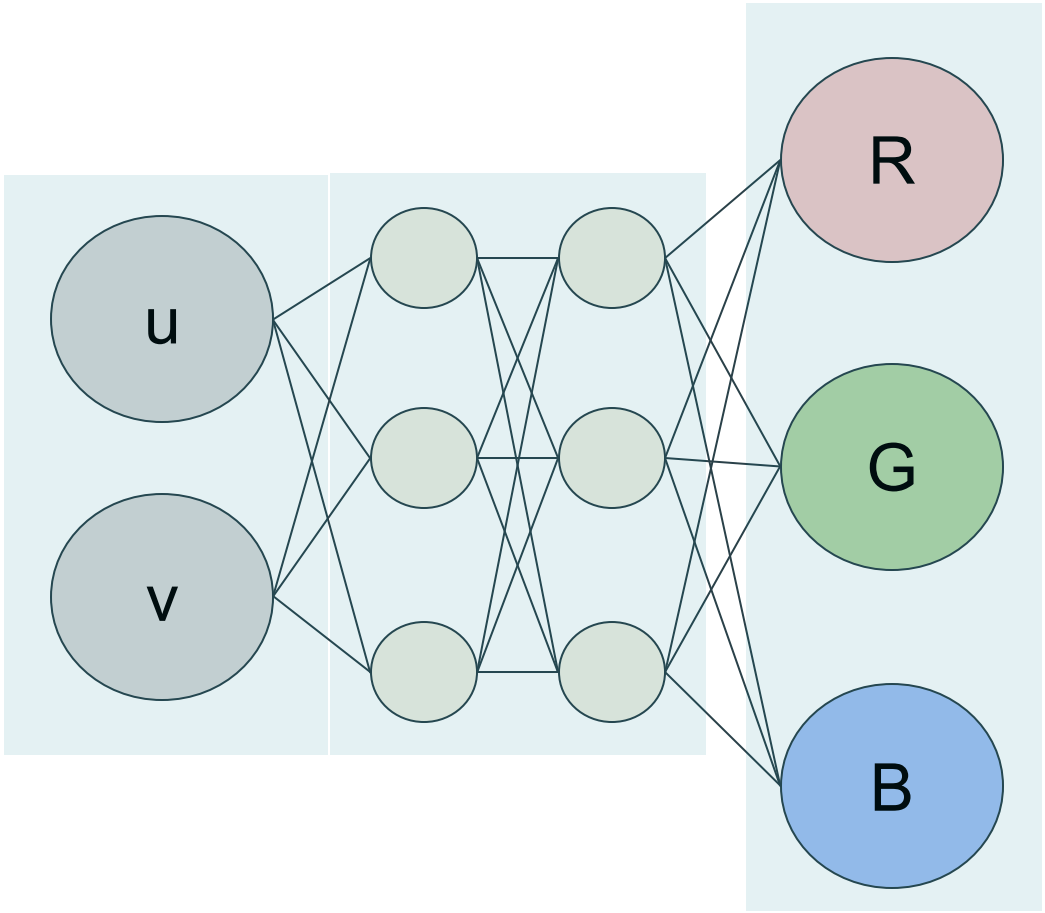
- **Just 9 parameters** and a bit of maths!
- Simple gradient fill is probably the best solution



HIDDEN LAYERS



- Inputs: Texture coordinate (uv)
- Outputs: Color channels (rgb)
- For 2 * 32 node hidden layers
  - 67 biases
  - 1184 weights
- **1251 parameters total**



IMPLEMENTATION



Slang: Connect the layer inputs/outputs

```
struct Network {
    NetworkParameters<2, 32>  layer0;
    NetworkParameters<32, 32> layer1;
    NetworkParameters<32, 3>  layer2;

    [Differentiable]
    float3 eval(no_diff float2 uv)
    {
        float inputs[2] = {uv.x, uv.y};
        float output0[32] = layer0.forward(inputs);
        [ForceUnroll]
        for (int i = 0; i < 32; ++i)
            output0[i] = activation(output0[i]);

        float output1[32] = layer1.forward(output0);
        [ForceUnroll]
        for (int i = 0; i < 32; ++i)
            output1[i] = activation(output1[i]);

        float output2[3] = layer2.forward(output1);
        [ForceUnroll]
        for (int i = 0; i < 3; ++i)
            output2[i] = activation(output2[i]);

        return float3(output2[0], output2[1], output2[2]);
    }
}
```

Python: allocate/optimize more layers

```
class Network(spy.InstanceList):
    def __init__(self):
        super().__init__(module["Network"])
        self.layer0 = NetworkParameters(2,32)
        self.layer1 = NetworkParameters(32,32)
        self.layer2 = NetworkParameters(32,3)

    # Calls the Slang 'optimize' function for the layer.
    def optimize(self, learning_rate: float, optimize_counter: int):
        self.layer0.optimize(learning_rate, optimize_counter)
        self.layer1.optimize(learning_rate, optimize_counter)
        self.layer2.optimize(learning_rate, optimize_counter)
```

RESULT!

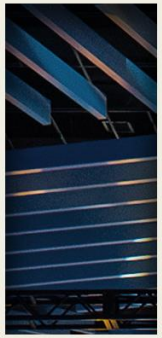


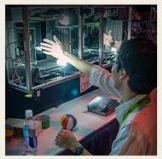
Reference



Better!









SIGGRAPH 2026
Los Angeles 19–23 JUL

Learning a Neural Texture

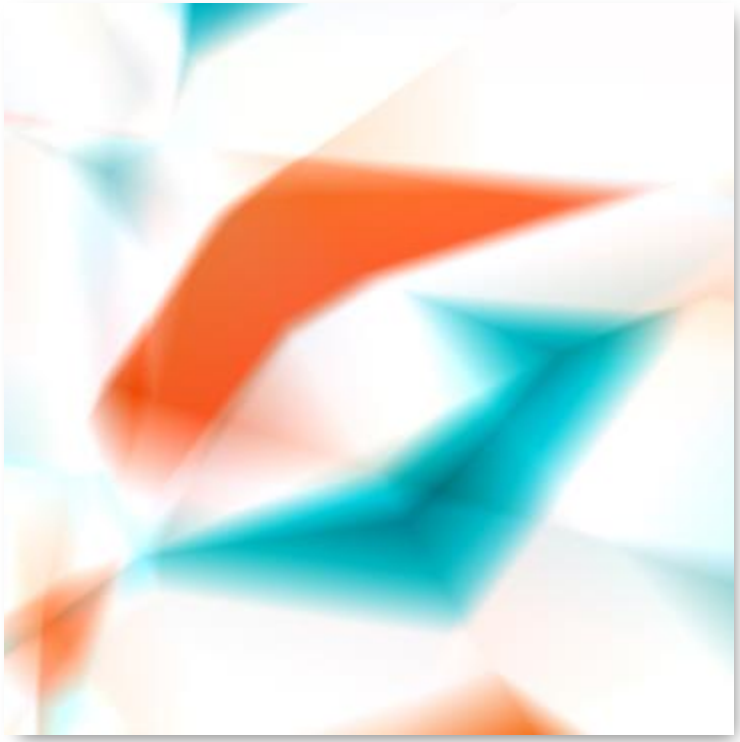
Result!



Reference



Better!



This does not look very good…

Adding more parameters?



Reference





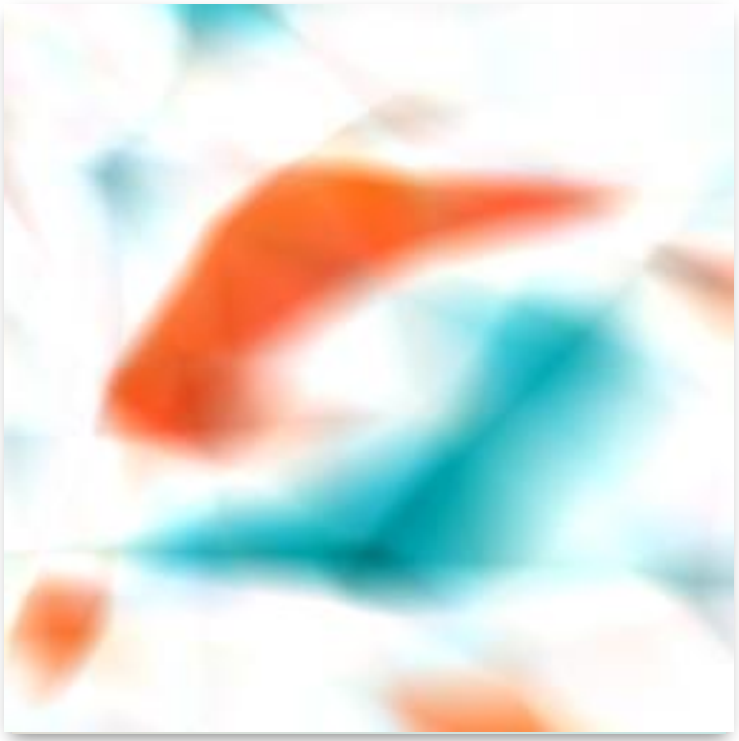
3 layers, 32 neurons

Adding more parameters?



Reference





3 layers, 64 neurons

Adding more parameters?



Reference





4 layers, 64 neurons

Adding more parameters?



- Given enough parameters, this MLP will *eventually* work
  - MLPs can approximate *anything*!
- …but we only have a finite amount of time per frame!
- Is this even practical for real-time?



4 layers, 64 neurons

Adding more parameters?

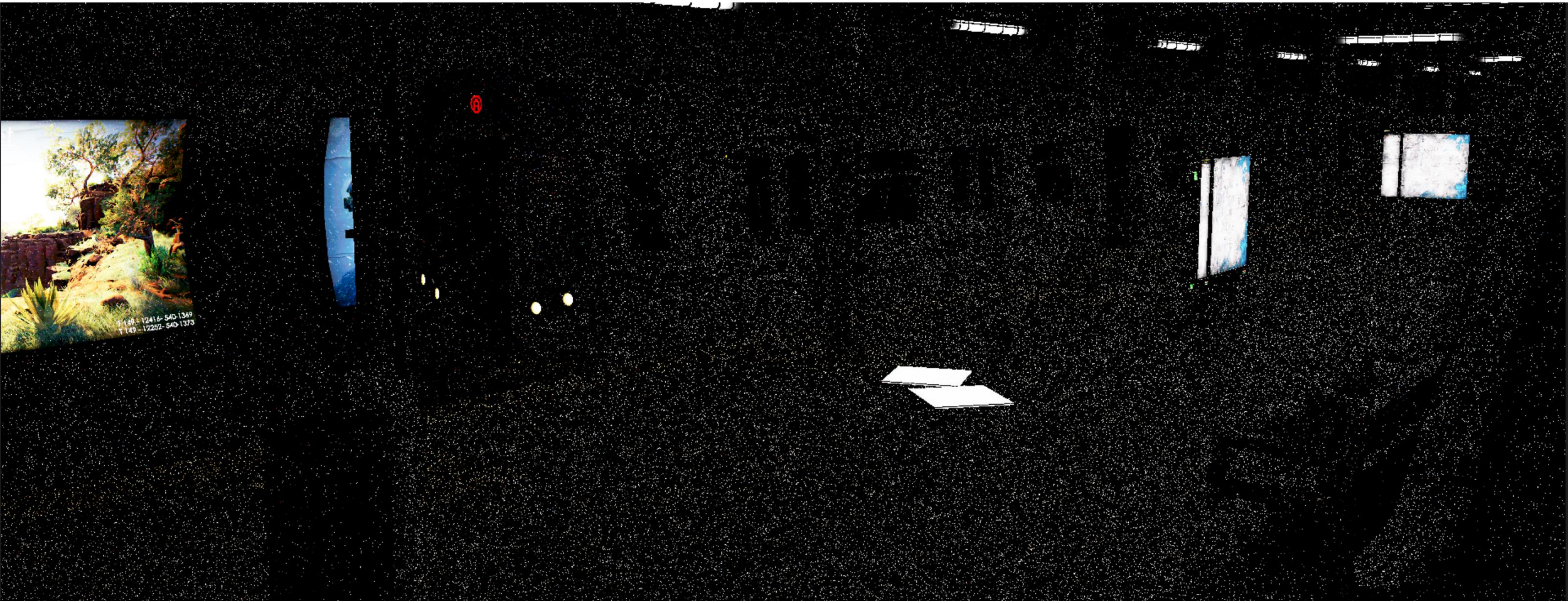


- Given enough parameters, this MLP will *eventually* work
  - MLPs can approximate *anything*!
- …but we only have a finite amount of time per frame!
- Is this even practical for real-time?
- This sounds familiar!



4 layers, 64 neurons

“Monte Carlo Raytracing can render anything”



…we just need to be smart about using it



…we just need to be smart about using it



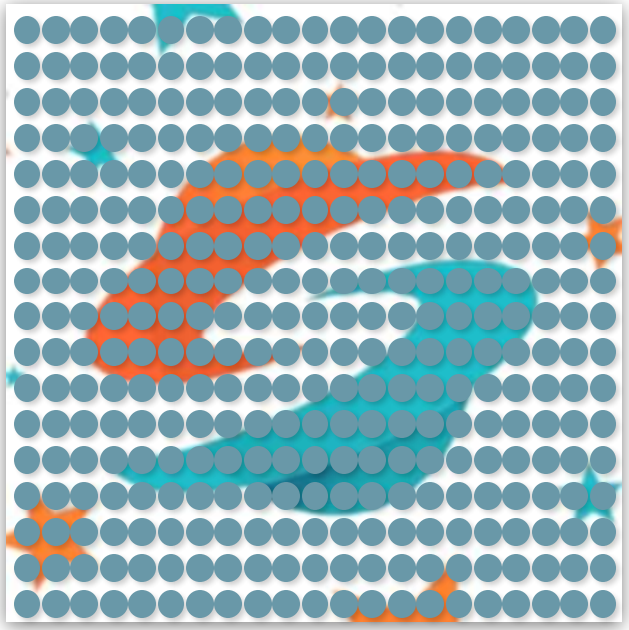
- Any “universal” tool needs careful use if we care about efficiency
- For ray tracing, we have DLSS, ReSTIR, path guiding, …
- Let’s talk about the machine learning tricks we can use for neural shading!

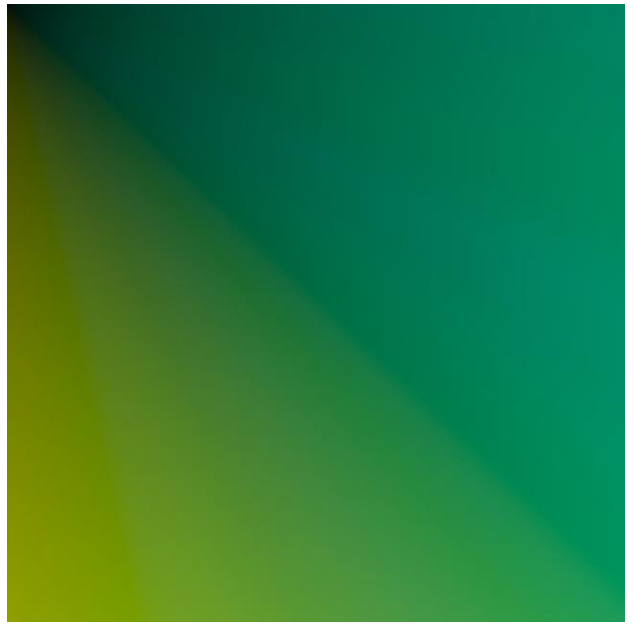
Speeding Up Training

SIGGRAPH



- Training is quite slow
- We are evaluating the loss for every pixel
- We don’t need to!





Speeding Up Training



- Training is quite slow
- We are evaluating the loss for every pixel
- We don’t need to!





Speeding Up Training



- Training is quite slow
- We are evaluating the loss for every pixel
- We don’t need to!
- Good baseline: Randomly jittered points



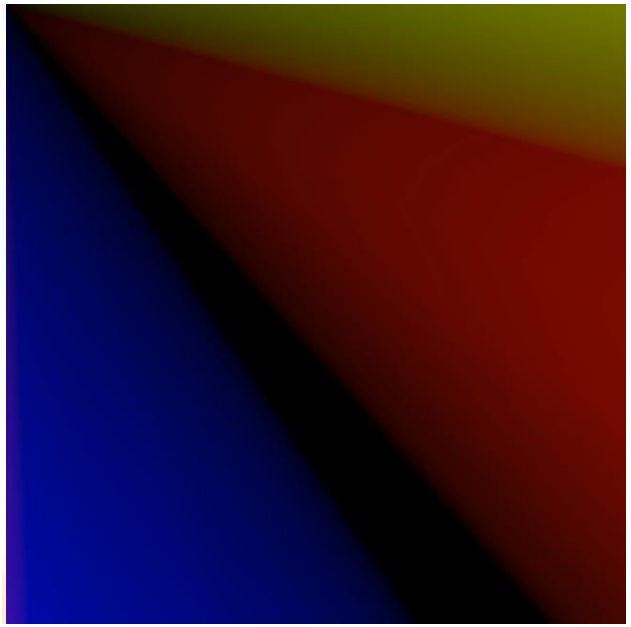


Speeding Up Training



- Training is quite slow
- We are evaluating the loss for every pixel
- We don’t need to!
- Good baseline: Randomly jittered points
- Extra credit: Slowly increase batch size



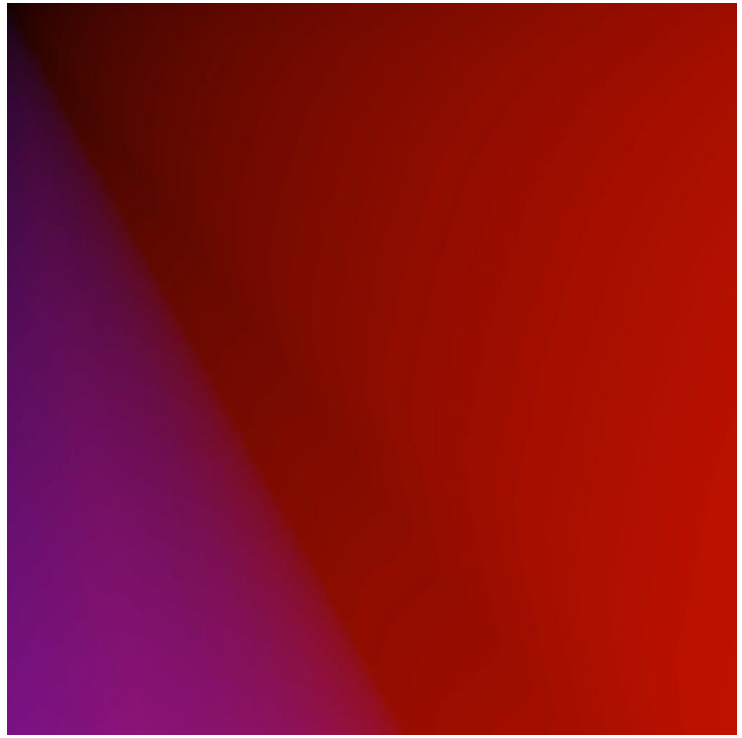


A closer Look at Activations

SIGGRAPH



- Sometimes training gets “stuck”



3 layers, 32 neurons

A closer look at activations



- Sometimes training gets “stuck”
- This is because of our choice of activation function:

```
for (int i = 0; i < 32; ++i)
    output0[i] = activation(output0[i]);
/* ... */
for (int i = 0; i < 32; ++i)
    output1[i] = activation(output1[i]);
/* ... */
for (int i = 0; i < 3; ++i)
    output2[i] = activation(output2[i]);
```

```
float activation(float x)
{
    return max(x, 0.0f);
}
```

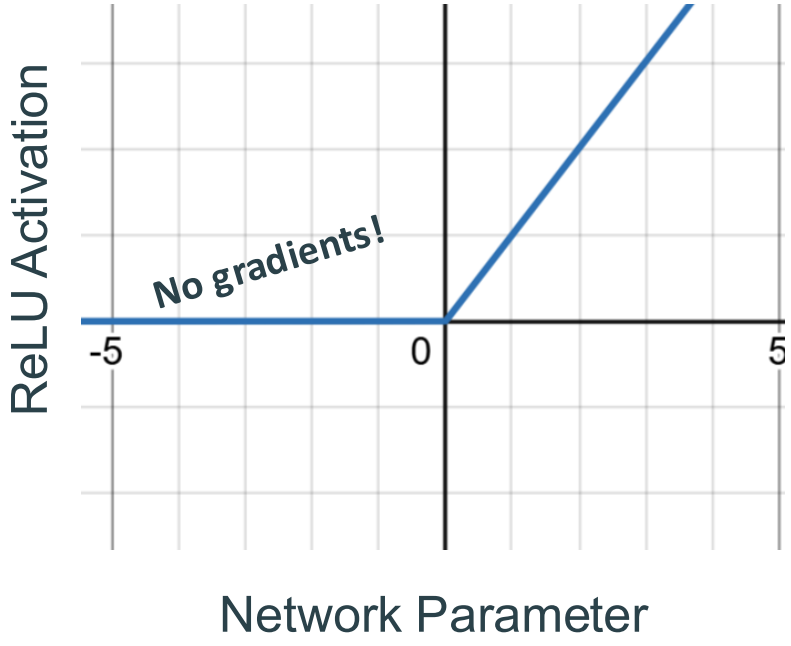


3 layers, 32 neurons

A closer look at activations



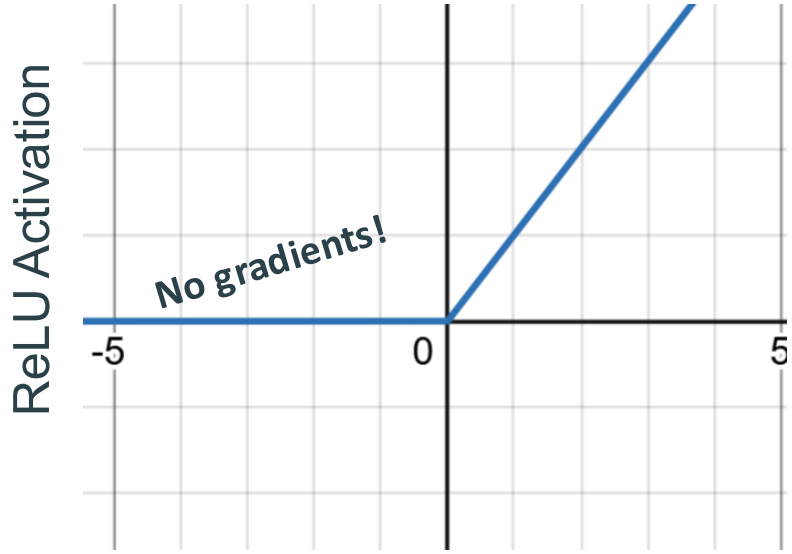
- This is a “ReLU” activation
- When the output is negative, the neuron receives no gradients
- This can cause small networks to get “stuck”



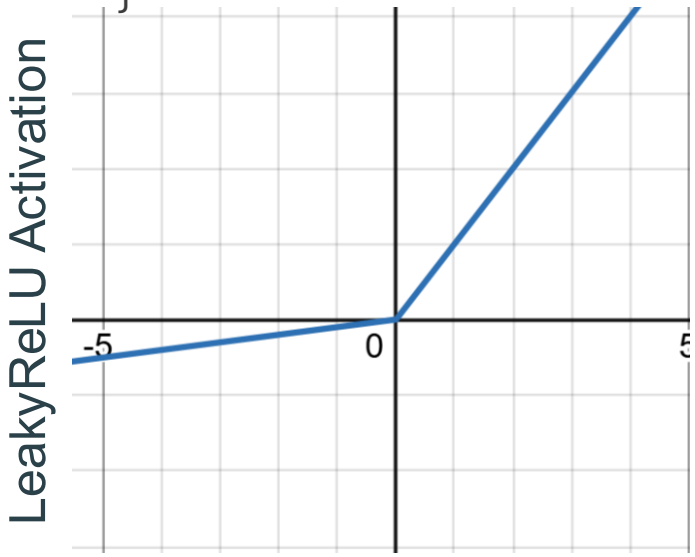
A closer look at activations



- This is a “ReLU” activation
- When the output is negative, the neuron receives no gradients
- A “leaky” activation can help:



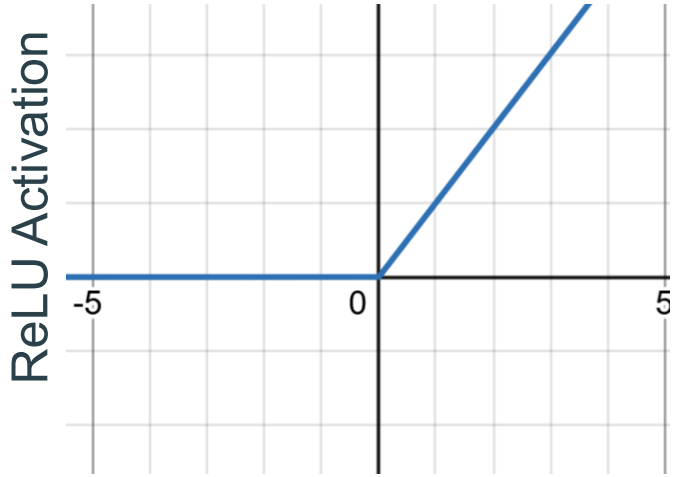

```
float leakyReLU(float x)
{
    if (x > 0.0f)
        return x;
    else
        return x * 0.01f;
}
```



A closer look at activations



- Why does our output look like it's made of triangles?
- A ReLU can only produce piecewise linear outputs
- We can do better by using a smoother activation



Network Layer Output

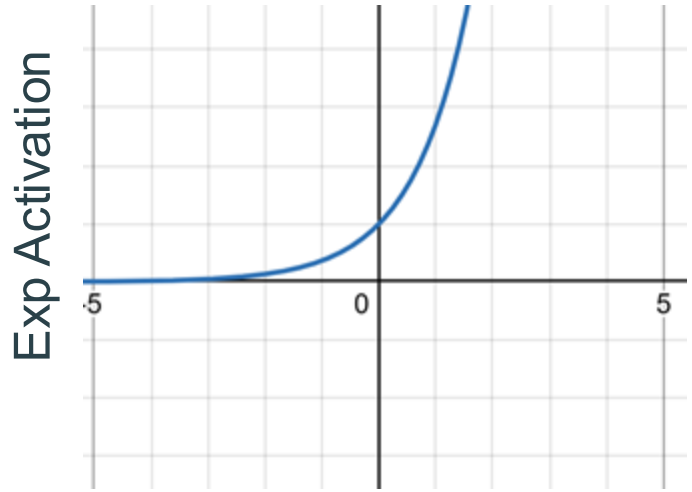


3 layers, 32 neurons

A closer look at activations



- Why does our output look like it's made of triangles?
- A ReLU can only produce piecewise linear outputs
- We can do better by using a smoother activation
- Let's try an exponential:



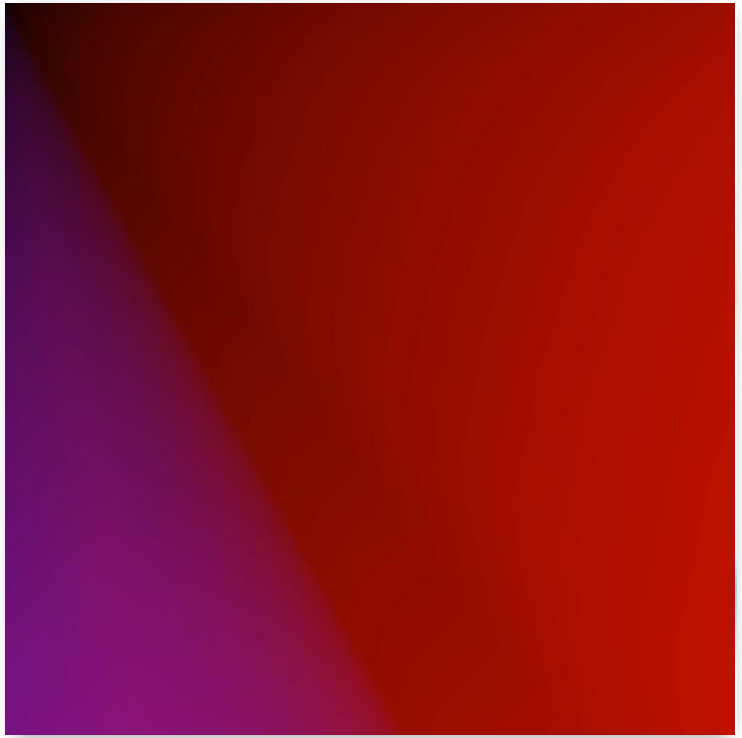


3 layers, 32 neurons

A closer Look at Activations



```
for (int i = 0; i < 32; ++i)
    output0[i] = activation(output0[i]);
/* ... */
for (int i = 0; i < 32; ++i)
    output1[i] = activation(output1[i]);
/* ... */
for (int i = 0; i < 3; ++i)
    output2[i] = activation(output2[i]);
```



3 layers, 32 neurons

A closer Look at Activations



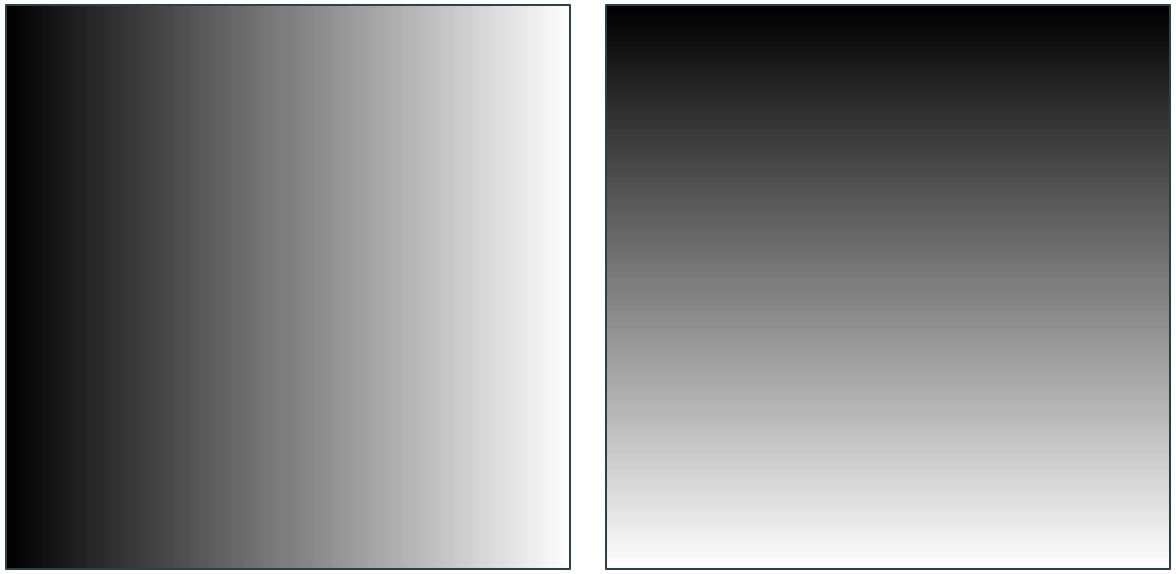
```
for (int i = 0; i < 32; ++i)
    output0[i] = leakyReLU(output0[i]);
/* ... */
for (int i = 0; i < 32; ++i)
    output1[i] = leakyReLU(output1[i]);
/* ... */
for (int i = 0; i < 3; ++i)
    output2[i] = exp(output2[i]);
```


3 layers, 32 neurons

Giving better inputs



- This is the task we're asking the model to solve:



”Image UVs”





“Image”

- i.e. “transform a gradient into an image”

Giving better inputs



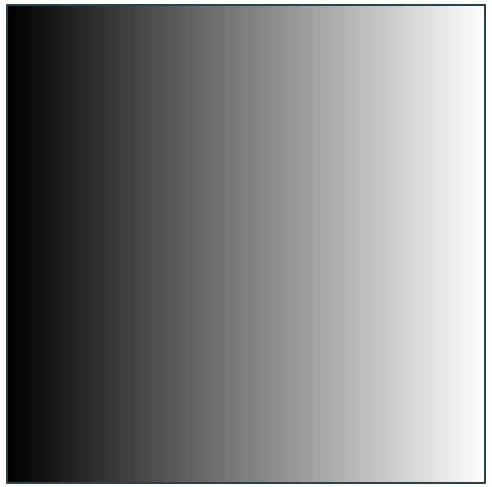
- This isn't usually how we do image representation!
- Image codecs use much more complex building blocks than gradients
  - E.g. cosines (JPEG), wavelets, BC6/BC7 blocks, …
- Given enough parameters, the network will eventually learn something similar
- For a small network, that's wasteful
  - Why learn what you already know?
- Let's try passing some JPEG inspired inputs!
  - "Extract" our domain knowledge from the network into a fixed-function input

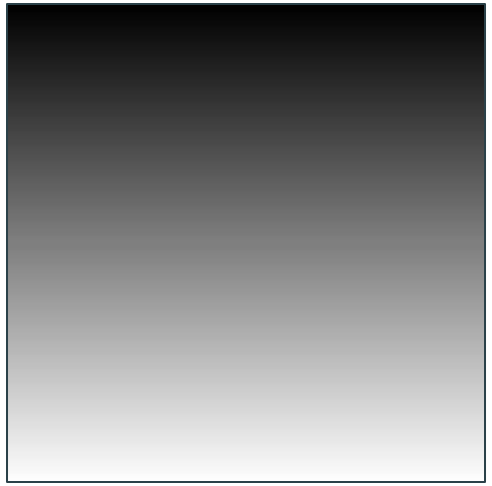
Giving better inputs



"Frequency encoding":

- Instead of passing u and v to the network…





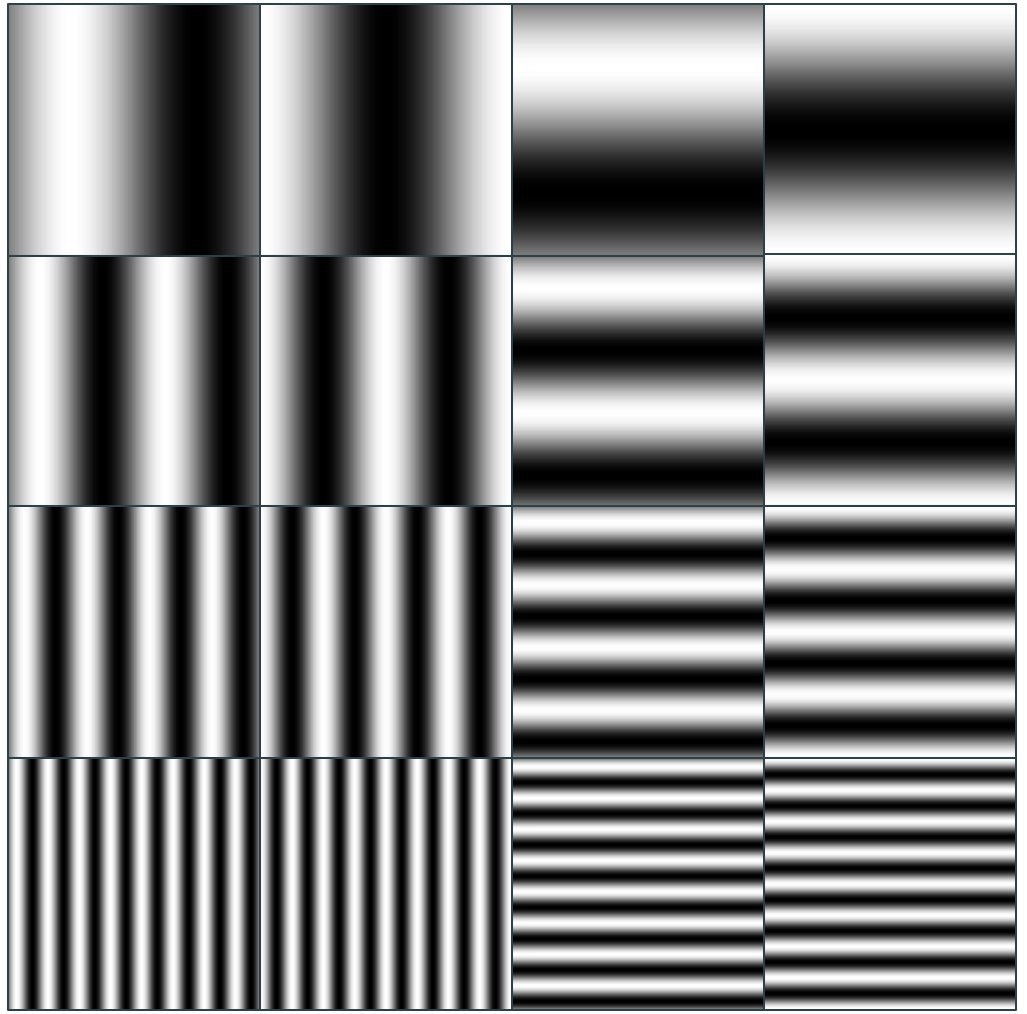
Giving better inputs



"Frequency encoding":

- Instead of passing u and v to the network…
- Pass

```
sin(u),  cos(u),  sin(v),  cos(v),
sin(2u), cos(2u), sin(2v), cos(2v),
sin(4u), cos(4u), sin(4v), cos(4v),
…
```



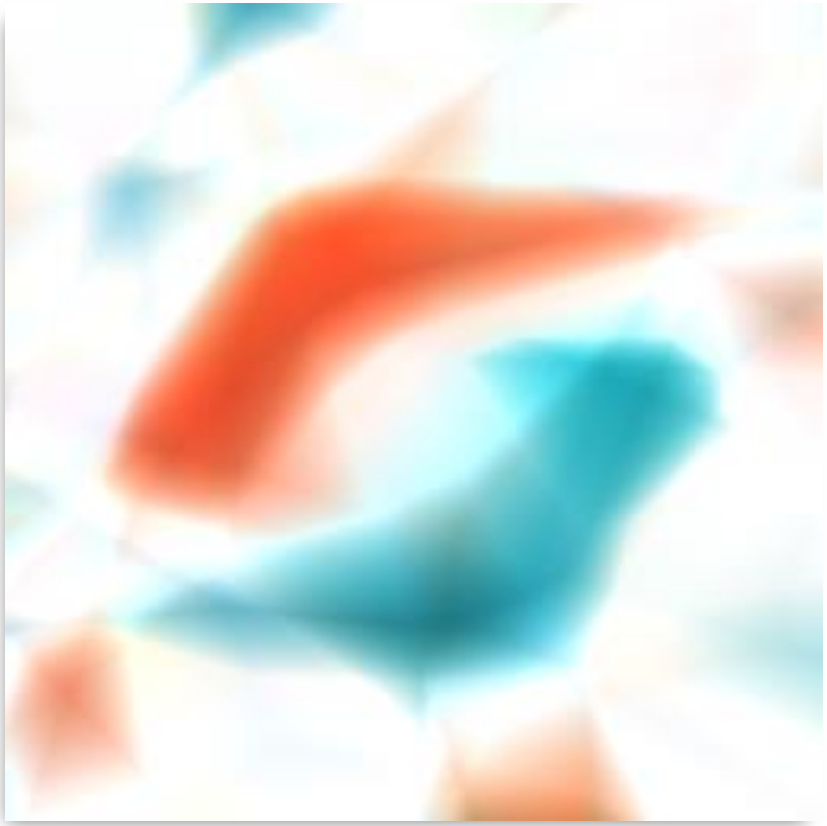
Giving better inputs



"Frequency encoding":

- Instead of passing u and v to the network…
- Pass

```
sin(u),  cos(u),  sin(v),  cos(v),
sin(2u), cos(2u), sin(2v), cos(2v),
sin(4u), cos(4u), sin(4v), cos(4v),
…
```



No Input Encoding

Giving better inputs



"Frequency encoding":

- Instead of passing u and v to the network…
- Pass

```
sin(u),  cos(u),  sin(v),  cos(v),
sin(2u), cos(2u), sin(2v), cos(2v),
sin(4u), cos(4u), sin(4v), cos(4v),
…
```



Frequency Encoding

Adding more parameters?



Reference





3 layers, 32 neurons

Adding more parameters?



Reference





3 layers, 32 neurons

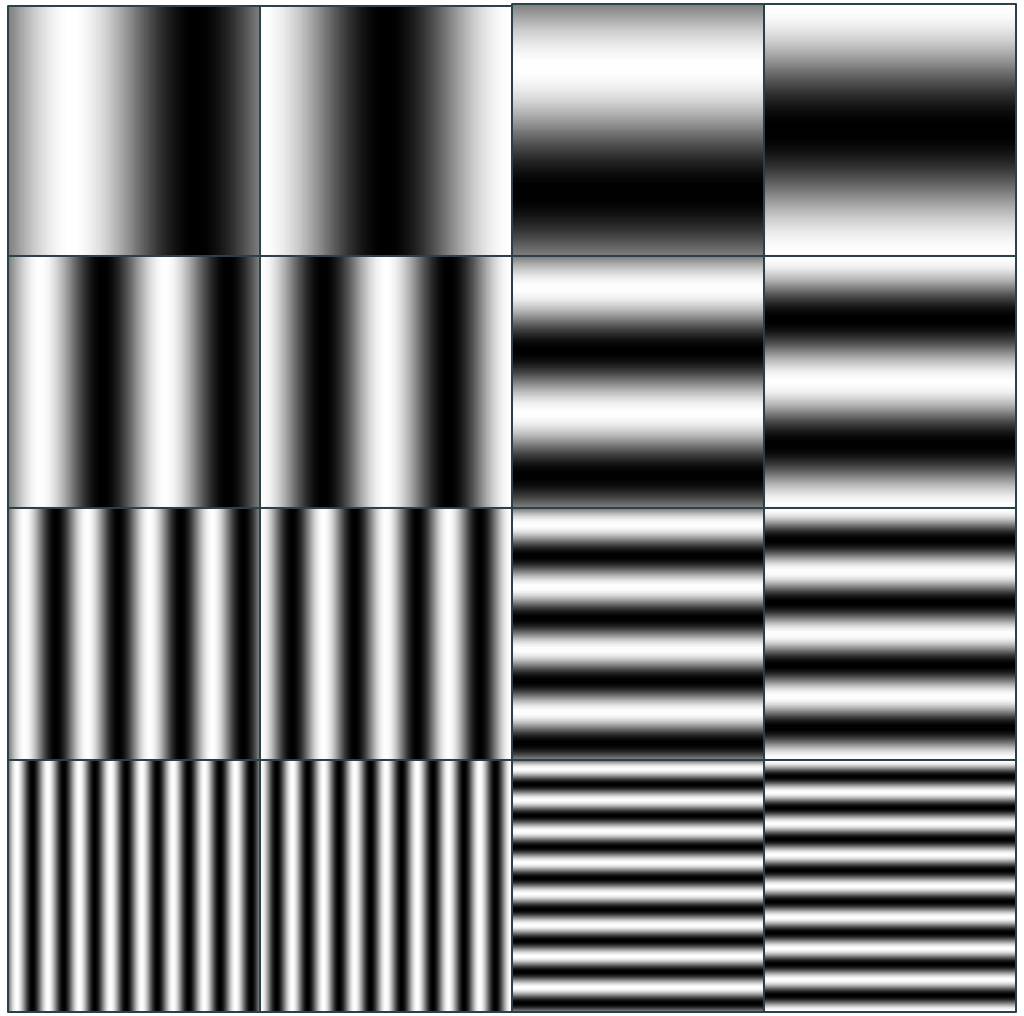
Giving better inputs



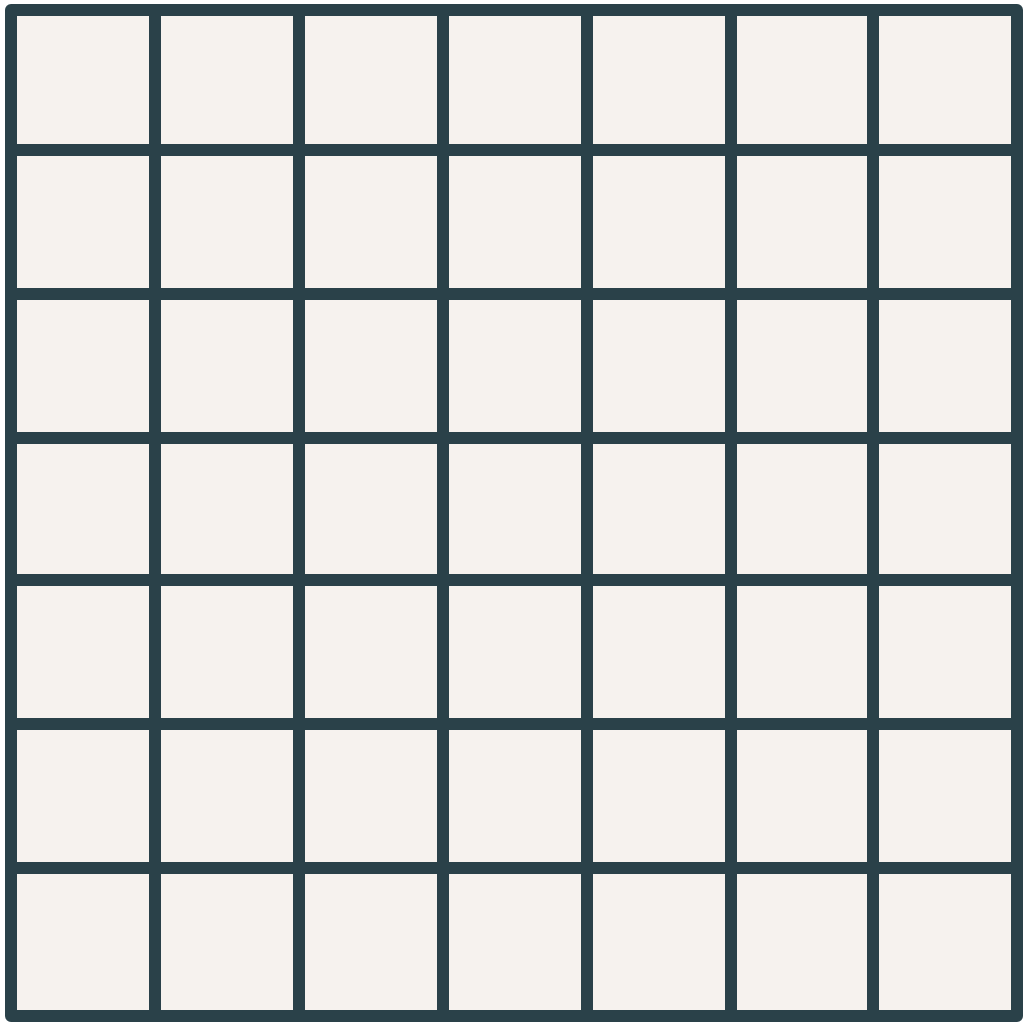
"Frequency encoding":

- Instead of passing u and v to the network…
- Pass

```
sin(u),  cos(u),  sin(v),  cos(v),
sin(2u), cos(2u), sin(2v), cos(2v),
sin(4u), cos(4u), sin(4v), cos(4v),
…
```



Giving better inputs



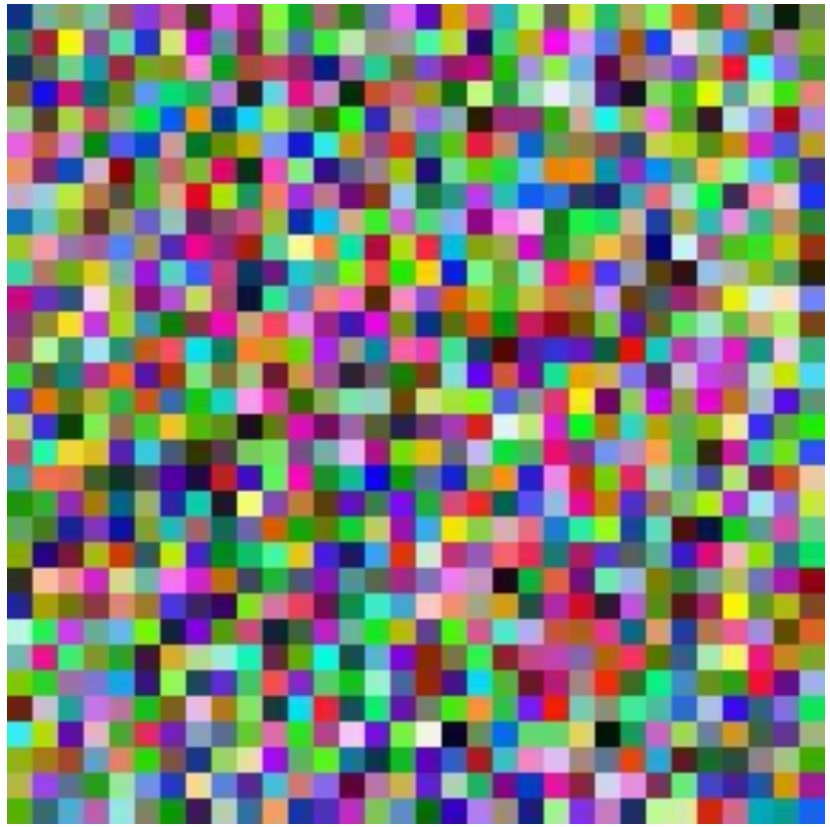
“Frequency encoding”:

- Instead of passing u and v to the network…
- Pass

```
latent_texture.Sample(uv)
```

Latent Texture

Neural Texture



Learned Latent Texture

Take-aways



- NOT: How to build a state-of-the-art neural texture compressor (that comes later in the course ☺)

Take-away #1



S*mall* trainable models need (your!) careful engineering to work well.

E.g.

- Don’t make it learn things you already know.
  - Turn your domain knowledge into fixed components of your model
- Build up a tool box of common optimization tricks:
  - Different architectures, optimizers, activations, data preprocessing, ensembling, …
  - Learn how to debug when things don’t work

A lot of problems go away when your model has 8 billion parameters

- But that’s not us ☺

Take-away #2



Your code *has* to be fast to iterate

- Solving a new task with a trainable model will rarely work right away
- You might have to put on your *applied researcher* hat ☺
  - Experiment, debug, iterate

Anything that stands in the way of fast prototyping might stop you from succeeding

- Manual derivatives
- Boilerplate (pipeline setup, shader variable binding, …)

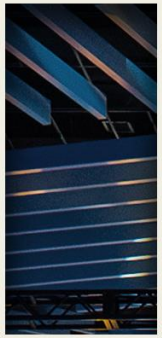
Take-away #2

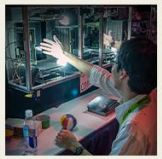


Your code *has* to be fast to iterate

The Slang ecosystem is built to tackle these challenges (auto-diff, slangpy, …)

- Driving Slang from Python is very friendly to this kind of rapid exploration
- Once you have a model that works – ship it in C++
  - But you can reuse the Slang code!







SIGGRAPH 2026
Los Angeles 19–23 JUL

Automatic Differentiation

A deep dive into slang's autodiff feature



In this section:

- Forward and backward gradient propagation
- How does auto-diff work with custom types
- Custom derivatives
- Propagate gradients to buffers

Consider a Simple Function



```
float square(float x, float y)
{
    return x * x + y * y;
}

void main()
{
    float x = 3.0f;
    float y = 4.0f;
    float result = square(x, y);
    printf("Result: %f\n", result);
}
```

```
D:\test> slangi example.slang
Result: 25.000000
```

Marking Functions as [Differentiable] Allows Slang to Generate Derivative Functions



```
[Differentiable]
float square(float x, float y)
{
    return x * x + y * y;
}
```

Computing Forward Derivatives



```
[Differentiable]
float square(float x, float y)
{
    return x * x + y * y;
}
```

$$f(x, y) = x^2 + y^2$$

$$\downarrow$$

$$f'(x, y, dx, dy) = 2xdx + 2ydy$$

Computing Forward Derivatives



```
[Differentiable]
float square(float x, float y)
{
    return x * x + y * y;
}
```

Given: $x = f(\theta) \qquad y = g(\gamma)$

$s = \text{square}(x, y)$

If we know:

"dx": $\frac{\partial x}{\partial \theta}$ and "dy": $\frac{\partial y}{\partial \gamma}$

Compute:

$$\frac{\partial s}{\partial \theta} + \frac{\partial s}{\partial \gamma}$$

i.e. $$\frac{\partial s}{\partial x}\frac{\partial x}{\partial \theta} + \frac{\partial s}{\partial y}\frac{\partial y}{\partial \gamma}$$

$$2x\ dx + 2y\ dy$$

Computing Forward Derivatives



```
[Differentiable]
float square(float x, float y)
{
    return x * x + y * y;
}

void main()
{
    let x = diffPair(3.0, 1.0); // x = 3, ∂x/∂θ = 1
    let y = diffPair(4.0, 1.0); // y = 4, ∂y/∂γ = 1
    let result = fwd_diff(square)(x, y);
    printf("dResult: %f\n", result.d);
}
```

```
D:\test> slangi example.slang
dResult: 14.000000
```

Given: $x = f(\theta) \qquad y = g(\gamma)$

$s = \text{square}(x, y)$

If we know:

"dx": $\frac{\partial x}{\partial \theta}$ and "dy": $\frac{\partial y}{\partial \gamma}$

Compute:

$$\frac{\partial s}{\partial \theta} + \frac{\partial s}{\partial \gamma}$$

i.e. $$\frac{\partial s}{\partial x}\frac{\partial x}{\partial \theta} + \frac{\partial s}{\partial y}\frac{\partial y}{\partial \gamma}$$

$$2x\ dx + 2y\ dy = 6+8 = 14$$

$x = 3,\ dx = 1,\ y = 4,\ dy = 1$

Computing Forward Derivatives



```
[Differentiable]
float square(float x, float y)
{
    return x * x + y * y;
}

void main()
{
    let x = diffPair(3.0, 1.0); // x = 3, ∂x/∂θ = 1
    let y = diffPair(4.0, 0.0); // y = 4, ∂y/∂γ = 0
    let result = fwd_diff(square)(x, y);
    printf("dResult: %f\n", result.d);
}
```

```
D:\test> slangi example.slang
dResult: 6.000000
```

Given: $x = f(\theta)$

$s = \text{square}(x, y)$

How do we find out just $\frac{\partial s}{\partial x}$ **?**

$$\frac{\partial s}{\partial x}\frac{\partial x}{\partial \theta} + \frac{\partial s}{\partial y}\frac{\partial y}{\partial \gamma}\,0$$

$\frac{\partial x}{\partial \theta}$: 1

Computing Forward Derivatives



```
[Differentiable]
float square(float x, float y)
{
    return x * x + y * y;
}

void main()
{
    let x = diffPair(3.0, 0.0); // x = 3, ∂x/∂θ = 0
    let y = diffPair(4.0, 1.0); // y = 4, ∂y/∂γ = 1
    let result = fwd_diff(square)(x, y);
    printf("dResult: %f\n", result.d);
}
```

```
D:\test> slangi example.slang
dResult: 8.000000
```

Given: $x = f(\theta)$

$s = \text{square}(x, y)$

How do we find out just $\frac{\partial s}{\partial y}$ **?**

$$\frac{\partial s}{\partial x}\frac{\partial x}{\partial \theta} + \frac{\partial s}{\partial y}\frac{\partial y}{\partial \gamma}$$

$\frac{\partial x}{\partial \theta}$ = 0, $\frac{\partial y}{\partial \gamma}$ = 1

Computing Backward Derivatives



```
[Differentiable]
float square(float x, float y)
{
    return x * x + y * y;
}
```

Given: $s = \text{square}(x, y)$

$l = f(s)$

If we know:

$$\frac{\partial l}{\partial s}$$

Compute:

$$\frac{\partial l}{\partial x} \quad \text{and} \quad \frac{\partial l}{\partial y}$$

i.e. $$\frac{\partial l}{\partial s}\frac{\partial s}{\partial x} \quad \text{and} \quad \frac{\partial l}{\partial s}\frac{\partial s}{\partial y}$$

Computing Backward Derivatives



```
[Differentiable]
float square(float x, float y)
{
    return x * x + y * y;
}

void main()
{
    var x = diffPair(3.0, 0.0);
    var y = diffPair(4.0, 0.0);
    let dLdSquare = 1.0f;
    bwd_diff(square)(x, y, dLdSquare);
    printf("dL/dx: %f, dL/dy: %f\n", x.d, y.d);
}
```

```
D:\test> slangi example.slang
dL/dx: 6.000000, dL/dy: 8.000000
```

Given: $s = \text{square}(x, y)$

$l = f(s)$

If we know:

$$\frac{\partial l}{\partial s}$$

Compute:

$$\frac{\partial l}{\partial x} \quad \text{and} \quad \frac{\partial l}{\partial y}$$

i.e. $\frac{\partial l}{\partial s}\frac{\partial s}{\partial x}$ and $\frac{\partial l}{\partial s}\frac{\partial s}{\partial y}$

$\frac{\partial l}{\partial s}$: 1 $\quad$ $\frac{\partial l}{\partial s}$: 1

Forward vs backward derivative



Forward Derivative

- Computes gradient of output with regard to **all** input parameters
- Each call is **cheaper**
- Needs **many** separate calls to find out the gradients for each input
- Uses: compute normal from SDF, du dv etc.

Backward Derivative

- Computes gradient of output with regard to **each** input parameter.
- Each call is more **expensive**
- Only need **one** call to find out gradients for each input.
- Uses: compute gradients for network weights

Using Custom Types with Auto-diff



What if `square` function takes in a user defined type?

```
struct Point
{
    float x;
    float y;
}

[Differentiable]
float square(Point p)
{
    return p.x * p.x + p.y * p.y;
}
```

Differentiable and non-differentiable types



```
[Differentiable]
float run(int op, float a, float b)
{
    if (op == 0)
        return a + b;
    else
        return a
}

void test()
{
    fwd_diff(run)( ? );
}
```

Non-differentiable argument is passed in as-is.

```
func fwd_diff(run)(op: int, a: DifferentialPair<float>,
b: DifferentialPair<float>) -> DifferentialPair<float>
```

Defined in test.slang(2)

Differentiable and non-differentiable types



```
[Differentiable]
float run(int op, float a, float b)
{
    if (op == 0)
        return a + b;
    else
        return a
}

void test()
{
    fwd_diff(run)(???);
}
```

Differentiable argument is passed in as a pair of primal and gradient values.

```
func fwd_diff(run)(op: int, a: DifferentialPair<float>,
b: DifferentialPair<float>) -> DifferentialPair<float>
```

Defined in test.slang(2)

IDifferentiable interface



Slang defines a built-in `IDifferentiable` interface to represent differentiable types.

```
interface IDifferentiable
{
    associatedtype Differential : IDifferentiable
    static Differential dzero();
    static Differential dadd(Differential, Differential);
};
```

By default, these builtin types are differentiable:

`half`, `float`, `double`

vector and matrix of floating point types

`T[]` (if `T` is differentiable).

Using Custom Types with Auto-diff



```
struct Point
{
    float x;
    float y;
}

[Differentiable]
float square(Point p)
{
    return p.x * p.x
}

void main()
{
    fwd_diff(square)(??)
}
```

`Point` is treated as non-differentiable by the type system.

```
func fwd_diff(square)(p: Point) ->
DifferentialPair<float>
```

Defined in test.slang(8)

Using Custom Types with Auto-diff



```
struct Point : IDifferentiable
{
    float x;
    float y;
}

[Differentiable]
float square(Point p)
{
    return p.x * p.x
}

void main()
{
    fwd_diff(square)(???)
}
```

Mark the type as differentiable, and the compiler will auto-synthesize the implementation to `IDifferentiable`

```
func fwd_diff(square)(p: DifferentialPair<Point>) ->
DifferentialPair<float>

Defined in test.slang(8)
```

Defining Custom Derivatives



Consider the `square` function:

```
[Differentiable]
float square(float x, float y)
{
    return x * x + y * y;
}
```

Custom Forward Derivative:

```
[ForwardDerivativeOf(square)]
DifferentialPair<float> squareFwd(
    DifferentialPair<float> x,
    DifferentialPair<float> y)
{
    return diffPair(square(x.p, y.p),
                    2.0f * (x.p * x.d + y.p * y.d));

}
```

Defining Custom Derivatives



Consider the `square` function:

```
[Differentiable]
float square(float x, float y)
{
    return x * x + y * y;
}
```

Custom Backward Derivative:

```
[BackwardDerivativeOf(square)]
void squareBwd(
    inout DifferentialPair<float> x,
    inout DifferentialPair<float> y,
    float dOut)
{
    x = diffPair(x.p, 2.0f * x.p * dOut);
    y = diffPair(y.p, 2.0f * y.p * dOut);
}
```

Propagate gradients into buffers



Consider the `square` function:

```
RWStructuredBuffer<float> yBuffer;

[Differentiable]
float square(float x, int yIdx)
{
    let y = yBuffer[yIdx];
    return x * x +  y * y;
}

[numthreads(128, 1, 1)]
void computeMain(int tid
{
    var x = diffPair(3.0
    bwd_diff(square)(x, tid, 1.0f);
}
```

What if y is read from a buffer at yIdx?

How does gradient propagate to y?

```
func bwd_diff(square)(x:
InOut<DifferentialPair<float>>, yIdx: int,
resultGradient: float) -> void
```

Defined in test.slang(3)

Propagate gradients into buffers



Consider the `square` function:

```
RWStructuredBuffer<float> yBuffer;

[Differentiable]
float square(float x, int yIdx)
{
    let y = getY(yIdx);
    return x * x +  y * y;
}

float getY(int yIdx) { return yBuffer[yIdx]; }
```

Step 1: wrap buffer access into a function.

Propagate gradients into buffers



Consider the `square` function:

```
RWStructuredBuffer<float> yBuffer;

[Differentiable]
float square(float x, int yIdx)
{
    let y = getY(yIdx);
    return x * x +  y * y;
}

float getY(int yIdx) { return yBuffer[yIdx]; }
```

```
RWStructuredBuffer<Atomic<float>> yGradBuffer;

[BackwardDerivativeOf(getY)]
void getYBwd(int yIdx, float dOut)
{
    yGradBuffer[yIdx] += dOut;
}
```

Step 1: wrap buffer access into a function.

Step 2: provide custom derivative for `getY`.

Debugging with custom derivatives



```
[Differentiable]
float square(float x, float y)
{
    let x2 = x * x;
    let y2 = y * y;
    return x2 + y2;
}

[numthreads(128, 1, 1)]
void computeMain(int tid : SV_DispatchThreadID)
{
    var x = diffPair(3.0);
    var y = diffPair(4.0);
    // compute dL/dx and dL/dy
    bwd_diff(square)(x, y, 1.0f);
}
```

Want to check the gradient of y2 when debugging? How to `printf` it out?

Debugging with custom derivatives



```
[Differentiable]
float square(float x, float y)
{
    let x2 = x * x;
    let y2 = debugGrad(y * y);
    return x2 + y2;
}

[numthreads(128, 1, 1)]
void computeMain(int tid : SV_DispatchThreadID)
{
    var x = diffPair(3.0);
    var y = diffPair(4.0);
    // compute dL/dx and dL/dy
    bwd_diff(square)(x, y, 1.0f);
}
```

```
float debugGrad(float x) { return x; }

[BackwardDerivativeOf(debugGrad)]
void debugGradBwd(
    inout DifferentialPair<float> x, float dOut)
{
    printf("Gradient is %f\n", dOut);
    x = diffPair(x.p, dOut);
}
```

Backward custom derivatives can be viewed as a “callback” on propagated gradients.

More resrouces on autodiff



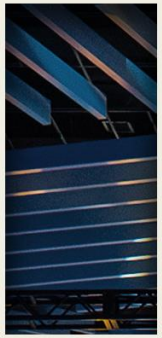
What we covered:

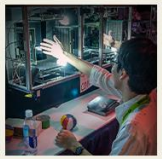
- Definition of forward/backward derivative propagation functions
- Initiating forward or backward mode differentiation
- Custom differentiable types
- Custom Derivatives and their uses

Checkout out Slang's documentation to learn more:

https://docs.shader-slang.org/en/latest/external/slang/docs/user-guide/07-autodiff.html









SIGGRAPH 2026
Los Angeles 19–23 JUL

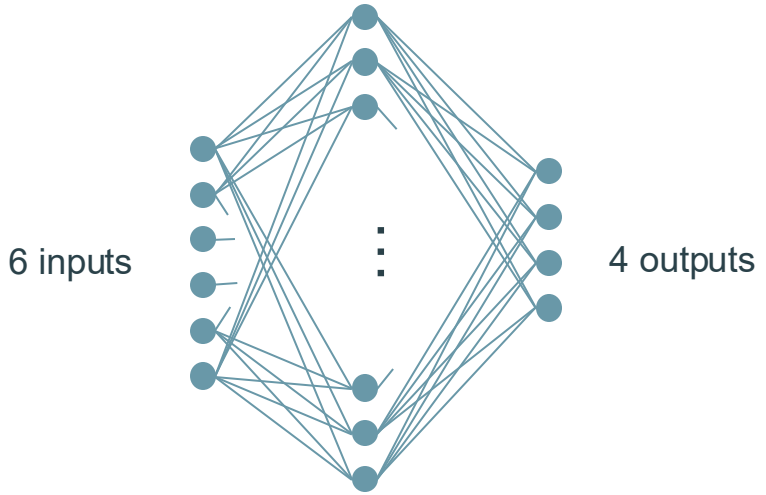
Hardware Acceleration

Implementing our toy neural texture example



```
struct NeuralTexture2D
{
    NetworkParameters<6, 32> layer1;
    NetworkParameters<32, 4> layer2;

    float4 sample(float2 uv, float2 du, float2 dv)
    {
        float input[6] = {uv.x, uv.y, du.x, du.y, dv.x, dv.y};
        let latentResult = layer1.eval(input);
        let output = layer2.eval(latentResult);
        return float4(output[0], output[1], output[2], output[3]);
    }
}
```



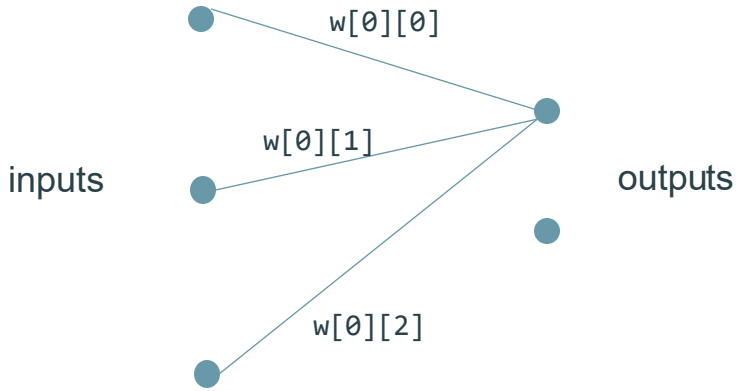
Computation of a 3-input, 2-output layer



```
outputs[0] = biases[0] +
             inputs[0] * w[0][0] +
             inputs[1] * w[0][1] +
             inputs[2] * w[0][2];
outputs[0] = max(0.0, outputs[0]);
```

Weighted Sum of Inputs

Activation



Computation of a 3-input, 2-output layer

```
outputs[0] = biases[0] +
             inputs[0] * w[0][0] +
             inputs[1] * w[0][1] +
             inputs[2] * w[0][2];
outputs[0] = max(0.0, outputs[0]);

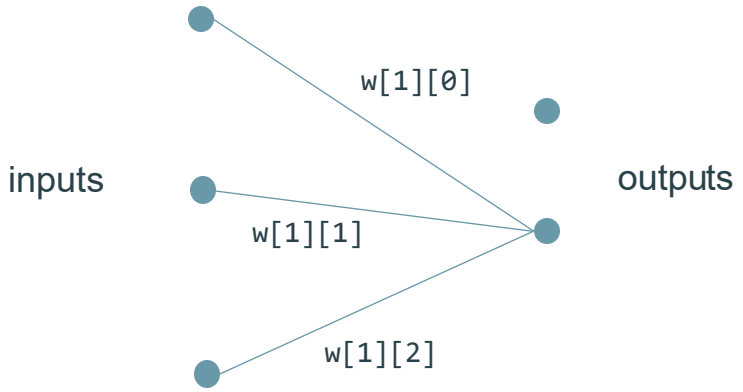
outputs[1] = biases[1] +
             inputs[0] * w[1][0] +
             inputs[1] * w[1][1] +
             inputs[2] * w[1][2];
outputs[1] = max(0.0, outputs[1]);
```

Weighted Sum of Inputs

Activation

Weighted Sum of Inputs

Activation

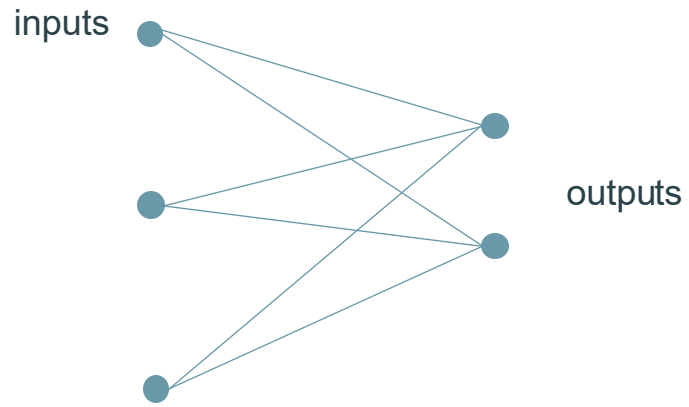


Computation of a 3-input, 2-output layer

```
outputs[0] = biases[0] +
             inputs[0] * w[0][0] +
             inputs[1] * w[0][1] +
             inputs[2] * w[0][2];
outputs[0] = max(0.0, outputs[0]);

outputs[1] = biases[1] +
             inputs[0] * w[1][0] +
             inputs[1] * w[1][1] +
             inputs[2] * w[1][2];
outputs[1] = max(0.0, outputs[1]);
```

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} + \begin{bmatrix} w_{00} & w_{01} & w_{02} \\ w_{10} & w_{11} & w_{12} \end{bmatrix} \begin{bmatrix} i_0 \\ i_1 \\ i_2 \end{bmatrix}$$



Simplified view of a Feed-Forward Layer



```
struct NetworkParameters<int InputSize, int OutputSize>
{
    float weights[InputSize][OutputSize];
    float biases[OutputSize];
    Array<float, OutputSize> eval(float input[InputSize])
    {
        float output[OutputSize] = biases;
        output += MatMulVec(weights, input);
        for (int i = 0; i < OutputSize; ++i)
            output[i] = max(output[i] * 0.01f, output[i]); // ReLU
        return output;
    }
}
```

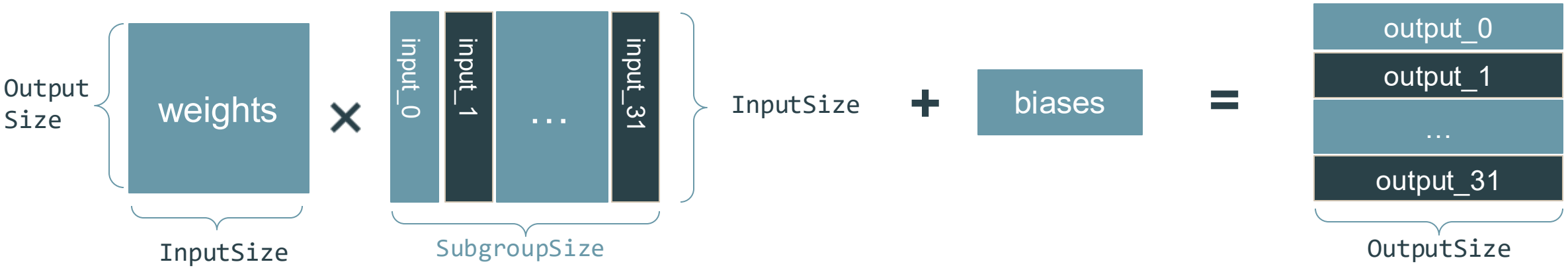
Each subgroup is computing a matrix-matrix multiplication



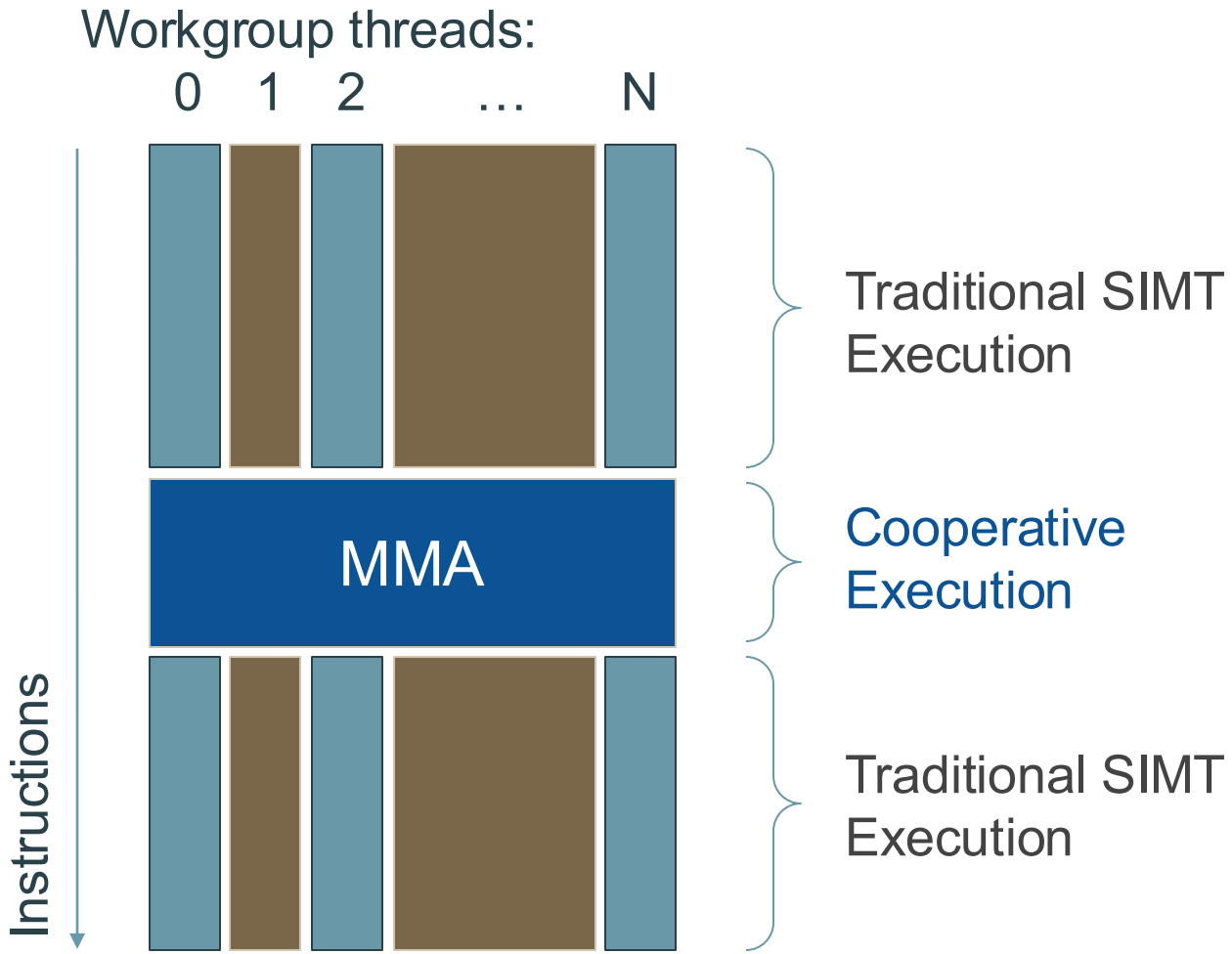
```
Array<float, OutputSize> eval(float input[InputSize])
{
   float output[OutputSize] = biases;
   output += MatMulVec(weights, input);
   ...
   return output;
}
```

Thread	Computation
0	MatMulVec(weights, input_0)
1	MatMulVec(weights, input_1)
2	MatMulVec(weights, input_2)
3	MatMulVec(weights, input_3)
…	
31	MatMulVec(weights, input_31)

Computation done by the entire subgroup: **M**atrix **M**ultiply **A**ccumulate (a.k.a. **GE**neral **M**atrix **M**ultiply)



Modern GPUs have hardware support for Matrix-Multiply-Accumulation



MMA exposed as intrinsics in various APIs



DirectX® 12 — LinAlg Matrix Preview

Vulkan™ — VK_KHR_cooperative_matrix

VK_NV_cooperative_matrix2



SIMD Group Matrix

MMA intrinsics restrictions



- Matrix operands in a Matrix-Multiply-Accumulation operation must be the same across all threads in a subgroup.
- Input vectors must be explicitly packed into a matrix and loaded in one operation.
  - Often require explicit use of group shared memory, which isn't available outside compute shaders.

Cooperative Vectors: Using MMA hardware from SIMT code



DirectX® 12

Shader Model 6.10

Vulkan™

VK_NV_cooperative_vector



Metal 4

Machine Learning

Simplified view of a Feed-Forward layer



```
struct NetworkParameters<int InputSize, int OutputSize>
{
    float weights[InputSize][OutputSize];
    float biases[OutputSize];
    Array<float, OutputSize> eval(float input[InputSize])
    {
        float output[OutputSize] = biases;
        output += MatMulVec(weights, input);
        for (int i = 0; i < OutputSize; ++i)
            output[i] = max(output[i] * 0.01f, output[i]); // ReLU
        return output;
    }
}
```

Feed-Forward layer using Cooperative Vectors



```
struct NetworkParameters<int InputSize, int OutputSize>
{
    half* weights;
    half* biases;
    CoopVec<half, OutputSize> eval(CoopVec<half, InputSize> input)
    {
        let output = coopVecMatMulAdd<half, OutputSize>(
            input, CoopVecComponentType.Float16, // input and format
            weights, CoopVecComponentType.Float16, // weights and format
            biases, CoopVecComponentType.Float16, // biases and format
            CoopVecMatrixLayout.RowMajor, // matrix layout
            false, // transpose matrix before multiply?
            sizeof(half) * InputSize); // matrix stride
        return max(output * 0.01h, output); // Leaky ReLU activation
    }
}
```

Using 16-bit floats because 32-bit float is not supported by HW.

Training with Cooperative Vectors



```
struct NetworkParameters<int InputSize, int OutputSize>
{
    half* weights;
    half* biases;

    CoopVec<half, OutputSize> eval(CoopVec<half, InputSize> input)
    {
        let output = coopVecMatMulAdd<half, OutputSize>(
            input, CoopVecComponentType.Float16, // input and format
            weights, CoopVecComponentType.Float16, // weights and format
            biases, CoopVecComponentType.Float16, // biases and format
            CoopVecMatrixLayout.RowMajor, // matrix layout
            false, // transpose matrix before multiply?
            sizeof(half) * InputSize); // matrix stride
        return max(output * 0.01h, output); // Leaky ReLU activation
    }
}
```

1. Where should we store the gradients of the weights?

2. How to make this differentiable?

Storing the gradients of the weights



```
struct NetworkParameters<int InputSize, int OutputSize>
{
    half* weights;
    half* biases;
    half* weightsGrad;
    half* biasesGrad;

}
```

1. Where should we store the gradients of the weights?

Training with Cooperative Vectors



```
struct NetworkParameters<int InputSize, int OutputSize>
{
    ...

    CoopVec<half, OutputSize> eval(CoopVec<half, InputSize> input)
    {
        let output = coopVecMatMulAdd<half, OutputSize>(
            input, CoopVecComponentType.Float16, // input and format
            weights, CoopVecComponentType.Float16, // weights and format
            biases, CoopVecComponentType.Float16, // biases and format
            CoopVecMatrixLayout.RowMajor, // matrix layout
            false, // transpose matrix before multiply?
            sizeof(half) * InputSize); // matrix stride
        return max(output * 0.01h, output); // Leaky ReLU activation
    }
}
```

How to make `eval` differentiable?

- `coopVecMatMulAdd` intrinsic is non-differentiable.

Make eval differentiable



```
struct NetworkParameters<int InputSize, int OutputSize>
{
    ...
    half* weightsGrad;
    half* biasesGrad;

    [Differentiable]
    CoopVec<half, OutputSize> eval(CoopVec<half, InputSize> input)
    {
        let output = matMulAdd(input);
        return max(output * 0.01h, output); // Leaky ReLU activation
    }

    CoopVec<half, OutputSize> matMulAdd(CoopVec<half, InputSize> input)
    {
       let output = coopVecMatMulAdd<half, OutputSize>(
            input, CoopVecComponentType.Float16, // input and format
            weights, CoopVecComponentType.Float16, // weights and format
            biases, CoopVecComponentType.Float16, // biases and format
            CoopVecMatrixLayout.RowMajor, // matrix layout
            false, // transpose matrix before multiply?
            sizeof(half) * InputSize); // matrix stride
```

Step:

- Decorate `eval` as `Differentiable`
- Provide wrapper `matMulAdd` for the non-differentiable intrinsic
- Define custom derivative for the `matMulAdd`

Make matMulAdd differentiable with custom derivative



```
struct NetworkParameters<int InputSize, int OutputSize>
{
    ...
    half* weightsGrad;
    half* biasesGrad;

    ...
    CoopVec<half, OutputSize> matMulAdd(CoopVec<half, InputSize> input)
    { ... }

    [BackwardDerivativeOf(matMulAdd)]
    void matMulAddBwd(
        inout DifferentiablePair<CoopVec<half, InputSize> input,
        CoopVec<half, OutputSize> resultGrad)
    {

        // What goes here??

    }
}
```

Task: propagate gradients from each output element (`resultGrad`) to:

- weight and bias parameters (to store in `weightsGrad` and `biasesGrad`).
- Each input element (return via `input.d`)

Gradient propagation for matrix-vector multiplication



Given matrix-vector multiplication

$$\boldsymbol{u} = \mathrm{M}\boldsymbol{v} + B$$

We have:

$$d\mathrm{M} = d\mathbf{u} \otimes_{\text{outer}} \boldsymbol{v}$$

$$d\mathrm{B} = d\mathbf{u}$$

$$d\mathbf{v} = \mathrm{M}^T\, d\mathbf{u}$$

$\otimes_{\text{outer}}$: outer vector product

Accumulating gradients across threads



Given matrix-vector multiplication

$$\boldsymbol{u} = \mathrm{M}\boldsymbol{v} + B$$

We have:

$$d\mathrm{M} = d\mathbf{u} \otimes_{\text{outer}} \boldsymbol{v}$$

$$d\mathrm{B} = d\mathbf{u}$$

$$d\mathbf{v} = \mathrm{M}^T\, d\mathbf{u}$$

$\otimes_{\text{outer}}$: outer vector product

M

dM

B

dB

Thread	Computation
0	u = malMulVecAdd(M, v0, B)
1	u = malMulVecAdd(M, v1, B)
2	u = malMulVecAdd(M, v2, B)
…	
N	u = malMulVecAdd(M, vN, B)

Each thread accesses different v, but the same M and B

Accumulating gradients across threads

Given matrix-vector multiplication

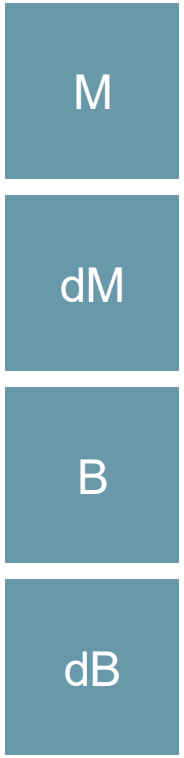
$$\boldsymbol{u} = \mathrm{M}\boldsymbol{v} + B$$

We have:

$$d\mathrm{M} \mathrel{+}= d\mathbf{u} \otimes_{\text{outer}} \boldsymbol{v}$$

$$d\mathrm{B} \mathrel{+}= d\mathbf{u}$$

$$d\mathbf{v} = \mathrm{M}^T\, d\mathbf{u}$$



Thread	Computation
0	u = malMulVecAdd(M, v0, B) dM += outerProduct(…) dB += dU
1	u = malMulVecAdd(M, v1, B) dM += outerProduct(…) dB += dU
2	u = malMulVecAdd(M, v2, B) dM += outerProduct(…) dB += dU
…	
N	u = malMulVecAdd(M, vN, B) dM += outerProduct(…) dB += dU

Needs to be atomic to prevent race conditions!

Intrinsics to speedup atomic gradient accumulation



Given matrix-vector multiplication

$$\boldsymbol{u} = \mathrm{M}\boldsymbol{v} + B$$

We have:

$$d\mathrm{M} \mathrel{+}= d\mathbf{u} \otimes_{\text{outer}} \boldsymbol{v}$$

$$d\mathrm{B} \mathrel{+}= d\mathbf{u}$$

$$d\mathbf{v} = \mathrm{M}^T\, d\mathbf{u}$$

Each step has its corresponding coop-vector intrinsic:

`coopVecOuterProductAccumulate`

`coopVecReduceSumAccumulate`

`coopVecMatMul`

Back-prop gradients through eval



```
[BackwardDerivativeOf(eval)]
void matMulAddBwd(
    inout DifferentialPair<CoopVec<half, InputSize>> input,
    CoopVec<half, OutputSize> resultGrad)
{
    // Back-prop gradients to the weights matrix.
    coopVecOuterProductAccumulate(
        resultGrad.data,
        input.v.data,
        weightsGrad, 0,
        CoopVecMatrixLayout.TrainingOptimal, CoopVecComponentTy...Float16);

    // Back-prop gradients to the biases vector.
    coopVecReduceSumAccumulate(resultGrad.data, biasesGrad, biasesOffset);

    ...
```

`coopVecOuterProductAccumulate` **requires** `weightsGrad` to be stored in `TrainingOptimal` layout.

$$d\mathrm{M} \mathrel{+}= d\mathbf{u} \otimes_{\mathrm{outer}} \boldsymbol{v}$$

$$d\mathrm{B} \mathrel{+}= d\mathbf{u}$$

Back-prop gradients through eval



```
[BackwardDerivativeOf(eval)]
void matMulAddBwd(
    inout DifferentialPair<CoopVec<half, InputSize>> input,
    CoopVec<half, OutputSize> resultGrad)
{
    ...
    // Back-prop gradients to the input vector.
    let dInput = coopVecMatMul<half, InputSize>(
        resultGrad.data, CoopVecComponentType.Float16,
        weights, CoopVecComponentType.Float16,
        CoopVecMatrixLayout.ColumnMajor, false, sizeof(half)*InputSize);
    input = {input.p, {dInput}};
}
```

$$d\mathbf{v} = \mathrm{M}^T \, d\mathbf{u}$$

Compute
i.e. `transpose(weights) * resultGrad`
by setting the layout of weights to **`ColumnMajor`**.

Callers of Feed-Forward layer can now be autodiff'ed



```
struct NeuralTexture2D
{
    NetworkParameters<6, 32> layer1;
    NetworkParameters<32, 4> layer2;

    [Differentiable]
    float4 sample(float2 uv, float2 du, float2 dv)
    {
        float input[6] = {uv.x, uv.y, du.x, du.y, dv.x, dv.y};
        let latentResult = layer1.eval(input);
        let output = layer2.eval(latentResult);
        return float4(output[0], output[1], output[2], output[3]);
    }
}
```

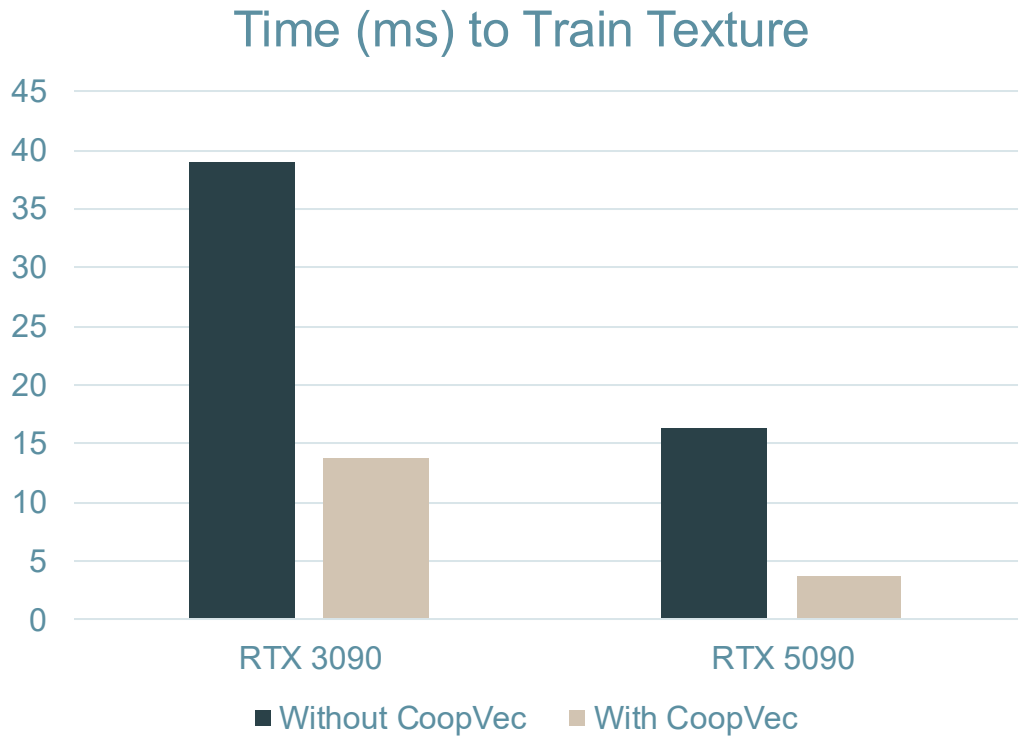
Performance improvements with Coop-Vector



- **Benchmark:** Neural Texture Training
- **Input:** **100MB** PNG



Over **3x** speedup compared to highly optimized code without coop-vec.



Targets have no Coop-Vector: Coop-Matrix intrinsic APIS



Tensor Core

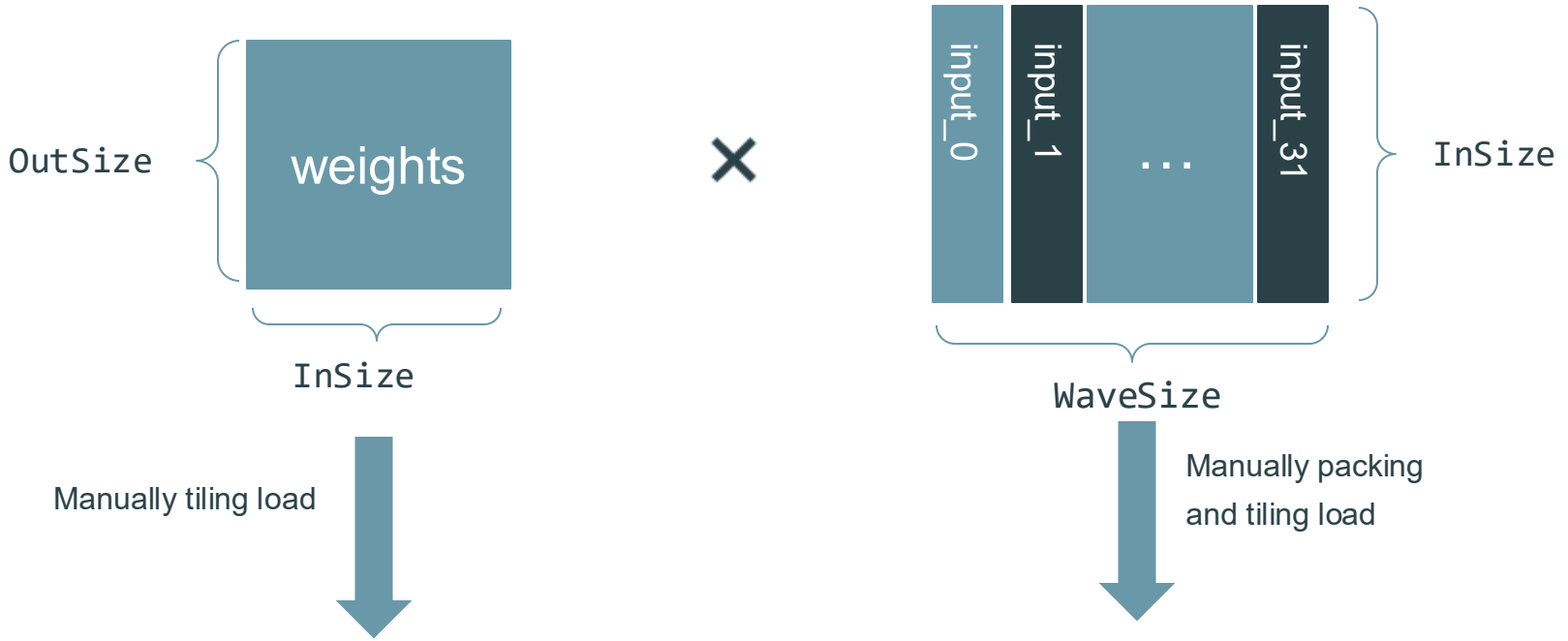


VK_NV_cooperative_matrix



SIMD Matrix

Targets have no Coop-Vector: Coop-Matrix intrinsic APIS

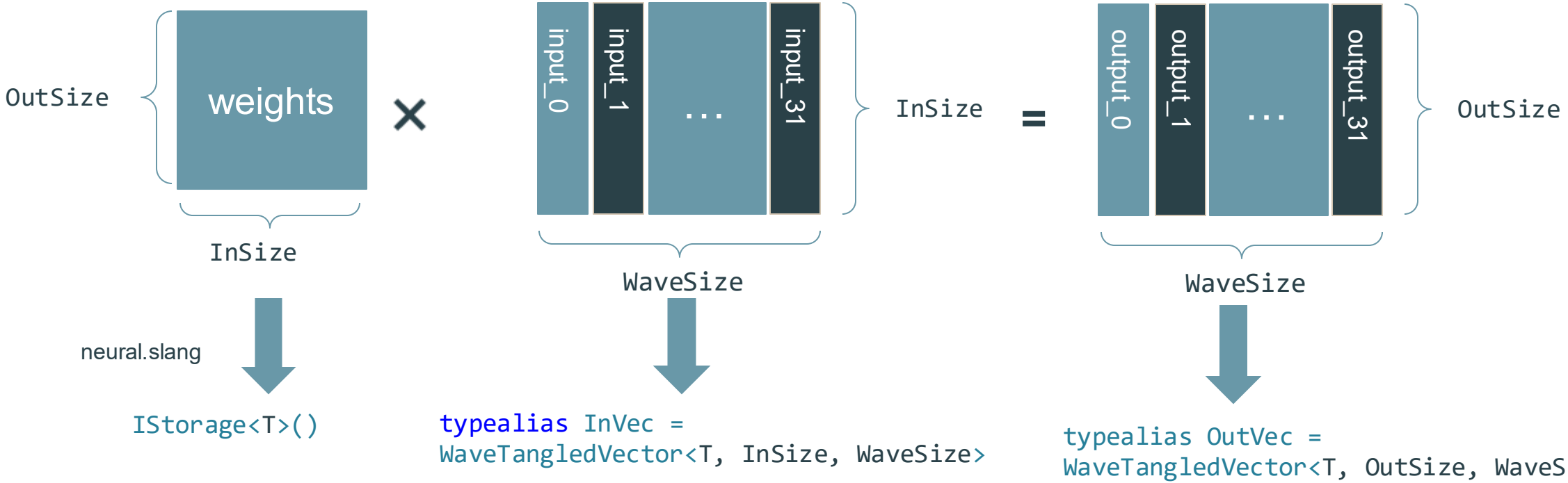



```
CoopMat<T, OutSize/M, InSize/K>[M][k]
```

```
CoopMat<T, InSize/K, WaveSize/N>[K][N]
```

```
for each tile K:
   for each tile M:
      for each tile N:
         coopMatMulAdd(matA, matB, matC)
```

Use neural.slang to abstract your MLP inferencing and training



Inferencing:

```
FFLayer<T, InVec, OutVec, Layout, ReLu> layer;
layer.eval();
```

Training:

```
FFLayer<T, InVec, OutVec, Layout, ReLu> layer;
diff_bwd(layer.eval)();
```

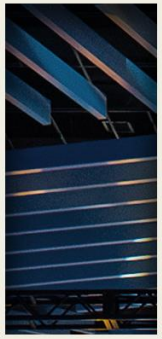
Resources



- **Source code and other resources:**
  - Course page: shader-slang.org/landing/siggraph-26/
  - Course Material: github.com/shader-slang/neural-shading-s26
  - Check out the “MLP-Training-CoopVec” example for both shader (Slang) and host-side (C++) code.
  - NVIDIA Neural Shading SDK: github.com/NVIDIA-RTX/RTXNS
  - neural.slang examples: github.com/shader-slang/slangpy-samples/tree/main/experiments/neural_slang



Course Landing Page







SIGGRAPH 2026
Los Angeles 19–23 JUL

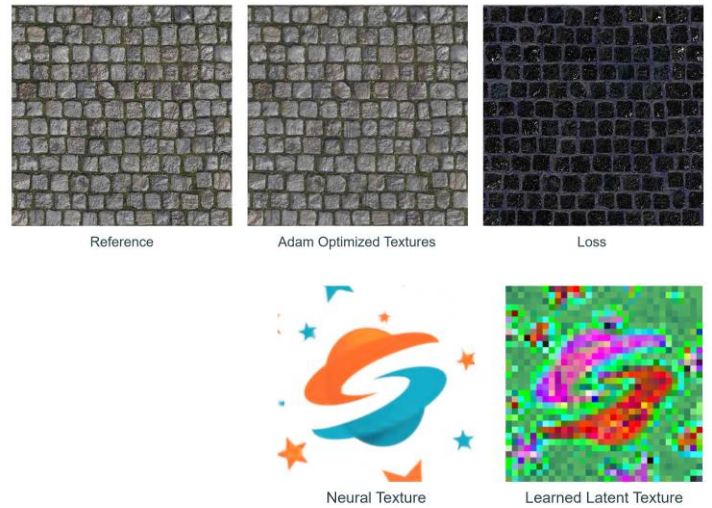
Neural Materials

© 2026 SIGGRAPH. ALL RIGHTS RESERVED.

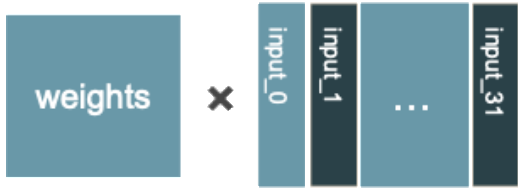
Quick Recap



- We can learn the parameters (aka textures) of a ‘fixed function’ material
- We can learn the parameters (aka weights/biases) of a neural network to represent a texture.
- Slang makes both of these tasks much simpler with auto diff
- And we now have hardware that can perform neural calculations very fast




```
bwd_diff(loss)( /* ... */ );
```



Introducing neural materials

Real-Time Neural Appearance Models

TIZIAN ZELTNER*, NVIDIA, Switzerland
FABRICE ROUSSELLE*, NVIDIA, Switzerland
ANDREA WEIDLICH*, NVIDIA, Canada
PETRIK CLARBERG*, NVIDIA, Sweden
JAN NOVÁK*, NVIDIA, Czech Republic
BENEDIKT BITTERLI*, NVIDIA, USA
ALEX EVANS, NVIDIA, United Kingdom
TOMÁŠ DAVIDOVIČ, NVIDIA, Czech Republic
SIMON KALLWEIT, NVIDIA, Switzerland
AARON LEFOHN, NVIDIA, USA



Fig. 1. Close-up renderings of a TEAPOT asset with our neural BRDF. Our model learns the intricate details and complex multi-layered material behavior of the ceramic, fingerprints, smudges, and dust which are responsible for the realism of the object while being faster to evaluate than traditional non-neural models of similar complexity. The system we present allows us to include such high-fidelity objects in real-time renderers in a scalable way.

We present a complete system for real-time rendering of scenes with complex appearance previously reserved for offline use. This is achieved with a combination of algorithmic and system level innovations.

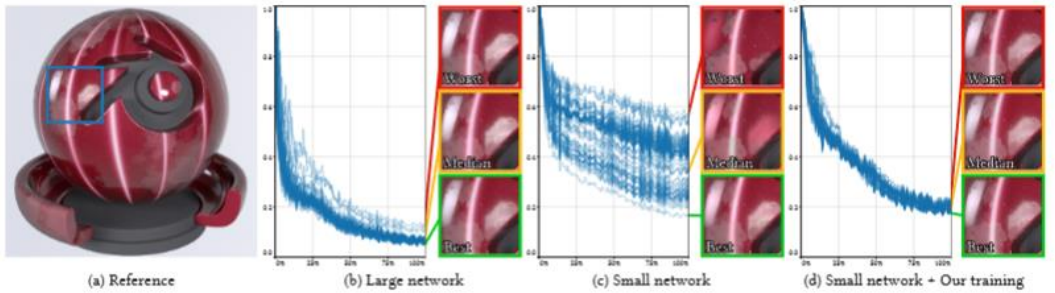
Our appearance model utilizes learned hierarchical textures that are

By exposing hardware accelerated tensor operations to ray tracing shaders, we show that it is possible to inline and execute the neural decoders efficiently inside a real-time path tracer. We analyze scalability with increasing number of neural materials and propose to improve performance using

Transactions on Graphics (Presented at SIGGRAPH 2024)

Taming optimization variance in compact neural shading networks

BENEDIKT BITTERLI, NVIDIA, USA
PETRIK CLARBERG, NVIDIA, Sweden
CHRIS CUMMINGS, NVIDIA, United Kingdom
AARON LEFOHN, NVIDIA, USA
STEVE MARSCHNER, Cornell University, USA and NVIDIA, USA
JAN NOVÁK, NVIDIA, Czech Republic
FABRICE ROUSSELLE, NVIDIA, Switzerland
ANDREA WEIDLICH, NVIDIA, Canada
TIZIAN ZELTNER, NVIDIA, Switzerland



(a) Reference (b) Large network (c) Small network (d) Small network + Our training

Fig. 1. We train 64 different initializations of a neural reflectance model for a challenging reference material with multiple layers **(a)**. For a decoder network with excess capacity **((b)**, 54k parameters), all training runs converge to a solution of similar quality. For an undersized network barely able to capture the material **((c)**, 947 parameters), there is significant variance in training outcomes, with many instances missing important features. Using our multi-instance training approach **(d)**, all runs converge to a solution of similar quality, at the same training cost as the baseline **(c)**. Although the loss is higher than **(b)** as expected, training outcomes are equally as consistent as the much larger network.

We present a training algorithm to mitigate optimization instabilities in small neural networks, like those used in real-time neural shading applications. While large, overparameterized models exhibit predictable convergence, smaller architectures often suffer from high optimization variance: differ-

model with significantly higher likelihood than optimizing a single instance only. We develop and analyze the algorithm in the context of learning reflectance functions for neural shading. This is a challenging task for small neural models due to the high dynamic range of the target function. In

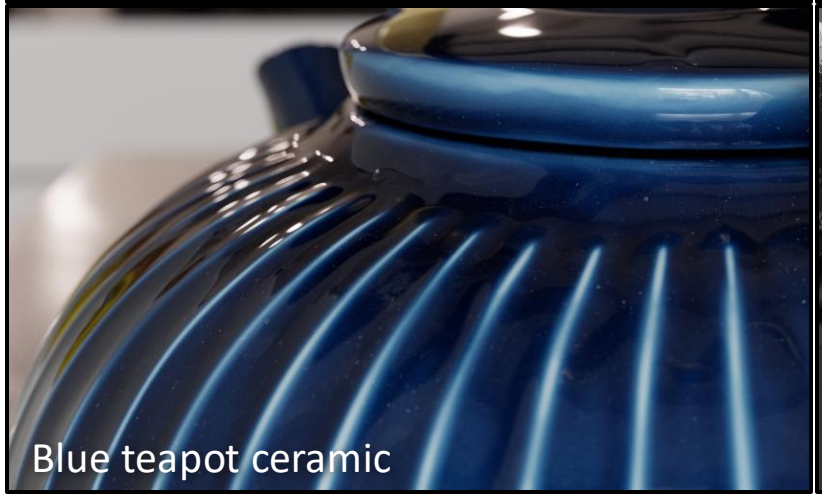
SIGGRAPH 2026 (Conference Track)

We can render these, but not in real-time



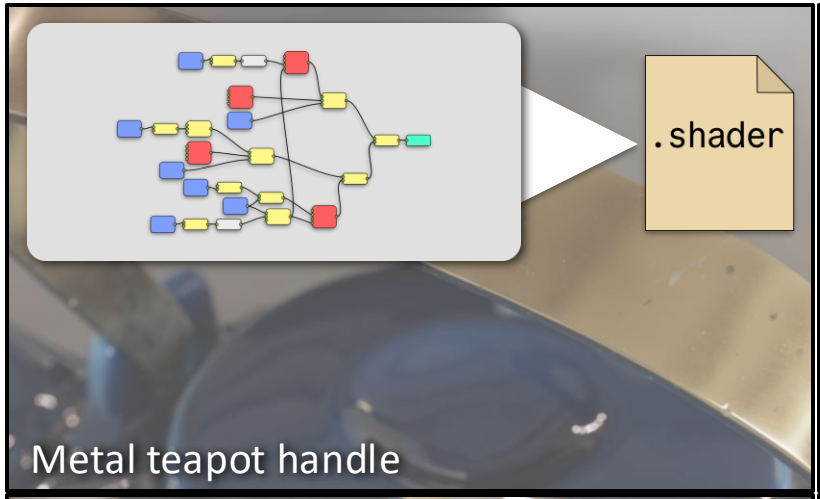


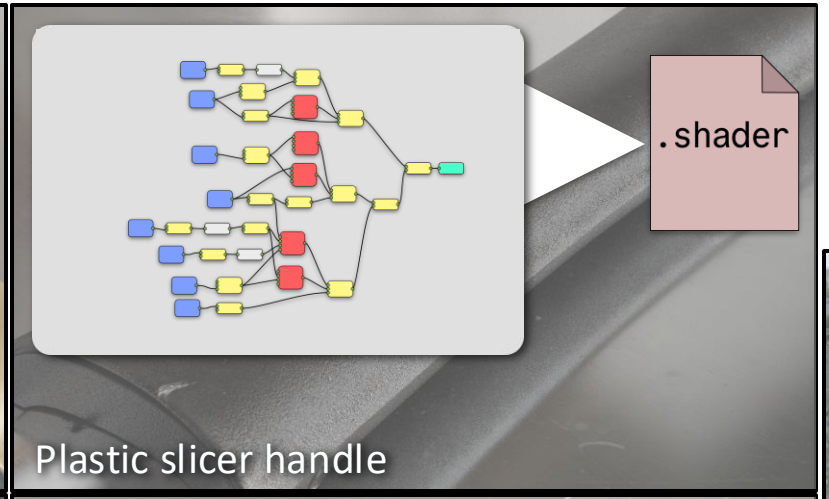


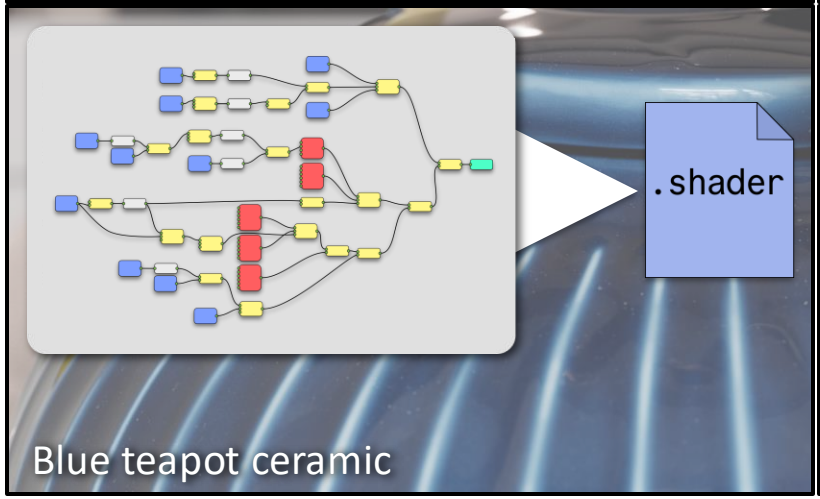


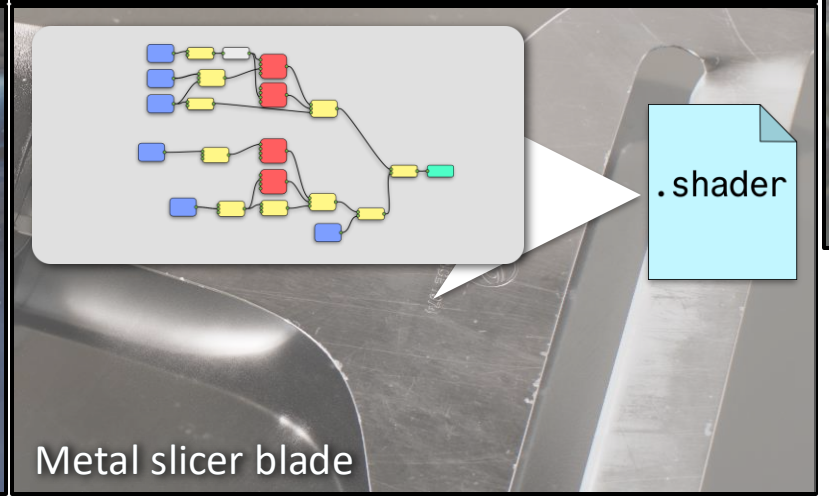


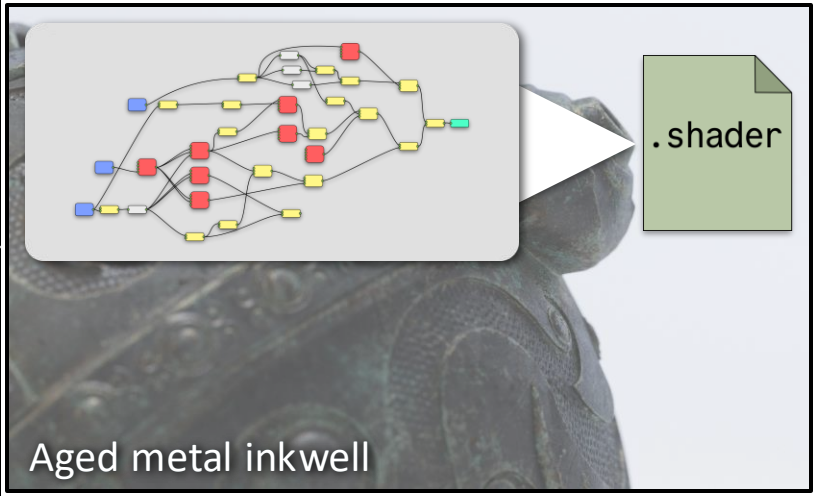
…and we don’t know how to simplify them well



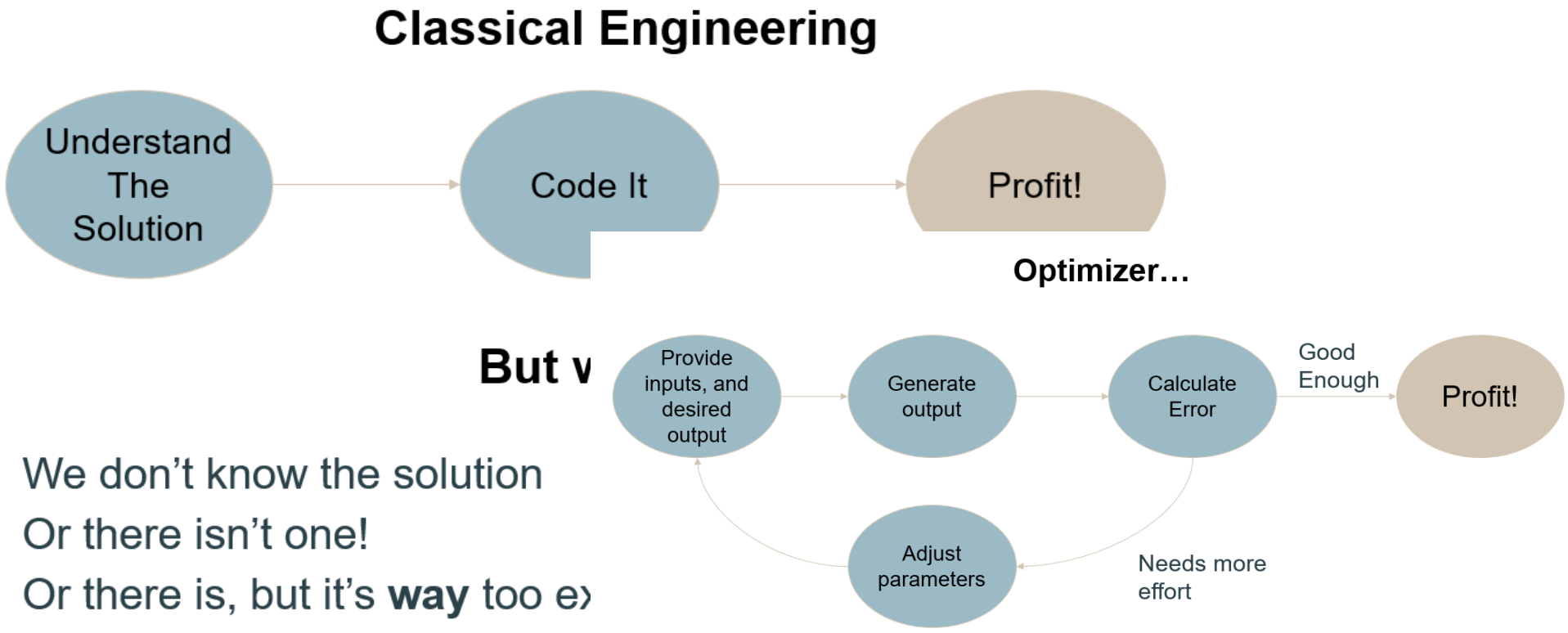








Sounds familiar?



- We don't know the solution
- Or there isn't one!
- Or there is, but it's **way** too ex

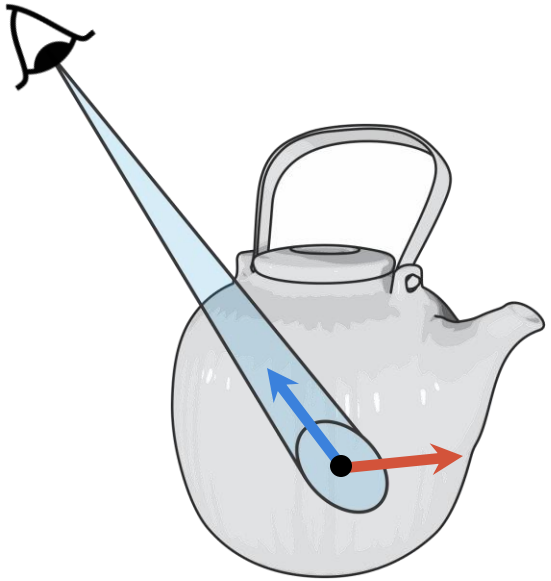
Samples

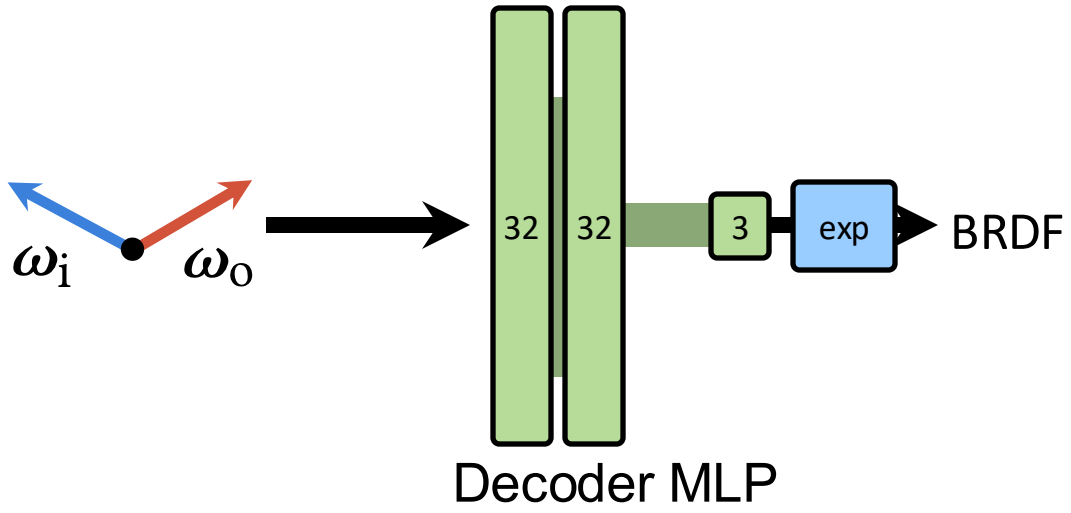


- In the **./neural** folder
- Network is now LinearLayer and provides a CoopVec implementation
- Now using lovely MDL material exported to Slang
- A pretty toy path tracer

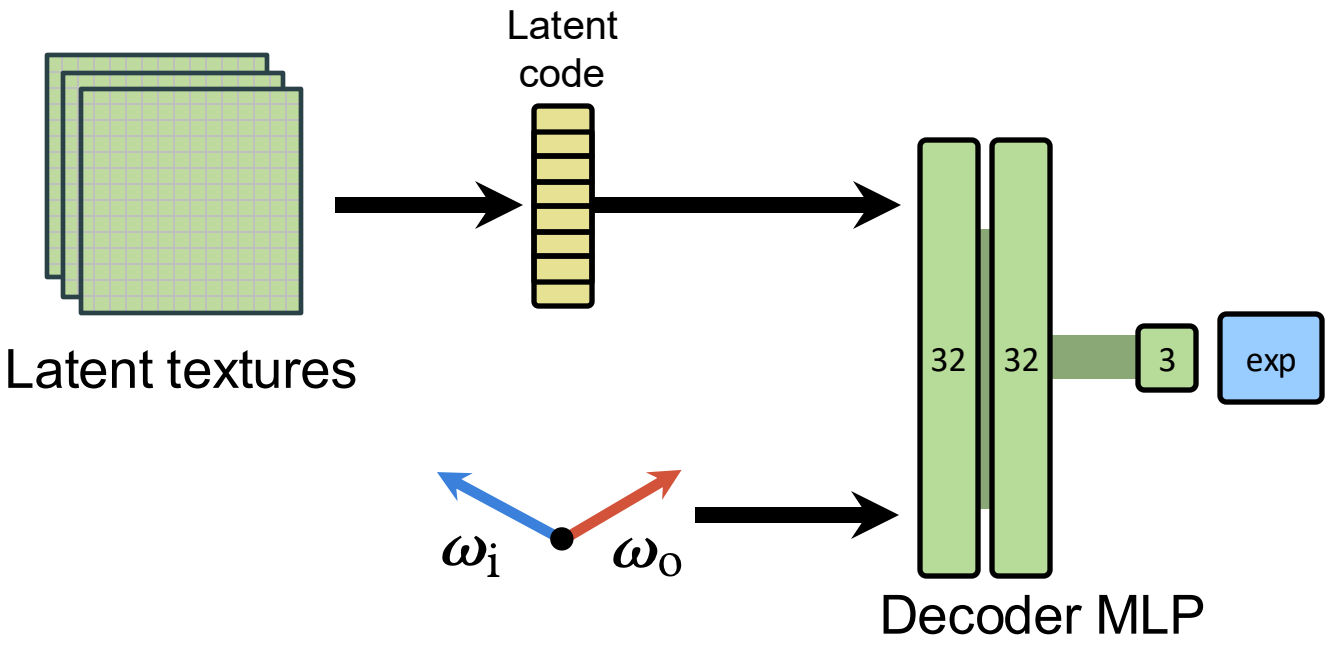


Simplest first: Pass directions to a network

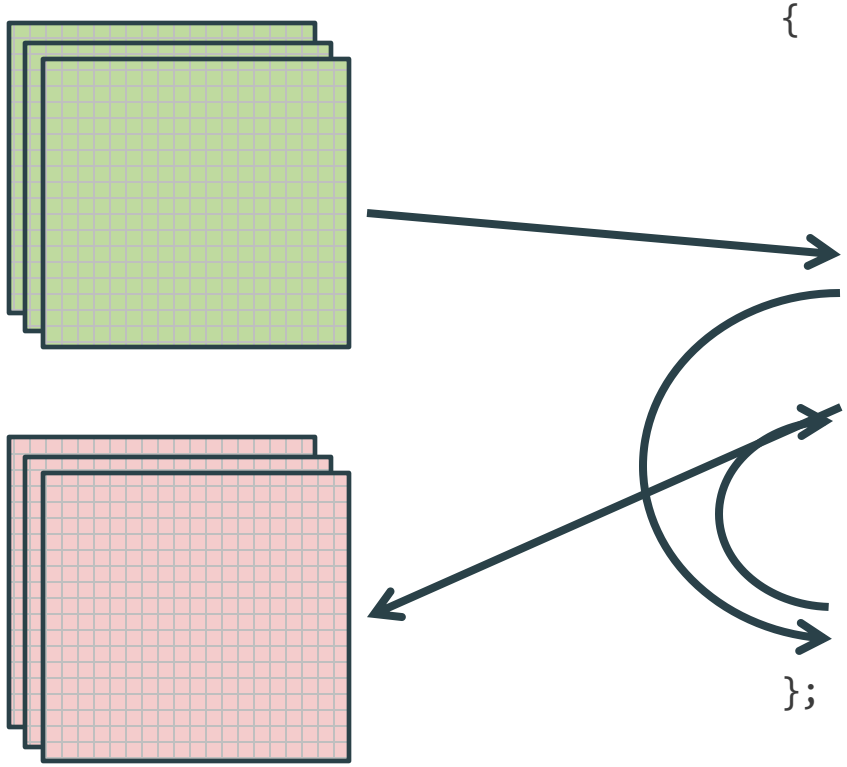




…and then texture it



Latent Texture



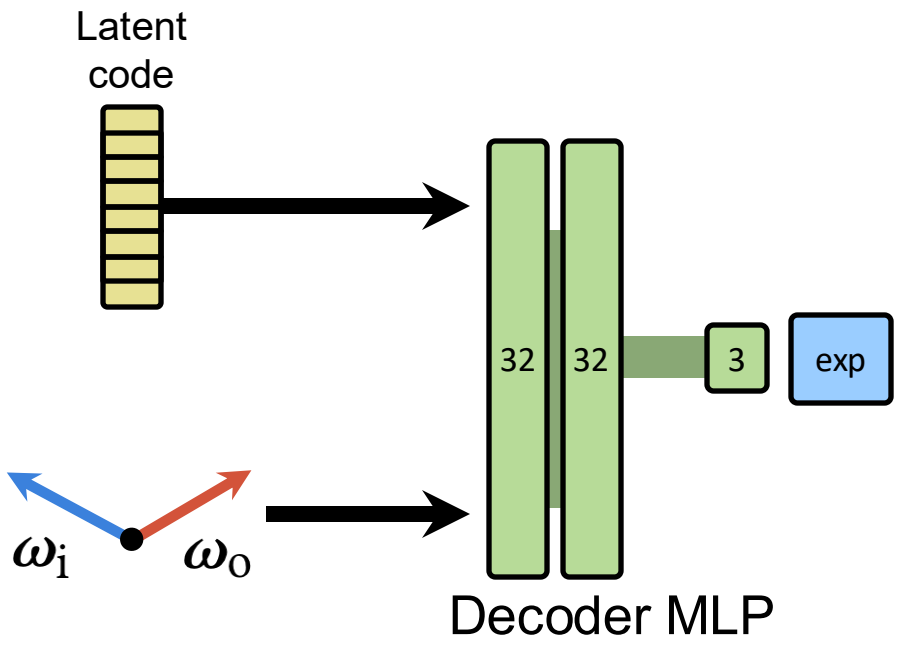
```
public struct LatentTexture<int NumLatents>
{
    RWTensor<float, 3> values;
    AtomicTensor<float, 3> gradients;

    // Read 1 channel from 1 pixel at a given coord
    [BackwardDerivative(get_value_bwd)]
    float get_value(int x, int y, int channel)

    // Atomic accumulate gradient for 1 channel/pixel
    void get_value_bwd(int x, int y, int channel, float grad)

    // Bilinear sample a uv, by calling get_value on 4
    // neighbouring pixels + interpolating.
    [Differentiable]
    float[NumLatents] sample(float2 uv)
};
```

Decoder (the MLP)




```
public struct Decoder {
    LinearLayer<4 + 6, 32> layer0;
    LinearLayer<32, 32> layer1;
    LinearLayer<32, 3> layer2;

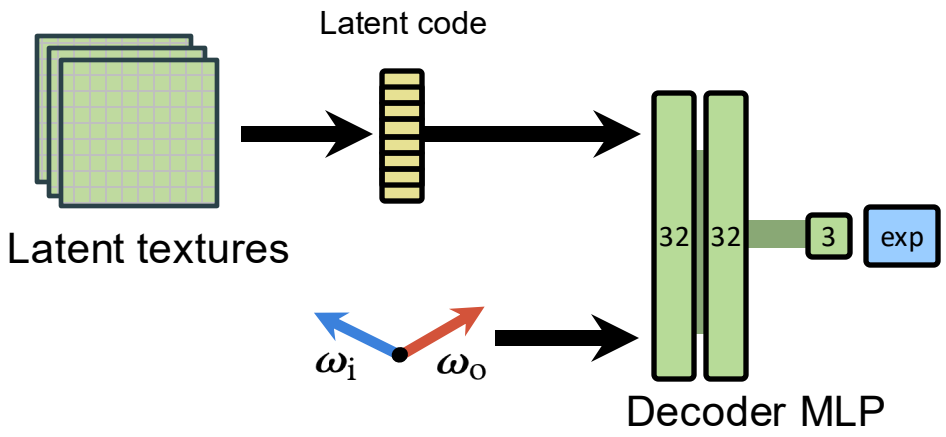
    [Differentiable]
    float3 forward(float[4] latents, float3 wi, float3 wo) {
        float inputs[4+6] = { latents[0], ..., wi.x, ... , wo.z }
        float[32] output0 = layer0.forward(inputs);
        [ForceUnroll]
        for (int i = 0; i < 32; ++i)
            output0[i] = leaky_relu(output0[i]);

        float[32] output1 = layer1.forward(h0);
        [ForceUnroll]
        for (int i = 0; i < 32; ++i)
            output1[i] = leaky_relu(output1[i]);

        float output2[3] = layer2.forward(h1);
        [ForceUnroll]
        for (int i = 0; i < 3; ++i)
            output2[i] = exp(output2[i]);

        return float3(output[0], output[1], output[2]);
    }
};
```

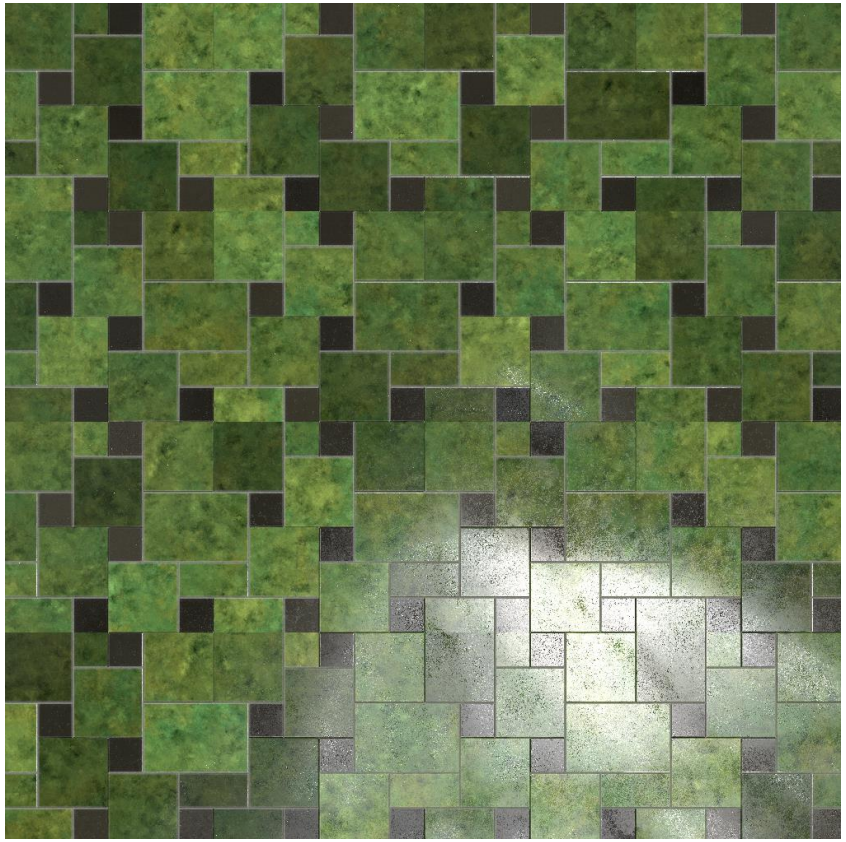
Glue them together




```
public struct Network
{
    LatentTexture<4> latent_texture;
    Decoder decoder;


    [Differentiable]
    float3 eval(float2 uv, float3 wi, float3 wo)
    {
        float[4] latents = latent_texture.sample(uv);
        return decoder.forward(latents, wi, wo);
    }
};
```

Render the reference

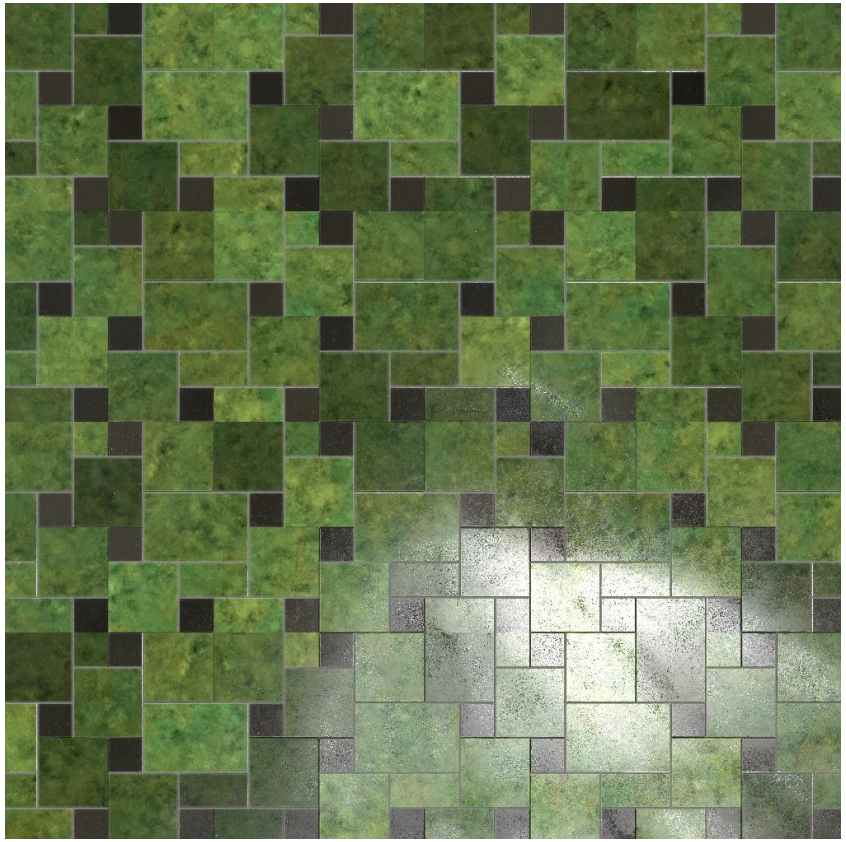


```
float3 target_eval(float2 uv, float3 wi, float3 wo, UniformSampleGenerator sg)
{
    // Fill out information that MDL material needs to evaluate.
    ShadingData sd = make_shading_data(uv, wi);
    sd.position = float3(uv, 0.0f);
    sd.view = wi;
    sd.uv = uv;
    sd.shading_frame = Frame(float3(0.0f, 0.0f, 1.0f), float3(1.0f, 0.0f, 0.0f));
    sd.geometric_normal = sd.shading_frame.normal;
    sd.geometric_tangent = float4(sd.shading_frame.tangent, 1.0f);
    sd.front_facing = true;

    // Construct the MDL material
    let material = MDLCeramicMaterialInstance(sd, true);

    // Read / return the colour
    float3 weights = material.eval(sd, wo, sg);
    return weights;
}
```

Render the reference



```
public float3 render_target(int2 pixel, inout float4 accumulator, int counter)
{
    // Calculate UV and inventing a view direction
    float2 uv = (float2(pixel) + 0.5f) / kImageResolution;
    float3 wi = normalize(float3(0.5f, 0.5f, 1.0f) - float3(uv, 0.0f));

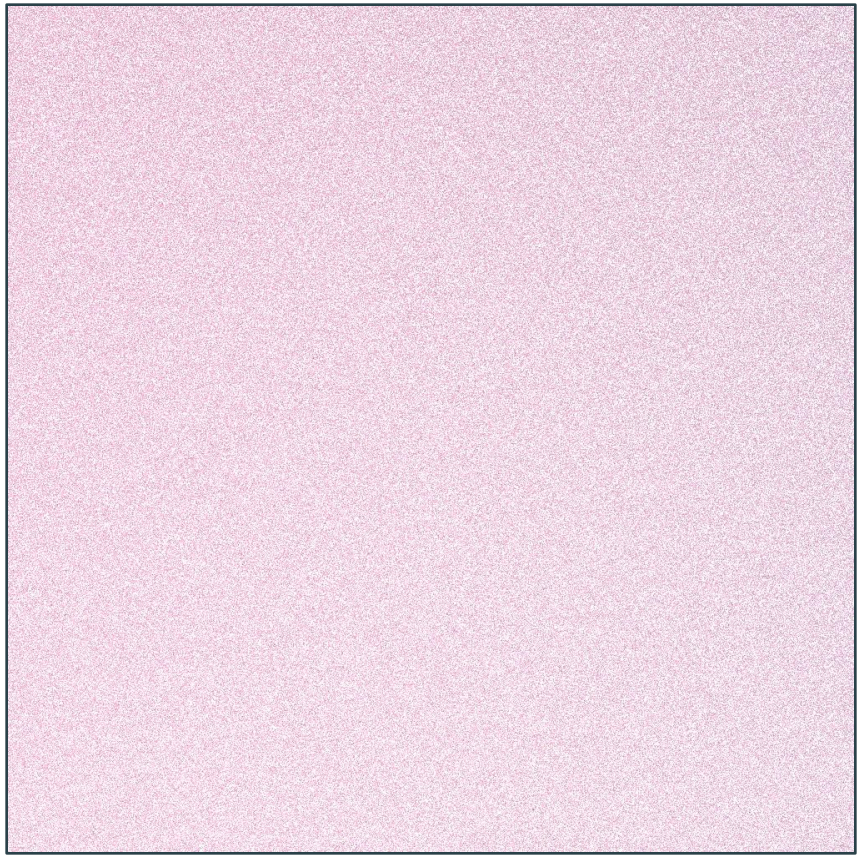
    // Randomly sample a light direction on a hemisphere
    UniformSampleGenerator sg = UniformSampleGenerator(pixel, counter * 128);
    float3 wo = next_dir_on_hemisphere(sg);

    // Sample the material
    float3 weights = target_eval(uv, wi, wo, sg);

    // Apply the lighting from an environment map
    float3 radiance = sample_env_map(wo) * weights / M_INV_TWO_PI;

    // Accumulate and return the result
    accumulator.xyz += radiance;
    accumulator.w += 1.0f;
    return accumulator.xyz / accumulator.w;
}
```

Render the reference



```
public float3 render_target(int2 pixel, inout float4 accumulator, int counter)
{
    // Calculate UV and inventing a view direction
    float2 uv = (float2(pixel) + 0.5f) / kImageResolution;
    float3 wi = normalize(float3(0.5f, 0.5f, 1.0f) - float3(uv, 0.0f));

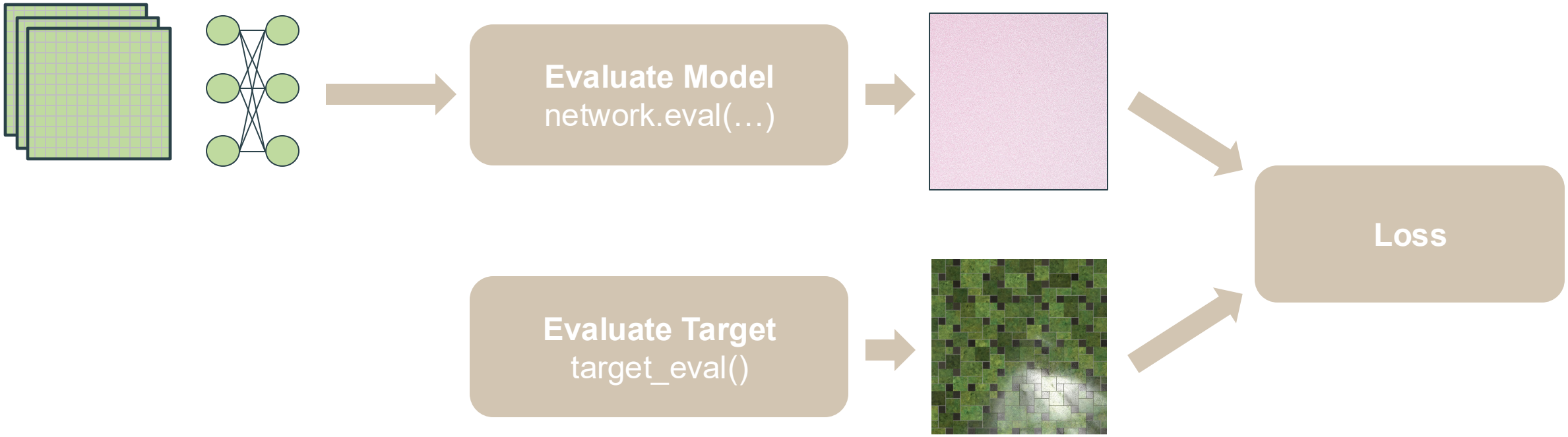
    // Randomly sample a light direction on a hemisphere
    UniformSampleGenerator sg = UniformSampleGenerator(pixel, counter * 128);
    float3 wo = next_dir_on_hemisphere(sg);

    // Sample the material
    float3 weights = target_eval(uv, wi, wo, sg);
    float3 weights = network.eval(uv, wi, wo);

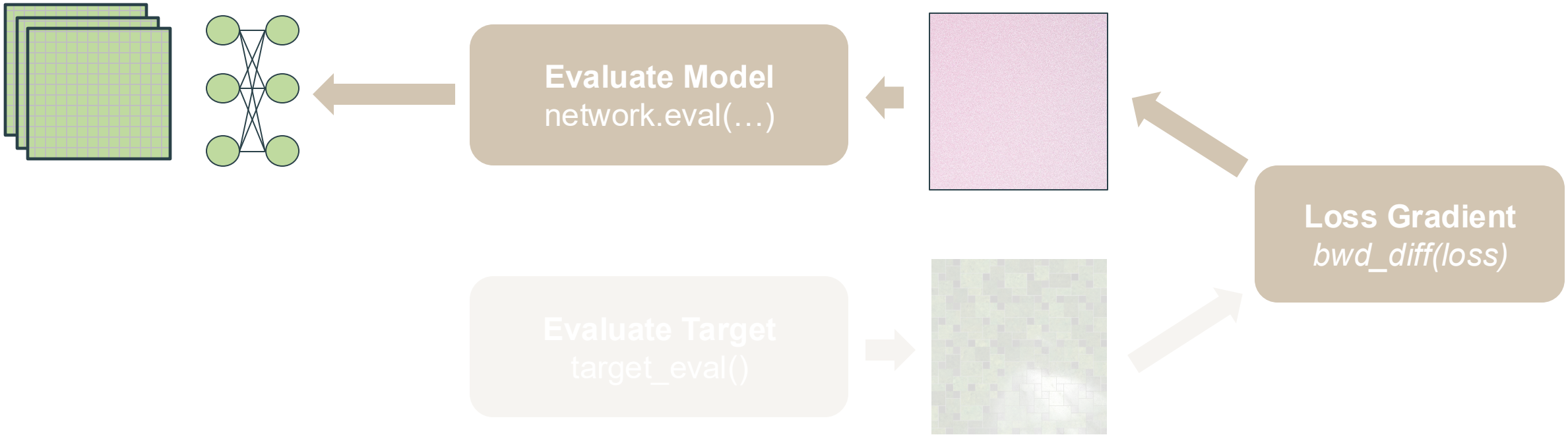
    // Apply the lighting from an environment map
    float3 radiance = sample_env_map(wo) * weights / M_INV_TWO_PI;

    // Accumulate and return the result
    accumulator.xyz += radiance;
    accumulator.w += 1.0f;
    return accumulator.xyz / accumulator.w;
}
```

Training!


Evaluate Model
network.eval(…)
Evaluate Target
target_eval()
Loss

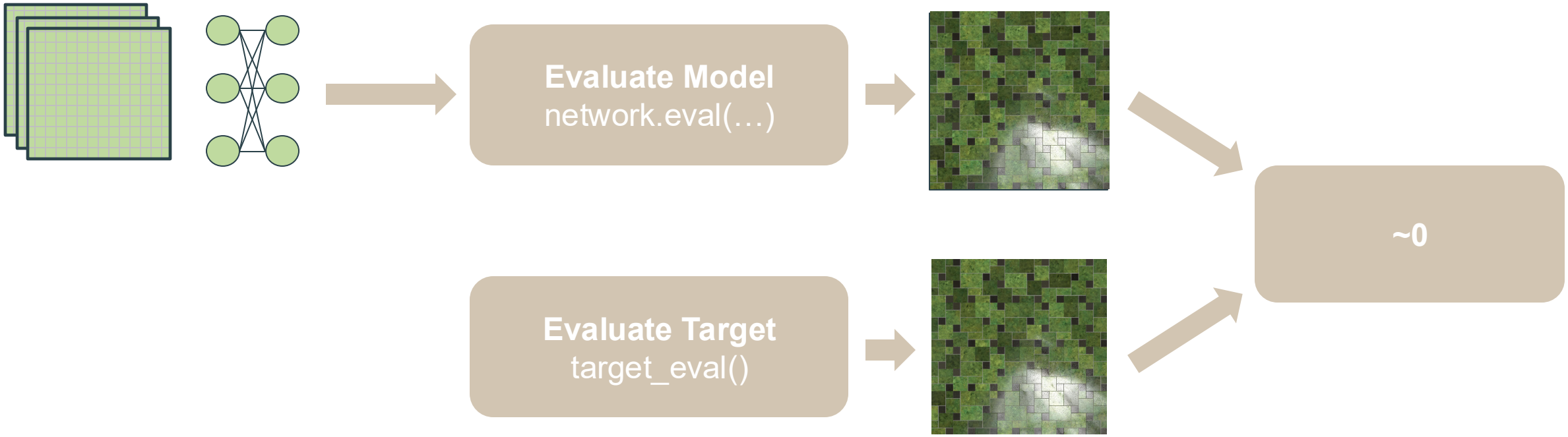
Training!


Evaluate Model
network.eval(…)
Evaluate Target
target_eval()
Loss Gradient
bwd_diff(loss)

Training!



Hopefully / Eventually…



Training!



```
public void calculate_grads(
    uint seed, int2 batch_index, int2 batch_size)
{
    let sg = UniformSampleGenerator(batch_index, seed);
    float2 uv = (float2(batch_index) + next_float2(sg))
              / float2(batch_size);

    float3 wi;
    float3 wo;
    sample_uniform_directions(sg, wi, wo));

    float3 reference = target_eval(uv, wi, wo, sg);

    bwd_diff(training_loss)(uv, wi, wo, reference, 1.0f);
}
```

```
[Differentiable]
float training_loss(
    float2 uv, float3 wi, float3 wo, float3 reference)
{
    float3 prediction = network.eval(uv, wi, wo);

    float loss = 0.0f;
    for (int i = 0; i < 3; ++i)
        loss += abs(power_log(prediction[i]) -
                    power_log(reference[i]));

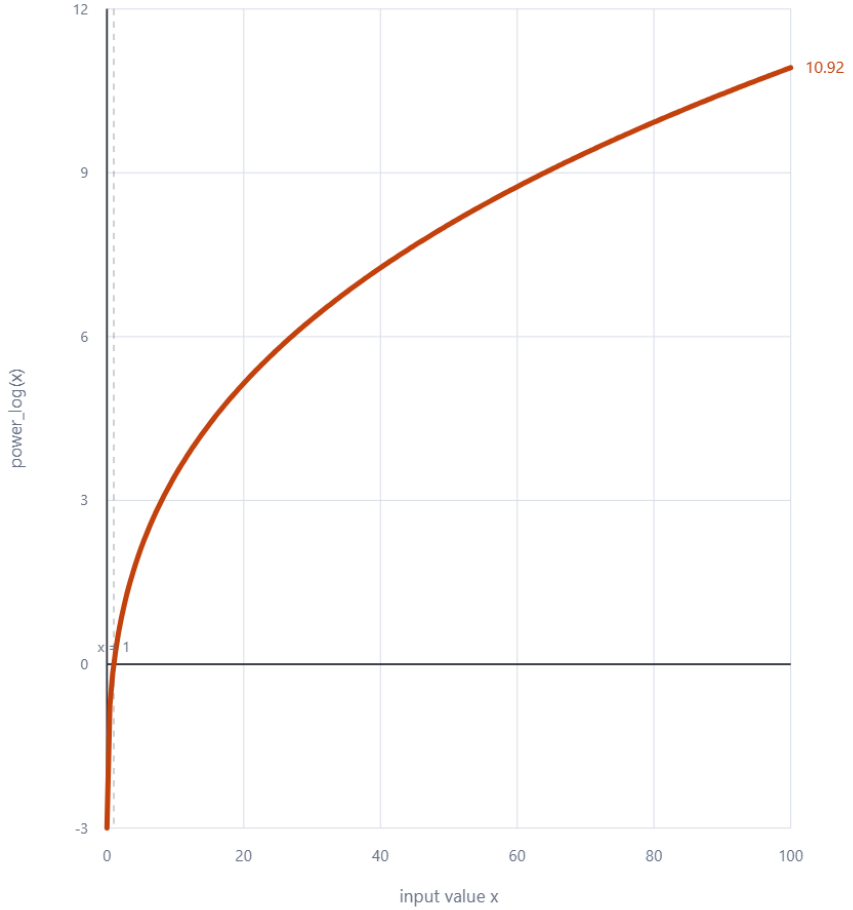
    return loss / 3.0f;
}
```

Why The New Loss?



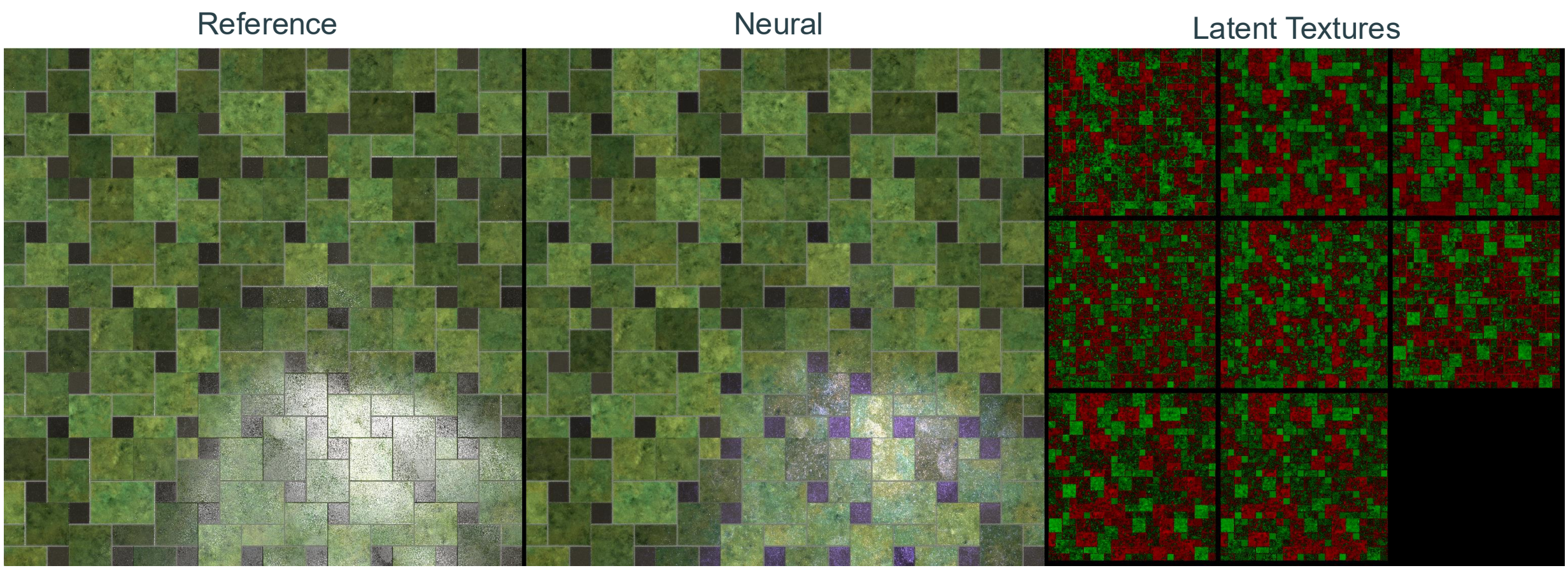
- BSDF does not output a color, and is not bounded to [0, 1]
- Eval describes the reflected radiance **per unit area** of the hemisphere for a given pair of directions
- Mirror material: concentrates almost all energy into 1 direction
- Consequence: BSDF outputs can get **very** high
- Solution: Apply power transform to reference and predictions

```
float power_log(float x)
{
    x = max(x, 0.0f);
    return 3.0f * (pow(x, 1.0f / 3.0f) - 1.0f);
}
```



© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Result



Result

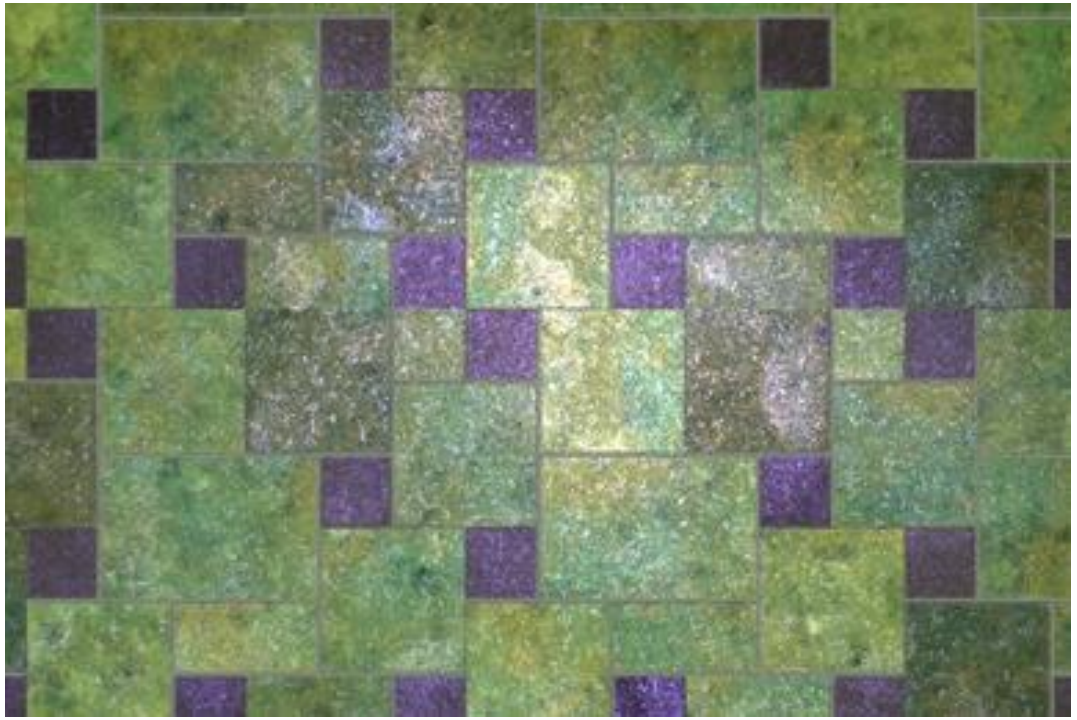


Result

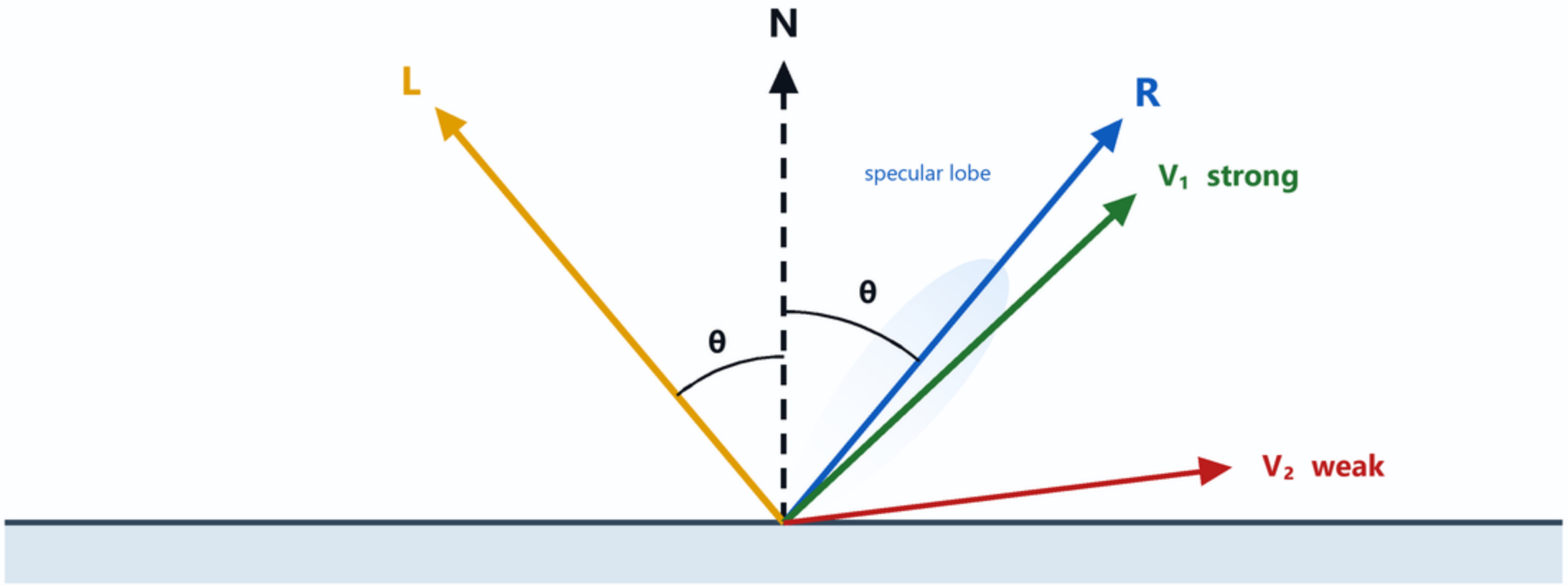


Where did we go wrong?

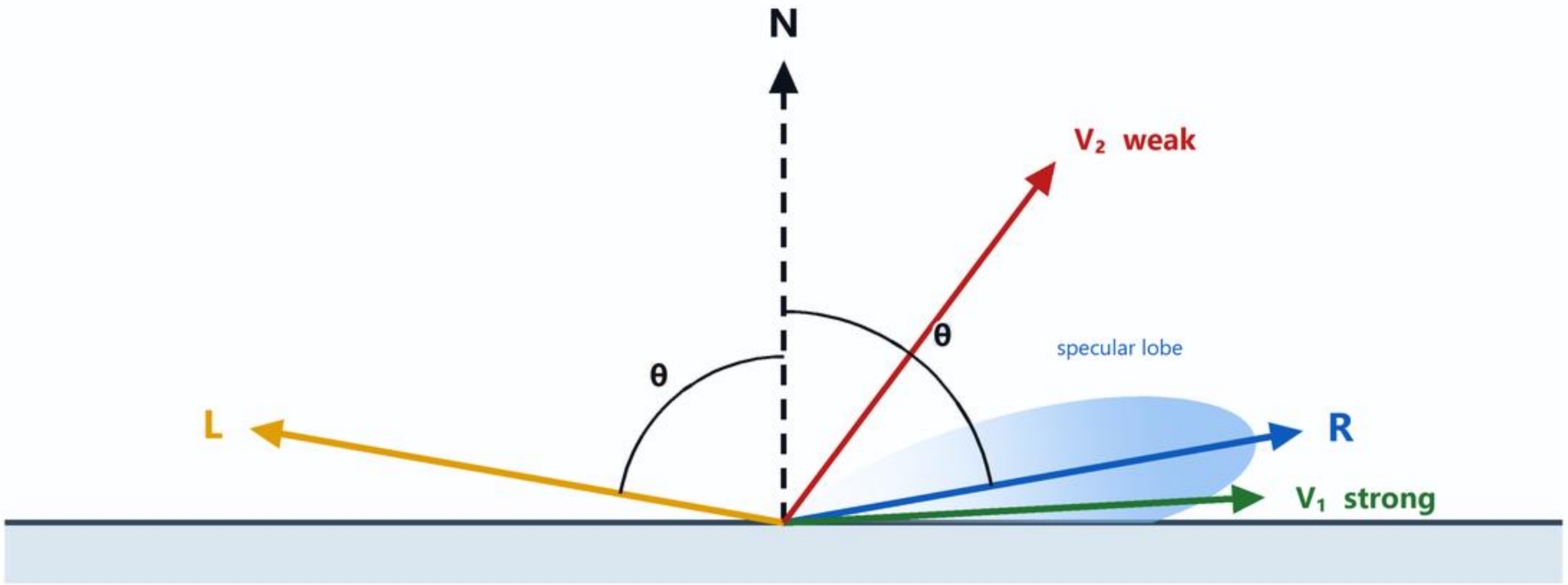




Where did we go wrong?



Where did we go wrong?



Where did we go wrong?



```
public void calculate_grads(
    uint seed, int2 batch_index, int2 batch_size)
{
    let sg = UniformSampleGenerator(batch_index, seed);
    float2 uv = (float2(batch_index) + next_float2(sg))
              / float2(batch_size);

    float3 wi;
    float3 wo;
    sample_uniform_directions(sg, wi, wo));

    float3 reference = target_eval(uv, wi, wo, sg);

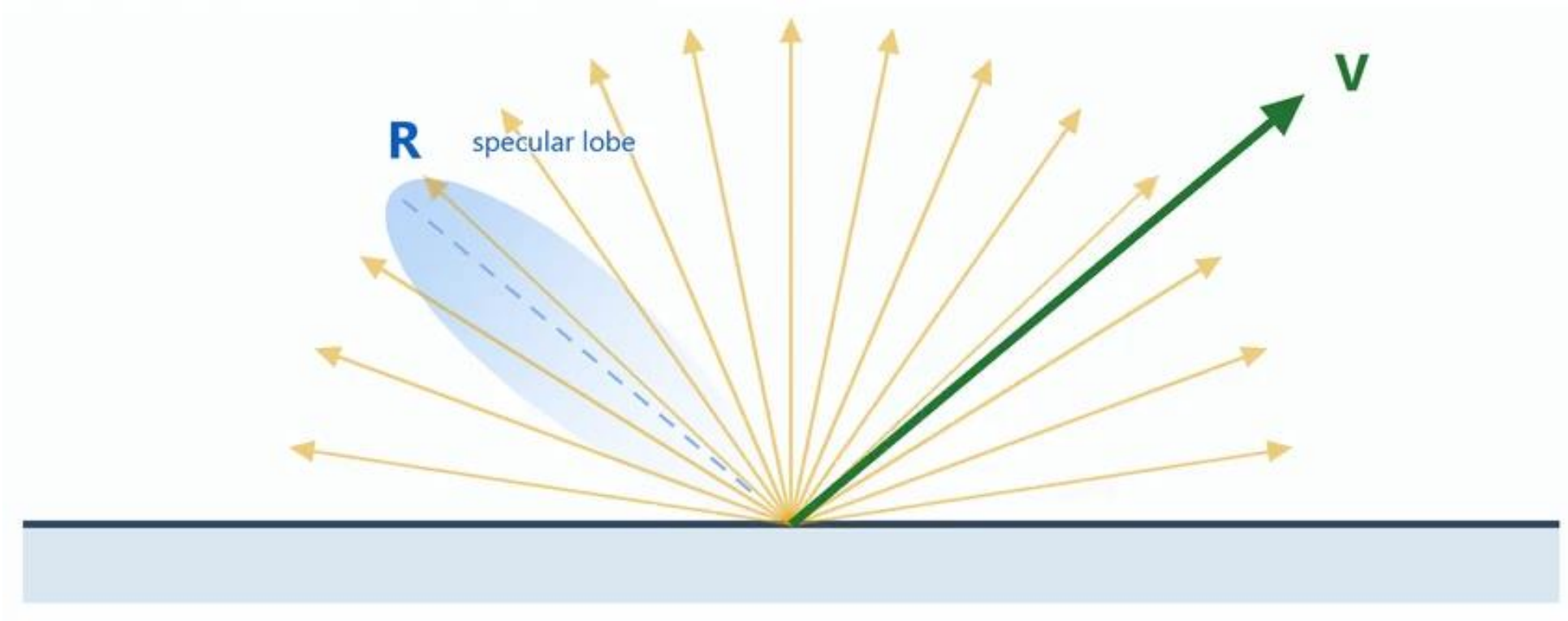
    bwd_diff(training_loss)(uv, wi, wo, reference, 1.0f);
}
```

“Pay equal attention to all view / light direction pairs”

Better sampling

Uniform sampling: almost all samples are 99% diffuse!

Sampling proportional to the specular lobe



Randomize Microfacet Normals



```
bool sample_microfacet_pair(inout UniformSampleGenerator sg, out float3 wi, out
float3 wo)
{
    // Uniform sample wi
    wi = next_dir_on_hemisphere(sg);

    // Choose random normal, using pow to bias towards the z axis
    float2 u = next_float2(sg);
    float phi = 2.0f * M_PI * u.x;
    float cos_theta = pow(u.y, 1.0f / (kSpecularGuidedExponent + 1.0f));
    float sin_theta = safe_sqrt(1.0f - cos_theta * cos_theta);

    // Convert to a cartesian vector
    float sin_phi, cos_phi;
    sincos(phi, sin_phi, cos_phi);
    float3 m = float3(sin_theta * cos_phi, sin_theta * sin_phi, cos_theta);

    // Reflect wi by normal to get wo
    wo = reflect(-wi, m);
    return min(wi.z, wo.z) > kMinCos;
}
```

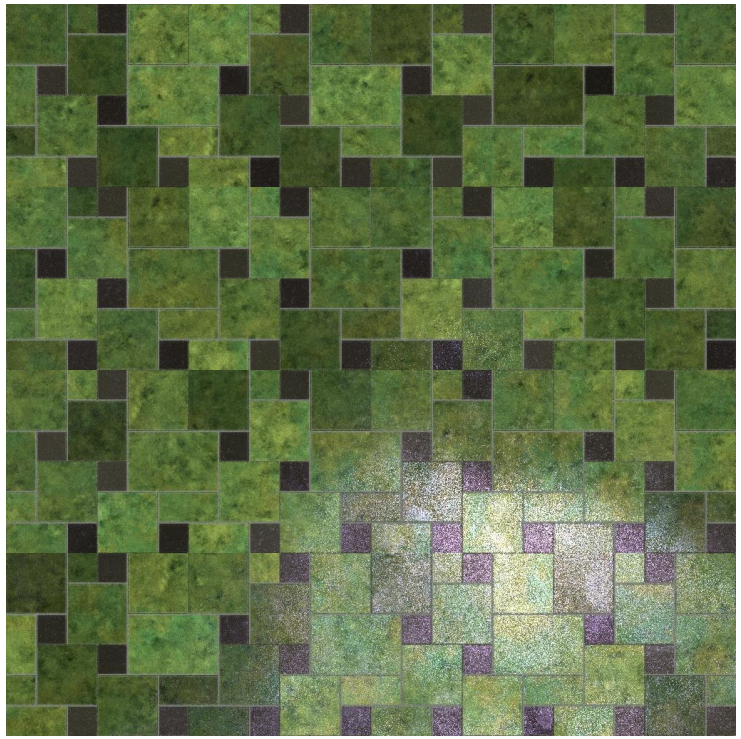
Randomize Microfacet Normals



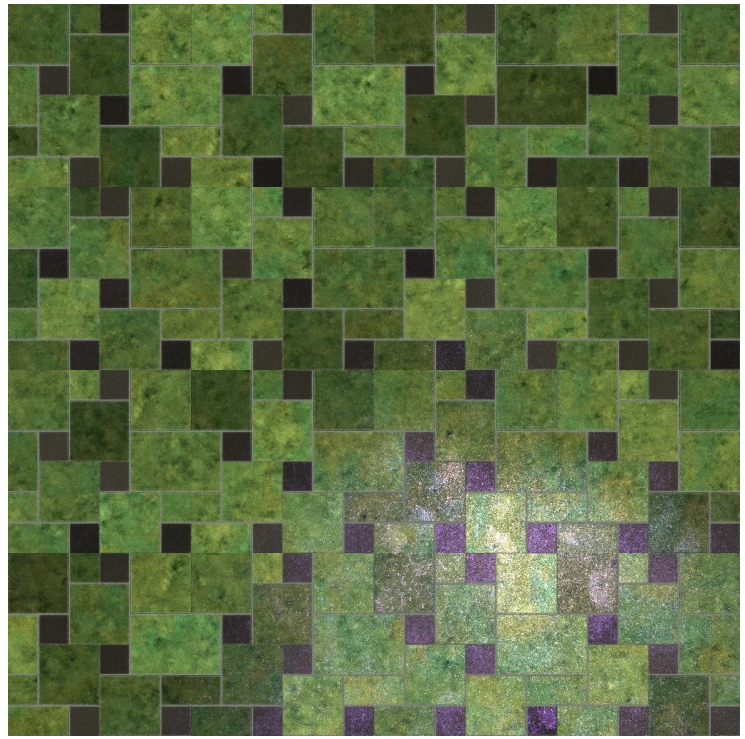
Reference



Microfacet (k=8)



Uniform



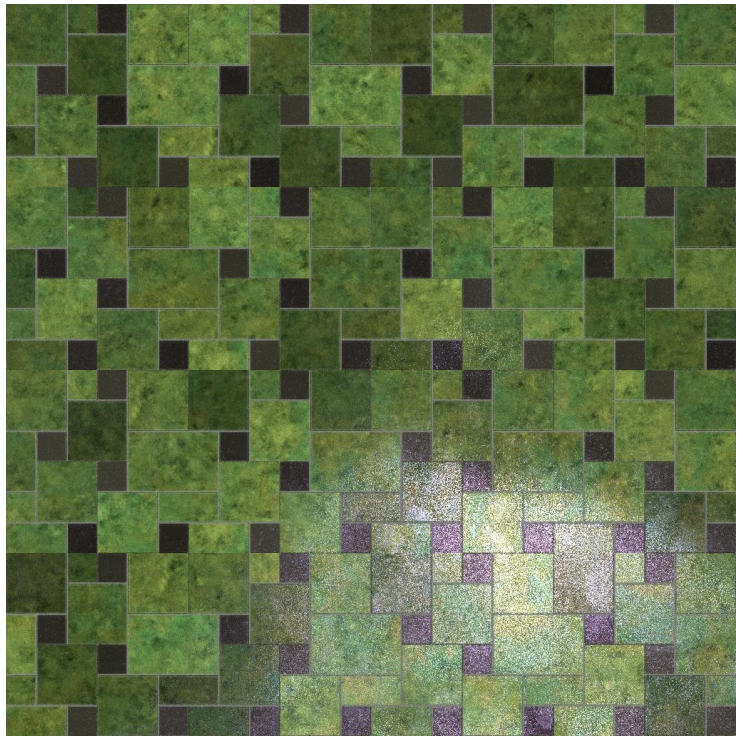
Randomize Microfacet Normals



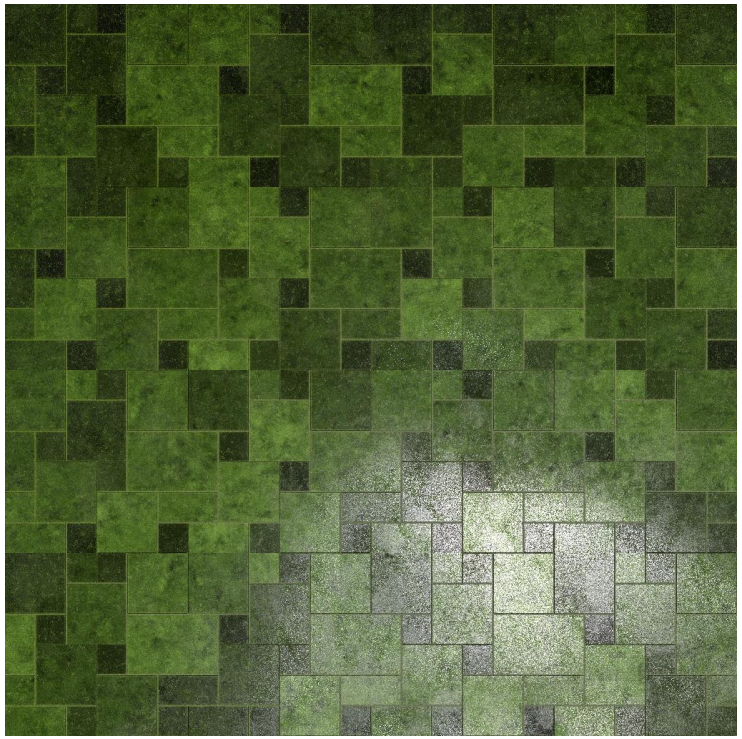
Reference



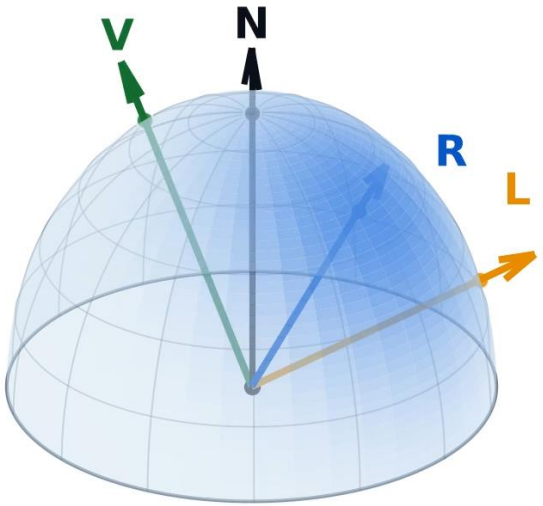
Microfacet (k=8)



Microfacet (k=128)

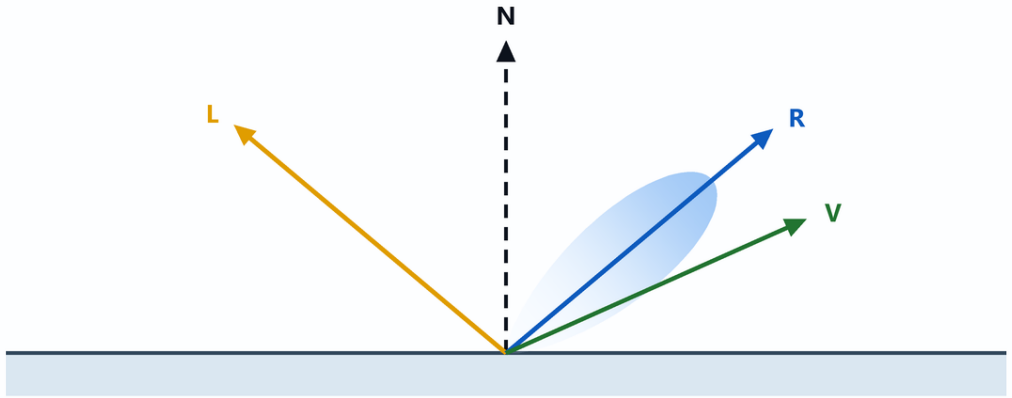


V and L are complicated



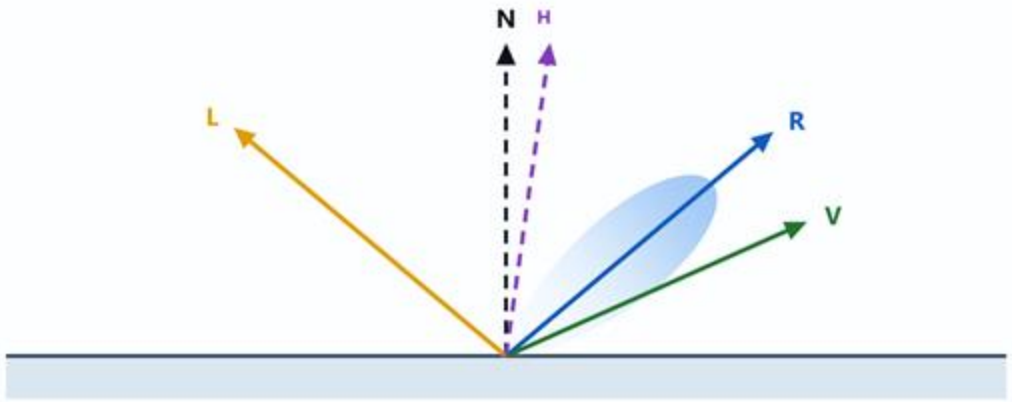
- Specular response stronger when R is ‘in the middle’ of V and L
- And changes at grazing angles due to Fresnel

V and L are complicated



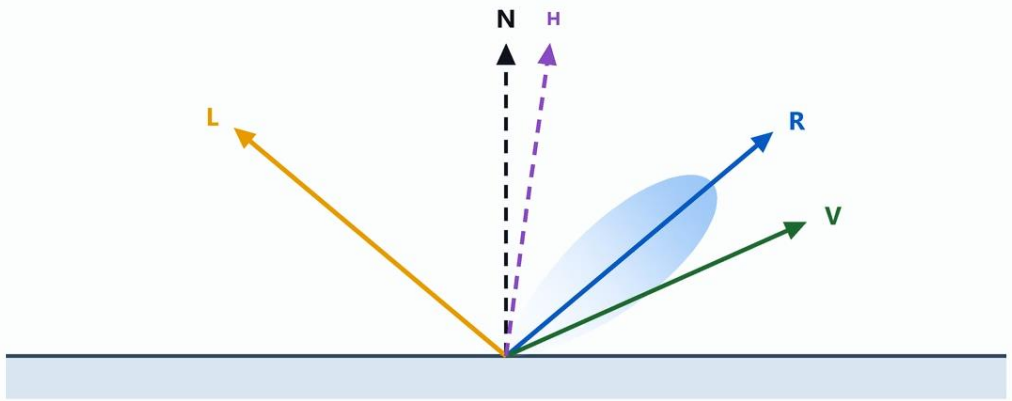
- Specular response stronger when R aligns with V
- And changes at grazing angles due to Fresnel

Half Vector



- Specular response stronger when R aligns with V
- And changes at grazing angles due to Fresnel
- Most shaders involve the half vector
- Lobe shape depends on normalize(V+L)
- Fresnel depends on V . H

Reparameterize



- Now have H = normalize(V+L)
- D = V rotated into frame of H
- D.z = V·H

  = V·normalize(V+L)

- ~~Lobe shape depends on normalize(V+L)~~
- Lobe shape depends on H
- ~~Fresnel depends on (V·normalize(V+L))~~
- Fresnel depends on D.z

Reparameterize



```
[Differentiable]
void rusinkiewicz_parameterize(float3 wi, float3 wo, out float3 wh, out float3 wd)
{
    // Calculate half vector
    wh = normalize(wi + wo);

    // Optimized 'rotate view into half vector coordinate system'
    float3 v = float3(wh.y, -wh.x, 0.0f);
    wd = wi * wh.z + cross(v, wi) + dot(v, wi) * v / (1.0f + wh.z);
}
```

Reparameterize



```
[Differentiable]
float3 eval(no_diff float2 uv, no_diff float3 wi, no_diff float3 wo)
{
    float[kNumLatents] latents = latent_texture.sample(uv);
    return decoder.forward(latents, wi, wo);
}
```

Reparameterize



```
[Differentiable]
float3 eval(no_diff float2 uv, no_diff float3 wi, no_diff float3 wo)
{
    float[kNumLatents] latents = latent_texture.sample(uv);

    float3 wh;
    float3 wd;
    rusinkiewicz_parameterize(wi, wo, wh, wd);

    return decoder.forward(latents, wh, wd);
}
```

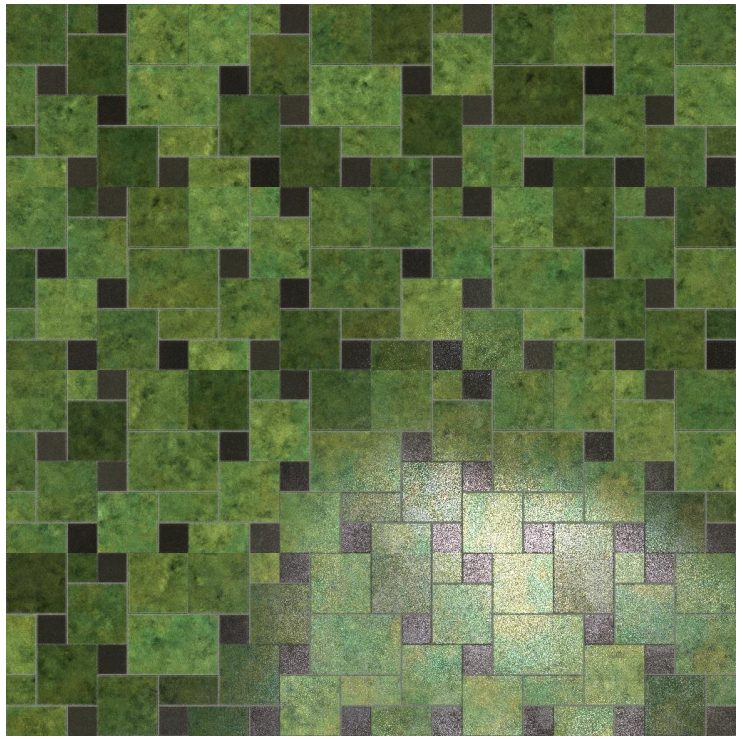
Result



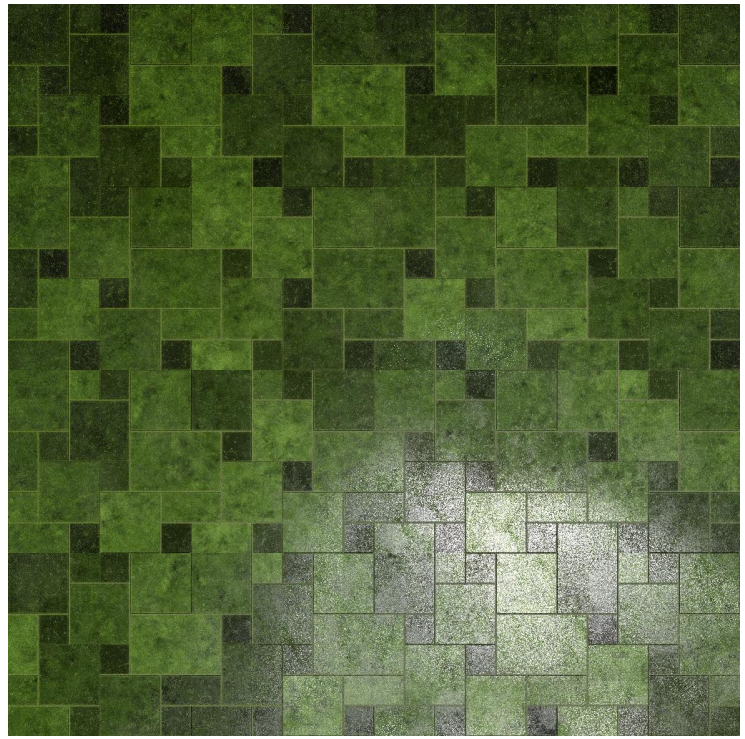
Reference



Rusinkiewicz
sampling + parameterization



Microfacet sampling



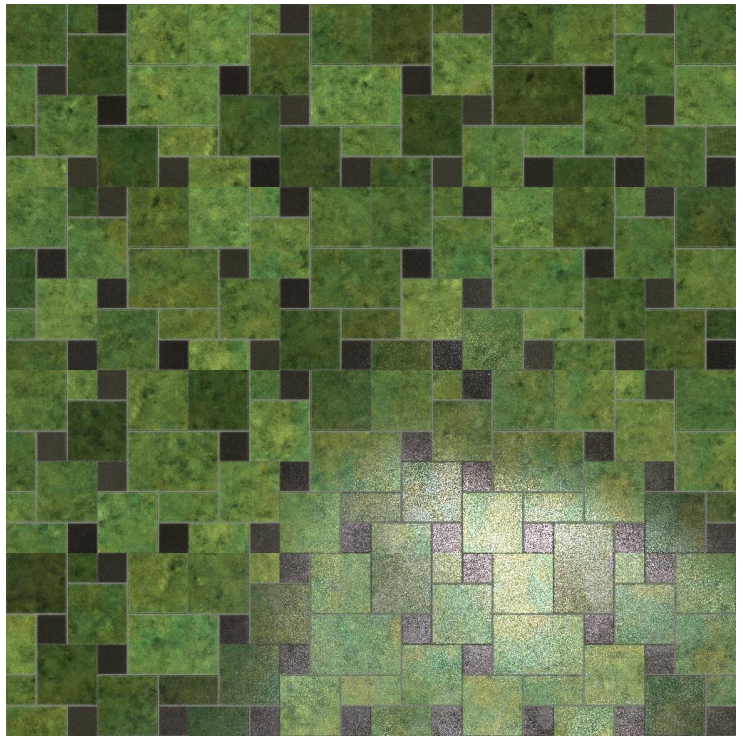
Result



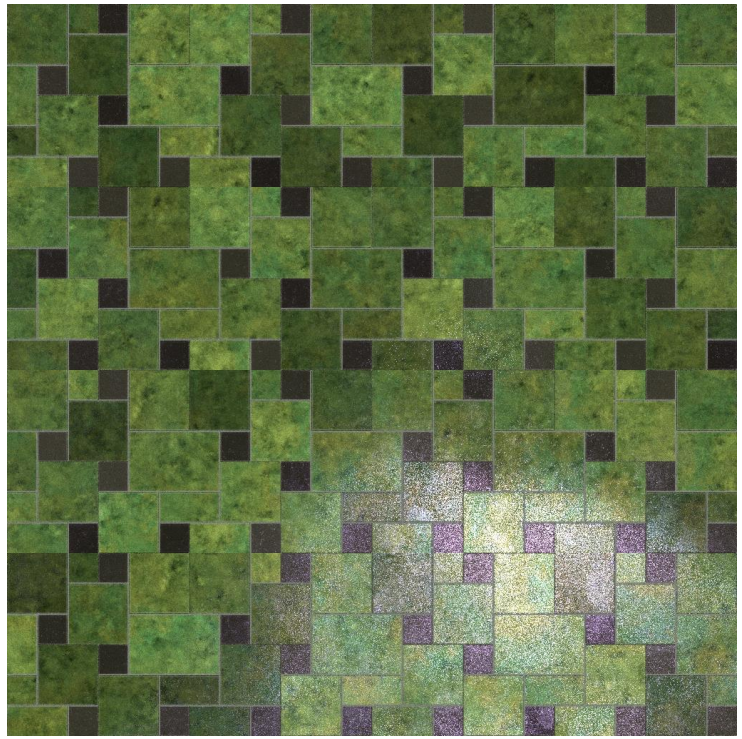
Reference



Rusinkiewicz
sampling + parameterization



Microfacet sampling



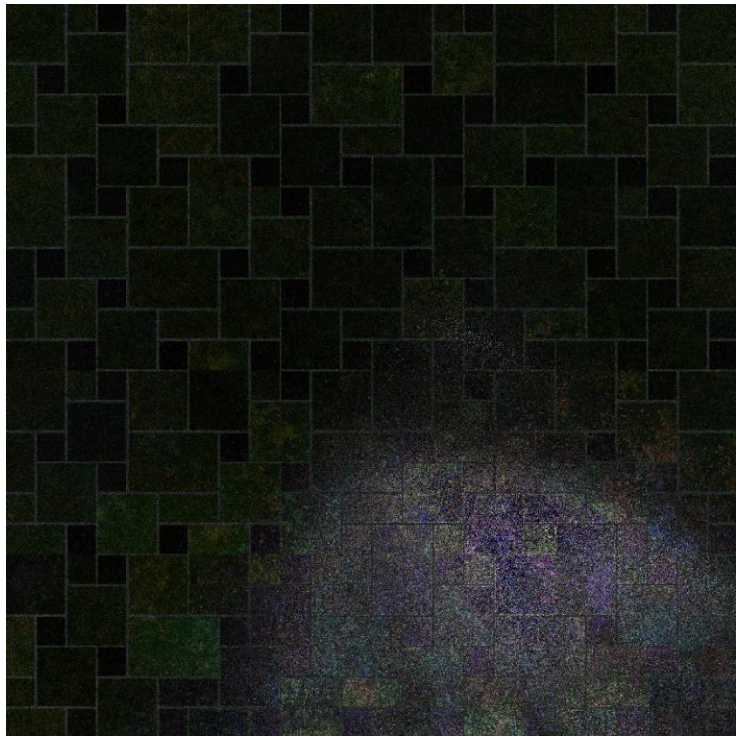
Result (dark == good)



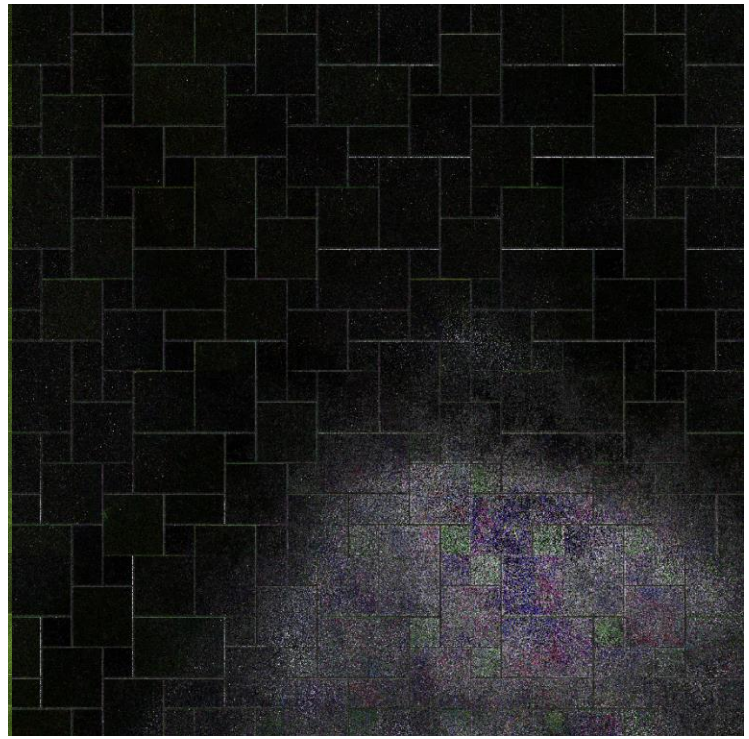
Reference



Rusinkiewicz
sampling + parameterization

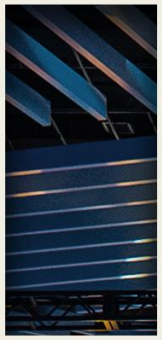


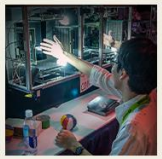
Microfacet sampling



Result





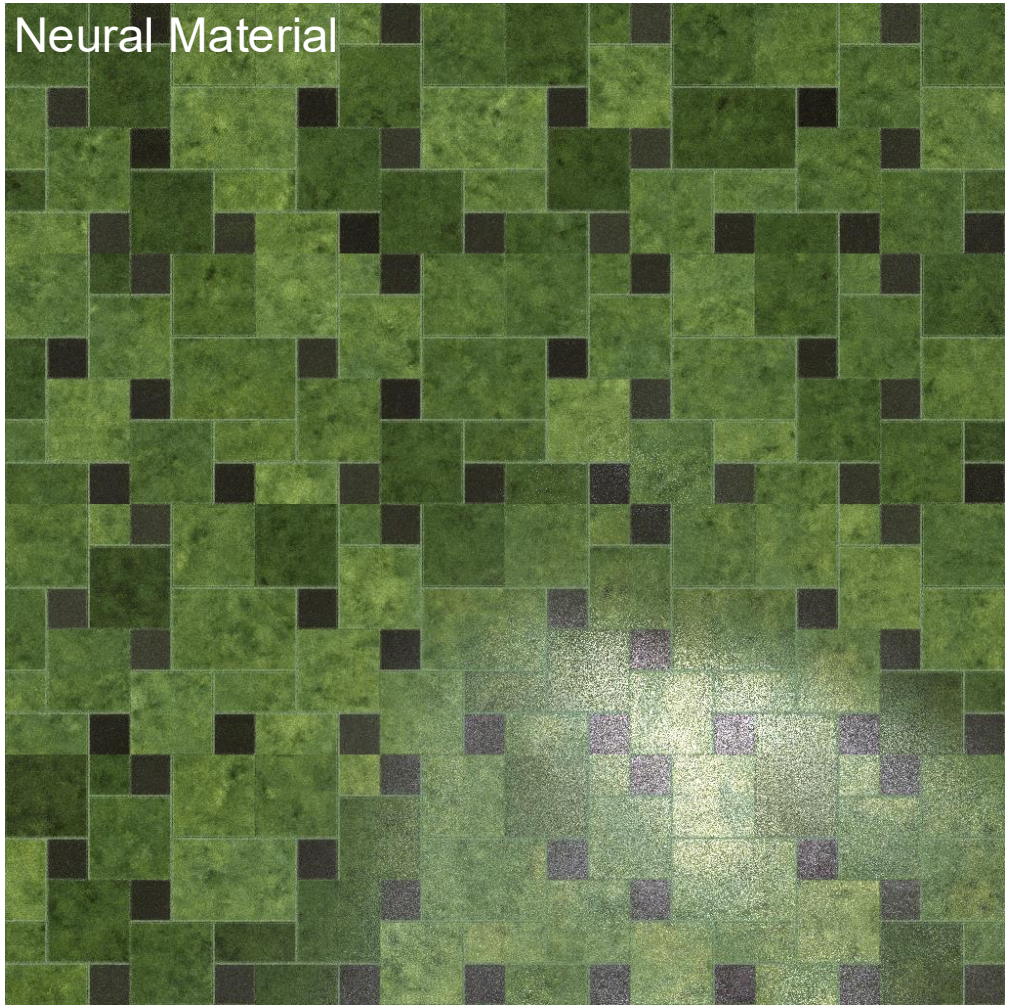




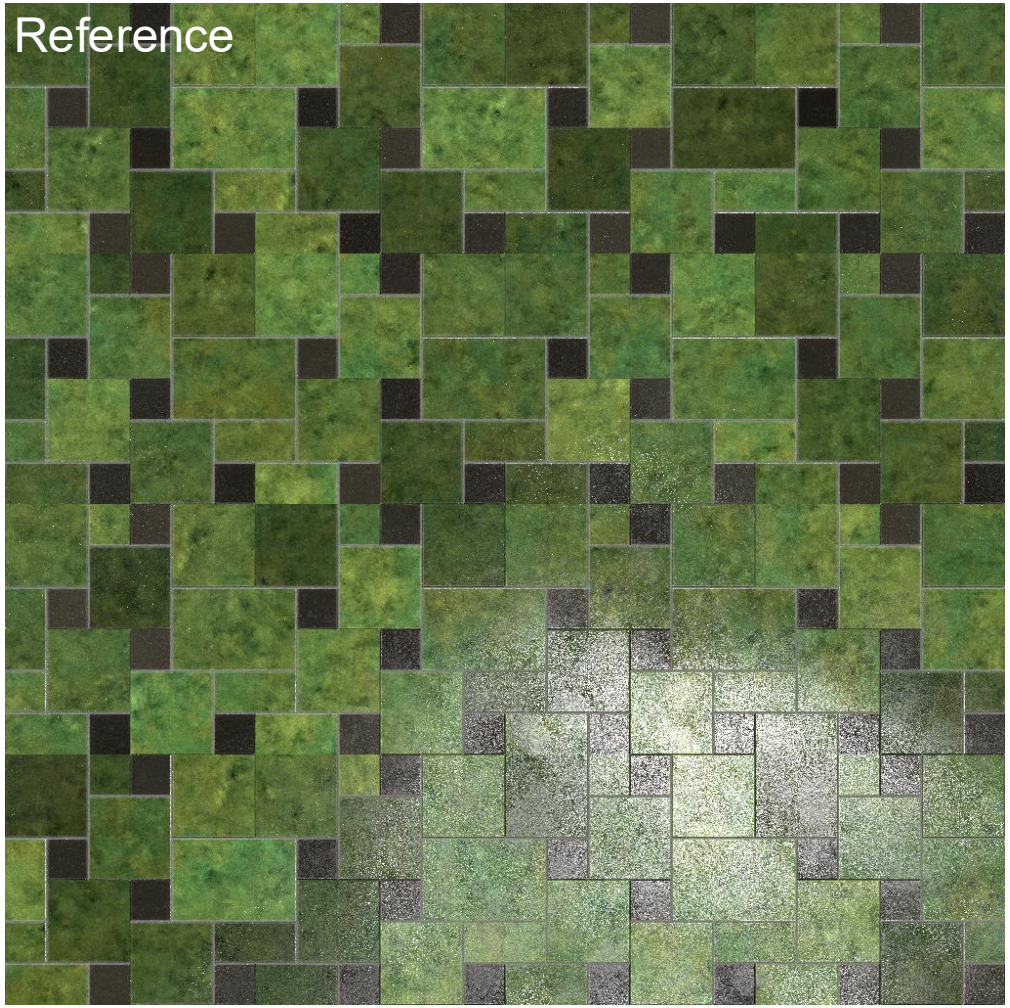
SIGGRAPH 2026
Los Angeles 19–23 JUL

Improving Neural Materials

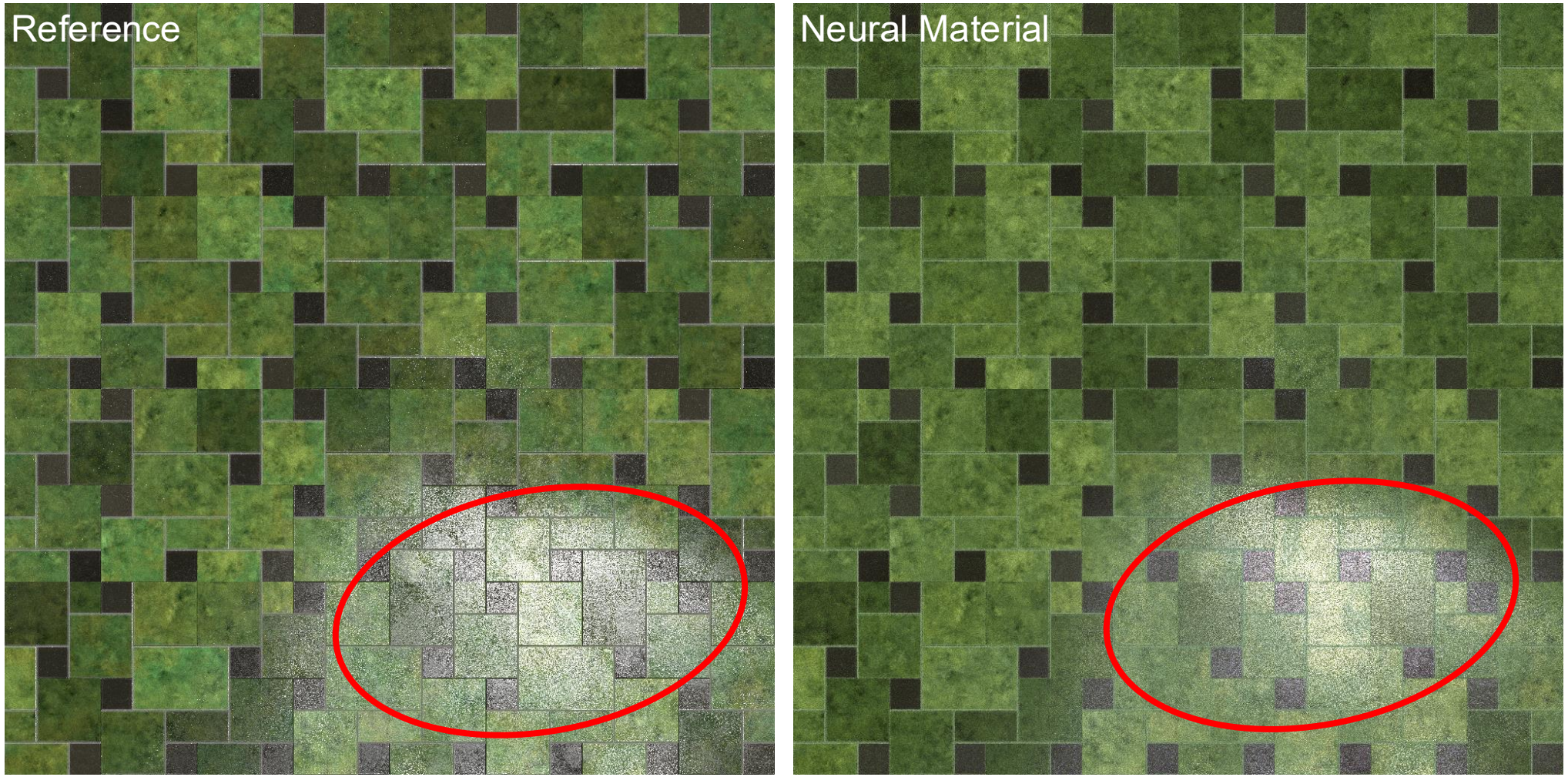
Reviewing the current state



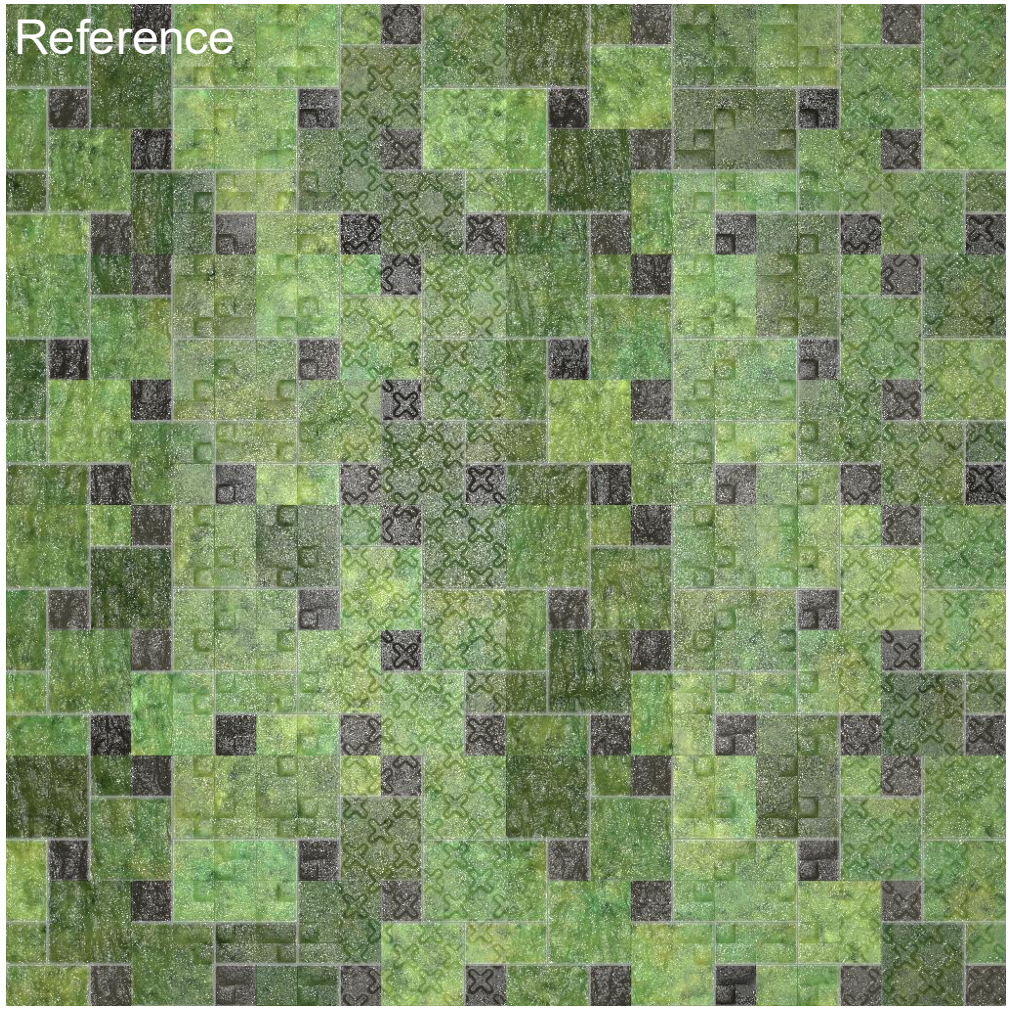
Reviewing the current state

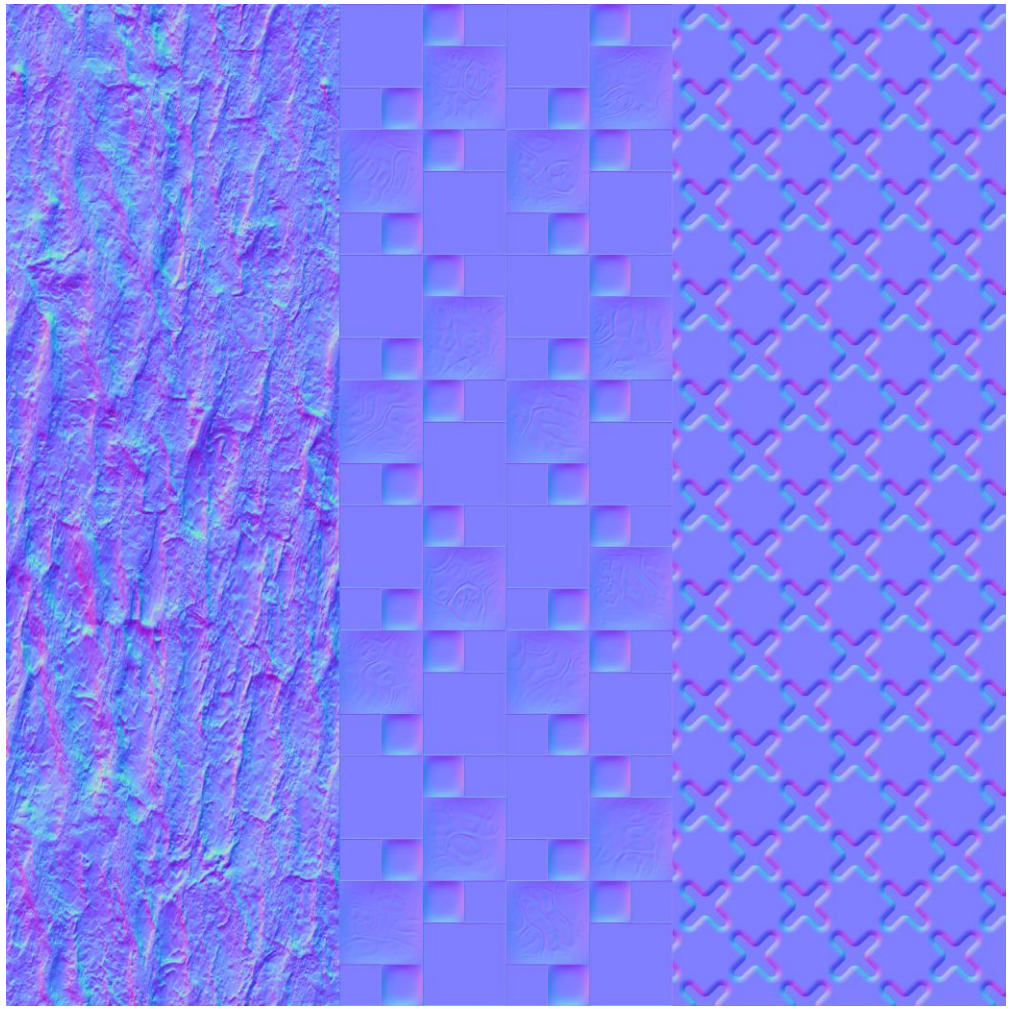


What's going on here?

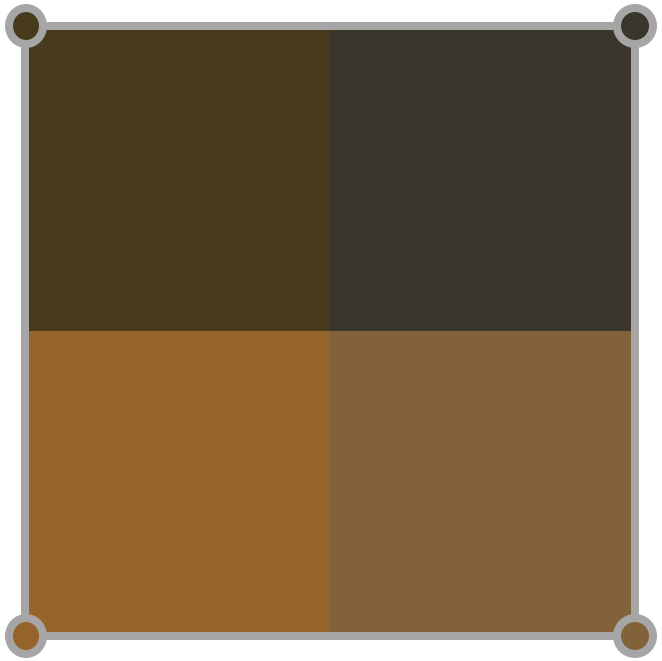


What's going on here?

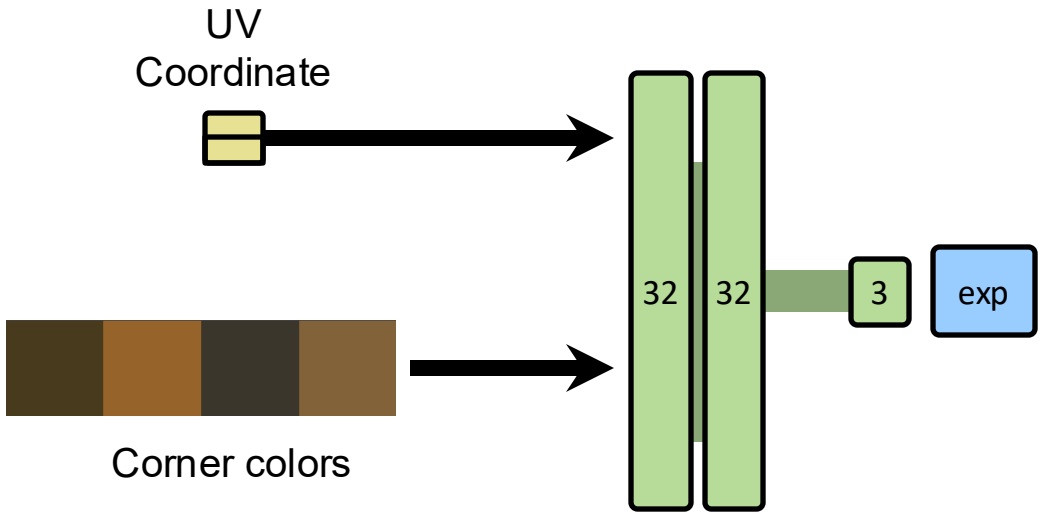


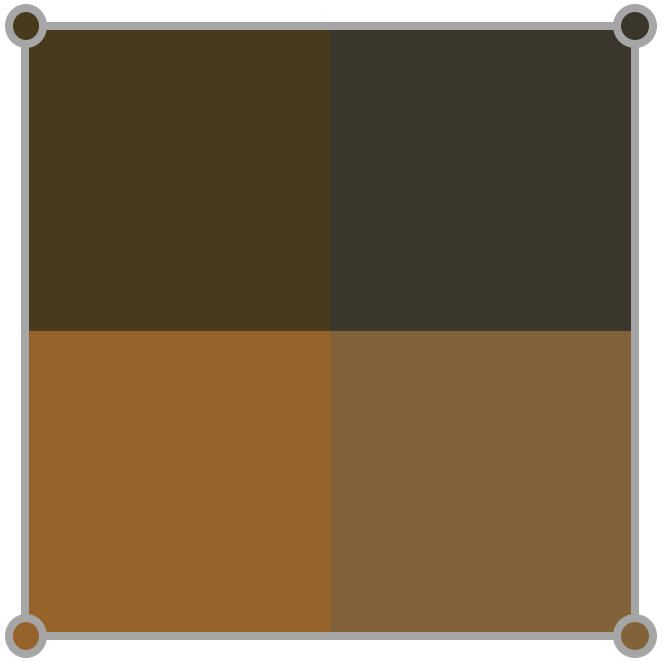


Can networks interpolate?



Can networks interpolate?





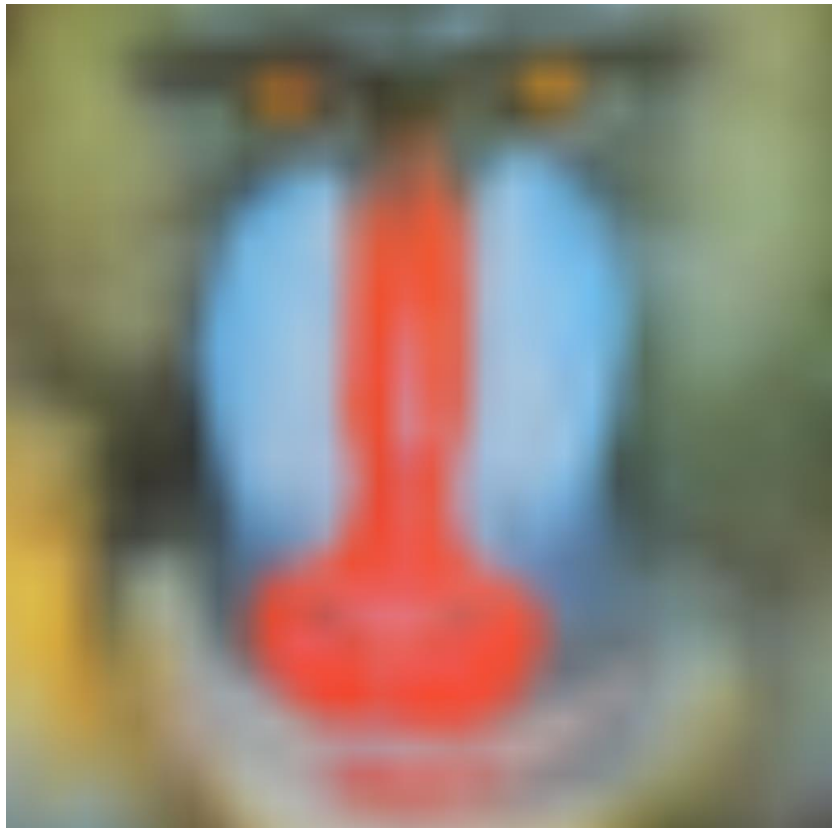
Can networks interpolate?



Can networks interpolate?



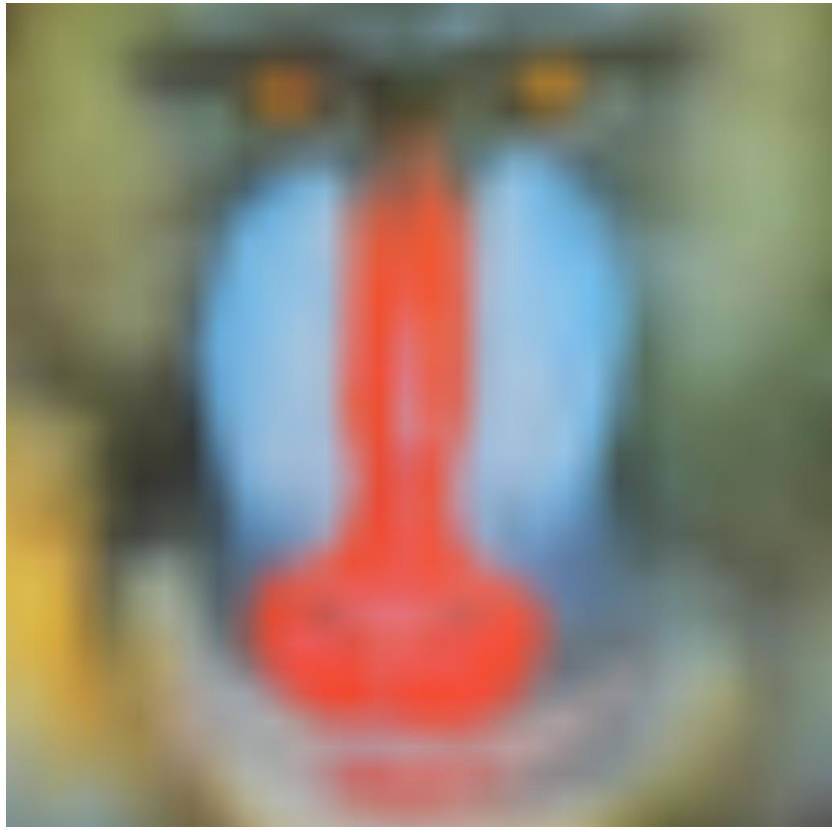
Reference



Can networks interpolate? No ☹



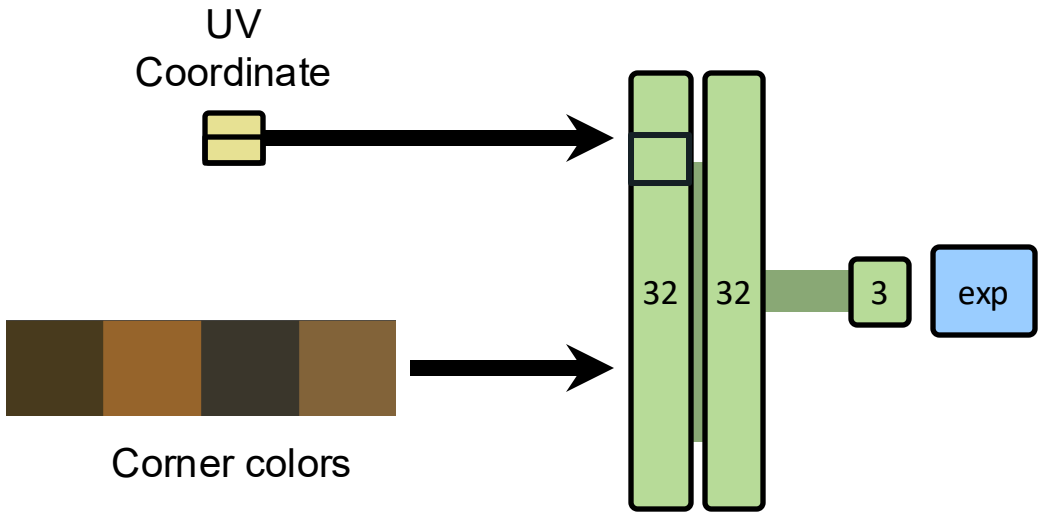
Reference

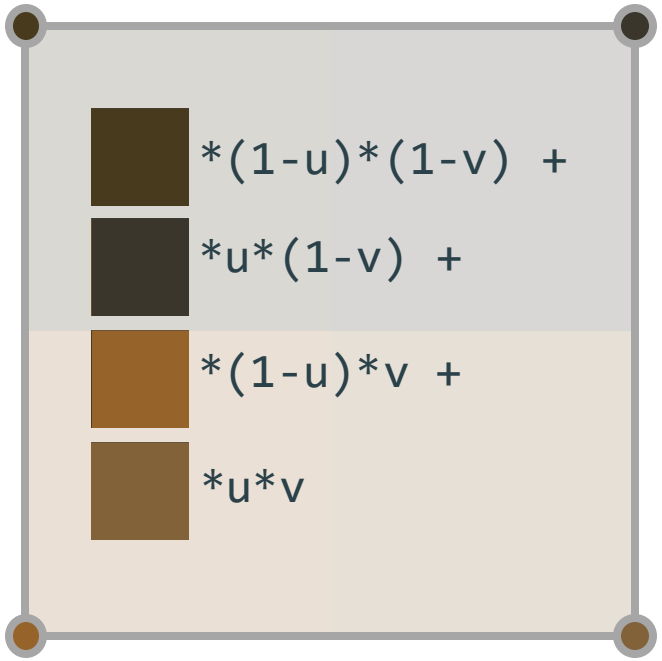


Neural Interpolator



Can networks interpolate?

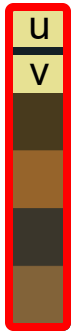


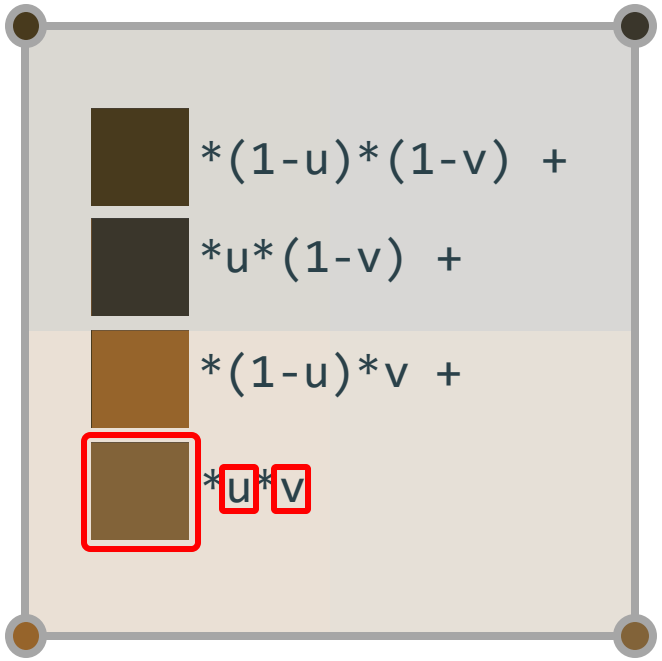


Networks can’t multiply!



```
float sum = BIAS;
for (int i = 0; i < Inputs; ++i)
{
    sum += W[i] * X[i];
}
return activation(sum);
```

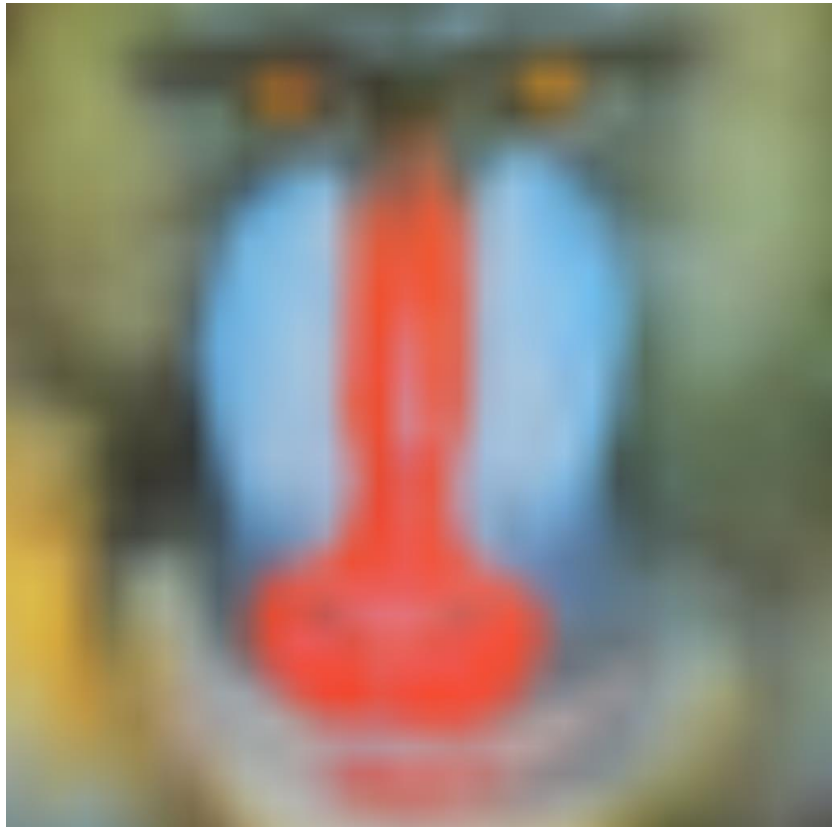




Can networks interpolate? No ☹



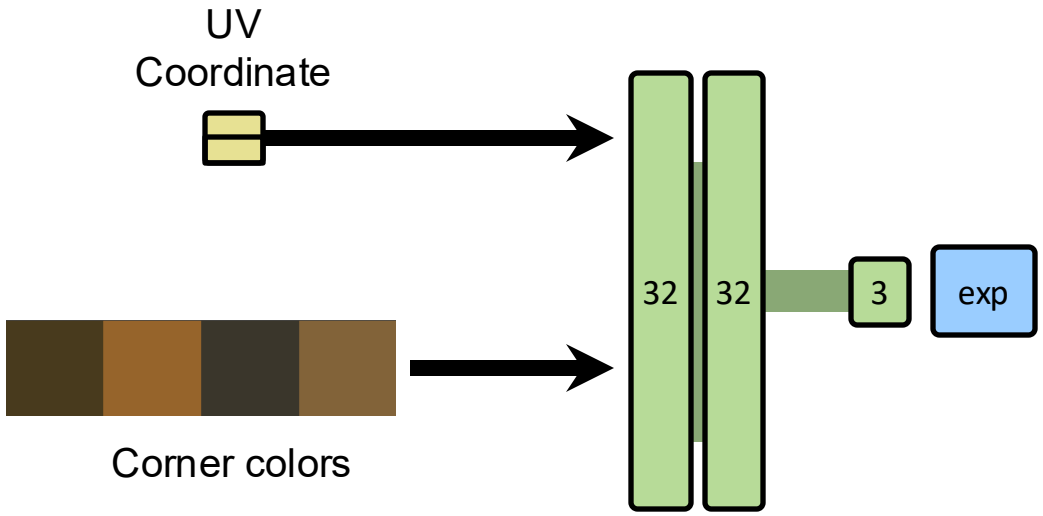
Reference

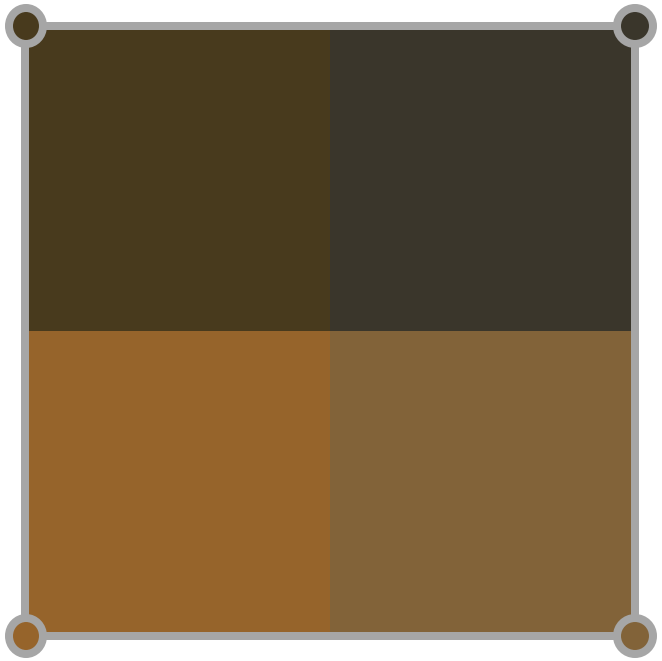


Neural Interpolator

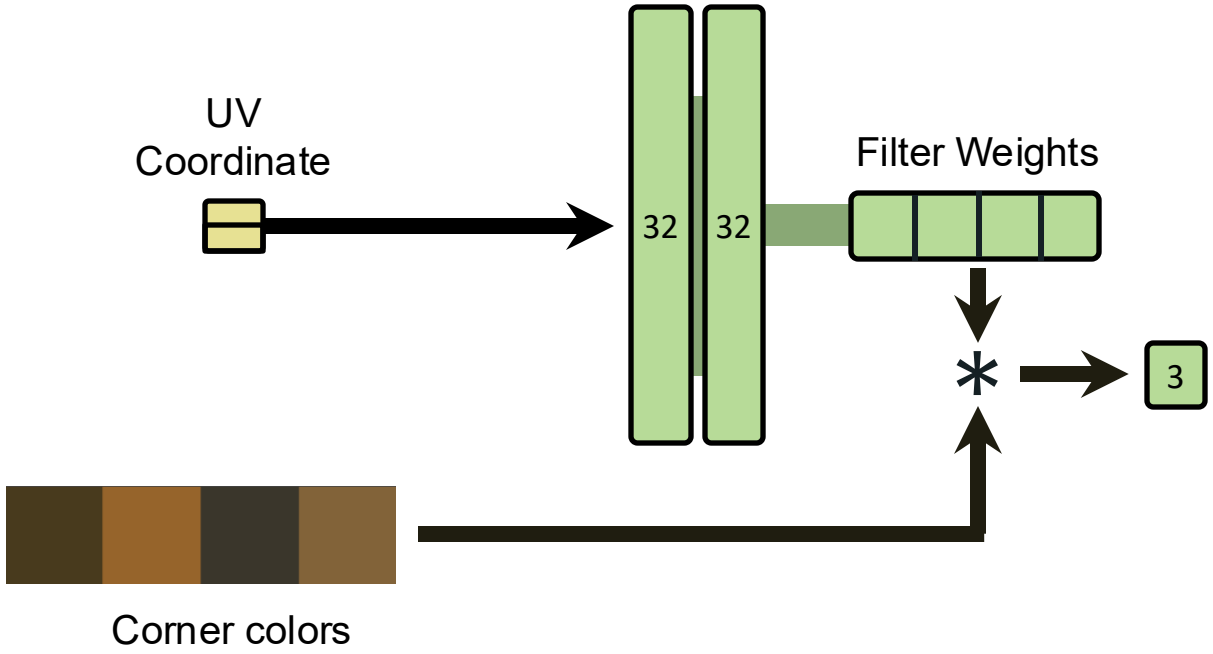


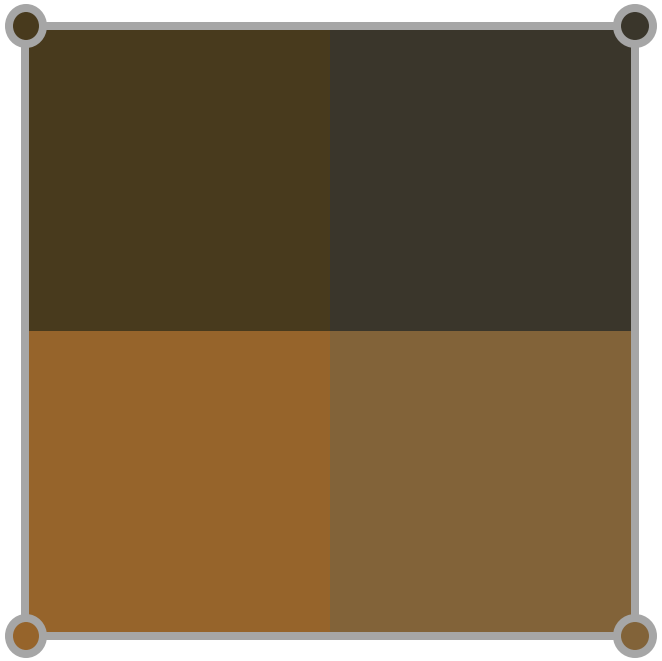
Working with the network





Working with the network

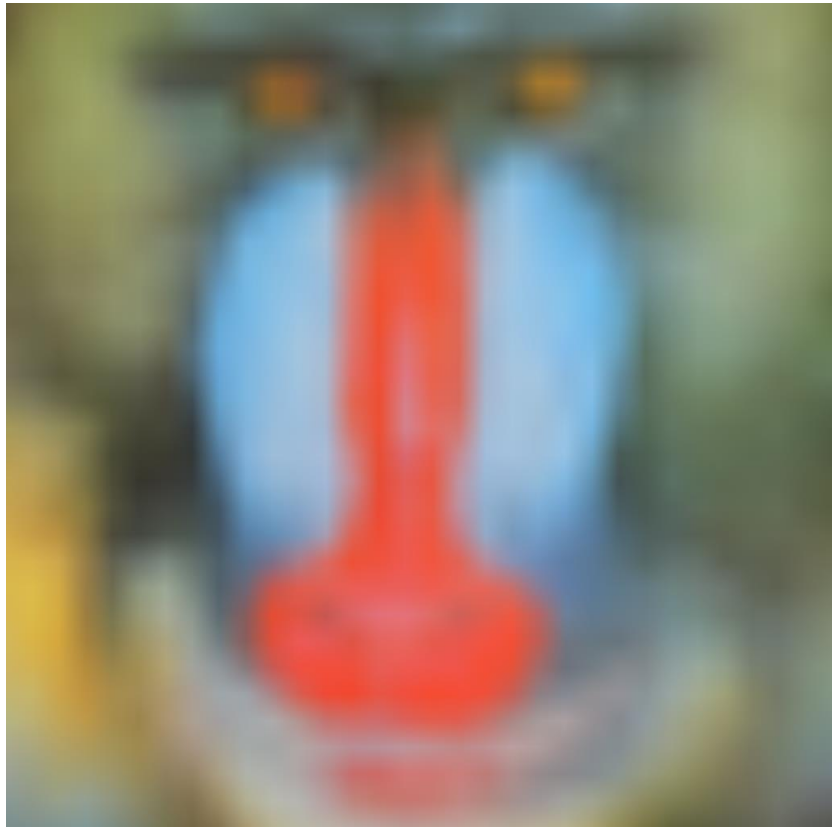




Can networks interpolate?



Reference



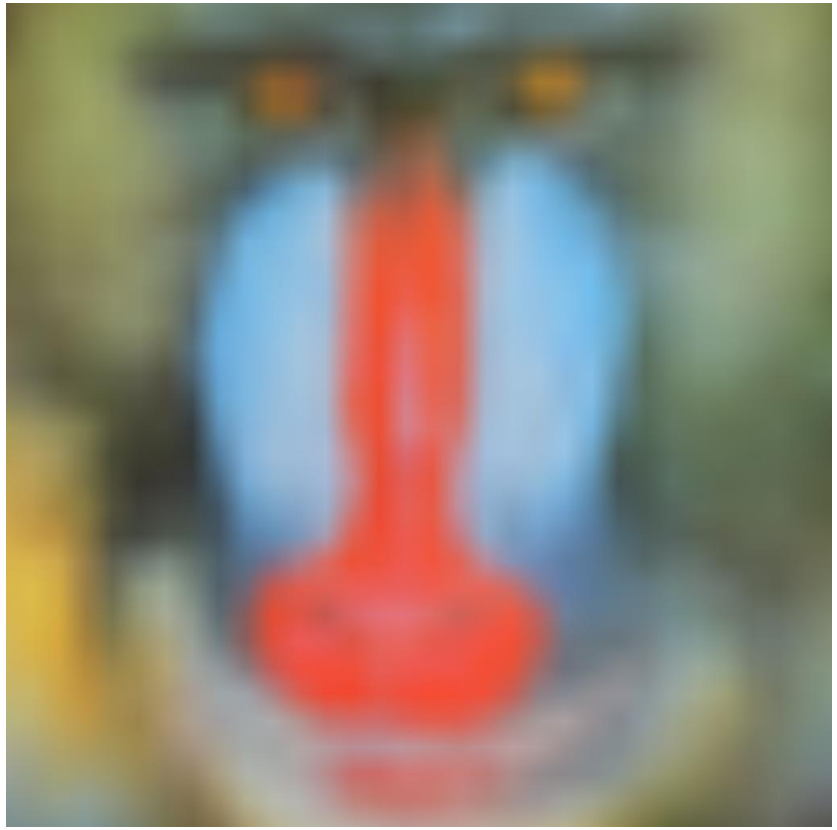
Neural Interpolator



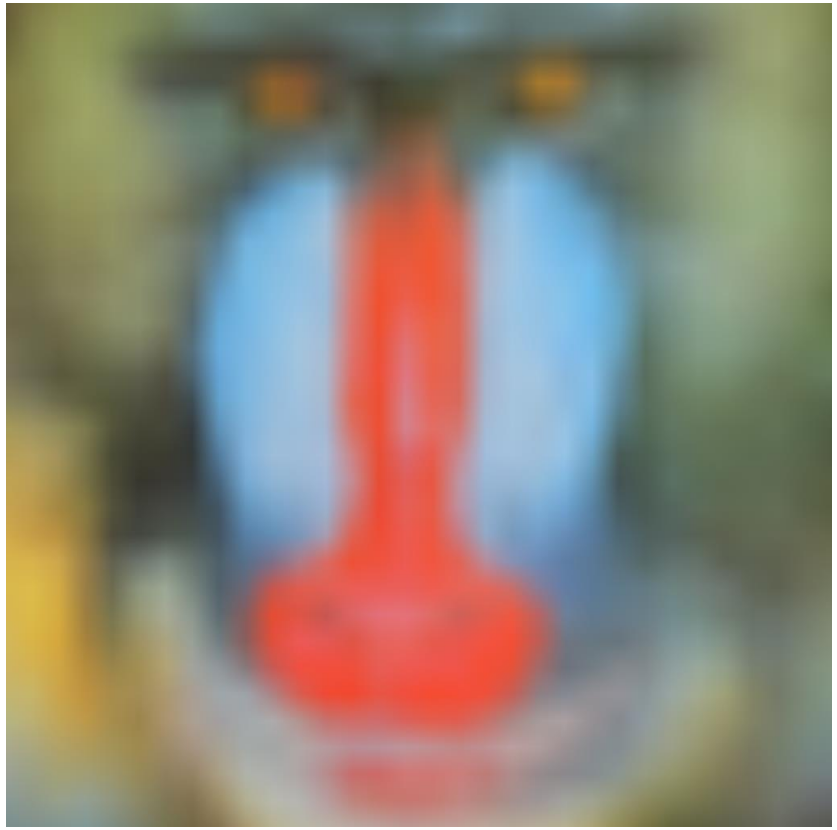
Can networks interpolate? Yes! ☺



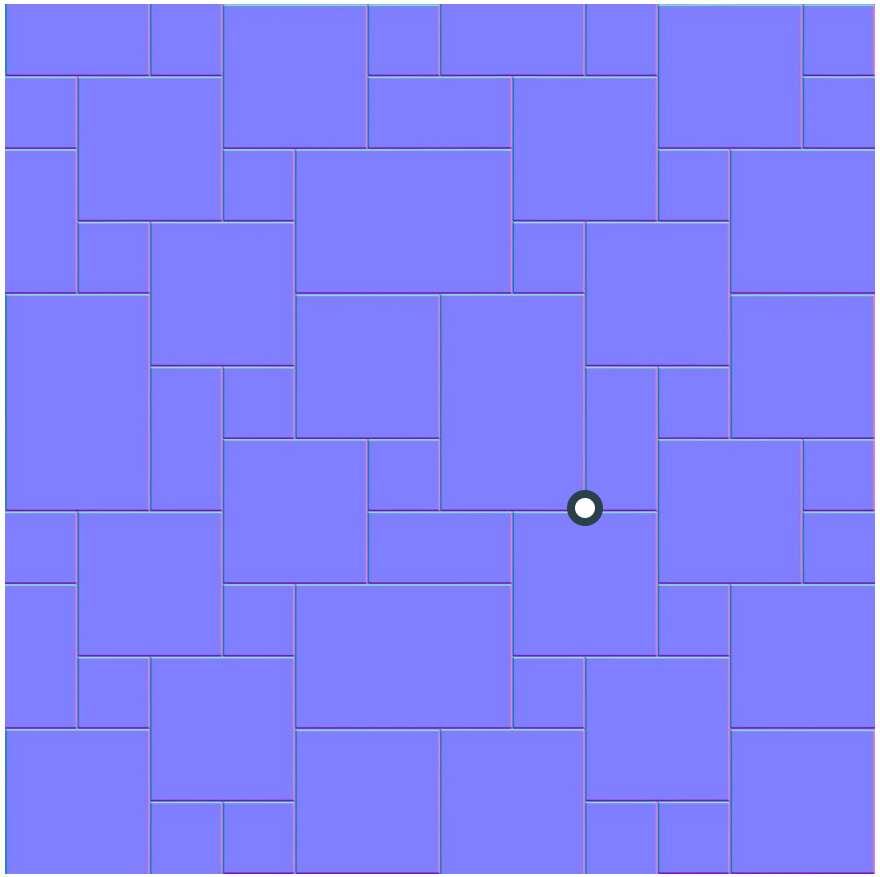
Reference



Hybrid Neural Interpolator



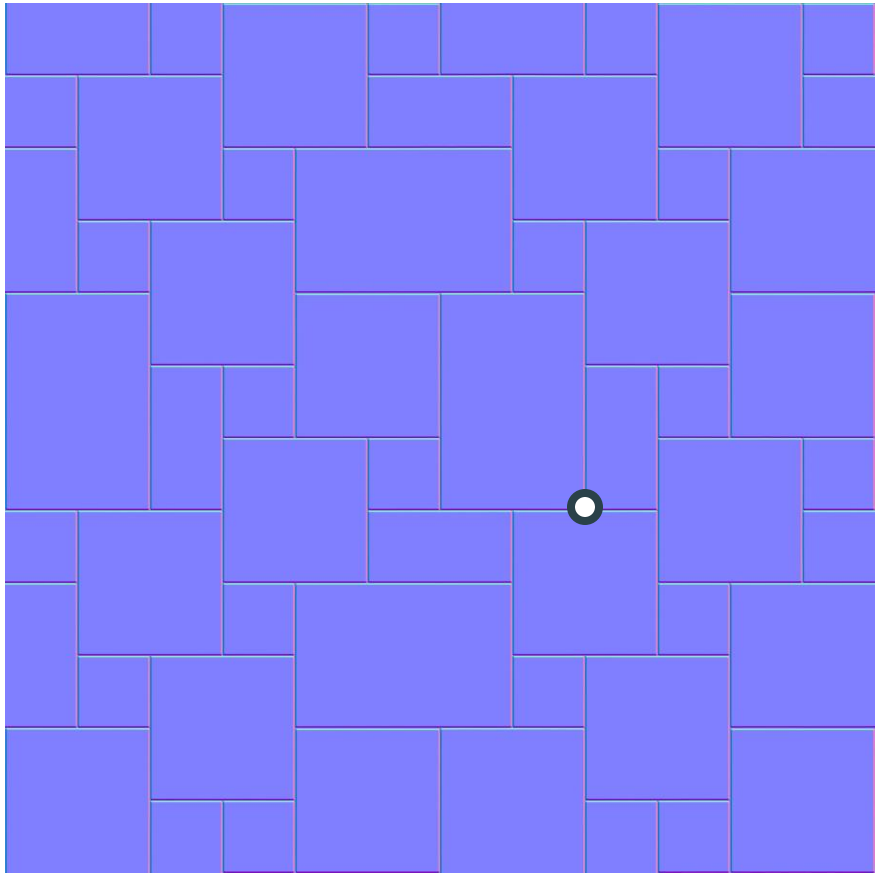
Can networks do normal mapping?



```
void apply_normal_map(
    float3 normal, float3 tangent,
    inout float3 wi, inout float3 wo
) {
    normal = normalize(normal);
    tangent = normalize(tangent);
    float3 bitangent = cross(normal, tangent);


    wi = float3(dot(wi, tangent), dot(wi, bitangent), dot(wi, normal));
    wo = float3(dot(wo, tangent), dot(wo, bitangent), dot(wo, normal));
}
```

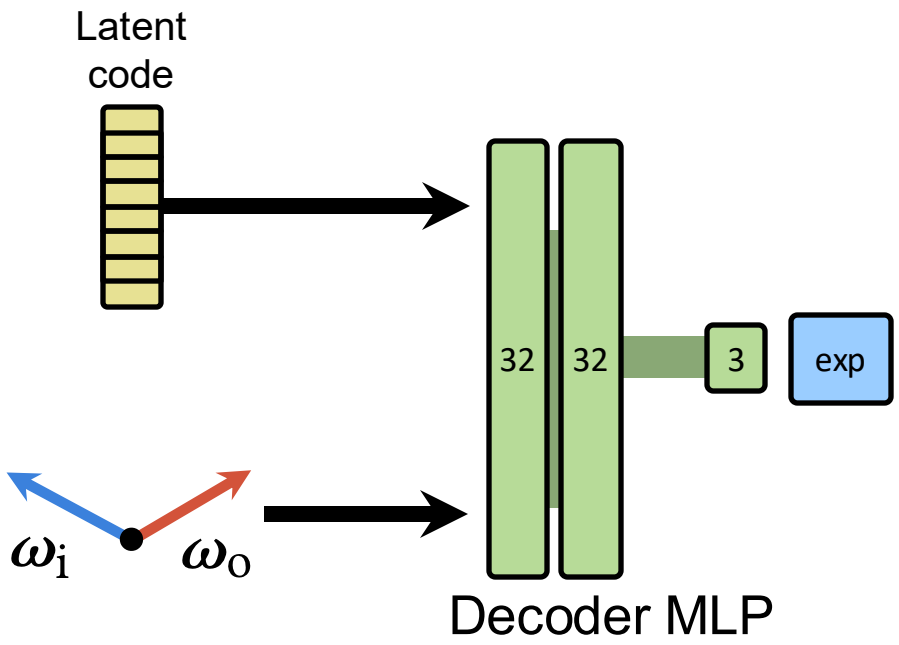
Can networks do normal mapping?



```
void apply_normal_map(
    float3 normal, float3 tangent,
    inout float3 wi, inout float3 wo
) {
    normal = normalize(normal);
    tangent = normalize(tangent);
    float3 bitangent = cross(normal, tangent);


    wi = float3(dot(wi, tangent), dot(wi, bitangent), dot(wi, normal));
    wo = float3(dot(wo, tangent), dot(wo, bitangent), dot(wo, normal));
}
```

Can networks do normal mapping?




```
public struct Network
{
    LatentTexture<kNumLatents> latent_texture;

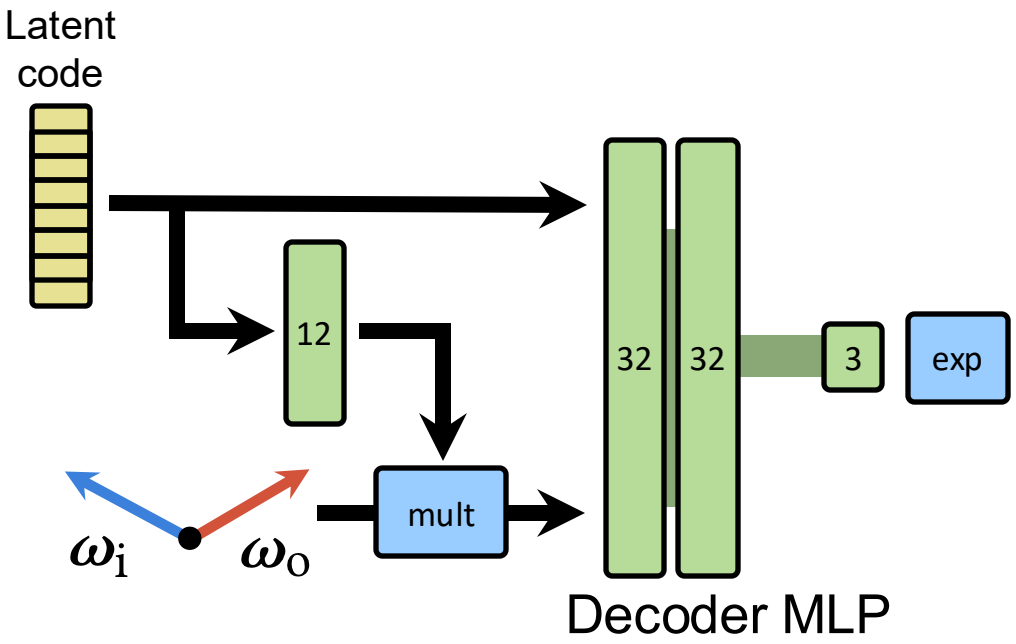
    Decoder decoder;

    [Differentiable]
    float3 eval(no_diff float2 uv, no_diff float3 wi, no_diff float3 wo)
    {
        float[kNumLatents] latents = latent_texture.sample(uv);

        float3 wh;
        float3 wd;
        rusinkiewicz_parameterize(wi, wo, wh, wd);

        return decoder.forward(latents, wh, wd);
    }
};
```

Can networks do normal mapping?




```
public struct Network
{
    LatentTexture<kNumLatents> latent_texture;

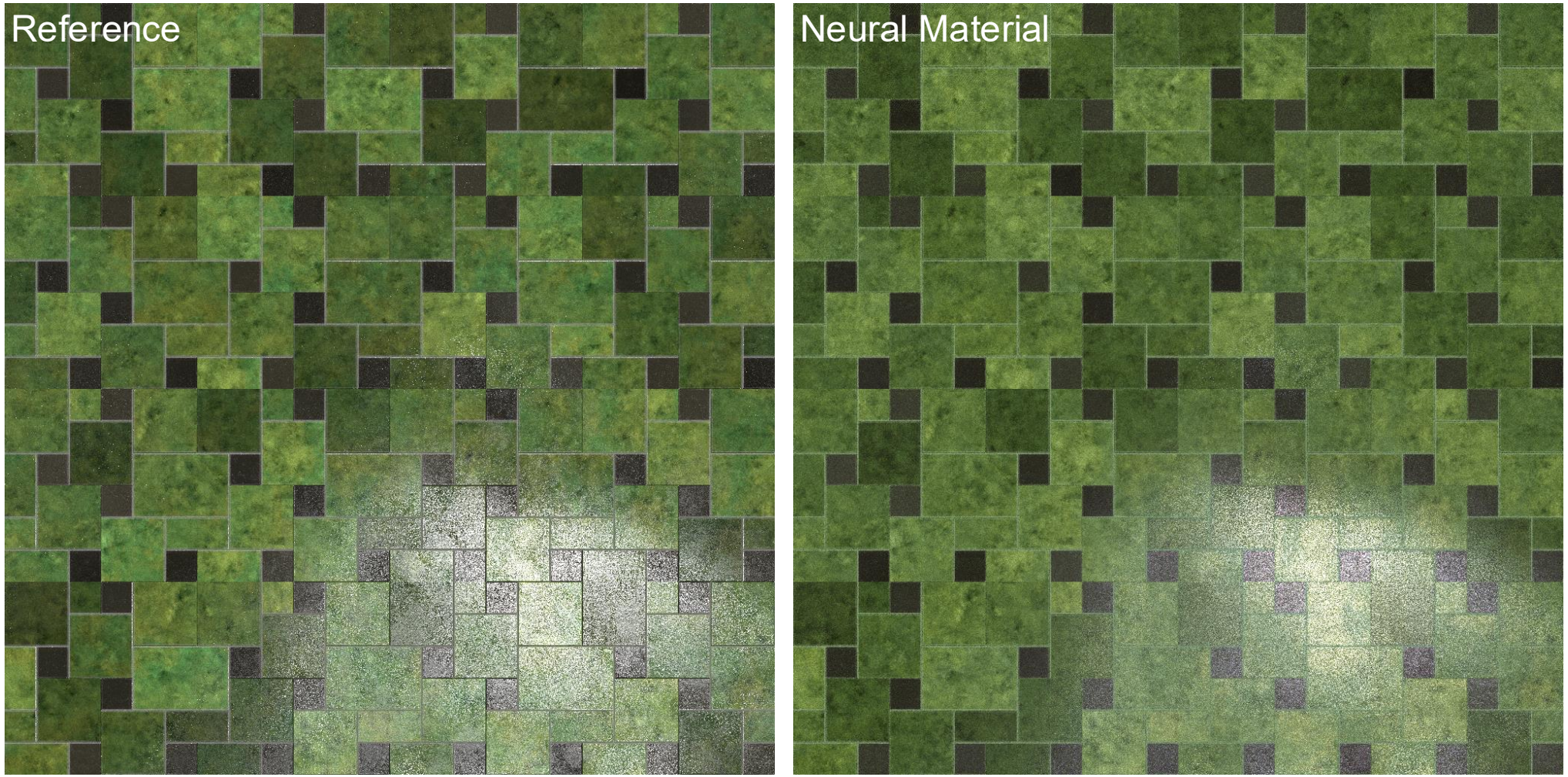
    Decoder decoder;

    [Differentiable]
    float3 eval(no_diff float2 uv, no_diff float3 wi, no_diff float3 wo)
    {
        float[kNumLatents] latents = latent_texture.sample(uv);

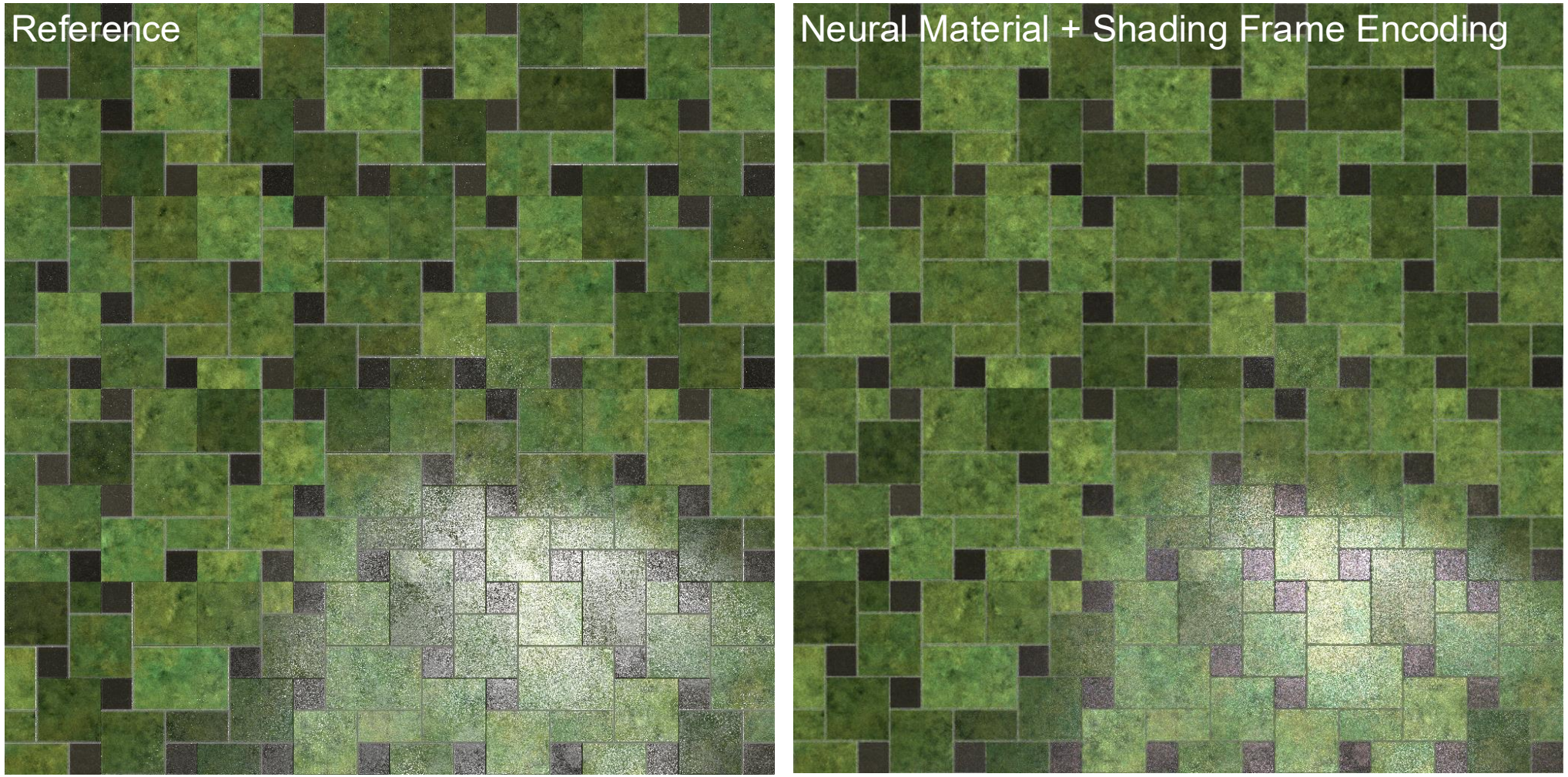
        float3 wh;
        float3 wd;
        rusinkiewicz_parameterize(wi, wo, wh, wd);

        return decoder.forward(latents, wh, wd);
    }
};
```

Can networks do normal mapping?



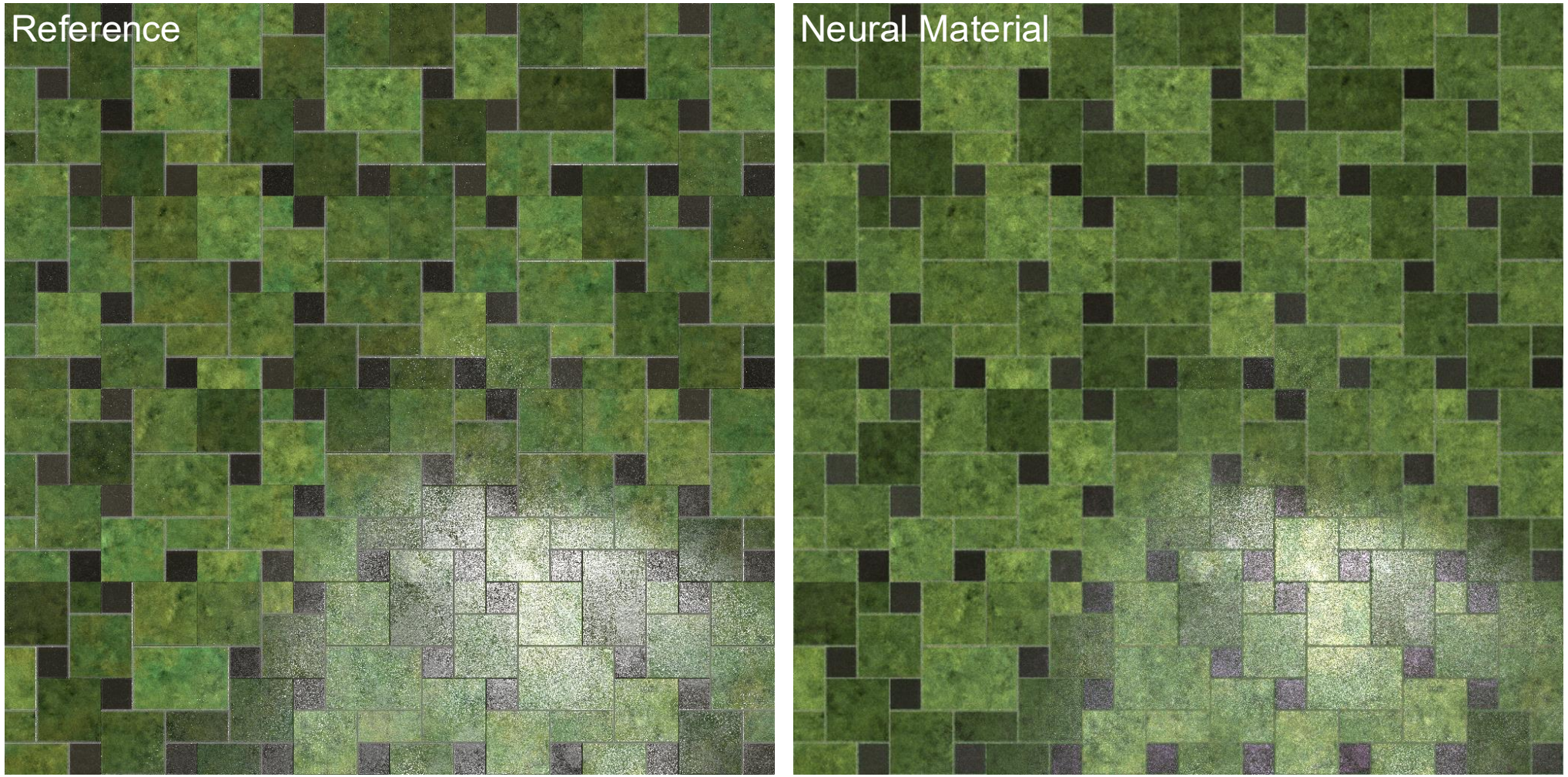
Can networks do normal mapping? Yes ☺



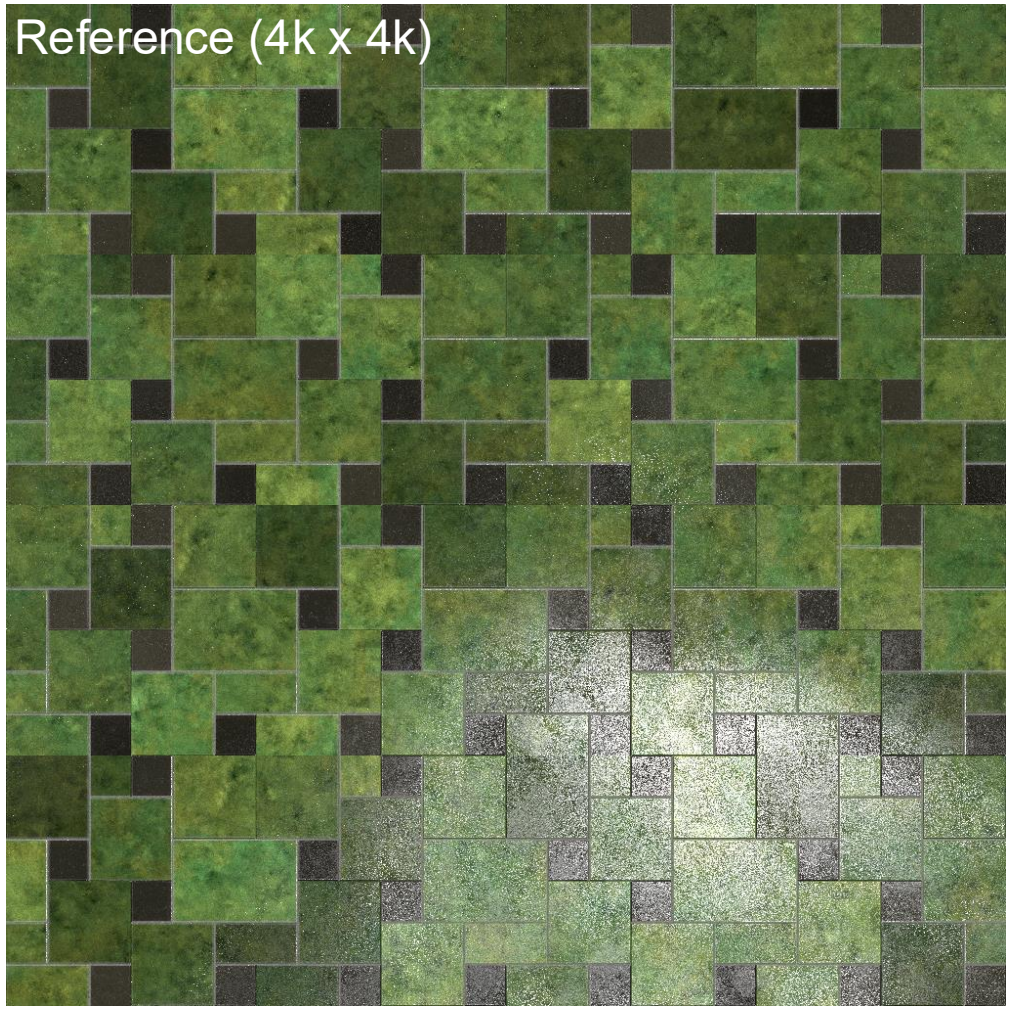
The resolution problem

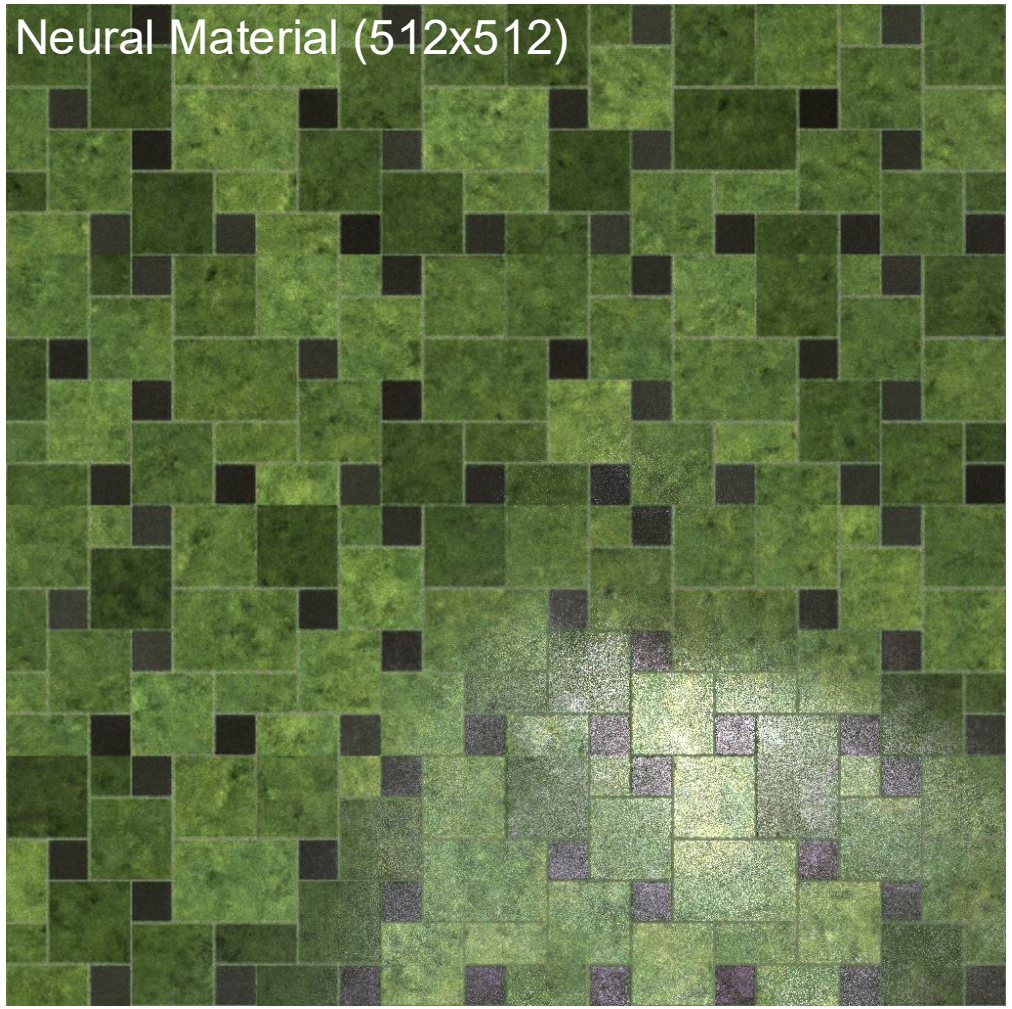


The resolution problem

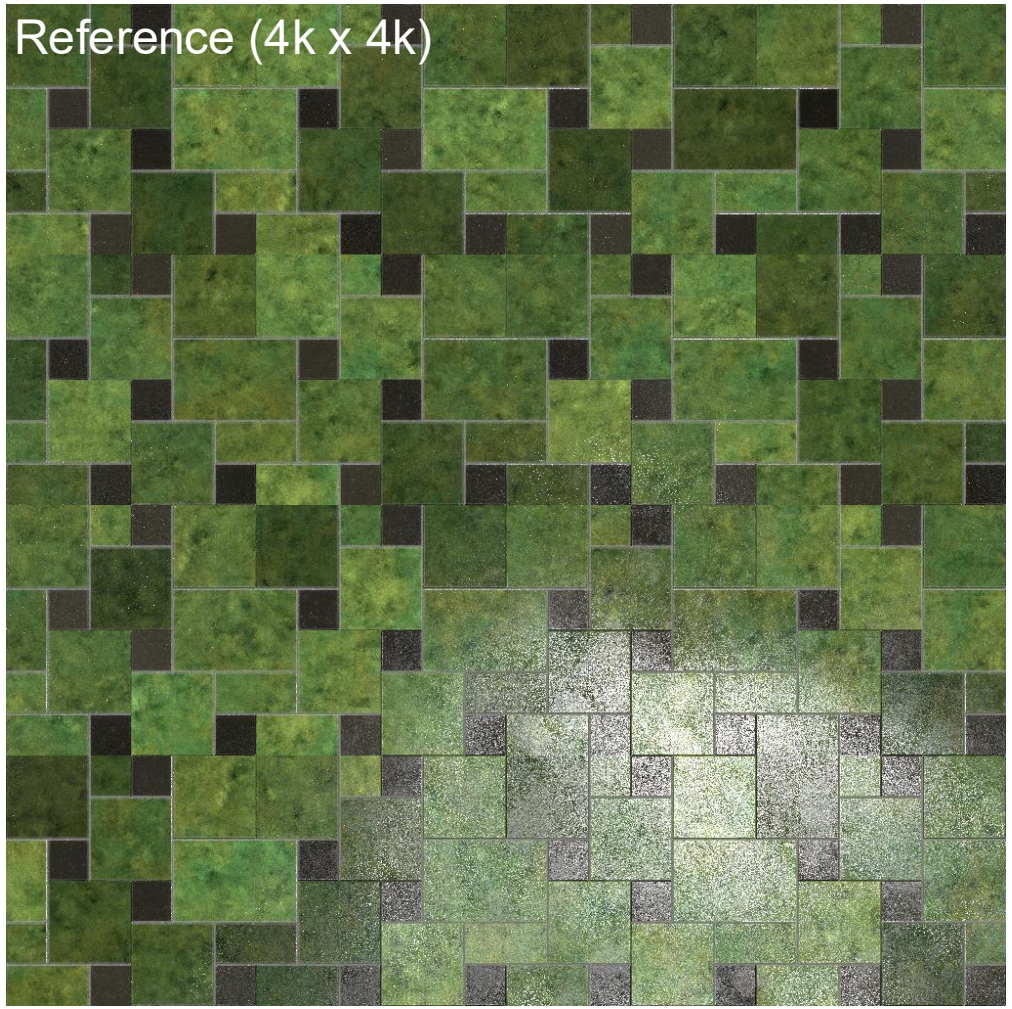


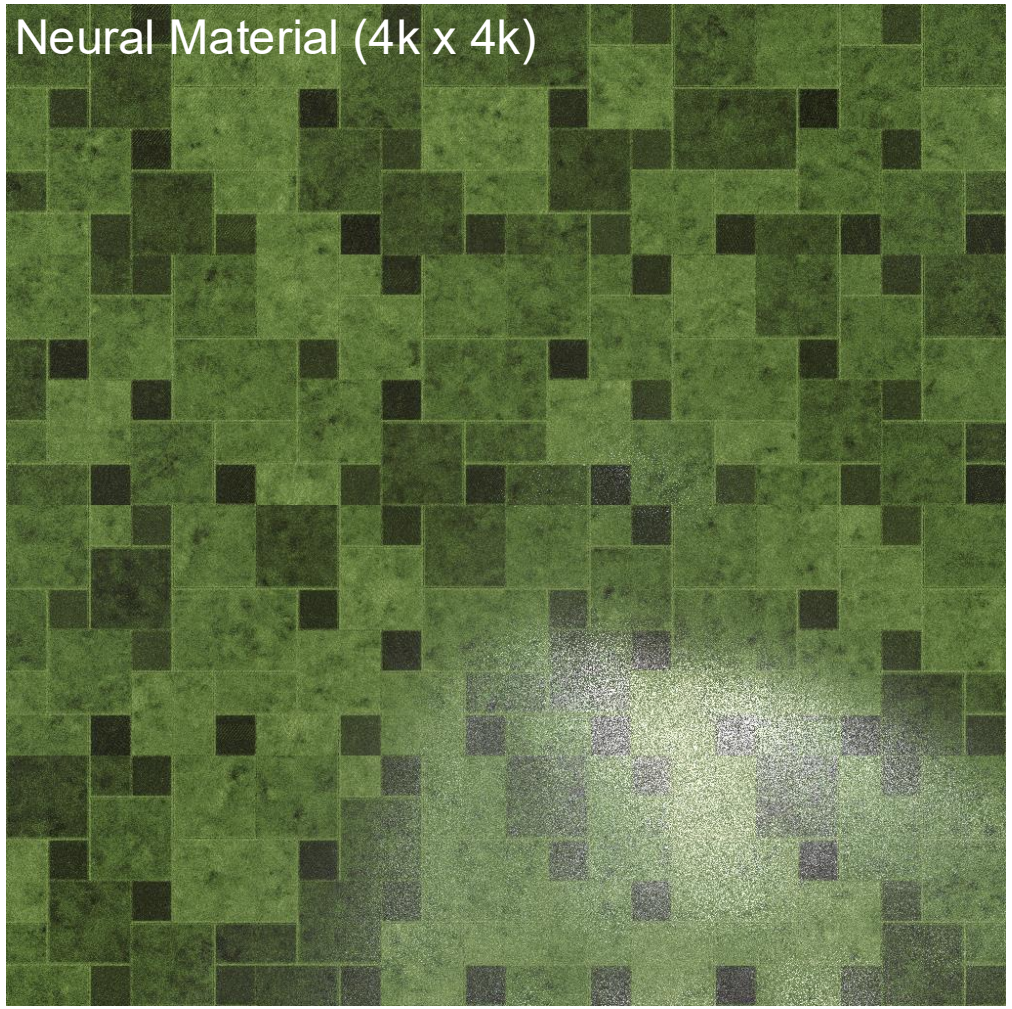
The resolution problem





The resolution problem





The resolution problem



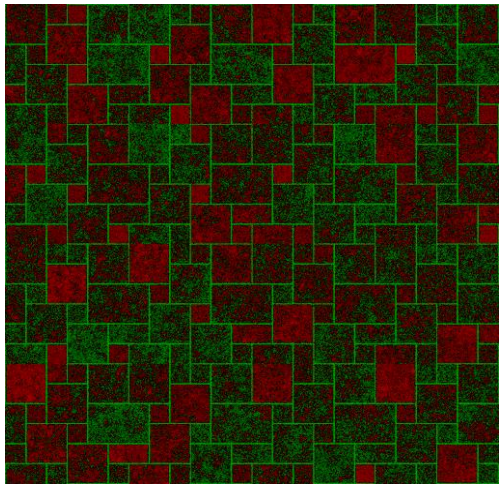
- Each pixel in the latent texture is a separate code to optimize
- $512^2 \rightarrow 4k^2$: 64x more pixels to optimize
  - Or: 64x fewer samples per pixel

This is inconvenient!

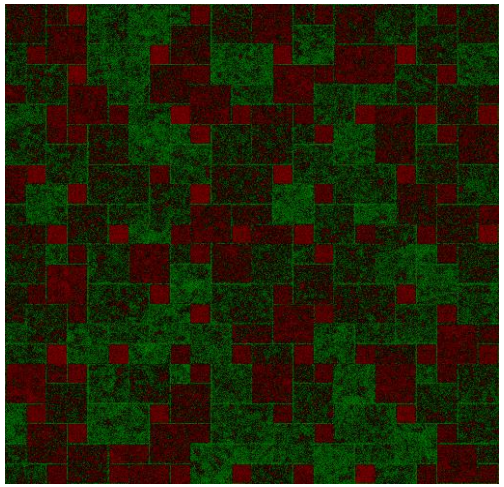
- Training time depends on texture resolution
- Latent texture evolves at a different speed than neural network

- Can we do better?

512 x 512

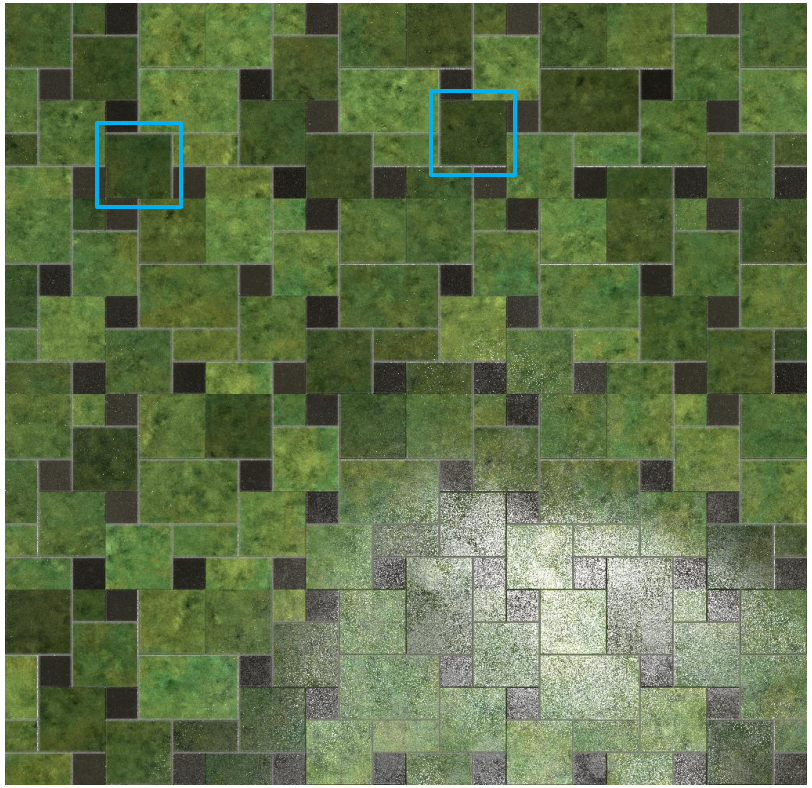


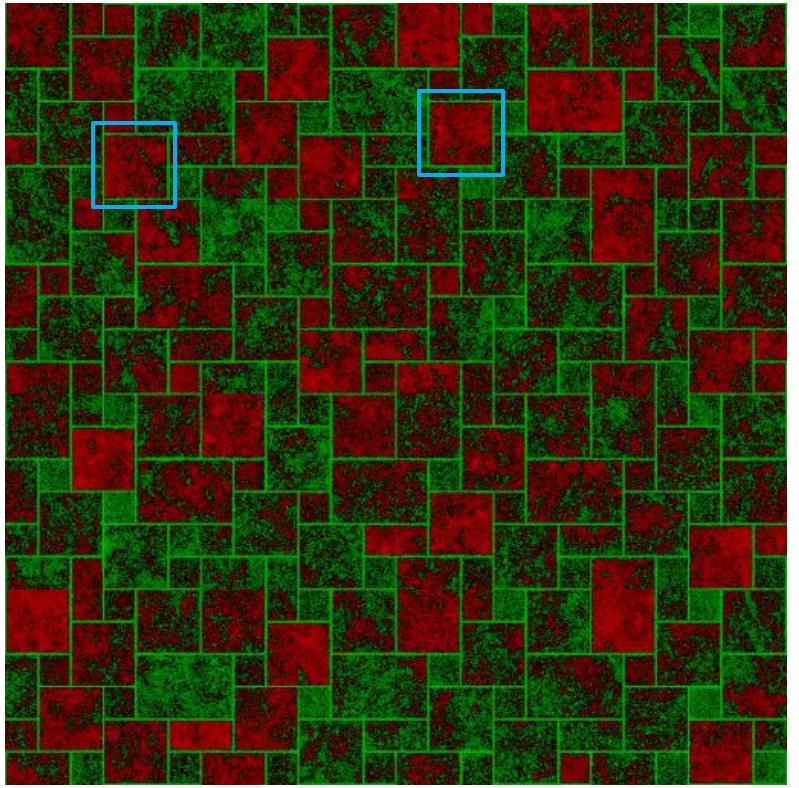
4k x 4k



© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Knowledge sharing





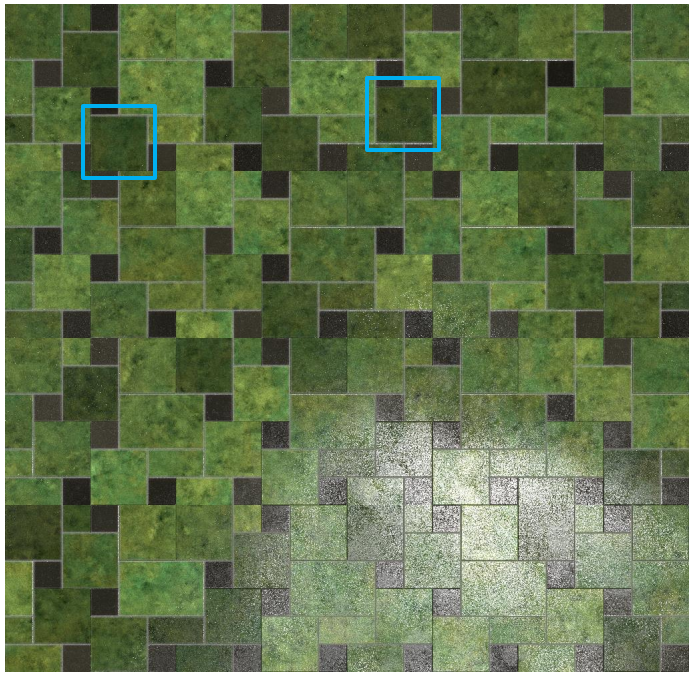
© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

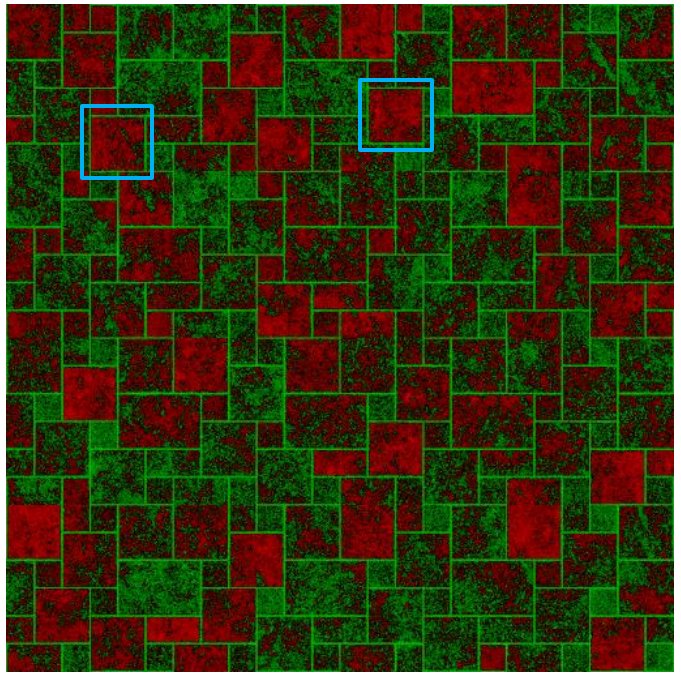
Knowledge sharing



- “Similar” texels will learn similar latent codes
- Can we learn their latent codes together?

- How do we know if two texels will look similar?





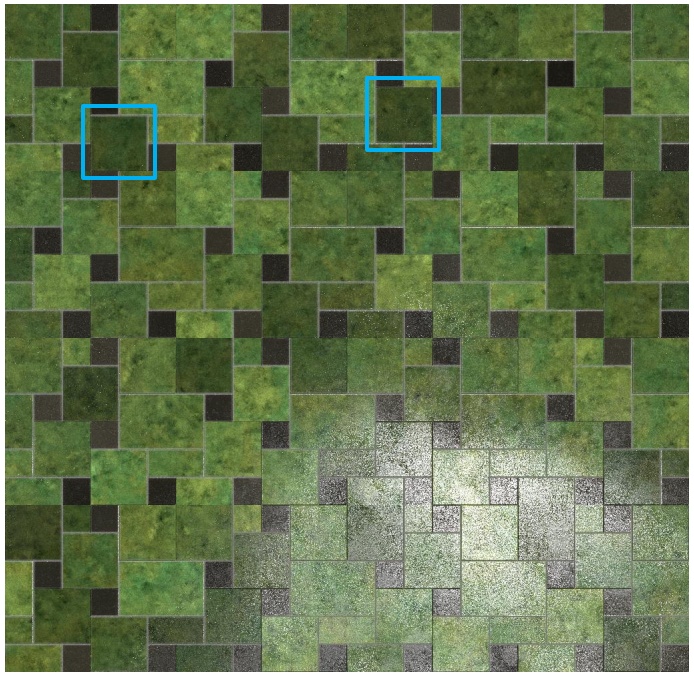
© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

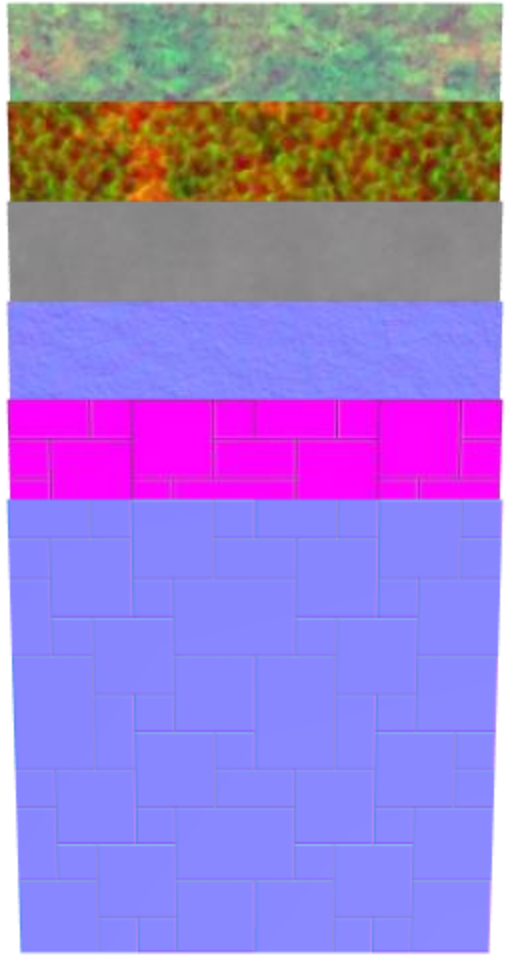
Knowledge sharing



- “Similar” texels will learn similar latent codes
- Can we learn their latent codes together?

- How do we know if two texels will look similar?





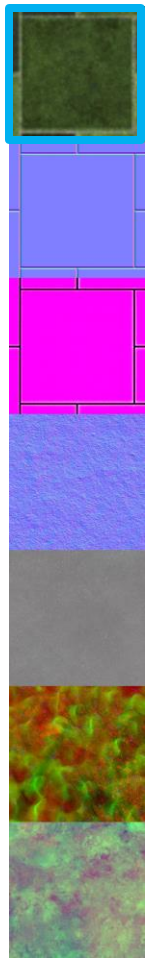
© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

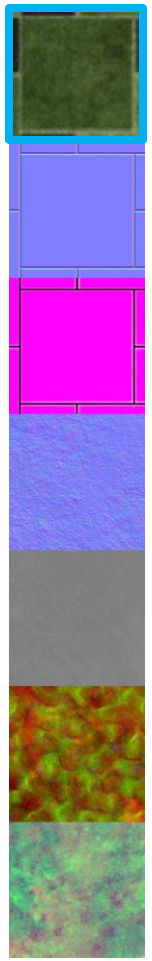
Knowledge sharing

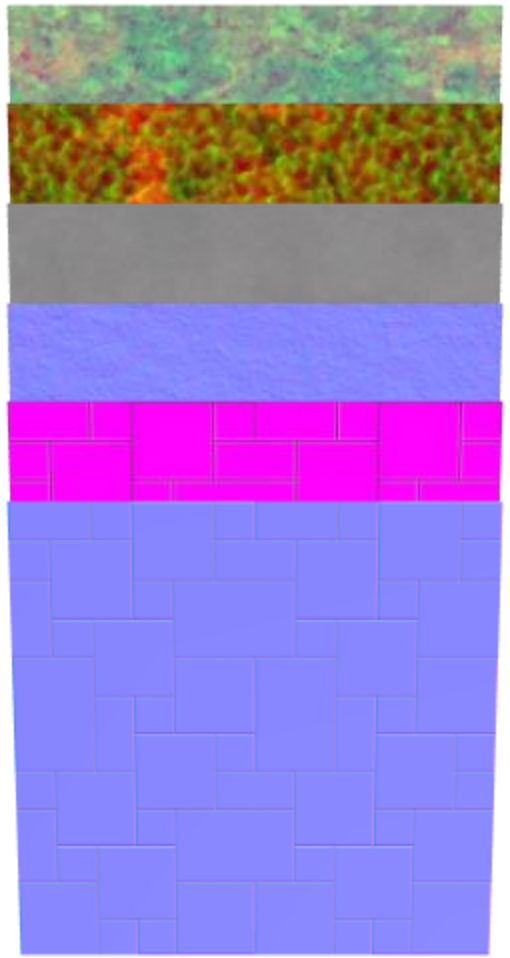


- “Similar” texels will learn similar latent codes
- Can we learn their latent codes together?

- How do we know if two texels will look similar?







© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

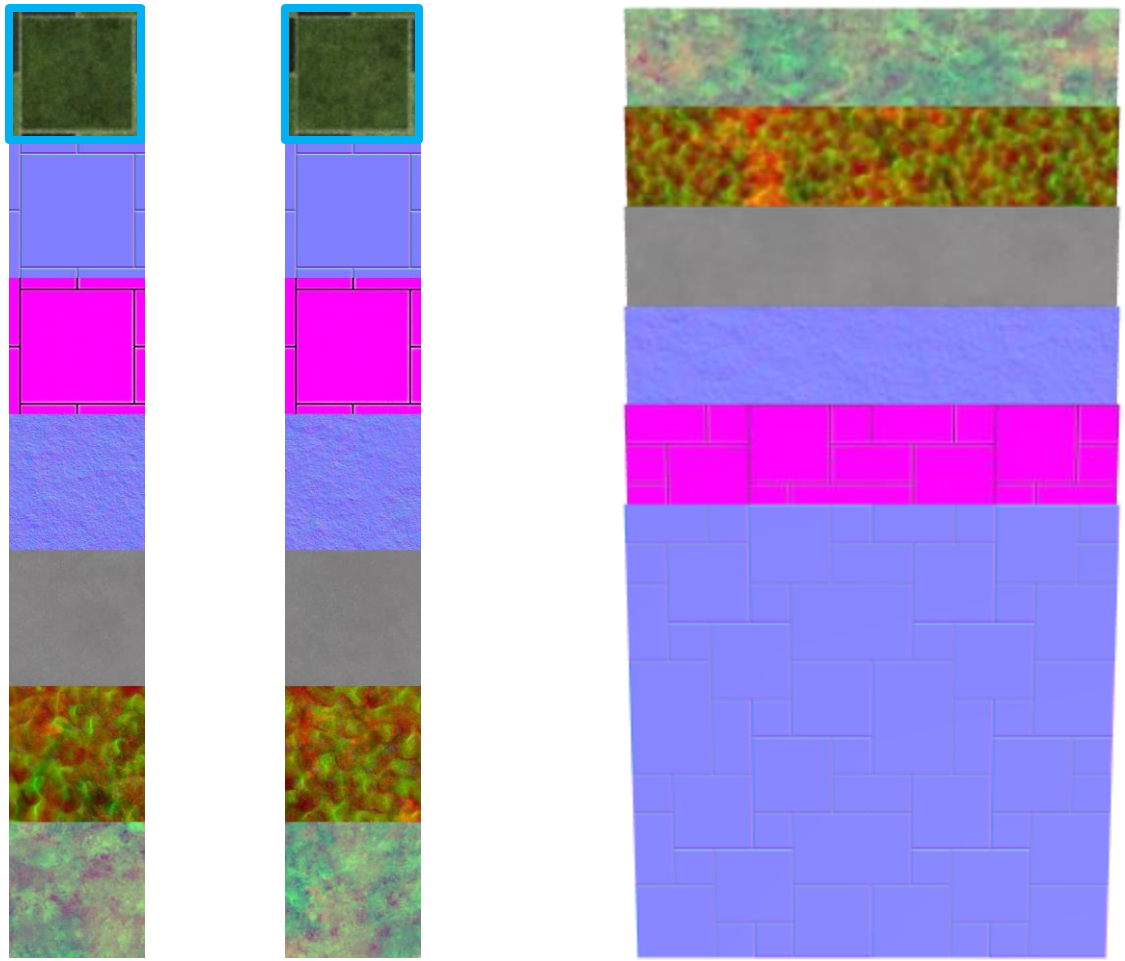
Knowledge sharing



- “Similar” texels will learn similar latent codes
- Can we learn their latent codes together?

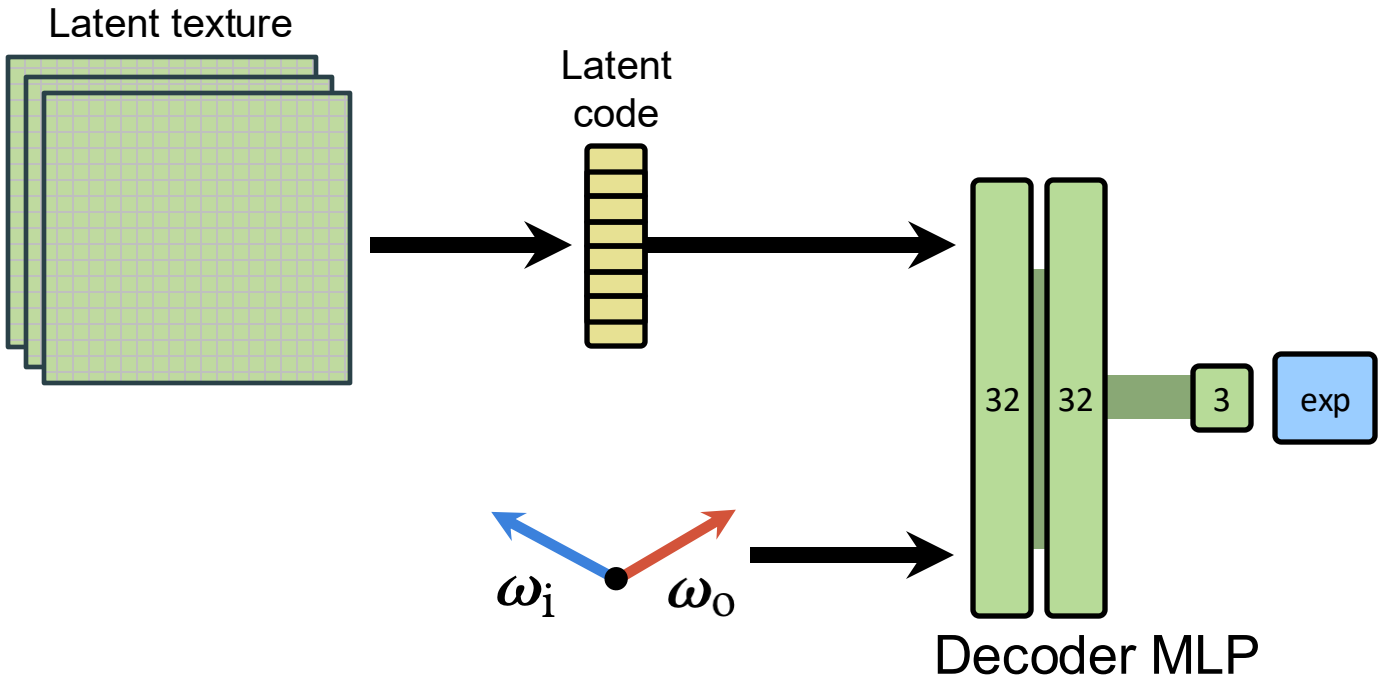
- How do we know if two texels will look similar?

- From the reference material textures!



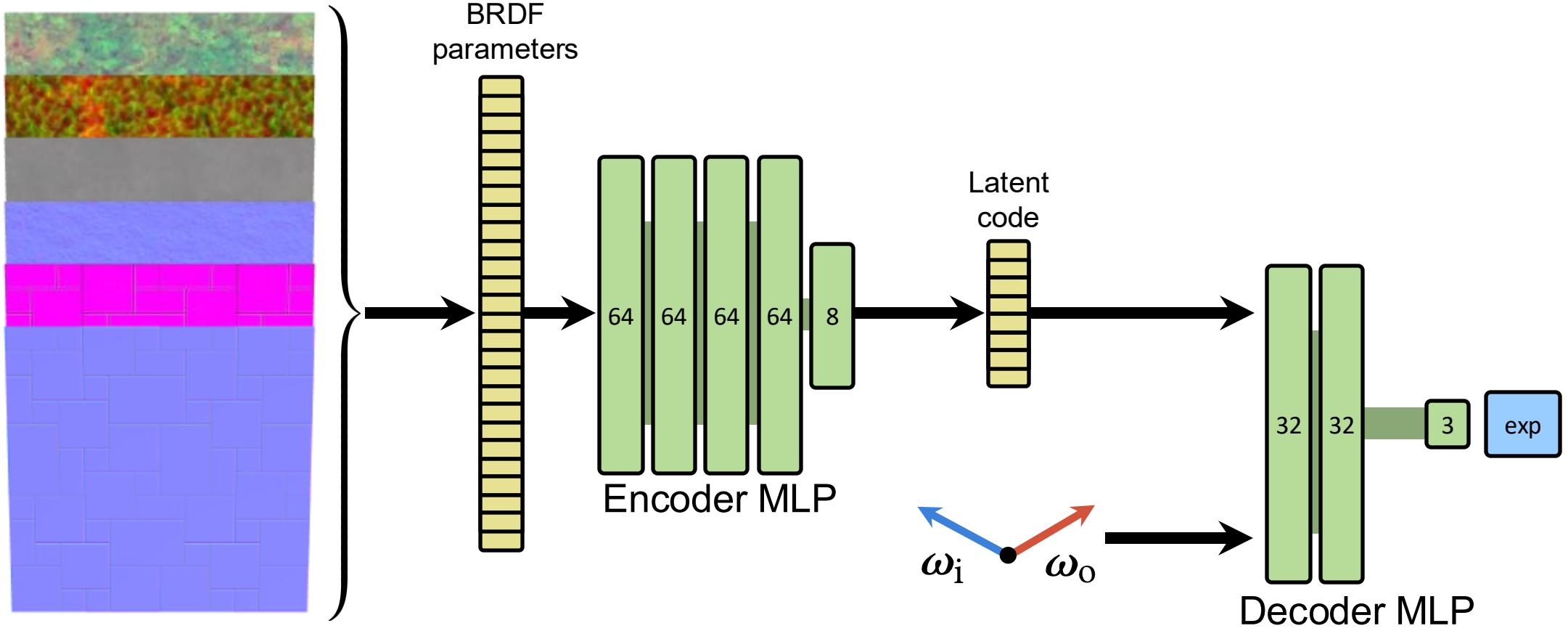
© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Encoding material textures



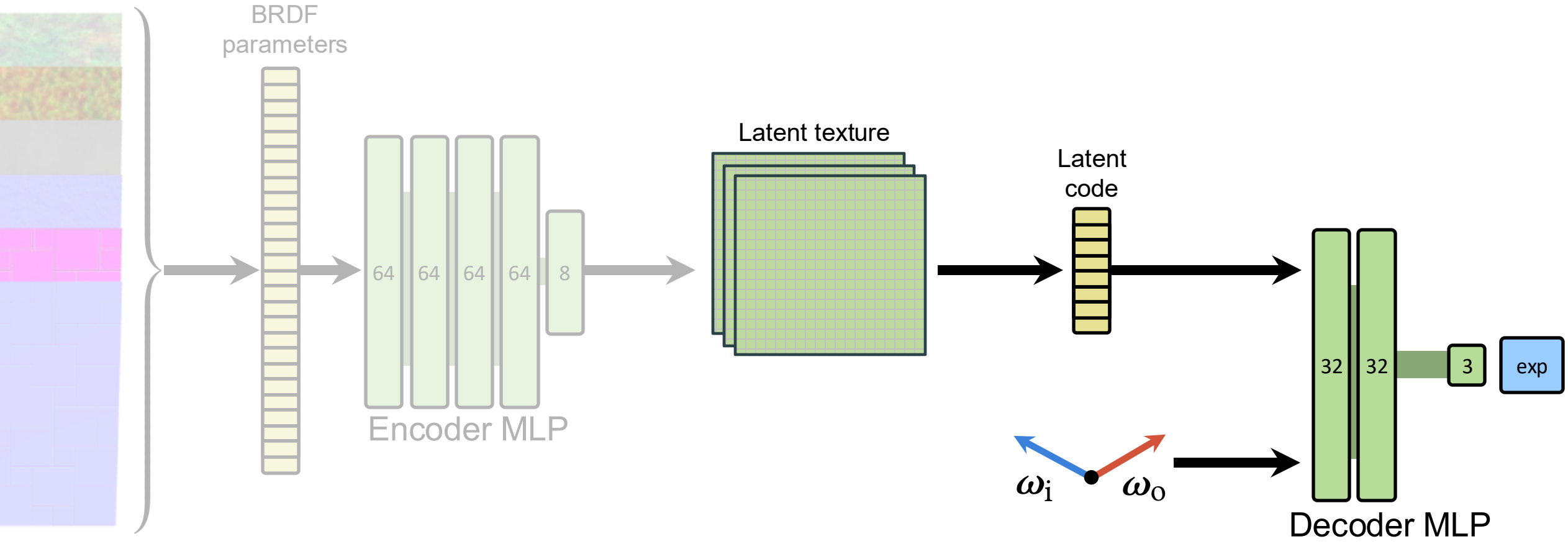
© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

The material Encoder



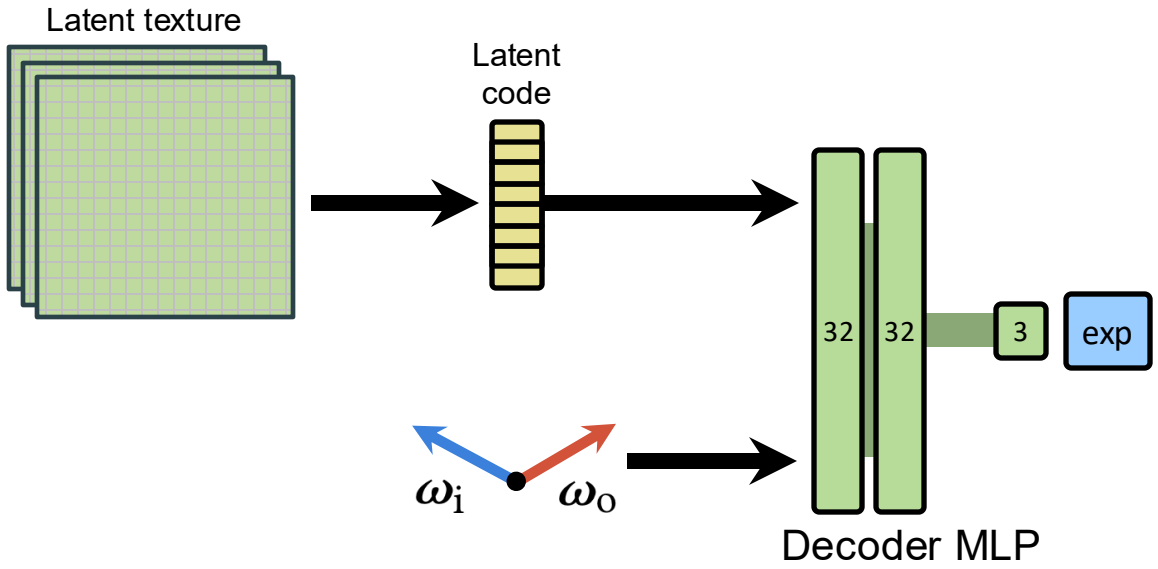
© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

After training: Bake your latents



© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Implementing the Encoder




```
public struct Network
{
    LatentTexture<kNumLatents> latent_texture;
    ShadingFrameRotation rotation;
    Decoder decoder;

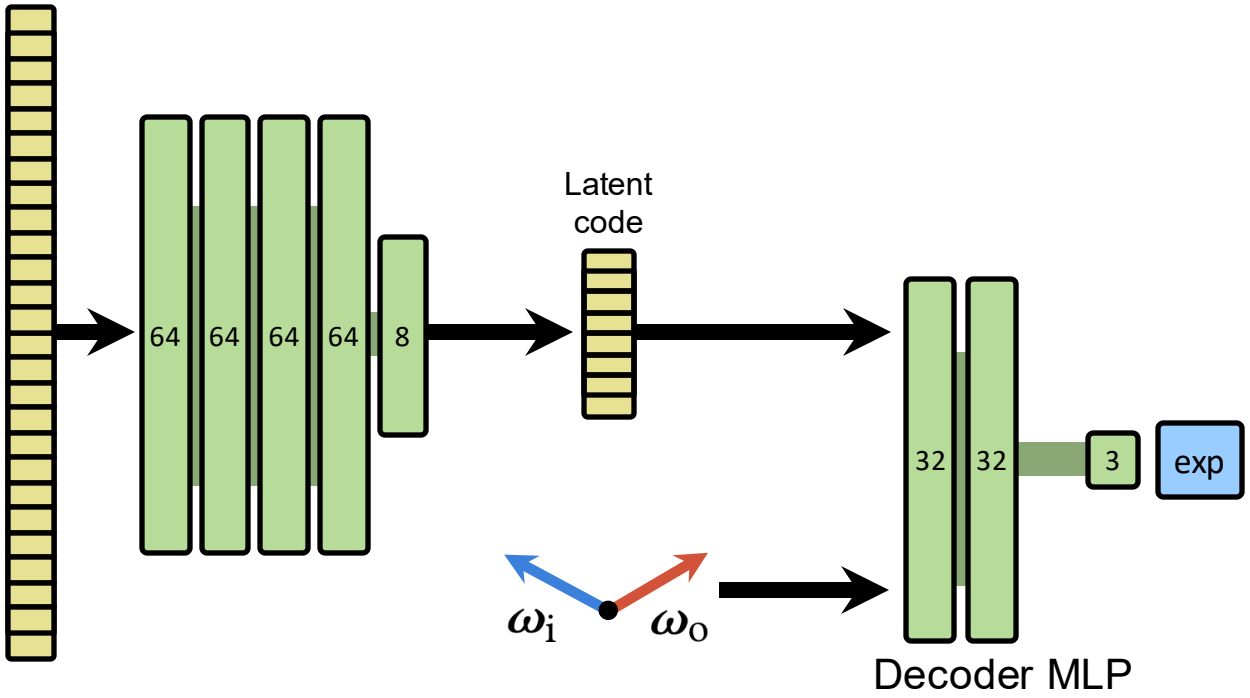

    [Differentiable]
    float3 eval(
            no_diff float2 uv,
            no_diff float3 wi,
            no_diff float3 wo
    ) {
        float[8] latents = latent_texture.sample(uv);

        /* .... */

    }
};
```

© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Implementing the Encoder




```
public struct Network
{
    Encoder encoder;
    ShadingFrameRotation rotation;
    Decoder decoder;

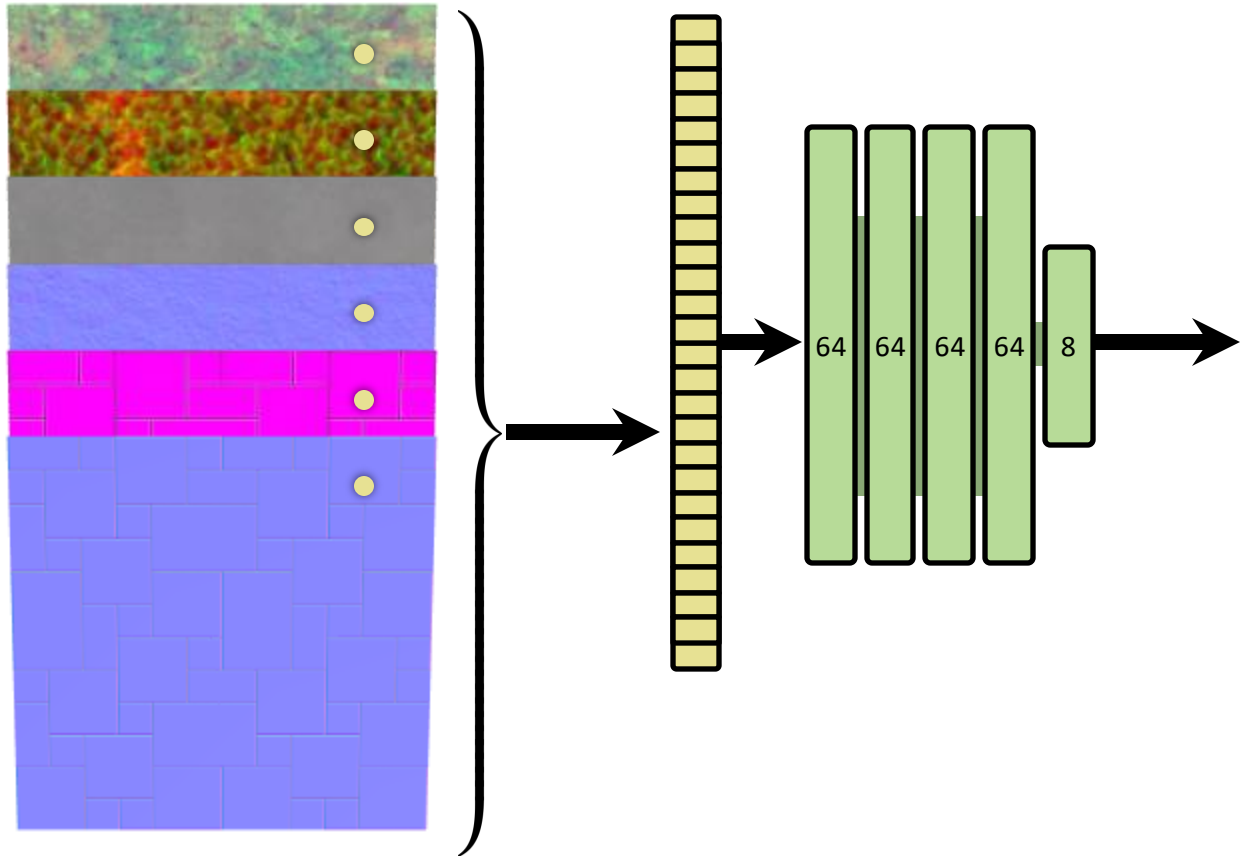

    [Differentiable]
    float3 eval(
            no_diff float[kEncoderInputs] encoder_inputs,
            no_diff float3 wi,
            no_diff float3 wo
    ) {
        float[8] latents = encoder.forward(encoder_inputs);

        /* .... */

    }
};
```

© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Implementing the Encoder




```
public struct Network
{
    Encoder encoder;
    ShadingFrameRotation rotation;
    Decoder decoder;

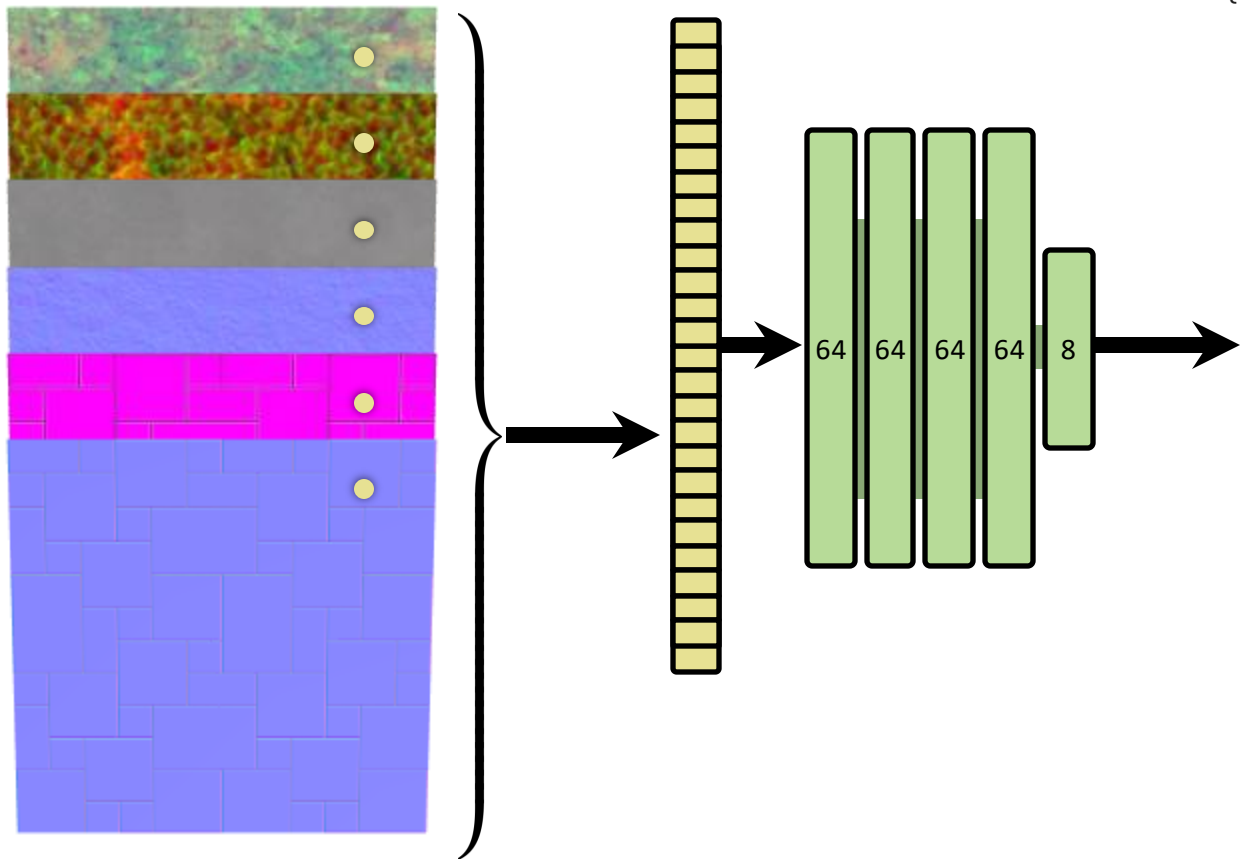

    [Differentiable]
    float3 eval(
            no_diff float[kEncoderInputs] encoder_inputs,
            no_diff float3 wi,
            no_diff float3 wo
    ) {
        float[8] latents = encoder.forward(encoder_inputs);

        /* .... */

    }
};
```

© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Implementing the Encoder




```
struct Encoder
{
    LinearLayer<kEncoderInputs, 64> layer0;
    LinearLayer<64, 64> layer1;
    LinearLayer<64, 64> layer2;
    LinearLayer<64, 8> layer3;

    [Differentiable]
    float[kNumLatents] forward(no_diff float[kEncoderInputs] inputs)
    {
        float[64] h0 = layer0.forward(inputs);
        [ForceUnroll]
        for (int i = 0; i < 64; ++i)
            h0[i] = leaky_relu(h0[i]);

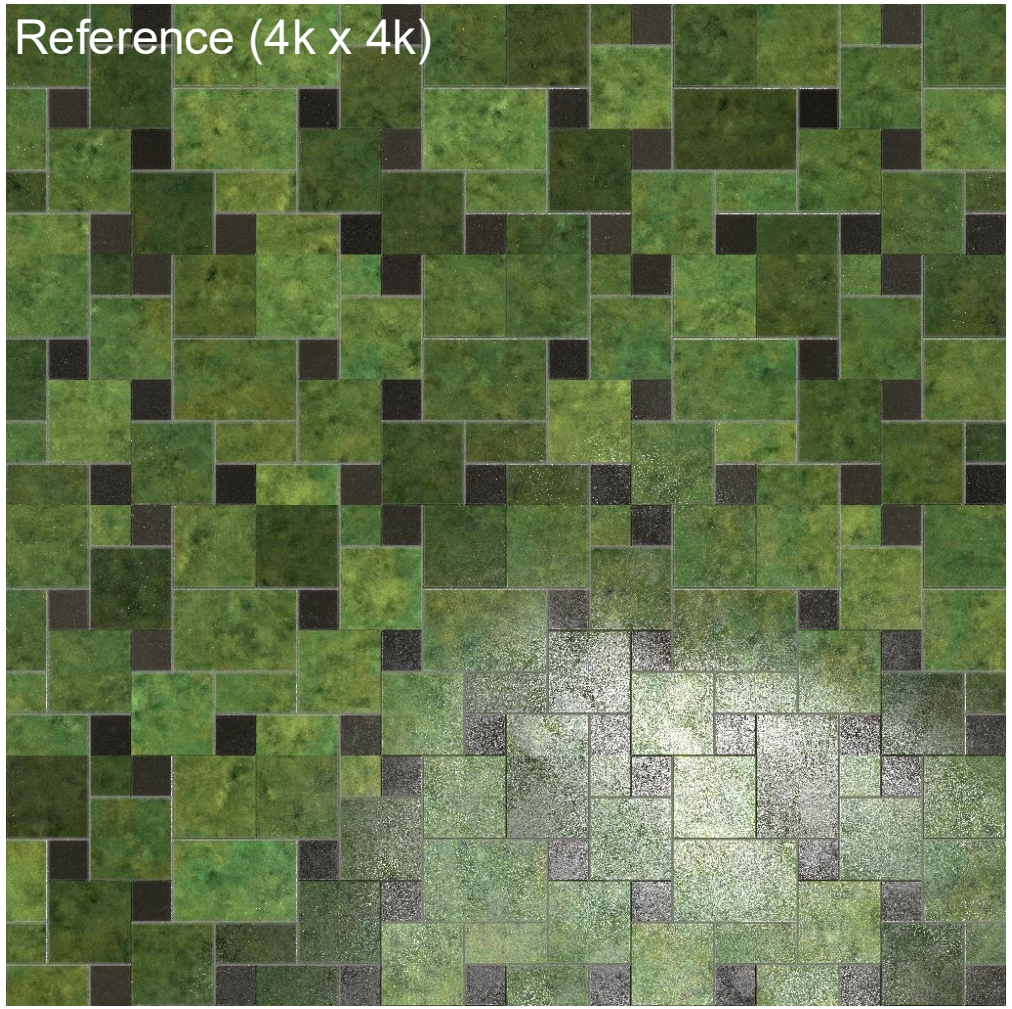
        float[64] h1 = layer1.forward(h0);
        [ForceUnroll]
        for (int i = 0; i < 64; ++i)
            h1[i] = leaky_relu(h1[i]);

        float[64] h2 = layer2.forward(h1);
        [ForceUnroll]
        for (int i = 0; i < 64; ++i)
            h2[i] = leaky_relu(h2[i]);

        return layer3.forward(h2);
    }
```

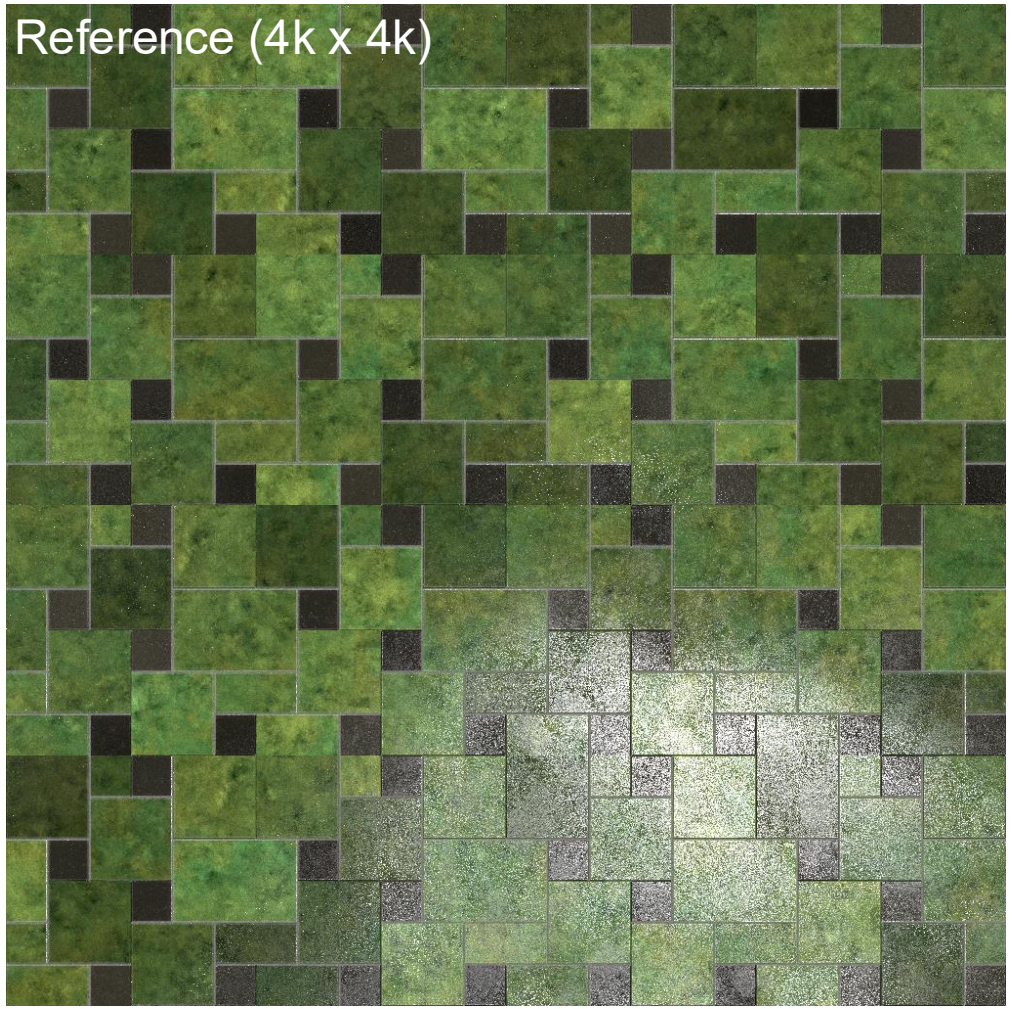
© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

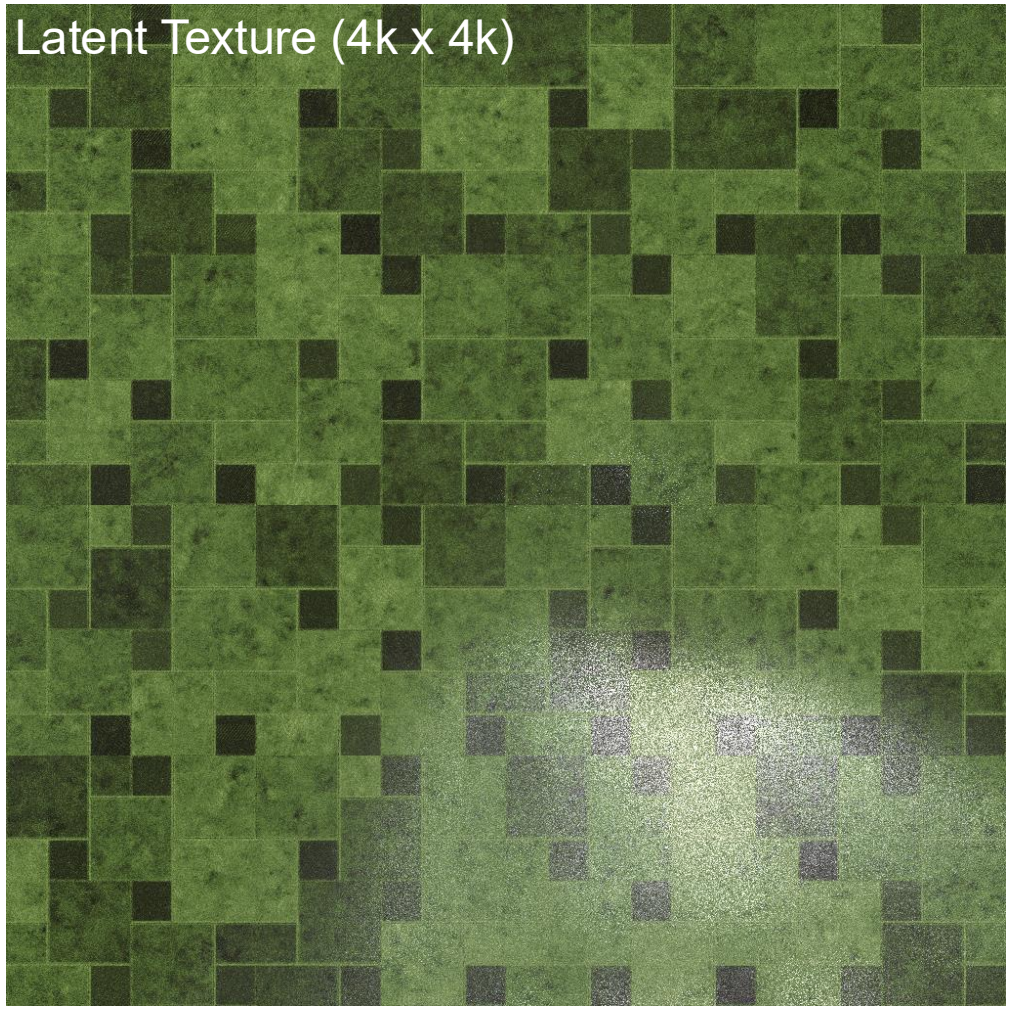
Results



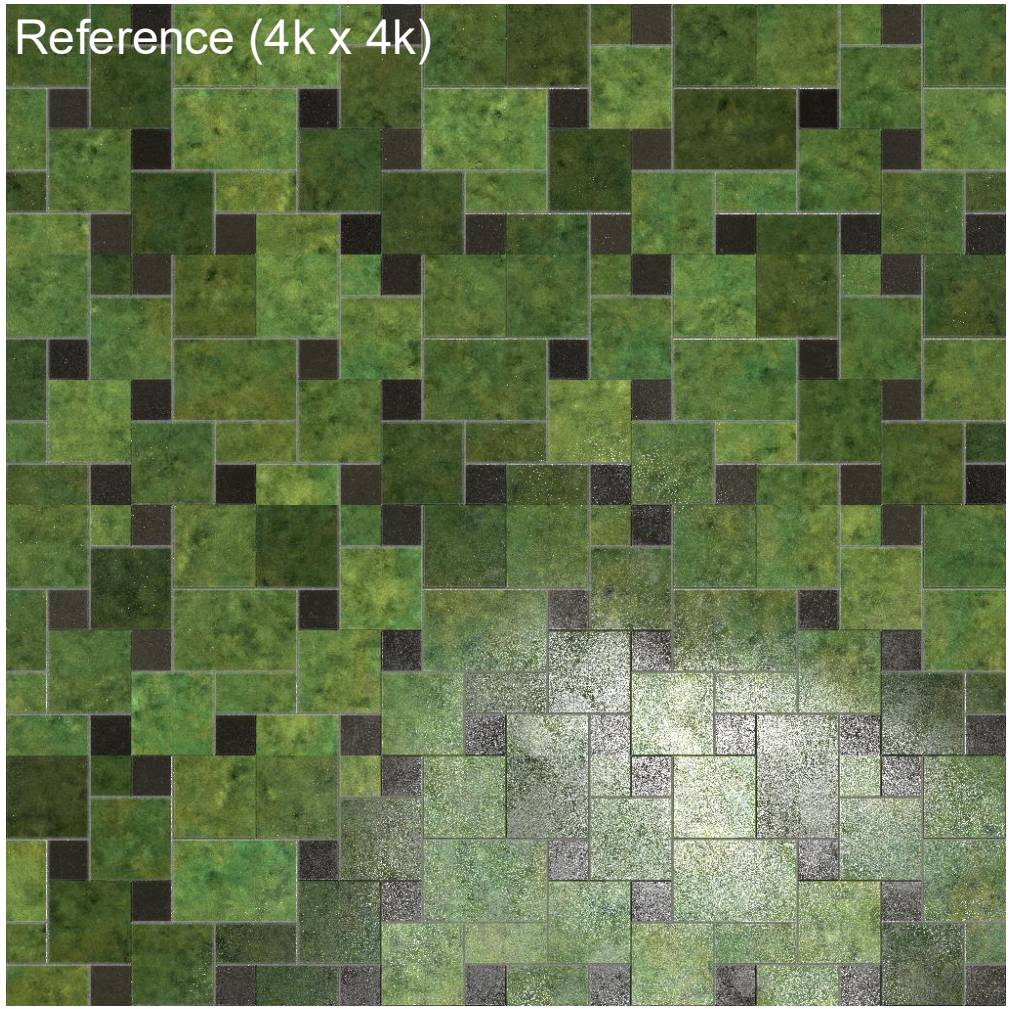


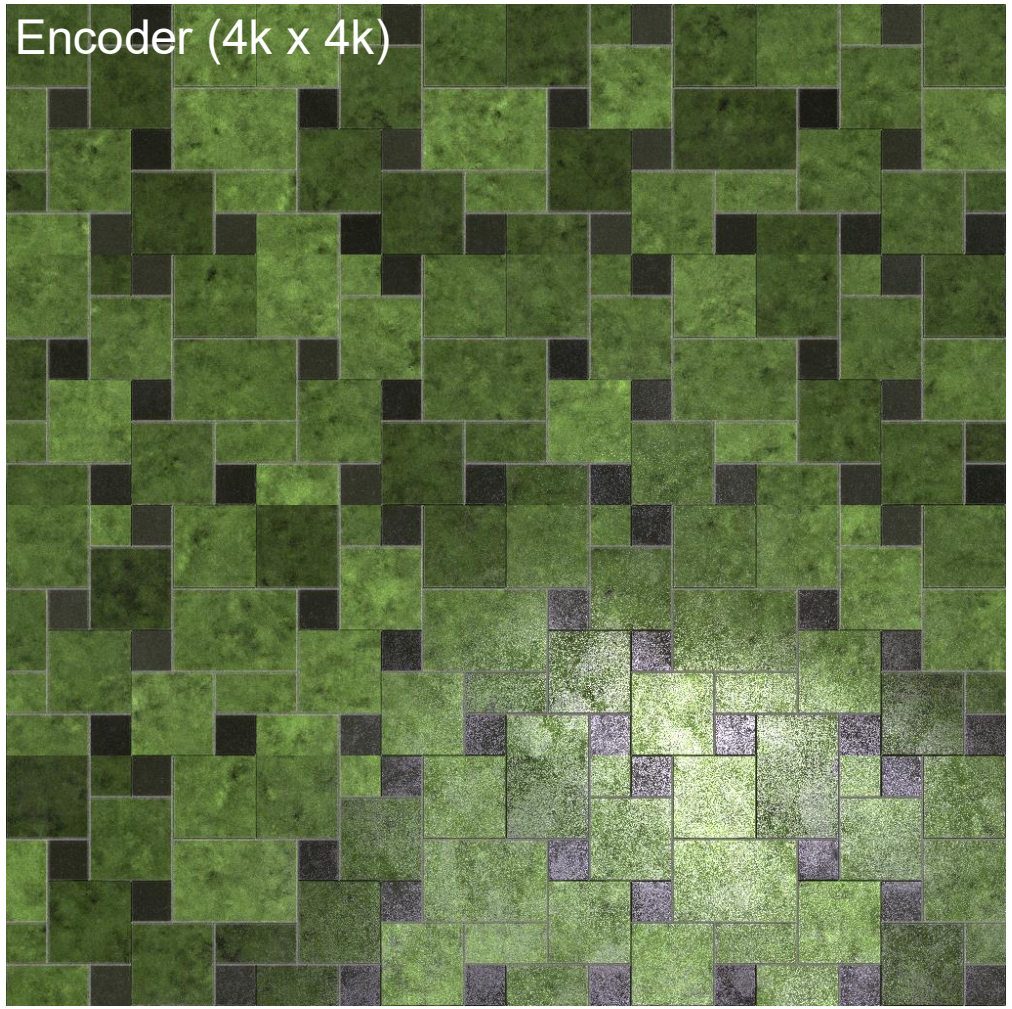
Results





Results





Variance in Neural Network training



Variance in Neural Network training



Reference



Neural Material

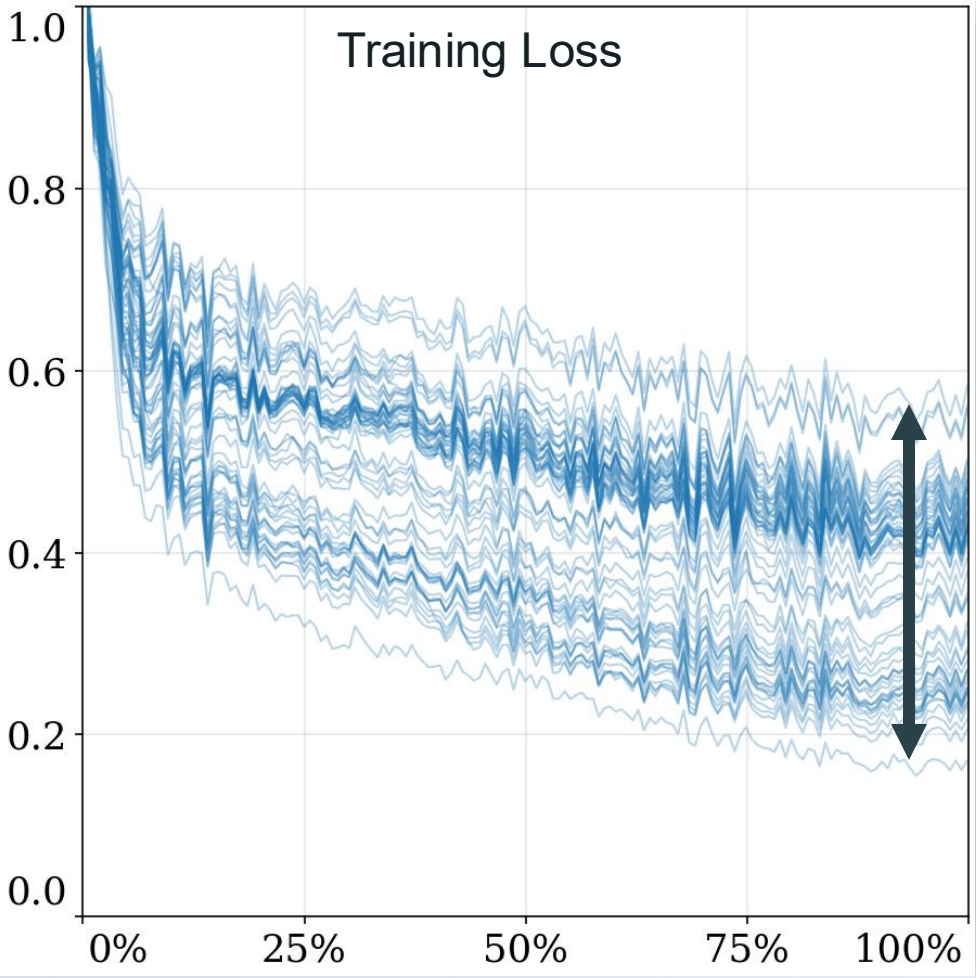


Variance in Neural Network training





Variance in Neural Network training

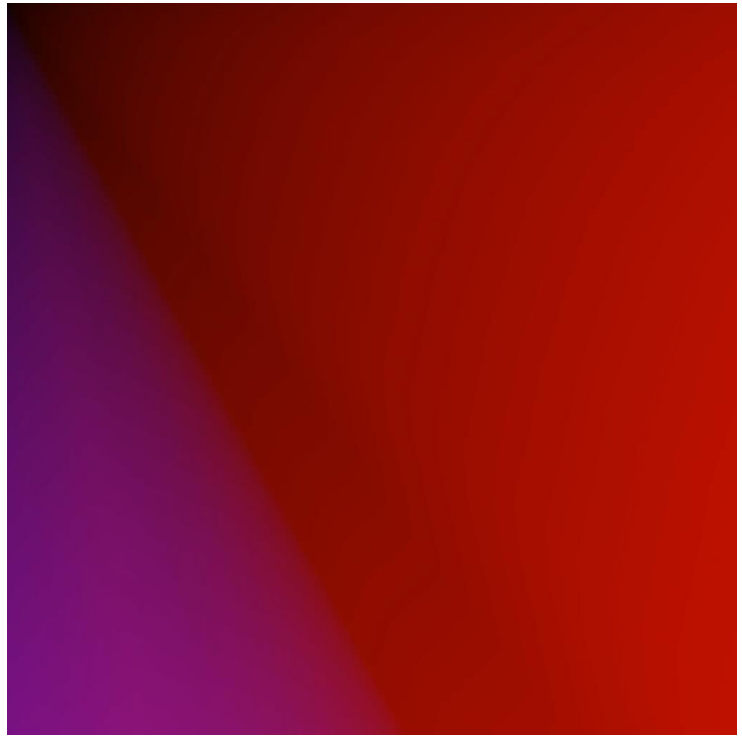




Callback: A closer Look at Activations



- Sometimes training gets “stuck”



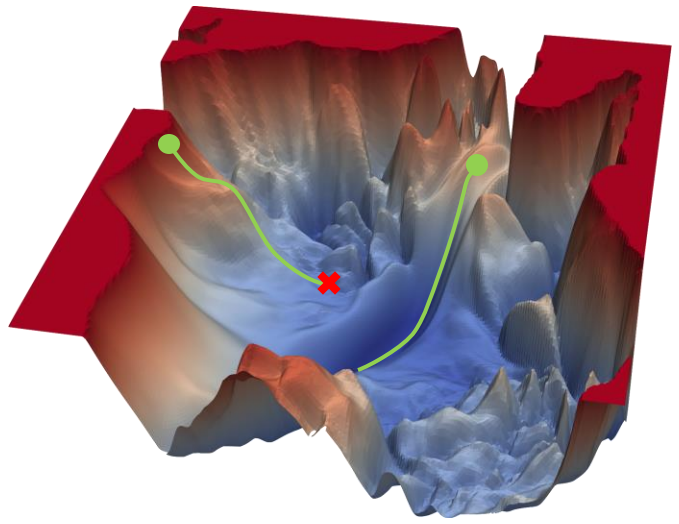
3 layers, 32 neurons

Variance in Neural Network training



- Sometimes training gets “stuck”!

- Loss is like a landscape
- Optimization tries to find a path from the initial weights to a loss minimum
- “Pits” in the loss landscape: The network gets stuck



Li et al.

Visualizing the Loss Landscape of Neural Nets

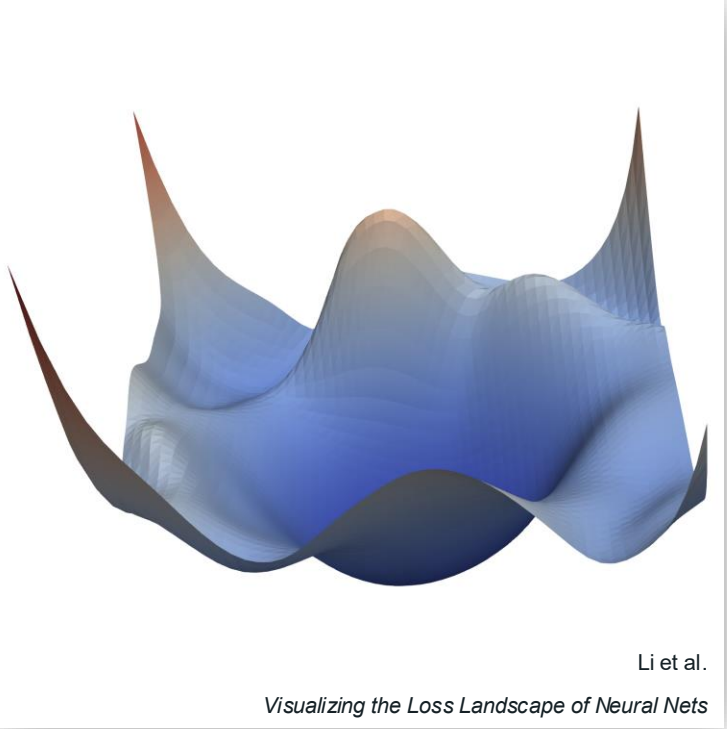
Variance in Neural Network training



- Sometimes training gets “stuck”!

- Loss is like a landscape
- Optimization tries to find a path from the initial weights to a loss minimum
- “Pits” in the loss landscape: The network gets stuck

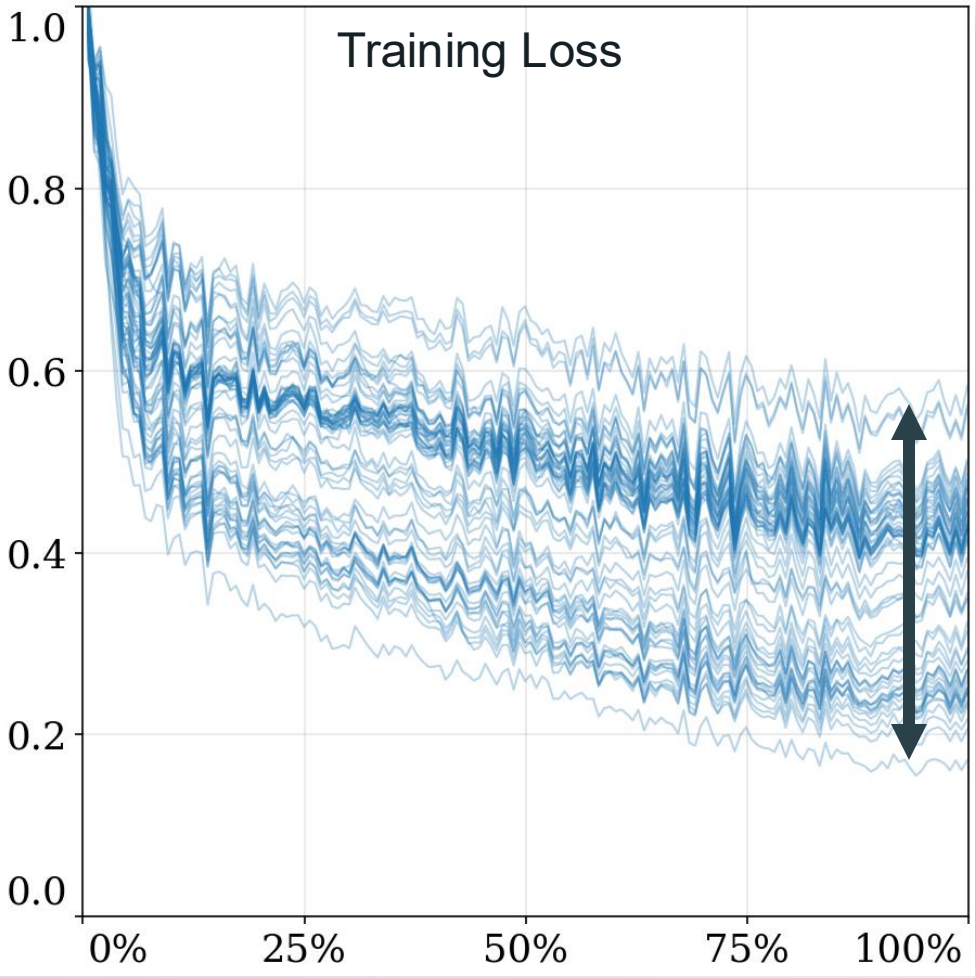
- Best solution: Make it easier to move around
  - Better activations, etc.
  - Larger models



Li et al.

Visualizing the Loss Landscape of Neural Nets

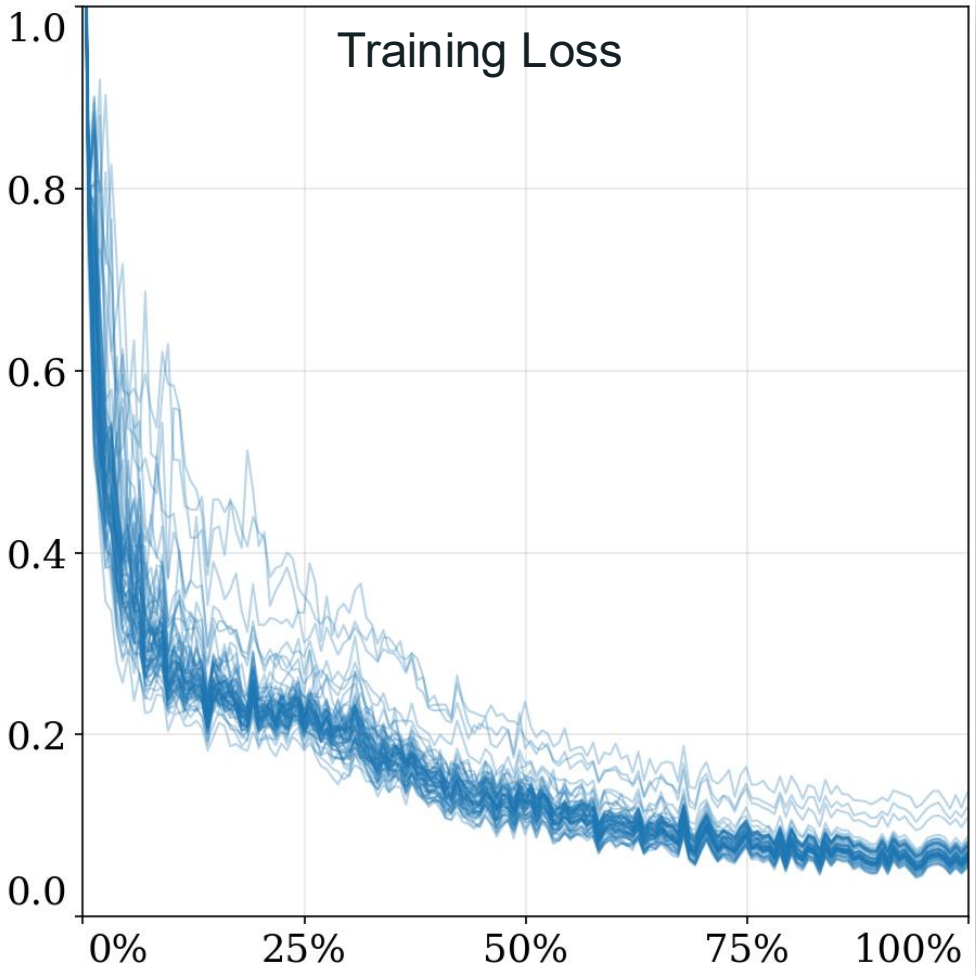
Variance in Neural Network training



Neural Material, 2x16



Variance in Neural Network training





Variance in Neural Network training



- Sometimes training gets “stuck”!

- Loss is like a landscape
- Optimization tries to find a path from the initial weights to a loss minimum
- “Pits” in the loss landscape: The network gets stuck

- Best solution: Make it easier to move around
  - Better activations, smooth gradients, etc.
  - Larger models

- In general: This problem can be difficult to avoid
  - We need small models
  - “Smarter” network modules (e.g. shading frame encoding)



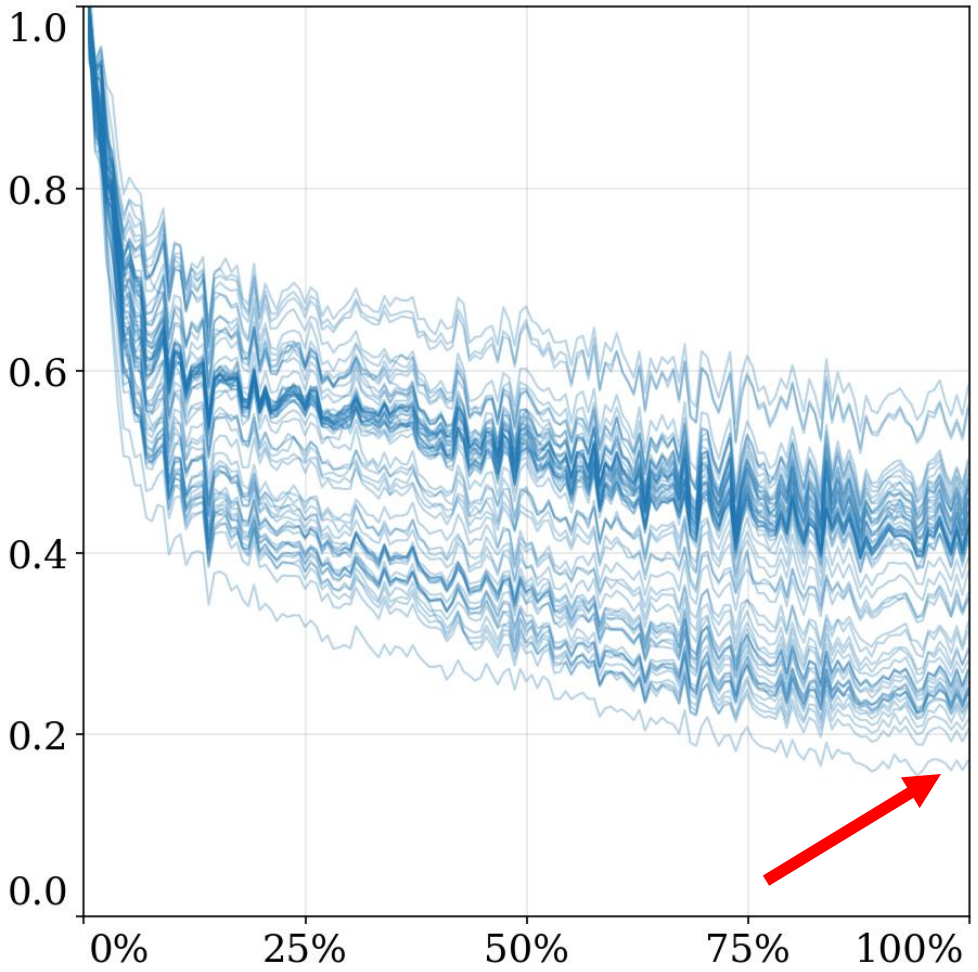
Reducing training variance



Brute force multi-instance training:

- Train the same model multiple times with different initial weights
- Pick the best one

Highly effective!



Reducing training variance

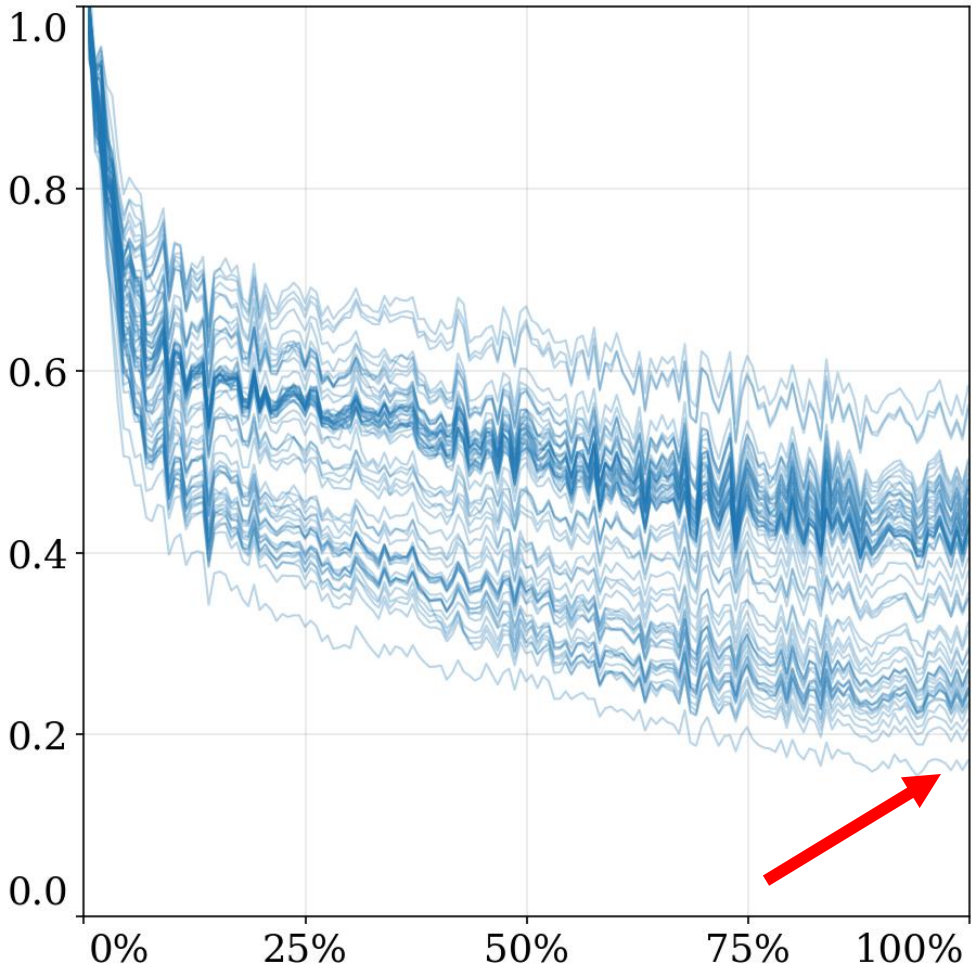


Brute force multi-instance training:

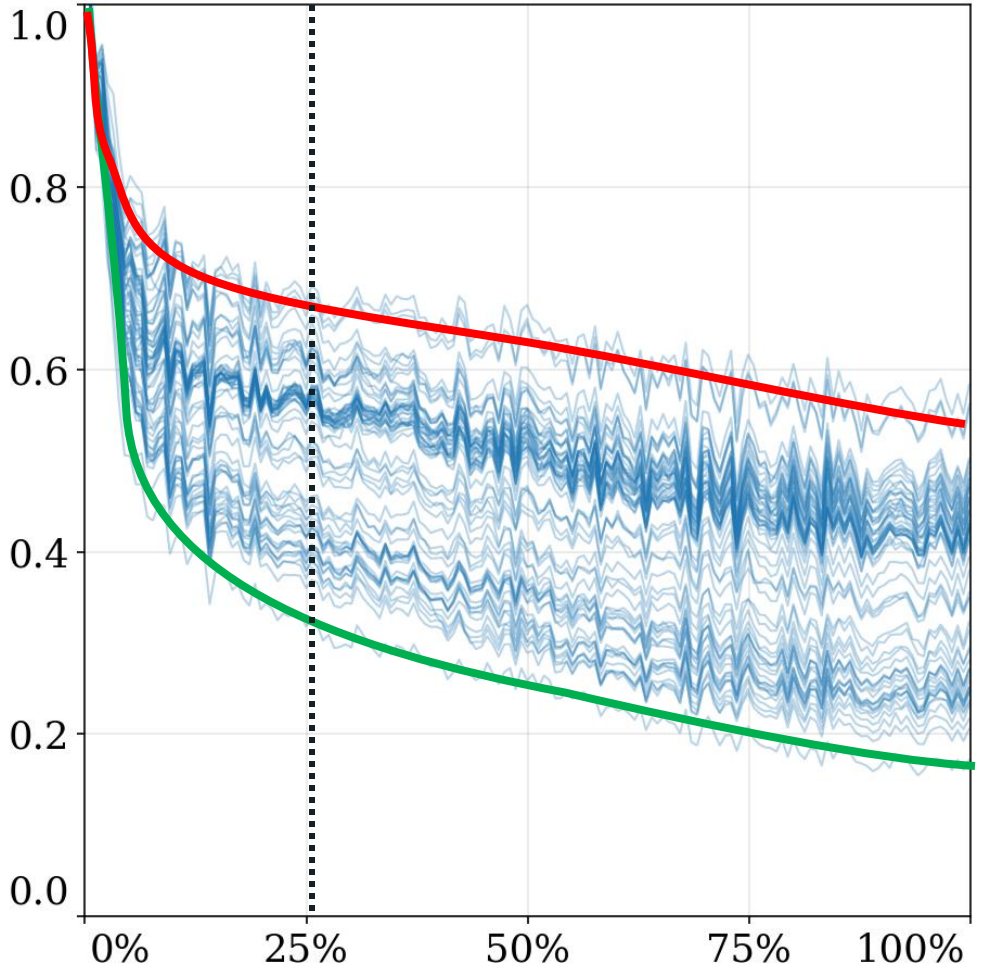
- Train the same model multiple times with different initial weights
- Pick the best one

Highly effective, but expensive!

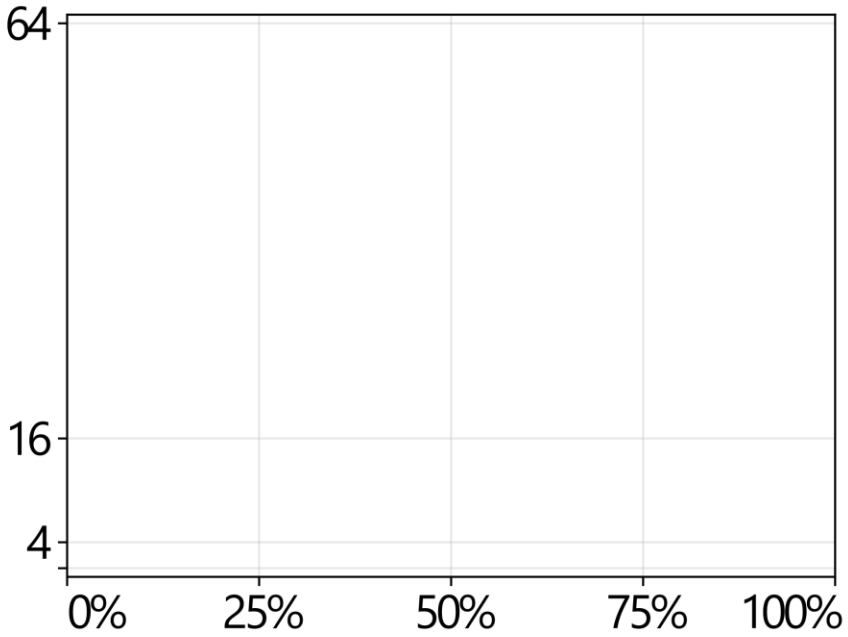
- Training 64 instances → 64x training cost

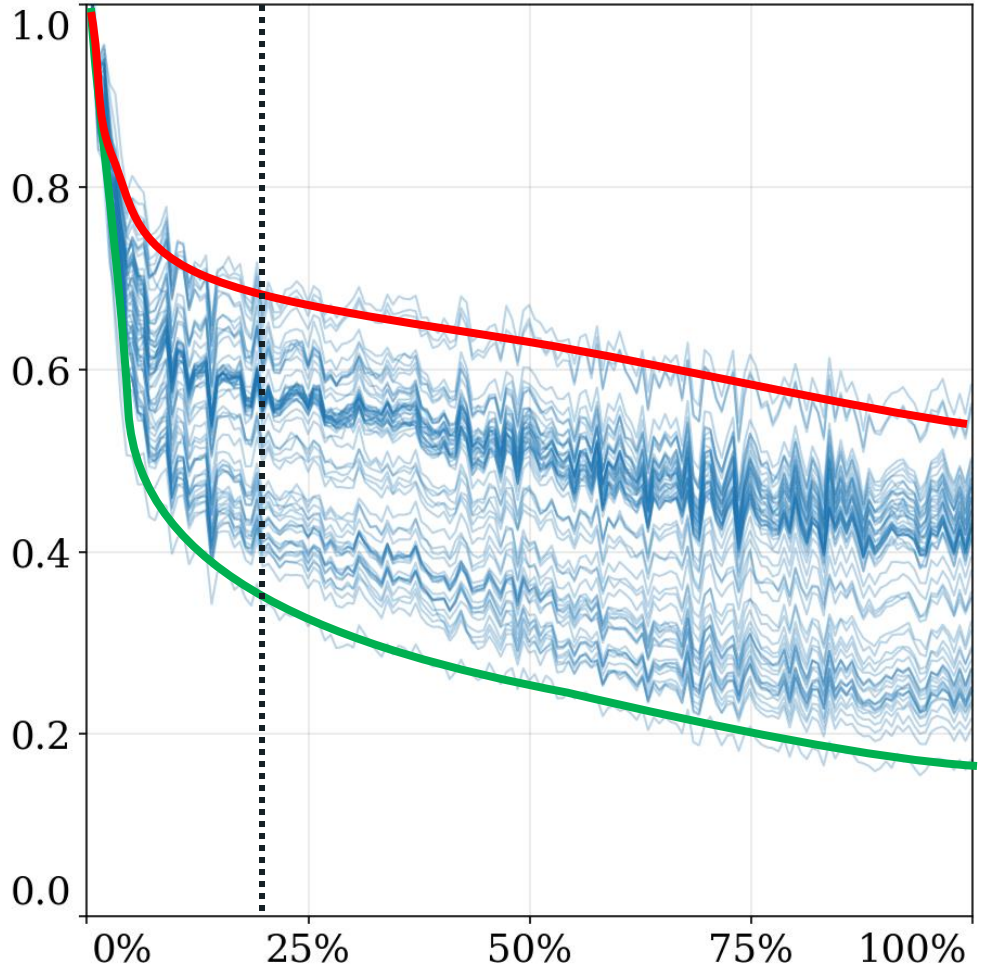


Do we have to wait until the end?

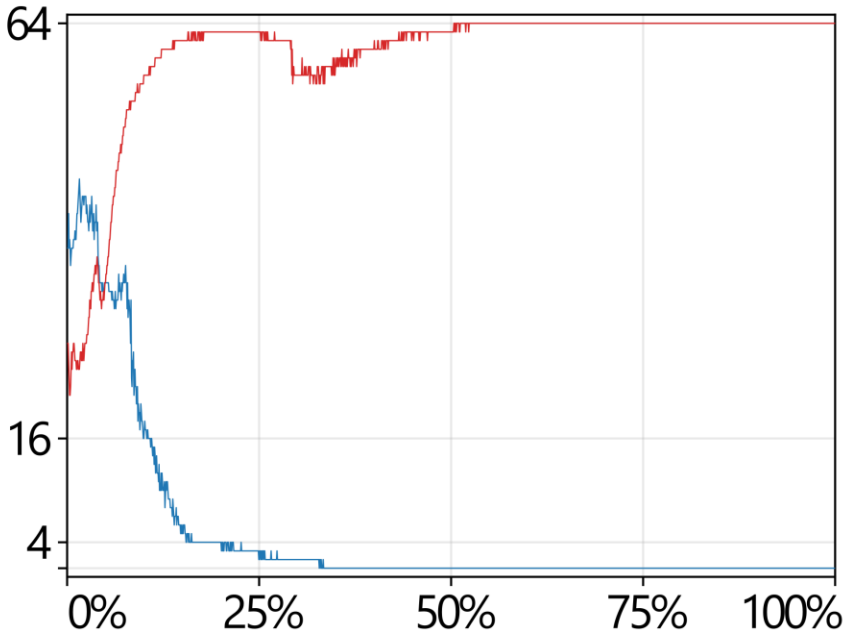


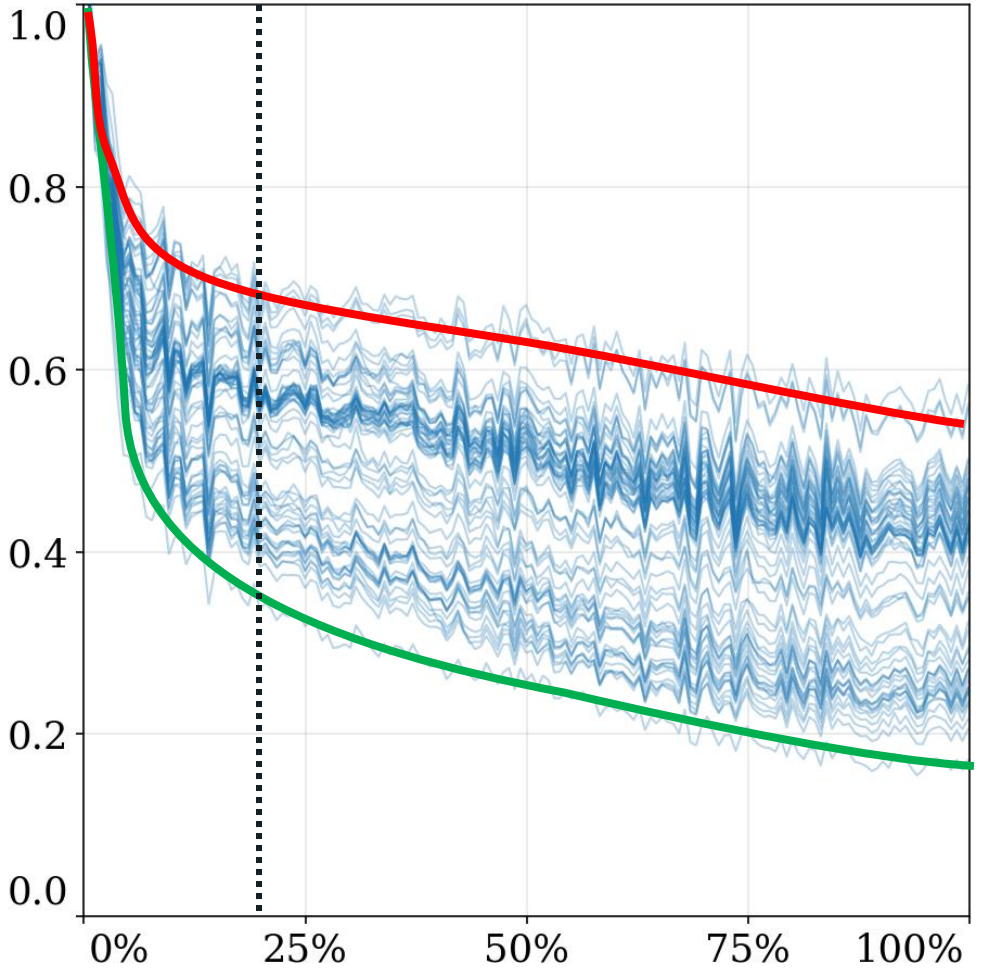
Do we have to wait until the end?



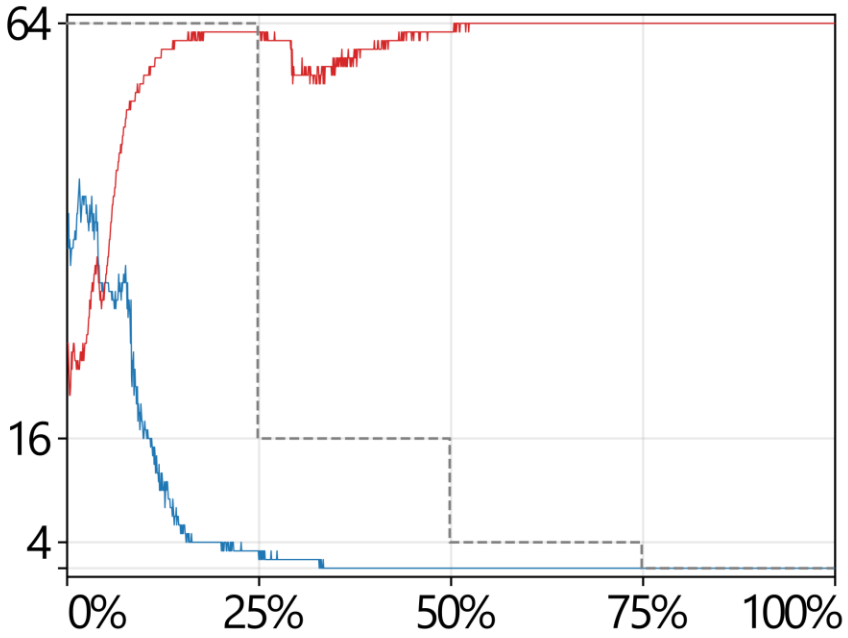


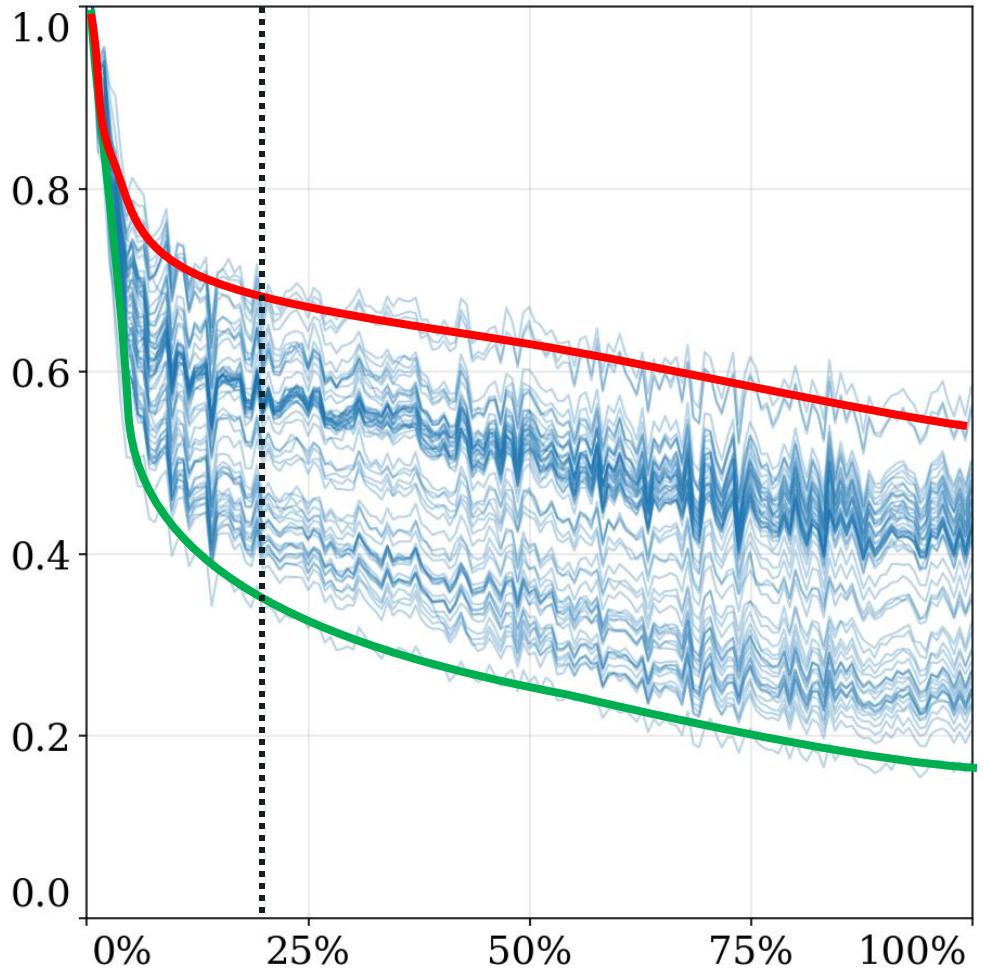
Do we have to wait until the end?





Do we have to wait until the end?



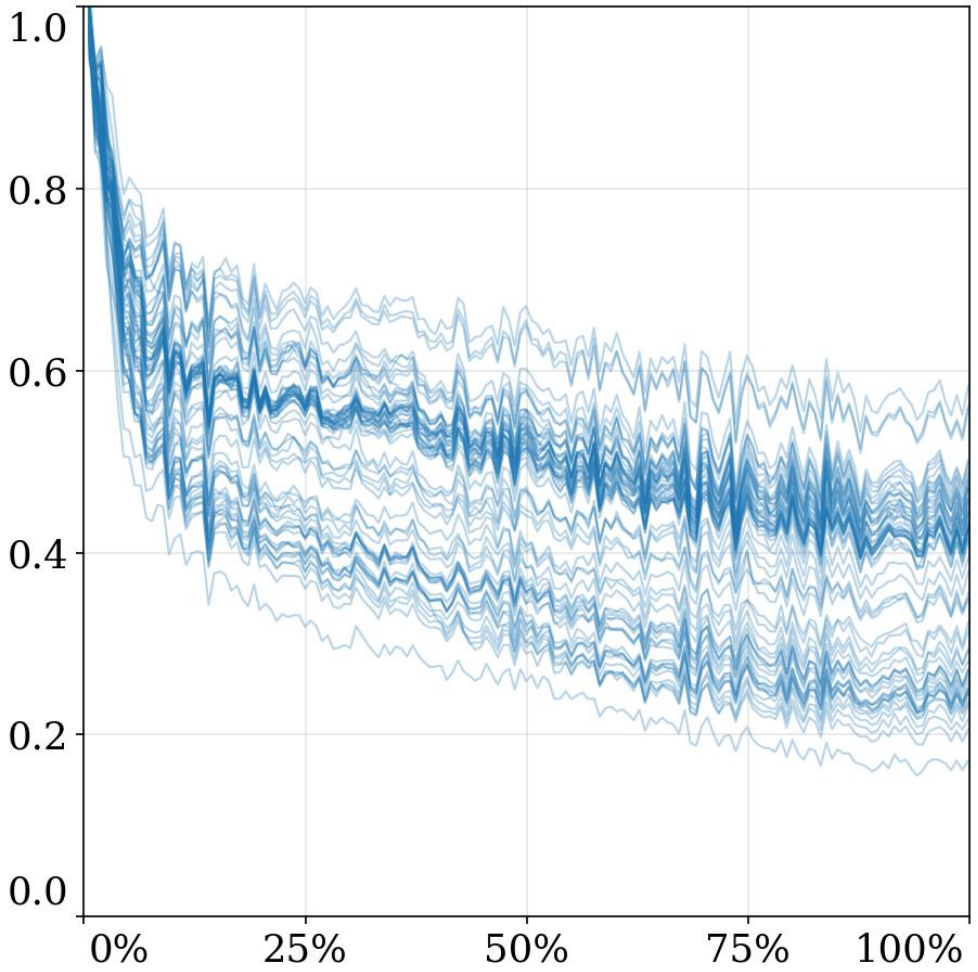


Reducing training variance



Multi-instance training with pruning:

- Train the same model multiple times with different initial weights



Reducing training variance

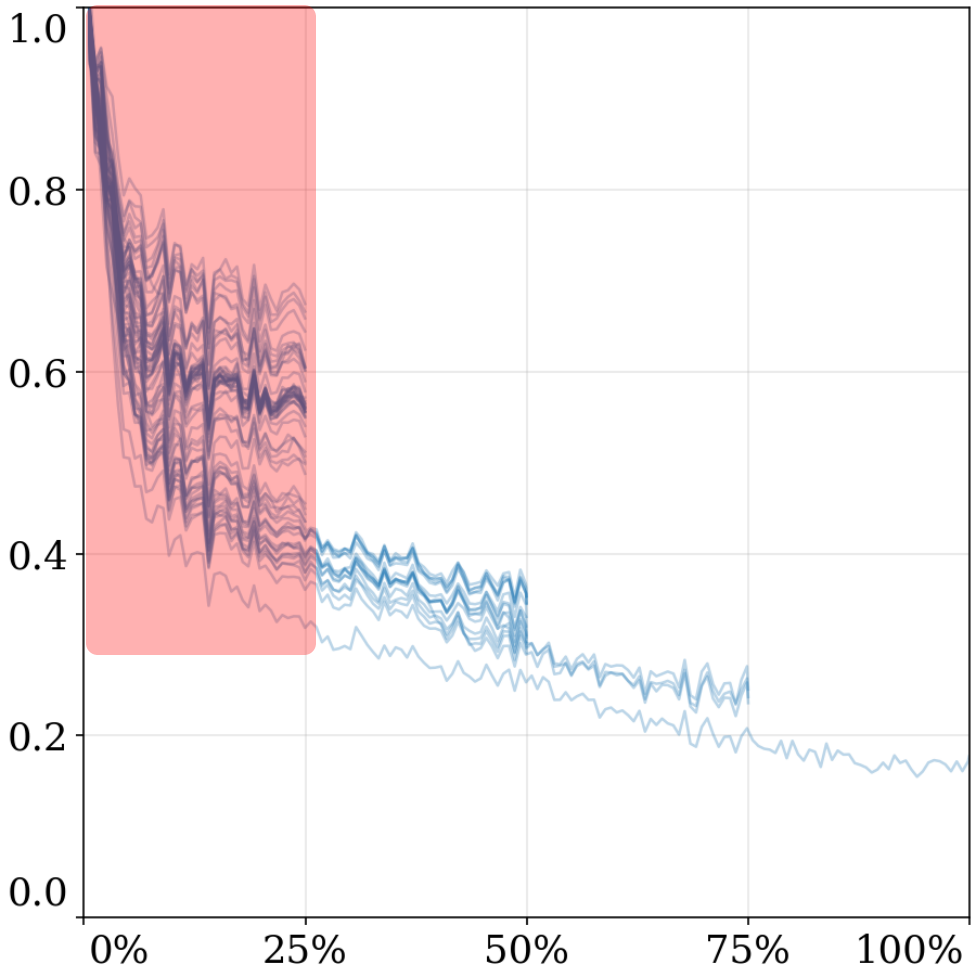


Multi-instance training with pruning:

- Train the same model multiple times with different initial weights
- At regular intervals, throw out the worst models

Training cost: 64x → ~21x training cost

- Most of the training cost is before we get a chance to prune
- Can we do better?



The impact of batch sizing



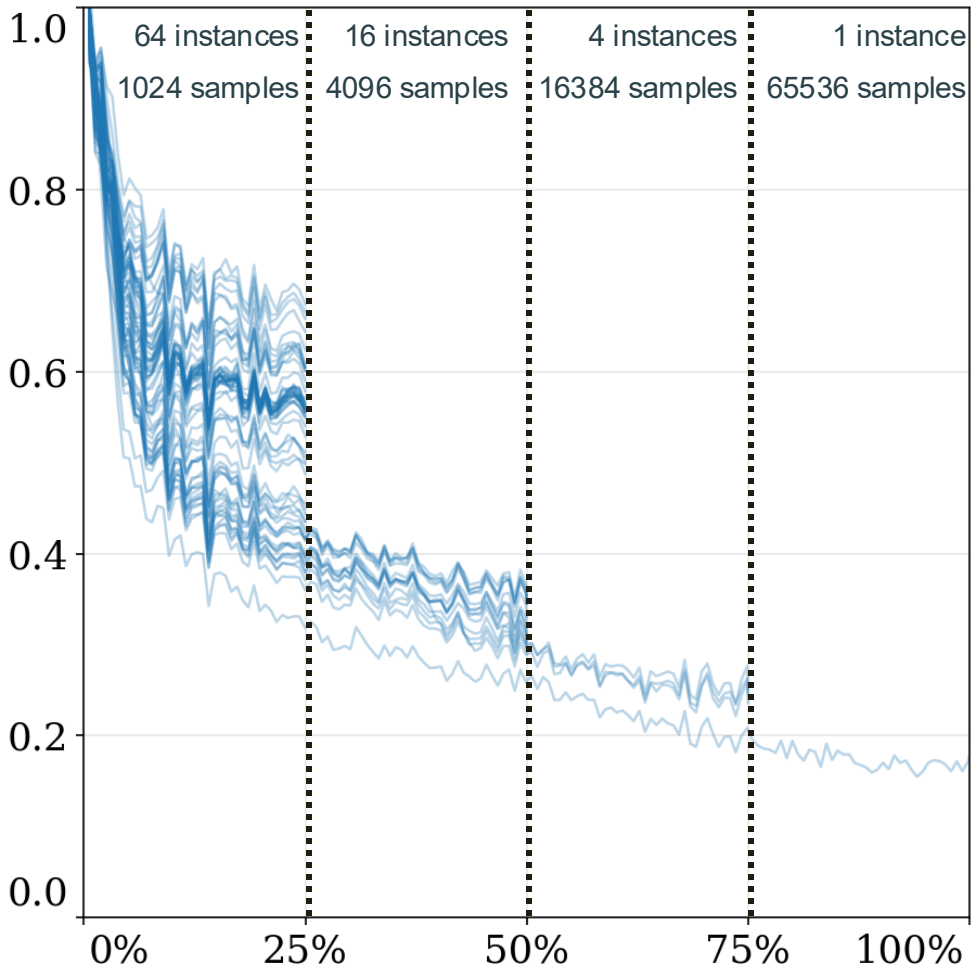
The size of the training batch can influence training behavior:

- Small batch → noisy gradients: More exploration
- Large batch → clean gradients: More exploitation

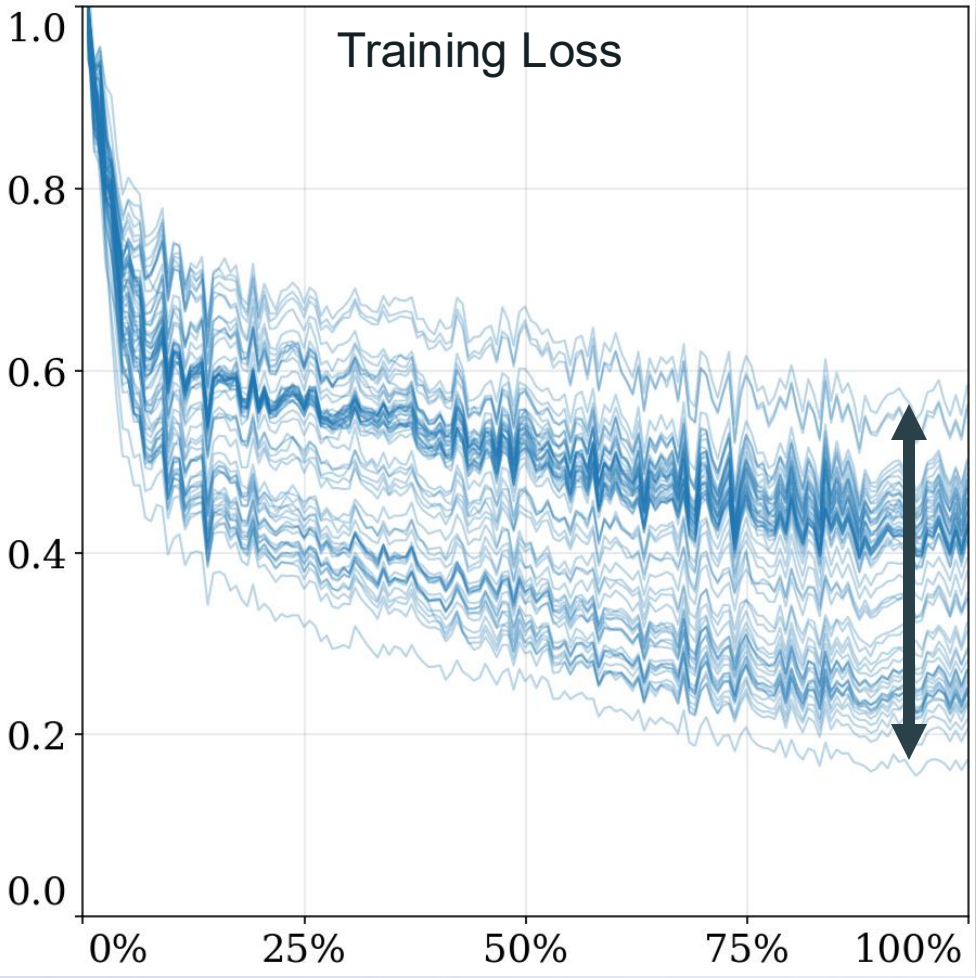
Middle ground: Ramp up batch size during training

- Small batches at beginning avoid local minima
- Large batches at the end find the best loss

Great synergy with multi-instance training!

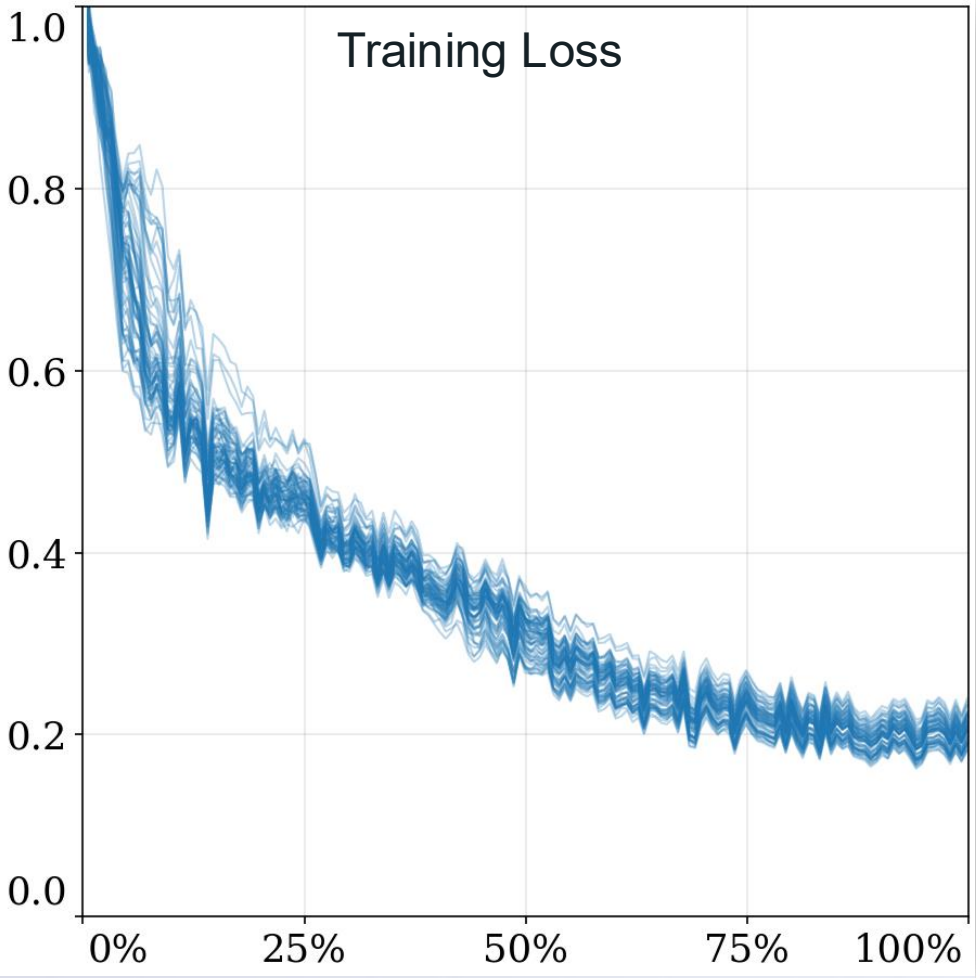


Results


Training Loss
1.0
0.8
0.6
0.4
0.2
0.0
0%
25%
50%
75%
100%

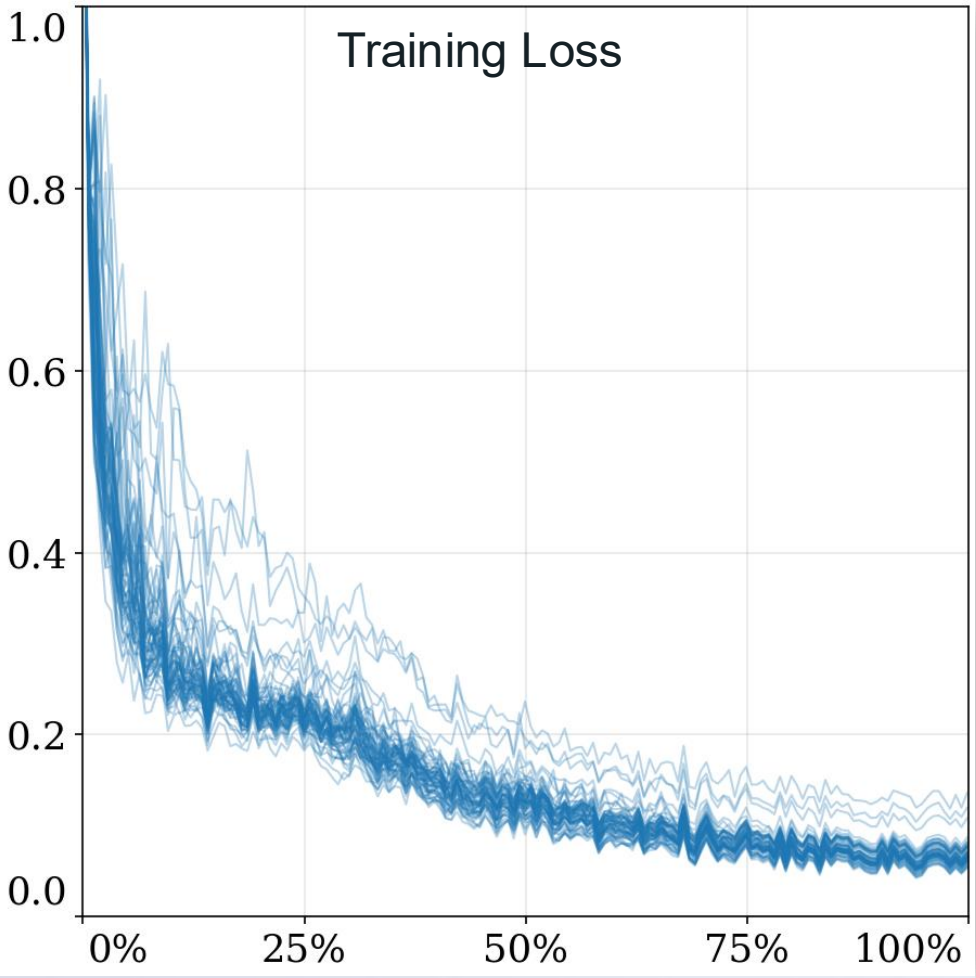

Neural Material, 2x16

Results

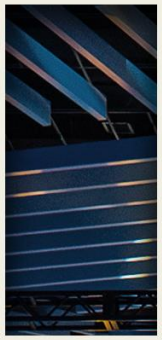

Training Loss
1.0
0.8
0.6
0.4
0.2
0.0
0%
25%
50%
75%
100%

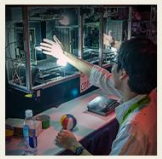

Neural Material, 2x16, Multi-Instance

Results


Training Loss
1.0
0.8
0.6
0.4
0.2
0.0
0%
25%
50%
75%
100%


Neural Material, 4x64







SIGGRAPH 2026
Los Angeles 19–23 JUL

Wrap-up

What we didn't cover



Low-level: Mechanics → **High-level: Applications**

Network components

- Network layers: Convolution, attention, residual, …
- Encodings: SIREN, hash grids, tri-planes …
- (Vector) quantization
- Normalization: Batch norm, group norm, layer norm, …

Training

- Optimizers: SOAP, Shampoo, Muon…
- Losses: L1/L2, KL-Div, perceptual (VGG, LPIPS, …)
- Data preparation

Architectures

- Autoencoders
- Transformers

Models

- Adversarial models
- Diffusion models
- Autoregressive models

Applications

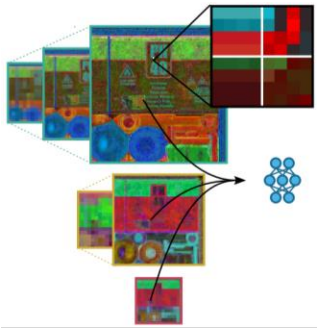
- Generative models
- World Models
- Inverse rendering: Splats, NeRFs, LRMs

Where to learn more

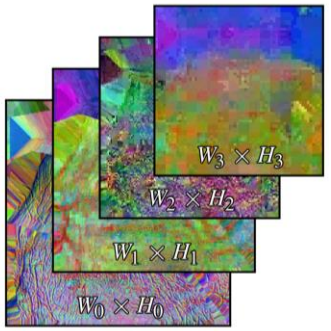
Neural Textures



Vaidyanathan et al. [2023]



Weinreich et al. [2024]



Belcour & Benyoub [2025]

Neural Materials



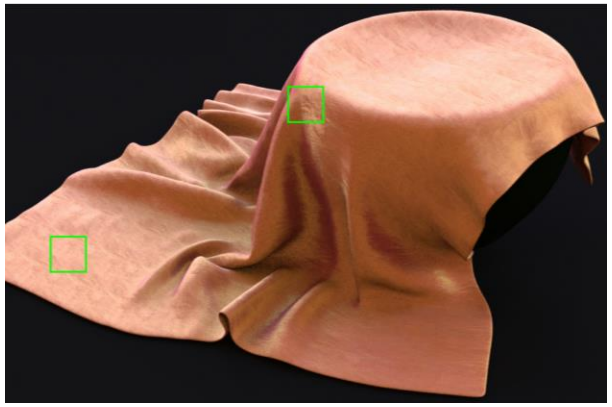
Rainer et al. [2020]



Kuznetsov et al. [2021]



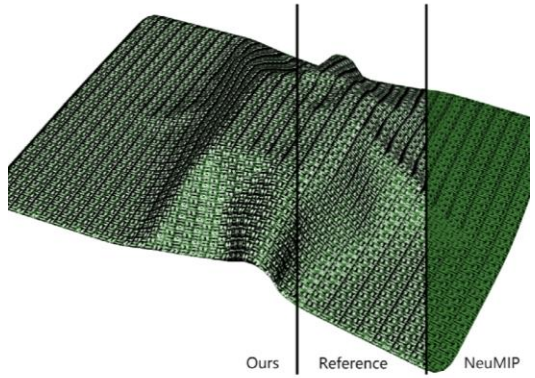
Zeltner et al. [2021]



Rainer et al. [2019]



Sztrajman et al. [2021]



Xue et al. [2021]

© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Where to learn more at SIGGRAPH

Technical Papers	
Monday 10:45am–12:35pm	Generative 3D & Editing
Tuesday 9:00am–10:30am	Generative 3D
Tuesday 10:45am–12:35pm	3D Generation
Tuesday 10:45am–12:15pm	Materials & Textures
Wednesday 2:00pm–3:30pm	Shading, Relighting and Volumes

Course Material:

https://shader-slang.org/landing/siggraph-26/

© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Where to learn more at SIGGRAPH

Technical Papers		Slang	
Monday 10:45am–12:35pm	Generative 3D & Editing	**Tuesday 10:15pm–11:45am**	Introduction to Slang
Tuesday 9:00am–10:30am	Generative 3D	**Tuesday 12:00pm–1:30pm**	Hands-On Machine Learning with Slang
Tuesday 10:45am–12:35pm	3D Generation	**Wednesday 1:30pm–2:00 p.m.**	Slang in Action: Modern Shading for Graphics, Compute, and Action
Tuesday 10:45am–12:15pm	Materials & Textures		
Wednesday 2:00pm–3:30pm	Shading, Relighting and Volumes		

Course Material:

https://shader-slang.org/landing/siggraph-26/

© 2025 SIGGRAPH. ALL RIGHTS RESERVED.

Where to learn more at SIGGRAPH

Technical Papers		Slang		Workshops & Talks	
Monday 10:45am–12:35pm	Generative 3D & Editing	**Tuesday 10:15pm–11:45am**	Introduction to Slang	**Wednesday 2:00pm–5:15pm**	Generative AI Meets Computer Graphics
Tuesday 9:00am–10:30am	Generative 3D	**Tuesday 12:00pm–1:30pm**	Hands-On Machine Learning with Slang	**Monday 2:00pm–5:15pm**	Controllable Generative Graphics for Games
Tuesday 10:45am–12:35pm	3D Generation	**Wednesday 1:30pm–2:00 p.m.**	Slang in Action: Modern Shading for Graphics, Compute, and Action	**Monday 3:45pm–4:45pm**	Neural Rendering, World Models, and Simulation
Tuesday 10:45am–12:15pm	Materials & Textures				
Wednesday 2:00pm–3:30pm	Shading, Relighting and Volumes				

Course Material:

https://shader-slang.org/landing/siggraph-26/

© 2025 SIGGRAPH. ALL RIGHTS RESERVED.



Q&A